

Handouts for S022 (Data Science in Education)

Luke Miratrix (with some others for some parts)

2025-02-07

Table of contents

Preface	11
A note on duplicated material	11
About Statistical Programming	11
The R Mystique	12
Do I take S-022? A self-assessment	13
A self-assessment quiz	14
Work Experience	14
Stat Courses	14
Programming	14
R Skills	15
Math	15
Other	15
Total the above	15
Assignment Formatting Guidelines	17
Formatting	17
Numbers	17
Plots	18
Avoiding bias	18
Conciseness	19
Self Grading Instructions	20
Acknowledgement:	20
I Getting Started	21
1 Getting Started with R	22
1.1 Some Resources and Directions	22
Handouts	22
Class Code	22
A Very Important Online Textbook	22
Class Sections	23
Office Hours	23
GSE Stat Help Desk	23

2	Tidy Projects for Analysis in R	24
	Organizing Projects for Research & Collaboration	25
	S022 Final Projects	25
	Organizational File Structure	25
	Using R Projects	26
	The <code>here</code> package	26
	Relative paths with <code>here()</code>	26
3	Resources on Learning R	28
3.1	Live Class Code and Sections	28
3.2	R for Data Science: The Very Important Online Textbook	28
3.3	Office Hours	29
3.4	GSE Stat Help Desk	29
3.5	R Cheat Sheets	29
3.6	Further information on using R Markdown	29
3.7	Useful links to materials on the web	29
3.8	Writing your own math	30
4	An R Code Style Guide (Miratrix version)	31
4.1	Why have coding style?	31
4.2	The naming of things	32
	4.2.1 Don't use common names	32
	4.2.2 Name constants for clarity	32
4.3	Syntax	33
	4.3.1 Appropriate spaces make code readable	33
	4.3.2 Judicious spaces group elements to increase readability	33
	4.3.3 Lots of spaces to line things up is nice	34
	4.3.4 Including names when calling functions	34
	4.3.5 No line of code should be longer than 80 characters	34
	4.3.6 Use <code><-</code> for assignment (if you are prissy)	35
	4.3.7 Double quotes are standard	36
	4.3.8 Comments are good	36
4.4	Pipes (<code>%>%</code>) with <code>magrittr</code>	36
	4.4.1 Surround the pipe with whitespace	37
	4.4.2 Formatting short pipes	37
	4.4.3 Do not pack pipes inside of other pipes	37
	4.4.4 Keep your parenthesis even if you have no arguments	38
	4.4.5 Rule 4.6: Assignment	38
4.5	A few final remarks	39

II	On markdown	40
5	Intro to R Markdown	41
5.1	Overview	41
5.1.1	Handy Resources:	41
5.2	Getting started: Formatting Your Document	42
5.3	Embedding R Code	43
5.3.1	Formatting Individual Code Chunks	44
5.3.2	Setting Global Options Code Chunks	45
5.3.3	Inline code	45
5.4	Embedding plots	45
5.5	Embedding tables	46
5.6	Embedding math	48
5.7	Help! R Markdown report generation doesn't work	48
6	Creating Reports in Quarto	50
6.1	What is Quarto?	50
6.2	Using Visual Editor with Rmd	50
6.3	Creating a Quarto Report	50
6.4	Rendering to PDF	52
6.5	A Couple Quick Tricks	53
6.6	Additional Quarto Resources	53
7	Working with Rmarkdown chunks	54
7.1	Markdown chunks	54
7.1.1	Options for including/suppressing code and output	54
7.1.2	Options for including/suppressing R messages	54
7.1.3	Options for modifying figure outputs	55
7.2	Changing your defaults	57
8	Making tables in Markdown	58
8.1	Making a “table one”	60
8.2	The stargazer package	61
8.3	The xtable package	62
III	On Plotting	64
9	Intro to ggplot	65
9.1	Summarizing	67
9.2	Grouping	69
9.3	Customization	69
9.4	Themes	71
9.5	Next steps	72

10	Some tips for making plots	73
11	Setting a theme	74
12	Formatting axes and making labels	76
13	Making legends using aes()	77
14	Cropping vs. zooming in	78
15	Controlling size in markdown files	80
16	Making the fonts of labels, etc., larger	81
17	Saving high resolution plots	84
18	Think about the height of a plot.	85
19	Think about the x and y axis of your plot	89
20	Controlling colors, etc	91
21	Tennessee Star Handout	92
21.1	Re-creating the Tennessee STAR Boxplot	92
21.1.1	(Optional) Recreating the boxplot without manipulating the original dataset	99
21.2	Bundling plots with grid.arrange()	100
21.3	Improvements on the Tennessee STAR Boxplot	102
21.3.1	What if we want to add a different color for the small class sizes to highlight the distributions in that category?	102
21.3.2	What if we want to annotate all the boxplots corresponding to the the small class size category?	103
21.4	What about adding text to the annotated Tennessee Star Boxplot?	105
22	Plotting Two Datasets at Once	107
23	Plotting aggregate data with ggPlot	110
23.0.1	Preface	110
24	The Situation	111
24.1	Categorical data and bar charts	114
24.2	A Closing Thought	116
25	Jitter and heatmap examples	117
26	Jitter examples	118

27 Making a heat map	127
28 Using pivot methods to make line graphs	128
28.1 Overview	128
28.2 Our data	128
28.3 Aggregating	129
28.4 Graphing (take 1)	130
28.5 Graphing (take 2)	131
28.6 Graphing (another road)	132
29 Prediction and Visualization with Models	134
29.1 Fit a model	135
29.2 Generate a Prediction Grid	135
29.3 Add Predictions to the Grid	138
29.4 Plot Predicted Trends	138
29.5 Extensions	139
29.5.1 Residuals and Diagnostics	139
29.5.2 Multiple Models	140
29.5.3 Fancy Plotting with <code>spread_predictions</code>	141
29.6 Handy Functions Recap	142
29.7 Final Thoughts	142
IV On Coding	143
30 Basic Data Manipulation with tidyverse	144
30.1 <code>filter()</code> (Grab the rows you want, 5.1)	144
30.2 <code>arrange()</code> (Sort your rows the way you want, 5.2)	145
30.3 <code>select()</code> (Grab the columns you want, 5.3)	145
30.4 <code>mutate()</code> (Make new variables out of your old ones, 5.4)	146
30.5 <code>group_by()</code> and <code>summarize</code> (Summarize your data by subgroup, 5.6)	146
30.6 Special: grouped mutates (Making new variables within subgroups, 5.6)	147
31 A Deep Dive on <code>group_by()</code>	149
32 Importing the 'dplyr' package	150
32.1 What does <code>group_by()</code> do?	150
32.2 Introducing <code>summarize()</code>	152
32.3 Combining <code>group_by()</code> with <code>summarize()</code>	152
32.3.1 Getting fancy: Grouping with more than one variable	153
32.4 Using <code>mutate()</code> with <code>group_by()</code>	154
32.5 Learning Check: What is the difference between pairing <code>group_by()</code> with <code>summarize()</code> vs <code>group_by()</code> with <code>mutate()</code> ?	155

32.6 Practical Application: Centering Data	156
32.6.1 Group Mean Centering	157
33 Intro to Regression	160
33.1 Simple Regression	160
33.2 Multiple Regression	164
33.3 Categorical Variables (and Factors)	165
33.3.1 Numbers Coding Categories.	165
33.3.2 Setting new baselines.	166
33.3.3 Testing for significance of a categorical variable.	166
33.3.4 Missing levels in a factor	167
33.4 Some extensions (optional)	169
33.4.1 Confidence Intervals	169
33.4.2 Prediction	170
33.4.3 Removing Outliers	171
33.4.4 Missing data	171
33.4.5 Residual plots and model fit	172
34 Doing things over and over again	176
35 The replicate() command	177
36 The rerun() command	179
36.1 Warning: replicate() doesn't do well with fancy functions	181
36.2 Take-away	182
37 map(), another way of repeating yourself	183
38 Repeating yourself when you are repeating yourself	185
39 Advanced stuff	187
39.1 relocate()	187
39.2 pull()	187
39.3 set_names(list, names)	188
39.4 unpack() and pack()	188
40 Exploring map and friends (tidyverse)	190
41 Overview	191
42 The map() function (R for DS 21.5)	192
42.1 Making data frames	193
42.2 The unlist() function	193
42.3 Extra arguments to map	194

43 Banging on Dataframes	195
43.1 Chaining <code>map</code>	196
43.2 Dataframes are lists of the columns	197
43.3 The <code>transpose()</code> function	198
44 List columns and notes on R for DS 25 (25.2)	200
44.1 List columns	200
44.2 <code>nest()</code> and <code>unnest()</code>	201
44.3 Using <code>summarise()</code>	203
V On Missing Data	205
45 An Introduction to Missing Data	206
45.1 Introduction	206
45.2 Visualizing missing data	207
45.2.1 Missing data plots with the VIM Package	209
45.3 Complete case analysis	211
45.4 Mean imputation	212
45.4.1 Doing Mean Imputation manually	213
45.4.2 Mean imputation with the Mice package	213
45.5 Regression imputation	218
45.5.1 Manually	218
45.5.2 Mice	219
45.5.3 Stochastic regression imputation	221
45.6 Multiple imputation	224
45.7 Extensions	229
45.7.1 Multi-level data	229
45.7.2 Longitudinal data	229
45.8 Further reading	232
45.9 Appendix: More about the mice package	233
45.10 Appendix: The amelia package	237
46 Using mice to handle categorical data	241
46.1 Adding a missing category	242
46.2 Mean imputation on dummy variables	243
46.3 Using Mice to impute values	244
46.3.1 A coding trick to get the defaults	246
46.4 References	246
VI On Machine Learning	247
47 A demonstration of different machine learning methods all fit to the same data	248

48 Overview	249
49 Setup	250
49.0.1 Load in the birthweight data.	250
50 Baseline linear model (OLS)	252
51 Forward stepwise selection	253
52 Ridge Regression	256
53 Lasso Regression	260
54 Single Tree (a CART)	262
55 Random Forests	265
55.1 Tuning the random forest	267
56 Comparing our machines	270
57 Reflections on using machine learning (for final projects)	274
58 Test/train splits, test/train/validate, and cross validation	275
58.1 Cross validation options for caret	276
58.2 Bootstrap vs. Cross validation	276
59 Determining what variables are important	277
60 Sensitivity check on trees	281
61 A cool and easy way to do causal inference type stuff	282
62 Interpreting LASSO models, and some other stuff	283
63 The Story	284
64 Fitting and interpreting LASSO	285
64.1 What variables to include?	288
64.2 Coefficient size	288
64.3 Assessing stability (a kind of inference)	290
64.4 Last words on writing up results	291
VII On the Bootstrap	293
65 Demonstration of the Cluster Bootstrap	294

66 The Cluster Bootstrap	295
66.1 The Tenn Star Data	296
66.2 The Wrong Method	297
66.3 Cluster robust standard errors	298
66.4 Cluster Bootstrap	298
 VIII Extra Stuff	 302
67 Cleaning Excel files	303
 68 Main effects, interactions in linear models, and prediction	 307
68.1 Getting the data ready	307
68.2 Fitting an interacted model	309
68.3 Plotting	311
68.4 Centering, an alternative approach	312
68.5 Confidence intervals for the gap?	313
68.6 A final investigation	314
68.7 Disclaimer	315
 69 Finding and Merging Data Online	 317
69.1 Datasets Posted on the Web	317
69.2 IPEDS	317
69.3 Finding IPEDS Data	318
69.4 Our Example Case	320
69.4.1 Merging the Admissions and Salary Data	321
 70 Applied Prediction Papers	 323
 71 <i>Surveys and reviews</i>	 324
 72 <i>Original applications</i>	 326

Preface

This “book” is basically a set of handouts generated for a Data Science in Education course taught at the Harvard Graduate School of Education by Luke Miratrix. They are loosely organized, but the primary purpose of this “book” is to make all the handouts easy to access and find.

The handouts were generated in response to student questions, and aim to short-circuit staring at the possibly confusing world of Stack Overflow. They are curated in the sense that these activities tend to come up over and over in the course of fundamental data science work.

I hope you find this resource useful. Please send feedback to lmiratrix@g.harvard.edu.

A note on duplicated material

Some of this material is taken from a sister “textbook” for my MLM course (S043/Stat151). That textbook is [here](#). In particular, you will find entire chapters of that textbook replicated here.

About Statistical Programming

Statistical programming entails using a computer to analyze data in a way where you design and control the analysis and sometimes aspects of data generation (simulation). This is a step (or possibly several steps) beyond simply running built-in commands and interpreting output. It includes cleaning and transforming data to suit your needs, designing visualizations and plots to illustrate points you want to make, and conducting analyses that are tailor-made to your questions of interest.

S022 will make extensive use of the statistical software package R, which runs on both PCs and Macs. The software is [free and available online](#). R is straightforward to learn, but is sufficiently powerful and versatile to be useful for real projects that you might carry out after this course. It is used widely in many other statistics courses and also in research in such fields as education, psychology, economics, medical research, epidemiology, public health, and political science. See below for further argument as to why you should learn R.

We highly recommend using RStudio, which makes using R easier. RStudio is an Integrated Development Environment (IDE) that structures your experience, helps keep things organized, and offers multiple time-saving features to make your programming experience better. You might also consider R Markdown. R Markdown allows for generating documents with embedded R code and R output in a clean format, which can greatly help report generation.

We will teach R and the associated tools as part of this course. We assume no background knowledge with statistical programming or any other programming.

The R Mystique

Many people seem to believe that R is particularly technically challenging and difficult to master. This probably stems from its extreme flexibility; it is a fully functional programming language as well as a statistical analysis package. R can do things that many other software packages (I'm looking at you, Stata) essentially can't. These more involved things are frequently hard to do because they require you to think like a programmer rather than a data analyst. As a result, R is perceived as a "hard" language to use.

However, for straightforward, off-the-shelf analyses, R is arguably easier to learn and use in many ways than programs such as Stata or SAS. This is especially true when R is combined with RStudio for a better programming experience.

Don't believe me? Take S022, have us teach you some R, and find out for yourself.

Do I take S-022? A self-assessment

I received numerous inquiries as to whether a given background is enough to take S022. Let me elaborate at length. The stated prerequisite to this course is S040 or equivalent. Ideally, you would have taken S052 as well, but enrolling in it concurrently is just as good. Technically, if you have *just* taken S040 or equivalent, you could take this course. Having gone a bit beyond means this course is a bit more manageable with more of these foundational skills. More specifically, we greatly prefer you to have the following skills:

- You can calculate summary statistics and visualizations for data (the mean, standard deviation, scatterplots, histograms) using some sort of statistical software.
- You can load data into some sort of statistical software, use that software to fit a regression model, and then interpret the results of the model.
- You know how to include categorical covariates in a linear regression.
- You know what an interaction term in a linear regression model is, and ideally can add one to a model you are fitting.
- You can interpret confidence intervals and p -values.
- You can write down a regression equation and identify what the covariates and parameters are in a presented regression equation.

You will have an easier time if you have some of the following skills and experiences as well:

- You have some experience with doing things in R (even a little helps here).
- You know what logistic regression is, and know how to fit a logistic regression model using some sort of statistical software.
- You have some experience doing quantitative research with real data in almost any capacity.

If you have a strong mathematical background, or strong comfort with math, or if you have a strong computer programming background, or strong comfort with that, then you can likely get a lot out of this course even if you do not have some of the above skills and knowledge.

People from many different backgrounds take S022. In general, the more background experience you have, the easier the course. Even if you don't have as solid a background, you can still get

a lot out of this course if you are willing to sign on for an intense experience. But *please* don't be surprised if you sign up for an intense experience... and it is really intense!

To get a very rough sense of what S022 might feel like, take the quiz below.

A self-assessment quiz

Trying to figure out how S022 will be for you? Add up the points across the following categories:

Work Experience

- +0 I have no work experience with quantitative data, or
- +1 I have some work experience with quantitative data, or
- +2 I have substantial work experience with quantitative data

Stat Courses

- 2 I have no prior stat experience under my belt, or
- +1 I have taken a very intro stat course (e.g., S012) or have a little stat knowledge, or
- +3 I have taken a linear regression course (e.g., S030, S040), or
- +5 I have taken an intermediate or advanced stats course (e.g., S052 or beyond)
- +1 bonus if concurrently enrolling in S052
- +1 bonus if prior quantitative classes were comfortable for you

Programming

- +0 I have basically no experience doing computer programming, or
- +1 I have some experience doing computer programming, or
- +2 I have a lot of experience doing computer programming

R Skills

- +0 I have no real experience with R, or
- +1 I have a small bit of experience with R (e.g., ran scripts of it in S040), or
- +2 I have some experience with R (e.g., played around with scripts a bit in S040), or
- +3 I have substantial experience with R (e.g., write my own R code for my work)
- +1 bonus if learning R has been comfortable for you
- +1 bonus if you are comfortable with something like STATA

Math

- +0 I don't recall much math from my past education, or
- +1 I have taken (and somewhat remember) Calculus, or
- +2 I have taken classes beyond Calculus
- +1 bonus if classes were comfortable for you

Other

- +2 I am content with getting a B or taking the class SAT/UNSAT

Total the above

A VERY ROUGH recommendation is:

- < 4: **Danger!** Please email the instructor to talk about how this might go for you.
- 4-6: **It will be really hard!** You could take this course, but will likely find it will take much more time than a typical course. We would support you through this, but be warned that this could feel like a lot to take on.
- 7-8: **It will be hard!** There are lots of folks like you who are taking this course. The course will likely be a fair bit of work and could feel confusing/overwhelming at times. We would support you through this. At the end you will have learned a lot if you stick with it.
- 9-12: **It will probably feel like a normal course.** No reservations. You can either work a reasonable amount and learn a lot about data science, or dig deeper to really go far with the skills we cover.

13+: **It will probably be a cake-walk.** You will learn some concepts but not have to work particularly hard in this course. We would still love to have you!

Students have historically said this course teaches you a lot; the question is just whether you have the time to allocate for the course. This quiz helps assess the time.

Assignment Formatting Guidelines

The following describes our expectations on what turned in assignments should look like. Please read this document carefully before your first assignment. Note that many of these requirements will be automatically fulfilled if you use R Markdown.

Guiding Principle

Your submitted work should present your work clearly and without undue clutter, be easy to navigate, and adhere to basic publication norms.

The following are some specific details that further the guiding principle. It is not an exhaustive list.

Formatting

- Start each question on a new page.
- Provide at least a brief title for the question or sub-question along with the question number. E.g. “(a). Association between graduation rates and school type.” You can copy the entire prompt for your future reference if you want, but abbreviated prompts are fine with us!
- Use headings or other means to highlight problem titles (e.g., bold, italic, etc.).
- Make sure you answer all parts of each question, and do so under the proper labeled subpart. For each question or sub-question, include any R code, R output and answer.
- Use different fonts/formatting for your R code, R output and answers and use font formatting consistently throughout each assignment.
- Use single line spacing and normal 1-inch margins.
- Include page numbers.
- *Let people say of your work: “There is no bombast, no similes, flowers, digressions, or unnecessary descriptions. Everything tends directly to the catastrophe.”* —Horace Walpole, The Castle of Otranto

Numbers

- Round your final answers, not intermediary steps.

- Put a zero in front of a decimal place (e.g. 0.2 instead of .2). This is optional for bounded numbers (e.g., p-values or proportions).
- Round to the nearest meaningful digit. What this means is a little hard to say, and different people can have different standards. However, if you have a statistic with a standard error of 1, it's not meaningful to report decimals because the estimate simply isn't precise enough to estimate numbers that small. Similarly, if you're reporting average salaries, it's usually not meaningful to report past the hundreds place regardless of your precision, because tens of dollars are too small to matter. Because this is so imprecise, we'll give a lot of lee-way, but spurious precision will annoy. If you're not sure what this means in practice, feel free to ask a TF.
- Format your p-values! Round p-values to 3 places, and never report a p-value of 0. Instead say, for example, " $p < 0.001$ " or " $p < 10^{-r}$ " for some r .
- *"Numbers have life; they're not just symbols on paper."* —Shakuntala Devi

Plots

- Make sure your plots are well labelled, including title, axis, legends and any other elements you choose to include.
- Your plots should be self-explanatory.
- Include notes and captions as necessary.
- Try to make plots easier to compare when you have multiple plots. For example, it is nice to have the same X -axis bounds if giving two histograms.
- Do not include best fit lines unless you have some reason, e.g. a significant p -value or a scientific basis for understanding an association.
- *"Above all else show the data."* —Edward Tufte

Avoiding bias

- Use plural phrases, nouns or pronouns, e.g. "children and their toys" for "a child and his toy." You may use the singular "they" pronoun ("a child and their toy"), but be warned that broader academic communities are still in flux regarding this usage.
- Try to avoid biased forms of language concerning race, gender, disability and sexuality. Walden University has [a reasonable list of case studies to consider on this topic](#). The APA also has [a guide for writing about race](#).
- *"I have yet to see a piece of writing, political or non-political, that does not have a slant. All writing slants the way a writer leans, and no man is born perpendicular."* — E.B. White

Conciseness

- “*Brevity is the soul of wit.*” —Hamlet

Self Grading Instructions

To self-grade an assignment, go through the solutions and compare the answer to your own work.

Grade each piece as following

- “+” - 1 point, basically nailed it.
- “check” - Half point, did at least part of it, but had some problems.
- “X” - zero points, didn’t get it.

A “piece” is each part of a problem, usually given a letter. So if you had 1(a), 1(b), and 1(c), that would be three pieces for a maximum total of three points for Problem 1.

As you go through the solutions, see if you can figure out where you went wrong, if you did so. If you still don’t understand a problem, make a note as to problem number.

Then, for the next ready-for-class check-in you will be asked to enter your score (this score just helps me understand how folks are doing with these mini assignments), how you feel about it, and some further commentary. In your commentary please include a sentence or two on what was tripping you up (list the items), if you got things wrong. Also note whether things are not making sense, even with the solutions.

For reporting your score, **report total number of points, not percent correct.** For example, if there are two questions, with the first having 3 points and the second 2, you would report a score between 0 and 5.

We primarily grade miniassignments on completion, meaning we will threshold your score to at least 90% if you (1) do the mini-assignment and (2) self-grade the miniassignment (regardless of the score you give yourself as long as your work shows reasonable effort).

Acknowledgement:

This idea of self-grading was inspired by Cora Wigger. She used to have some stuff on twitter, but she appears to have removed her account.

Part I

Getting Started

1 Getting Started with R

To get started with R, first download and install R and RStudio (the IDE, or integrated development environment, that makes using R much, much more pleasant). See links at [this HGSE software support page](#).

1.1 Some Resources and Directions

The following are ways of getting help with R.

Handouts

This textbook has many handouts written by the teaching team that illustrates how to use R for a variety of tasks. Also see the Resources page on Canvas for further commentary and organization.

Class Code

Each class has a R script that shows how to do the stuff from that class. See the Packets on Canvas for this code.

A Very Important Online Textbook

The online textbook [R for Data Science](#). provides important information on wrangling data, making plots, and doing statistical programming. It is full of examples and code snippets you can steal. There are two versions:

- [The First Edition](#) – this what we often refer to in these materials. But we are slowly updating to...
- [The Second Edition](#) – this one has a different organization, less weird modeling, and covers recent changes to the “tidyverse.”

Class Sections

Sections will typically have a hands-on component which will give you a chance to try things out yourself. These sections will also publish the final R code for future reference. See the section pages on Canvas to get this information.

Office Hours

Office hours are fine time to get help troubleshooting a specific or script you are working on.

GSE Stat Help Desk

Education students can write to stathelp@gse.harvard.edu to get help getting started with R. If groups of 3 or more want some tutorials, they can ask for them as well.

The Help Desk sometimes has Intro to R workshops. For example, grab some old workshop materials here: <http://its.gse.harvard.edu/gentle-introduction-r>

2 Tidy Projects for Analysis in R

Organizing Projects for Research & Collaboration

S022 Final Projects

In addition to submitting the slides for your final S022 project, you will also submit a zipped project folder containing all of the data, code, and analysis used to create your presentation. Below, we outline some best practices for keeping your projects organized as they develop, with the ultimate goal of sharing a clean, replicable project to the teaching team at the end of the semester.

Organizational File Structure

Often, complex projects involve several scripts that take a variety inputs and generate an array of outputs. Ultimately, an intentional file structure that is specific to your project can be extremely helpful in keeping your code organized in a way that facilitates collaboration and replication. As a starting point, consider using something like the basic file structure below:

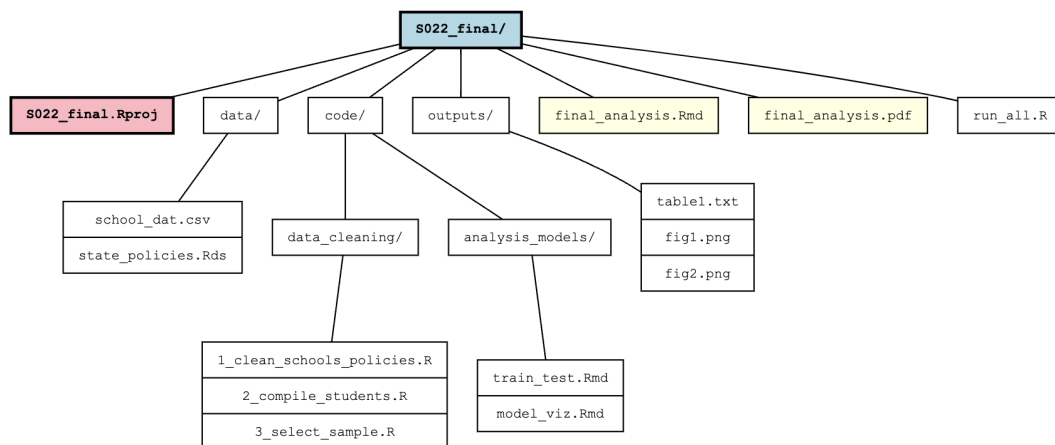


Figure 2.1: Figure 1: An organized research project

An organized project contains:

- All data files associated with the final analysis
- The cleaning scripts to prepare, merge, and process code for the final analysis
- All tables and outputs associated with the final analysis
- A markdown file (ex. `final_analysis.Rmd`) and rendered PDF (ex. `final_analysis.pdf`) report with the code and products for your research project
- An `.Rproj` file, which bundles the code together and manages relative file paths within the project.

The goal of your project's structure is to be clearly understood by collaborators or your future self.

Using R Projects

R Projects are an extremely useful tool for organizing code and keeping the variables and data in your environment contained across separate research projects. You can read more about Projects in [R for Data Science \(2e\), Chapter 6](#).

The main thing to know is that R Projects are one of the most effective ways to format code for collaborators to share across different computers and file locations.

The `here` package

In working with R files, you have probably come across the `getwd()` and `setwd()` functions. However, R Projects obviate the need for manually setting a working directory by identifying the location of your R project (and the associated `.Rproj` file) as the root directory that all of your code will reference. The `here` package leverages this feature by automatically detecting your project root, allowing you to build file paths that will work consistently across different computers.

For example, the code below will find the project root associated with this Quarto handout!

```
library(here)
here::here()
```

```
[1] "/Users/lmiratrix/Dropbox/S022 S2026/S022-textbook"
```

Relative paths with `here()`

By centering your code around the `.Rproj` file, you can write code that seamlessly runs on multiple collaborators' computers, regardless of the Project's location on those computers.

Good:

Uses the `here()` function to set relative file paths to locations in the project directory. Note that this example creates separate sub-directories for raw and cleaned data. A nice way to take it easy on your collaborators and your future self!

```
school_dat <- read_csv(here::here("data/raw_data/school_data_2025.csv"))
...
# Cleaning code here
saveRDS(clean_dat, here::here("data/clean_data/clean_school_data.Rds"))
```

Bad:

Imports data from far-flung locations, uses absolute file paths that will break on others' computers or if the project directory changes.

```
school_dat <- read_csv("/Users/noradelaney/Downloads/school_data_2025.csv")
...
# Cleaning code here
saveRDS(clean_dat,
        "/Users/noradelaney/Documents/s022_final_proj/data/clean_school_data.Rds")
```

3 Resources on Learning R

So, where can you get help with R?

When looking for help, first try to stick with this textbook as there are many ways of doing things in R, and if you leap into a broad internet search you can be led into strange lands that look very different from the land of S022. So first, check the table of contents of this book and look for things that can help you. Or search the book itself!

But sometimes you have to go further afield, or want a different kind of help. This index is designed to steer you away from getting lost by pointing you to various resources that are more tailored to the course.

3.1 Live Class Code and Sections

The labs and classes usually have R scripts that shows how to do the stuff from that class. Find the class by looking at the table of contents on Canvas, and then open the packet page for that week. The packets should provide all code used.

Sections will also typically have a hands-on component which will give you a chance to try things out yourself. These sections will also publish the final R code for future reference. This will also be on the packet page for the specified week.

3.2 R for Data Science: The Very Important Online Textbook

[R for Data Science \(2e\)](#) has the main coding concepts we use in our work. This textbook provides important information on wrangling data, making plots, and doing statistical programming. It is full of examples and code snippets you can steal. Note that we use the second edition of this textbook (2e).

There are also [solutions to this guide](#). It can be quite useful to do the exercises in this text!

A supplementary textbook Also check out Modern Dive’s [“Statistical Inference via Data Science”](#), especially for some beginning R information in Chapter 1. It is a more gentle introduction to R, which can help get ready for R for DS.

3.3 Office Hours

Office hours are fine time to get help troubleshooting a specific or script you are working on.

3.4 GSE Stat Help Desk

Education students can write to stathelp@gse.harvard.edu to get help getting started with R. If groups of 3 or more want some tutorials, they can ask for them as well.

The Stat Help Desk sometimes have intro to R workshops. For example, grab some old workshop materials here: <http://its.gse.harvard.edu/gentle-introduction-r>

3.5 R Cheat Sheets

R Cheat Sheets are one page sheets that give lots of R commands for specific tasks. VERY USEFUL! See <https://rstudio.com/resources/cheatsheets/> for a full compendium of cheat sheets for work in RStudio, such as the ggplot sheet.

We've covered/touched on a lot of the cheat sheet topics in class or section like using the dplyr package, R-Markdown, or RegEx, but there are also things that we haven't discussed, such as creating survival plots in ggplot or developing a web-based app.

3.6 Further information on using R Markdown

In addition to `?@sec-intro__markdown` (which has some initial thoughts and how-tos tailored to the course), here are a few additional external resources on using R Markdown:

- [rmarkdown-cheatsheet-2.0.pdf](#)
- [rmarkdown-reference.pdf](#)
- See the async lecture on configuring code chunks in Week 1, on Canvas, as well.

3.7 Useful links to materials on the web

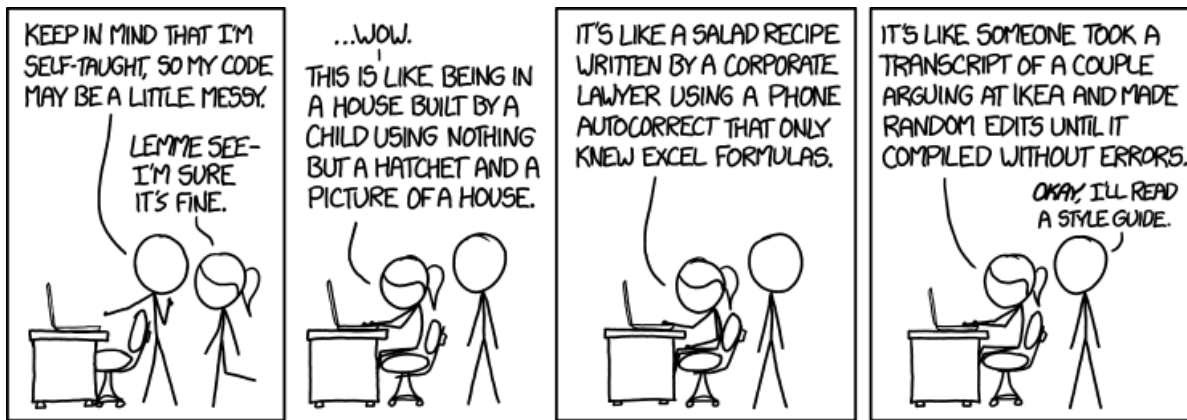
- R: <https://www.r-project.org/>
- RStudio: <https://www.rstudio.com/>
- Columbia RStudio overview: [here](#)
- TryR code school: <http://tryr.codeschool.com>
- Data Camp: <https://www.datacamp.com/courses/free-introduction-to-r>

- Princeton R tutorials: <http://data.princeton.edu/R/>
- D-Lab R training: <https://github.com/dlab-berkeley/R-for-Data-Science>
- GGPlot2: <http://ggplot2.org/>
- Google/stackoverflow

3.8 Writing your own math

- [Handout: math reference](#) – This handout, from my handout textbook for S043 (multilevel modeling) has the Latex you need to write equations easily for your reports. But what you really want is the Rmd version that has the code to make this handout, which is linked in the handout itself.

4 An R Code Style Guide (Miratrix version)



4.1 Why have coding style?

Many style decisions in coding are arbitrary, so why bother adhering to them? The primary reasons are that good style makes your code more readable, allows you to focus on writing quality code rather than worrying about inconsistencies, and helps you avoid being judged negatively for poor style.

More importantly, if you write in a clean style, you will be able to read your own code a few weeks later, when you have forgotten what you have been thinking. It also makes it *a lot* easier for the teaching team to help you understand possible problems in your code.

In this chapter, I lay out some tips for writing clean code. Much of this was initially taken from the Tidyverse style guide at <http://style.tidyverse.org>; we are primarily focusing on Chapters 2 and 4. But I will note that I depart substantially from some of their recommendations.

The cartoon is [xkcd](#); read those if you want to be an awesome nerd.

For further reading about coding practice, also see [Code and Data for the Social Sciences: A Practitioner's Guide](#), Gentzkow and Shapiro. Worth a skim!

4.2 The naming of things

“There are only two hard things in Computer Science: [cache invalidation](#) and naming things.” —Phil Karlton

- Variable and function names should be lowercase.
- Use an underscore to separate words within a name.
- Generally, variable names should be nouns and function names should be verbs.
- Ideally, your names should be self-explanatory and your code should be “self-documenting.”
- However, avoid making your variables too long (less than about 15 characters), unless you love typing.
- Avoid using numbers to store versions of a data frame (e.g., `df1`, `df2`, `df3`, ...).
- Names for variables, functions, files, etc. should generally consist of complete words.
- Naming is hard. more art than science.

```
# Good
day_one
first_day

# Bad
first_day_of_the_month
DayOne
dayone
djm1
```

4.2.1 Don't use common names

If R uses a name already, just avoid it.

```
# Bad
TRUE <- FALSE
pi <- 10
mean <- function(x) sum(x)
```

4.2.2 Name constants for clarity

```
# Good
winsor_upper <- 0.99
winsor_lower <- 0.01
diamonds <-
  diamonds %>%
  mutate(y_winsor = winsorize(y, probs = c(winsor_lower, winsor_upper)))
```



```
# Mediocre
diamonds_clean <-
  diamonds %>%
  mutate(y = winsorize(y, probs = c(0.01, 0.99)))
```

4.3 Syntax

4.3.1 Appropriate spaces make code readable

- Put a space before and after = when naming arguments in function calls.
- Always put a space after a comma, and never before (just like in regular English).

```
# Good
average <- mean(x, na.rm = TRUE)

# Also good
average <- mean( x, na.rm = TRUE )

# Bad
average<-mean(x, na.rm = TRUE)
average <- mean(x ,na.rm = TRUE)
```

4.3.2 Judicious spaces group elements to increase readability

- Most infix operators (==, +, -, <-, etc.) should be surrounded by spaces.
- The exception are those with relatively [high precedence](#): ^, :, ::, and ::: (“High precedence” means that these operators are evaluated first, like multiplication goes before addition.)

```
# Good
height <- (feet * 12) + inches
sqrt(x^2 + y^2)
x <- 1:10
base::get

# Bad
height<-feet*12 + inches
sqrt(x ^ 2 + y ^ 2)
x <- 1 : 10
base :: get
```

4.3.3 Lots of spaces to line things up is nice

Extra spacing (i.e., more than one space in a row) is ok if it improves alignment of equal signs or assignments (<-).

```
# Good
list(
  total = a + b + c,
  mean  = (a + b + c) / n
)
```

```
# Less good, but livable
list(
  total = a + b + c,
  mean  = (a + b + c) / n
)
```

4.3.4 Including names when calling functions

A function typically takes several arguments, namely the **data** to compute on and further arguments giving the **details** of the desired computation.

Omit names of common arguments (e.g. **data**, **aes**). However you can include these to make your code more readable to others.

If you override the default value of an argument, use the full name:

```
# Good
mean(1:10, na.rm = TRUE)

# Also Good
mean(x = 1:10, na.rm = TRUE)

# Bad
mean(x = 1:10, , FALSE)
mean(, TRUE, x = c(1:10, NA))
```

4.3.5 No line of code should be longer than 80 characters

- use one line each for the function name, each argument, and the closing ‘)’.
- use indentation to make it clear that the arguments are part of the function call.

```

# Good
do_something_very_complicated(
  something = "that",
  requires = many,
  arguments = "some of which may be long"
)

# Very bad
do_something_very_complicated(something = "that", requires = many, arguments = "some of which may be long")

# Still bad
do_something_very_complicated(
  something = "that", requires = many,
  arguments = "some of which may be long"
)

# Yup, still bad
do_something_very_complicated(
  something = "that", requires = many, arguments =
  "some of which may be long"
)

```

Exception to the above: short unnamed arguments can also go on the same line as the function name, even if the whole function call spans multiple lines.

```

# Good
map(x, f,
  extra_argument_a = 10,
  extra_argument_b = c(1, 43, 390, 210209)
)

```

4.3.6 Use <- for assignment (if you are prissy)

Use <-, not =, for assignment.

```

# Good
x <- 5

# Upsets the nitpickers
x = 5

```

4.3.7 Double quotes are standard

Use `"`, not `'`, for quoting text. The only exception is when the text already contains double quotes and no single quotes.

```
# Good
"Text"
'Text with "quotes"'
'<a href="http://style.tidyverse.org">A link</a>'

# Bad
"Text"
'Text with "double" and \'single\' quotes'
```

4.3.8 Comments are good

Comments can really help with code, and help a reader assess if the code is achieving the desired effect. Comments should give the purpose of the code, not just restate the code in English.

That said, comments can be dangerous because if you change your code without changing the comments, you can cause confusion. This happens a lot, potentially making comments a source of confusion rather than clarity.

4.4 Pipes (`%>%`) with `magrittr`

Use `%>%` (or `|>`, if you are modern) to emphasize a sequence of actions, rather than the object that the actions are being performed on.

Avoid using the pipe when:

- You need to manipulate more than one object at a time. Reserve pipes for a sequence of steps applied to one primary object.
- There are meaningful intermediate objects that could be given informative names (cf rule 2.9).

4.4.1 Surround the pipe with whitespace

- `%>%` should always have a space before it, and should usually be followed by a new line.
- After the first step, each line should be indented by two spaces.

Adhering to the above structure makes it easier to add new steps (or rearrange existing steps) and makes it harder to overlook a step.

```
# Good
iris %>%
  group_by(Species) %>%
  summarize_if(is.numeric, mean) %>%
  ungroup() %>%
  gather(measure, value, -Species) %>%
  arrange(value)
```

```
# Bad
iris %>% group_by(Species) %>% summarize_all(mean) %>%
ungroup() %>% gather(measure, value, -Species) %>%
arrange(value)
```

4.4.2 Formatting short pipes

It is ok to keep a one-step pipe in one line, if you want. But not doing so is also fine.

```
# Good
iris %>% arrange(Species)
```

```
# Also good
iris %>%
  arrange(Species)
```

```
arrange(iris, Species)
```

4.4.3 Do not pack pipes inside of other pipes

This is brain-hurty:

```
# Bad
x %>%
  select(a, b, w) %>%
  left_join(
```

```

    y %>% filter(!u) %>% gather(a, v, -b) %>% select(a, b, v),
    by = c("a", "b")
  )

```

Instead, separate the pipes into their own lines:

Good

```

x_join <- x %>%
  select(a, b, w)
y_join <- y %>%
  filter(!u) %>%
  gather(a, v, -b) %>%
  select(a, b, v)
left_join(x_join, y_join, by = c("a", "b"))

```

Generally when you are doing something to two datasets, put both inside the parenthesis, rather than having one in and one piped in.

4.4.4 Keep your parenthesis even if you have no arguments

magrittr allows you to omit () on functions that don't have arguments. Avoid this. This way data objects never have parentheses and functions always do.

Good

```

x %>%
  unique() %>%
  sort()

```

Bad

```

x %>%
  unique %>%
  sort

```

4.4.5 Rule 4.6: Assignment

The first line can have the assignment and also the starting dataframe of a pipe sequence.

Good

```

iris_long <- iris %>%
  gather(measure, value, -Species) %>%
  arrange(-value)

```

4.5 A few final remarks

- Style is awesome. Save a future researcher from spending months trying to disentangle your spaghetti!
- You don't need to memorize these rules! Just as you have spell check and grammarly on your computer for prose, there is a package `styler` to help you follow the code style guide.
- Just as you still need to learn to spell (since spell checker doesn't capture everything), you need to learn these rules as well.
- If you get tired of rules and typing, try using the [Tidyverse's 'styler' package](<https://www.tidyverse.org/blog/2017/12/styler-1.0.0/>). The package links to the “Addins” widget in RStudio and is a fast way to clean up your code according to Tidyverse conventions.

In closing:

“Good coding style is like correct punctuation: you can manage without it, but it sure makes things easier to read.” —Hadley Wickham

Part II

On markdown

5 Intro to R Markdown

5.1 Overview

This document will show you how to use R Markdown to create documents which draw directly on your data to produce reports.

[R Markdown](#) (and its newer cousin [Quarto](#)) is a simple but powerful markdown language which you can use to create documents with inline R code and results. This makes it much easier for you to complete homework assignments and reports; makes it much less likely that your work will include errors; and makes your work much easier to reproduce. For example, if you find you have to drop cases from your dataset, you can simply add that line of code to your document, and recompile your document. Any text that’s drawn directly from your analyses will be automatically updated.

To get started with R Markdown, watch Miratrix’s video on RMarkdown. You can find it on [this page](#), under the “Do it for Lab” tab.

To use a markdown document, you generally open it and work inside of R Studio. To compile (or “knit”) the document—meaning have the computer run all the code in the document, and then generate a new document with the results of the code and all the formatting specified carried out—click on the button that says ‘Knit’. You can make a new document by saying “new markdown document” and then immediately “knit” it to see what it does.

Other R packages, such as `Sweave` and `knitr`, allow you to do the same things, but R Markdown has the added advantage of being relatively simple to use.

5.1.1 Handy Resources:

When working with R Markdown, we highly recommend that you keep the [Posit Cheatsheet on R Markdown](#) close at hand—this is a great resource showing the various options and syntax you can use to customize your knitted document.

5.2 Getting started: Formatting Your Document

Every R Markdown document starts with a header, called the YAML header. YAML headers look like this:

```
---
title: "My perfect homework"
author: "R master"
output: pdf_document
---
```

A header can contain more or less information, as you see fit. Your computer needs to have a copy of LaTeX installed in order to output .pdf documents. If you don't, you should change `output: pdf_document` to `output: html_document` or `output: word_document`.

You can specify a lot of options in the header. The most important ones are the title of your document, the author, and the output format.

In the main part of your document, you identify sections of the document using hashtags; more hashtags indicate less important sections.

For example, this:

```
# A big section
```

produces a big header (large font, etc.)

while this

```
## A small section
```

produces a smaller header (still a large font, but less large).

Also, if your document includes a table of contents, the sections get used to automatically generate the table of contents.

You can *italicize* words by writing `*italicize*` or `_italicize_`. You can **bold** words with `**bold**` or `__bold__`.

You can add superscripts ($E=mc^2$) by writing `E=mc^2^`.

You can create unordered lists:

- Item 1
- Item 2
- Item 3

to get

- Item 1
- Item 2
- Item 3

Or ordered lists:

1. Item 1
2. Item 2
3. Item 3

to get

1. Item 1
2. Item 2
3. Item 3

To start a new page, just type `\newpage` (not relevant for HTML output).

As you may have noticed, one of the driving ideas behind R Markdown is that the text should be interpretable even if it's not compiled. A person should be able to read the initial text file and understand the basic organization and what all of the symbols denote.

You can also add links and images, and do many other things beyond what we'll show you in this class. There are many resources out there, but [here's](#) one place you can start.

We also note that newer versions of Markdown and Quarto have a [visual editor](#) that allows you to format things in the usual way, e.g., control-B for bold. Some people prefer to take that approach.

Regardless, to compile or knit the document, click on the button that says **Knit** or **Render**, or Shift + Ctrl/Cmd + K.

5.3 Embedding R Code

The power of R Markdown is you can include R code and that code will get run and the output included in your report automatically. There are two main ways to embed R code in R Markdown, code chunks (this is the primary way) or inline (this is if you want to get fancy).

5.3.1 Formatting Individual Code Chunks

To insert a code chunk click on **Insert** on the top right corner of your R Markdown file and select **R**. Or use keyboard shortcuts: Ctrl + Alt + I for PC and Cmd + Option + I for Mac.

A code chunk starts with three backticks and the letter **r** in curly braces, and ends with three more backticks on their own line. In between these lines you put the R code.

```
```${r}
YOUR CODE HERE
```
```

Code chunks have a number of different options you can put after the **r** in the opening line. The most important ones for us are:

- **eval** = TRUE, which means every time you knit the file, the code inside the R code chunk will get evaluated. This is the default.
- **echo** = TRUE, which means every time you knit the file, the code inside the R code chunk will be rendered, and you can see both the code itself and the results from evaluating the code.

```
```${r eval = FALSE}
The document will knit even if the code in this chunk has an error.

For example, the "bad" code below doesn't prevent the document from knitting
my_var <- !!198`123bx92
```
```

For class, you should keep **echo** = TRUE, so that we can see your code and be able to tell what went wrong, if something did. You can set **echo** = FALSE for code chunks that load and manipulate data in the setup of your analysis, if you don't want people to look at that part of your work.

Other code chunks options you may see in class are:

- **warnings** = FALSE, which means warning messages generated by the code will not be displayed.
- **results** = 'asis', which means results will not be reformatted when the file is compiled (useful if results return raw HTML).
- **fig.height** and **fig.width**, which specify the height and width (in inches) of plots created by the chunk.

Ultimately, the output of each code chunk is highly customizable. For granular information about how to format your chunk output exactly as desired, please take a look at [Ch. 11 of the R Markdown Cookbook by Xie, Dervieux, & Riederer \(2024\)](#).

5.3.2 Setting Global Options Code Chunks

Instead of specifying code chunks options every time, you can specify them globally in the setup chunk by using the `knitr` package. The code below sets the new default for code chunks in your project. Note that, because this chunk uses `echo=FALSE` in its header, this setup code will be hidden in the final rendered document.

```
```{r setup, echo=FALSE}
knitr::opts_chunk$set(echo = TRUE, eval = TRUE)
```
```

You can then add additional options **only** to relevant chunks using the chunk headers. For example, if you want to exclude specific chunks, you can re-set `echo = FALSE` and `eval = FALSE` for those specific chunks.

Running code chunks: A good practice for writing your code is to run individual code chunks to make sure they are doing what you want them to do as you write them. You can do this by executing individual lines of code, or whole chunks. Go to **Run** in the upper right corner and select what chunks to execute, e.g. **Run Current Chunk**, **Run Next Chunk**, etc.

5.3.3 Inline code

Code results can also be inserted directly in the text of your R Markdown file. This is particularly useful when you are extracting and interpreting model parameters. You can extract the coefficient from the model and use inline code to report it. If the data or model change, *the text will also change* when you knit the document.

To add inline code, enclose it in ``r ``. For example, to report the mean reading score, you can use ``r round(mean(dat$p7read))``, which will produce the number -0.04 in your rendered document. Here we used two commands: `round()` and `mean()`. You can use more commands and write more complex inline code, depending on what you want to report.

5.4 Embedding plots

Plots are easy to embed. For example, I use the chunk headings in the example below to control the width of the displayed output (75% of the full document width) using `out.width='75%'`. I also turn off pesky warnings and messages (since I know I can safely ignore them) to avoid cluttering the output using `message=FALSE` and `warnings=FALSE`.

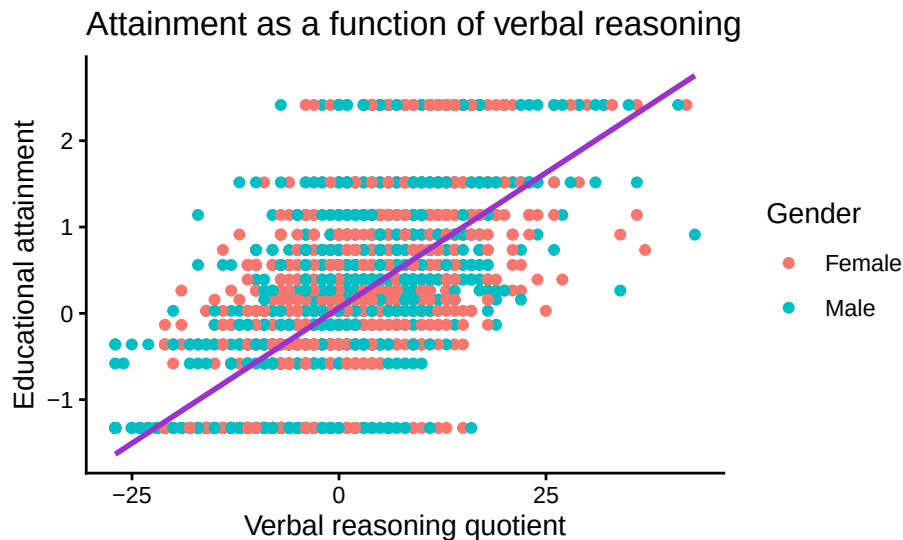
```
```{r out.width='75%', message=FALSE, warning=FALSE}
library(ggplot2)
```

```

dat$male <- factor(dat$male, levels = c(0, 1), labels = c("Female", "Male"))

ggplot(data=dat, aes(p7vrq, attain, colour=male)) +
 geom_point() +
 labs(title="Attainment as a function of verbal reasoning",
 x = "Verbal reasoning quotient",
 y = "Educational attainment", colour="Gender") +
 geom_smooth(method="lm", formula = y ~ x, se=FALSE, colour="darkorchid3")
...

```



Girls are rendered as coral, boys are rendered in turquoise, and the line of best fit is drawn in `darkorchid3` (because why not). Just because you have a lot of colors and plotting characters to work with doesn't mean you need to use them all. Notice that this command draws on `dat`, which we loaded in a previous chunk. When knitting the document, all code chunks with the default `eval=TRUE` option get executed in order and the results persist throughout the R Markdown document.

For the purposes of class, we want to see both your plot code and the plot itself. It's not uncommon to use wrong code to create a plot that looks correct (at least visually).

## 5.5 Embedding tables

You can directly render tables in R Markdown. The idea is, inside an R chunk, you call a command that prints out a table. The report then takes this printout and integrates it into your overall report. There are many different packages to make tables, but in class we'll mostly use `knitr`, `texreg`, `stargazer`, and the `tab_model()` function in `sjPlot`.

You can use these packages to create a descriptive table. For example:

```
head(dat) %>%
 knitr::kable(digits = 2)
```

neighid	schid	attain	p7vrq	p7read	dadocc	dadunemp	daded	momed	male	de- prive
675	0	0.74	21.97	12.13	2.32	0	0	0	Male	-0.18
647	0	0.26	-7.03	-12.87	16.20	0	0	1	Fe- male	0.21
650	0	-1.33	-11.03	-31.87	-23.45	1	0	0	Male	0.53
650	0	0.74	3.97	3.13	2.32	0	0	0	Male	0.53
648	0	-0.13	-2.03	0.13	-3.45	0	0	0	Fe- male	0.19
648	0	0.56	-5.03	-0.87	-3.45	0	0	0	Fe- male	0.19

See Chapter 8 for more on making various tables.

We can also use `texreg` or `stargazer` to create a taxonomy of regression models.

For example:

```
library(texreg)

fit some models
m1 <- lm(attain ~ male, data=dat)
m2 <- lm(attain ~ male + momed, data=dat)
m3 <- lm(attain ~ male + momed + daded, data=dat)

screenreg(list(m1,m2,m3),
 custom.coef.names=c("Intercept", "Male",
 "Maternal education", "Paternal education"))
```

	Model 1	Model 2	Model 3
Intercept	0.15 *** (0.03)	0.03 (0.03)	-0.02 (0.03)
Male	-0.12 ** (0.04)	-0.12 ** (0.04)	-0.12 ** (0.04)
Maternal education		0.49 ***	0.24 ***

		(0.05)	(0.05)
Paternal education			0.54 ***
			(0.06)
-----			
R^2	0.00	0.05	0.09
Adj. R^2	0.00	0.05	0.08
Num. obs.	2310	2310	2310
=====			
*** p < 0.001; ** p < 0.01; * p < 0.05			

Both packages include a lot of options and make it easy to produce publication-quality tables with little effort.

## 5.6 Embedding math

We'll be writing some mathematical models in class. R Markdown can use **LaTeX** style math-writing to display mathematical script. Similar to code chunks and inline code, you can use **LaTeX** for single or multiple equations, or for individual parameters embedded in the text.

For example, the following statement

```
$$Y_i = \beta_0 + \beta_1 X_i + \epsilon_i$$
```

compiles to

$$Y_i = \beta_0 + \beta_1 X_i + \epsilon_i$$

And the following statement `$$\mu$` compiles to  $\mu$ .

## 5.7 Help! R Markdown report generation doesn't work

If your markdown will not render, first try restarting R and then selecting run all chunks from the menu. If that does not work, then try fixing your code.

If that does work, but knitting still does not work, then something stranger is going on. First, check to see if you have a “View()” command in your Markdown file. This will cause your document to not be able to knit.

Also watch for the `skim()` command—it can crash report generation as well.



If you can't knit PDFs you need to install latex (tex). Once you do, reboot your computer. If things don't work, then knit to Microsoft word (or, failing that, html as a last resort), print to pdf, and turn that in. But then ask a teaching fellow to help get things set up, since PDFs make for much more readable reports.

## 6 Creating Reports in Quarto

### 6.1 What is Quarto?

Quarto is the “next-generation” of RMarkdown. Most tutorials on Quarto are intended for people who have experience with RMarkdown, so a great place to start is to read the prior chapter before reading this one.

Once you’re familiar with RMarkdown, the tricks you know will almost always work in Quarto. The major difference is that Quarto will render *inside* RStudio, as you write your document, which makes it a little nicer to work with. There are other new (and very cool) features, but they aren’t essential for anything in this course.

### 6.2 Using Visual Editor with Rmd

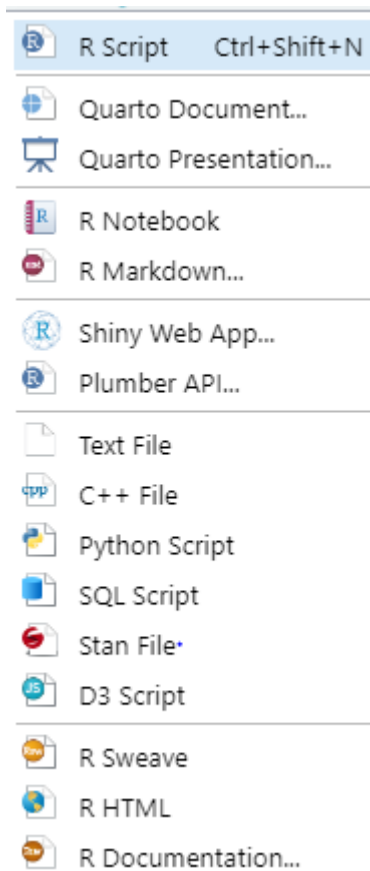
An obvious, cool thing about Quarto is the fancy visual editor, but you can turn this feature on for regular old RMarkdown files, too. Just click on this , and click “use visual editor.” You can also add `editor: visual` to the yaml (top-matter) of an rmd file and it’ll open in the visual editor by default.

### 6.3 Creating a Quarto Report

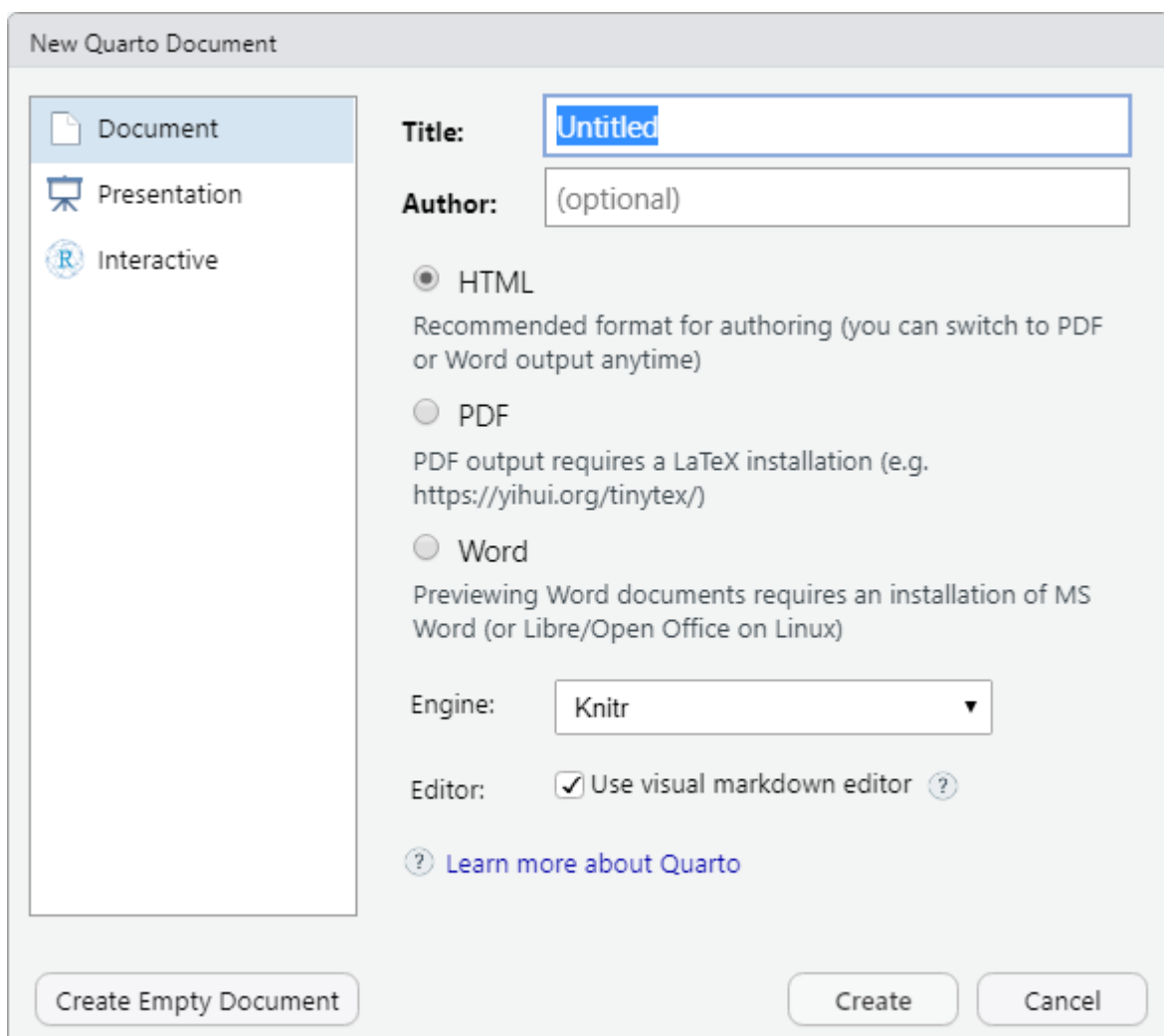
In RStudio, click this icon in the upper left:



That’ll give you the following drop-down menu, where you can select “Quarto Document”:



When you click on “Quarto Document...” RStudio might take a few seconds to load. Then you’ll see this pop-up:



Fill out the document title and author (just like for RMarkdown). You can always change the title and author later. You'll want to render your reports as PDFs, so select that option. Finally, hit the "Create" button at the bottom.

RStudio will load up a new Quarto doc (with some boilerplate markdown in it). From here, you can treat it like an RMarkdown file.

If you want a more thorough introduction to Quarto, check out [this tutorial](#).

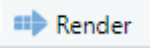
## 6.4 Rendering to PDF

To render your report to PDF, you'll need to have an installation of LaTeX. You can set this up from within RStudio.

Down by your console, there's a tab called "Terminal." Click on it to open the terminal. Inside the terminal type the following:

```
quarto install tool tinytex
```

I recommend restarting your computer after this.

Now you should be able to click  and get a pdf version of your report. By default, a copy of the pdf will be saved in the same folder as your Quarto document.

## 6.5 A Couple Quick Tricks

### 6.5.0.1 Making Code Chunks

Use the keyboard shortcut `ctrl+alt+i` or `command + option + i` to create a new R code chunk.

### 6.5.0.2 Adding Images

You can copy-paste or drag-and-drop images into a Quarto doc. That's how I put the above screenshots into this document.

## 6.6 Additional Quarto Resources

Here are a couple more links if you'd like to learn more:

- The [RMarkdown Cheatsheet](#) has great information. Remember, most things that work in RMarkdown work in Quarto. Also, there are a lot of other great cheatsheets at this site as well.
- A [Quarto Intro Video](#) that walks through using Quarto for the first time.
- A [In Depth Quarto Intro](#) video that covers many amazing new features (like making interactive html reports).

## 7 Working with Rmarkdown chunks

When you write R Markdown (or Quarto) reports, you are going to have a lot of “chunks” of code. These are the things that start with “`{r chunk_name, blah}`” or “`{r, blah}`.” When you render your report, these are run and the output is then taken and put in your report depending on how the chunk is configured.

This file gives some options for how to control these chunks.

### 7.1 Markdown chunks

#### 7.1.1 Options for including/suppressing code and output

**include:** Should chunk be included in knit file? Defaults to `TRUE`. If `FALSE`, code chunk is run, but chunk and any output is not included in the knit file.

**eval:** Should chunk be evaluated by R? Defaults to `TRUE`. If `FALSE`, code chunk is included in the knit file, but not run.

**echo:** Should the code from this chunk be included in knit file along with output? Defaults to `TRUE`. If `FALSE`, the output from the chunk is included, but the code that created it is not. Most useful for plots.

#### 7.1.2 Options for including/suppressing R messages

R has “errors,” meaning it could not run your code, “warnings,” meaning that the code was wrong, but there are some potential issues with it, and “messages,” which are simply information about what your code ran. You can include or suppress each of these types of message.

**error:** Should R continue knitting if code produces an error? Defaults to `FALSE`. Generally don’t want to change this because it means you can miss serious issues with your code.

**warning:** Should R include warnings in knit file? Defaults to `TRUE`.

**message:** Should R include informational messages in knit file? Defaults to `TRUE`. Easy way to clean up your markdowns.

For example, the following code chunk produces an error, but because the header is `error = TRUE`, `warning=FALSE`, `message=FALSE`, only the error is shown:

```
#This code produces an error
dat %>%
 filter(dest = 1)
```

```
Error in `filter()`:
! We detected a named input.
i This usually means that you've used `=` instead of `==`.
i Did you mean `dest == 1`?
```

```
#Example warning
parse_number(c("1", "$3432", "tomato"))
```

```
[1] 1 3432 NA
attr(,"problems")
A tibble: 1 x 4
 row col expected actual
 <int> <int> <chr> <chr>
1 3 NA a number tomato
```

```
#Example message
library(gridExtra)
```

### 7.1.3 Options for modifying figure outputs

`out.width`: What percentage of the page width should output take?

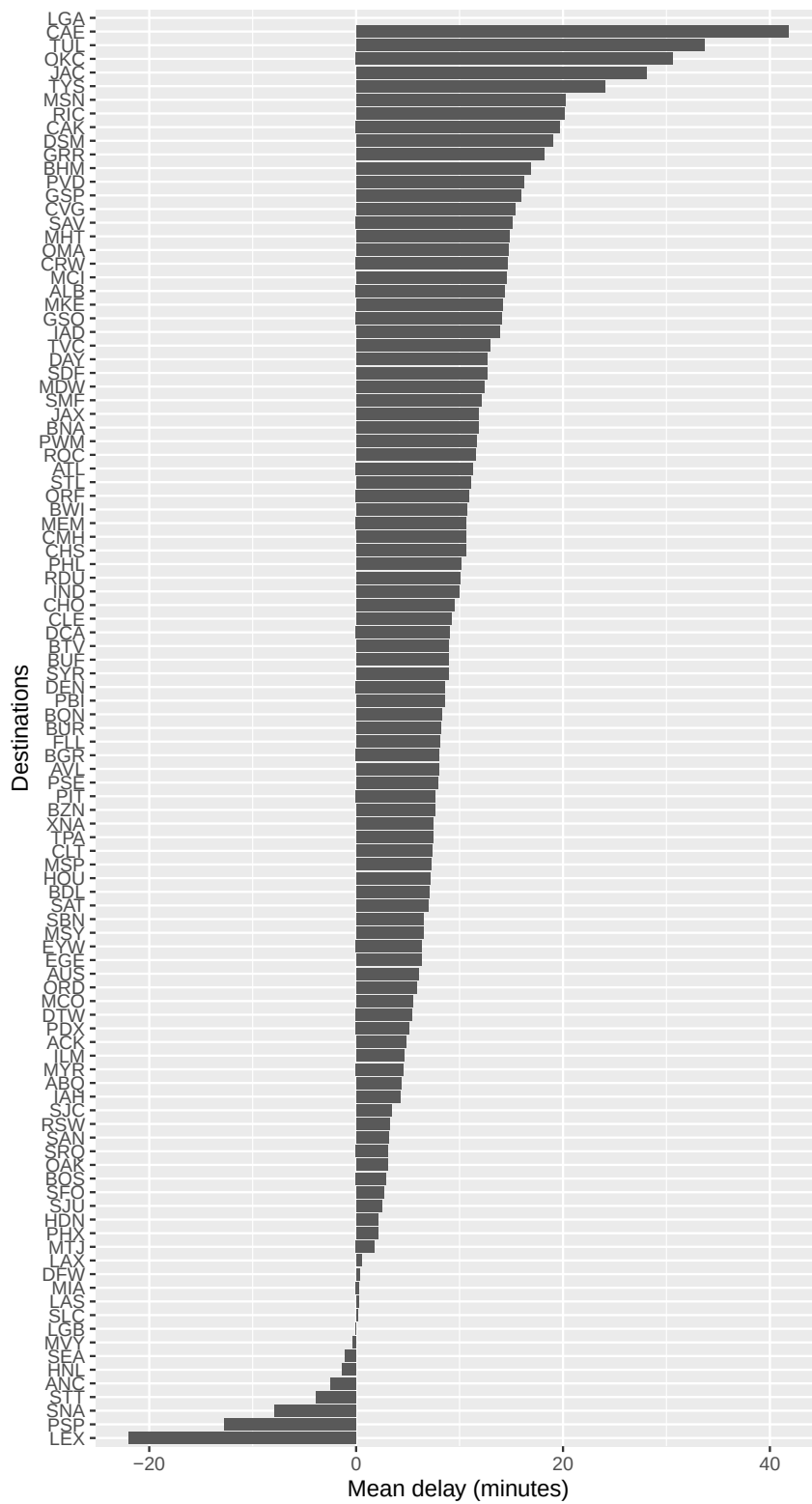
`fig.height`: What should be the height of figures?

`fig.width`: What should be the width of figures?

`fig.asp`: What should be the aspect ratio of figures?

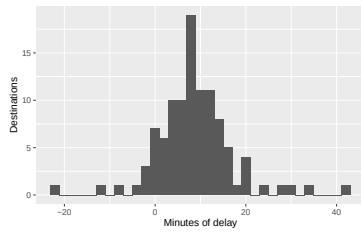
`fig.align`: How should figures be aligned?

We might want a bigger plot for this:





And a smaller plot for this:



## 7.2 Changing your defaults

At the beginning of your code, you can set custom defaults so all your chunks will render the same way (unless you override by specifically adding arguments to a chunk itself). This is handy in that you will then not need to repeat the custom arguments in each code chunk. For example, you can set a default figure size.

Here is an example:

```
knitr::opts_chunk$set(echo = TRUE,
 fig.width = 5,
 fig.height = 3,
 out.width = "5in",
 out.height = "3in", fig.align = "center")
```

## 8 Making tables in Markdown

When writing reports you may, from time to time, need to include a table. You should probably make a chart instead, but every so often a table actually is a nice thing to have. This chapter focuses on two key aspects: creating the table itself, and formatting it for a report or presentation.

Many table-making packages and functions in R produce basic tables that display nicely in a monospace font on the screen. This is a good starting point, but you'll often need additional formatting to make the table publication-ready. R offers several excellent packages to help with this, catering to different needs. Some are particularly suited for HTML documents (such as websites), while others are better for PDF documents (such as reports and papers). Finding the right package for your specific use case can take some trial and error.

To illustrate these concepts, let's start with some fake data.

```
library(tidyverse)
dat = tibble(G = sample(LETTERS[1:5], 100, replace=TRUE),
 X = rnorm(100),
 rp = sample(letters[1:3], 100, replace=TRUE),
 Z = sample(c("tx", "co"), 100, replace=TRUE),
 Y = rnorm(100))
```

We can make summary of it by our grouping variable:

```
sdat <- dat %>% group_by(G) %>%
 summarise(EY = mean(Y),
 pT = mean(Z == "tx"),
 sdY = sd(Y))
```

Our intermediate results:

```
sdat
```

```
A tibble: 5 x 4
 G EY pT sdY
<chr> <dbl> <dbl> <dbl>
1 A 0.357 0.45 0.764
2 B -0.165 0.364 1.01
```

```

3 C 0.117 0.278 1.19
4 D -0.0900 0.722 1.07
5 E 0.181 0.364 0.885

```

We can print this out in a much cleaner form using the `kable()` method from the `knitr` package:

```
knitr::kable(sdat, digits = 2)
```

G	EY	pT	sdY
A	0.36	0.45	0.76
B	-0.16	0.36	1.01
C	0.12	0.28	1.19
D	-0.09	0.72	1.07
E	0.18	0.36	0.89

Say our grouping variable is a set of codes for something more special. We can merge in better names by first making a small “cross-walk” of the ID codes to the full names, and then merging them to our results:

```

names = tribble(~ G, ~ name,
 "A", "fred",
 "B", "doug",
 "C", "xiao",
 "D", "lily",
 "E", "unknown")

```

```
names
```

```
A tibble: 5 x 2
```

```

 G name
 <chr> <chr>
1 A fred
2 B doug
3 C xiao
4 D lily
5 E unknown

```

```

sdat = left_join(sdat, names, by = "G") %>%
 relocate(name)

```

Again, the easiest way to make a nice clean table is with the `kable` command.

```
knitr::kable(sdat, digits=2)
```

name	G	EY	pT	sdY
fred	A	0.36	0.45	0.76
doug	B	-0.16	0.36	1.01
xiao	C	0.12	0.28	1.19
lily	D	-0.09	0.72	1.07
unknown	E	0.18	0.36	0.89

`kable()` is a great workhorse table-making tool! There are expansion R packages as well, e.g. `kableExtra`, which can do lots of fancy customization stuff.

## 8.1 Making a “table one”

The “table one” is the first table in a lot of papers that show general means of different variables for different groups. Perhaps not surprisingly, the `tableone` package is useful for making such tables:

```
library(tableone)
```

```
sample mean
```

```
CreateTableOne(data = dat,
 vars = c("G", "Z", "X"))
```

```

Overall
n 100
G (%)
 A 20 (20.0)
 B 22 (22.0)
 C 18 (18.0)
 D 18 (18.0)
 E 22 (22.0)
Z = tx (%) 43 (43.0)
X (mean (SD)) -0.02 (0.98)
```

```
you can also stratify by a variables of interest
```

```
tb <- CreateTableOne(data = dat,
 vars = c("X", "G", "Y"),
 strata = c("Z"))
```

```
tb
```

Stratified by Z				
	co	tx	p	test
n	57	43		
X (mean (SD))	0.06 (0.93)	-0.13 (1.04)	0.333	
G (%)			0.066	
A	11 (19.3)	9 (20.9)		
B	14 (24.6)	8 (18.6)		
C	13 (22.8)	5 (11.6)		
D	5 ( 8.8)	13 (30.2)		
E	14 (24.6)	8 (18.6)		
Y (mean (SD))	0.18 (0.95)	-0.05 (1.03)	0.249	

You can then use `kable` on your table as so:

```
print(tb$ContTable, printToggle = FALSE) %>%
 knitr::kable()
```

	co	tx	p	test
n	57	43		
X (mean (SD))	0.06 (0.93)	-0.13 (1.04)	0.333	
Y (mean (SD))	0.18 (0.95)	-0.05 (1.03)	0.249	

## 8.2 The stargazer package

You can easily make pretty tables using the **stargazer** package. You need to ensure the data is a `data.frame`, not `tibble`, because **stargazer** is old school. It appears to only do continuous variables. Stargazer is probably best known for making regression tables, but it can make other kinds of tables as well, such as data summaries.

When using **stargazer** to summarize a dataset, you can specify that it should include only some of the variables and you can omit stats that are not of interest:

```
to include only variables of interest
stargazer(as.data.frame(dat), header=FALSE,
 omit.summary.stat = c("p25", "p75", "min", "max"),
 # to omit percentiles
 title = "Table 1: Descriptive statistics",
 type = "text")
```

Table 1: Descriptive statistics

Statistic	N	Mean	St. Dev.
X	100	-0.019	0.978
Y	100	0.080	0.985

See the `stargazer` help file for how to set/change more of the options: <https://cran.r-project.org/web/packages/stargazer/stargazer.pdf>

**Warning:** `stargazer` does not work well with tibbles (the data frames you get from tidyverse commands), so you need to convert your data to a `data.frame` before using it. In particular, you have to “cast” your data to a `data.frame` to make it work:

```
library(stargazer)

to include all variables
stargazer(as.data.frame(dat), header = FALSE, type="text")
```

To use `stargazer` in a PDF or HTML report, you will want the report to format the table so it doesn’t look like raw output. To do so, you would not set `type="text"` but rather `type="latex"` or `type="html"`, and then in the markdown chunk header (the thing that encloses all your R code) you would say “results=‘asis’” in your code chunk header like so:

```
::: {.cell layout-align="center" messages='false'}

:::
```

This will ensure the output of `stargazer` gets formatted properly in your R Markdown.

Unfortunately, it is hard to dynamically make a report that can render to either html or a pdf, so you will have to choose one or the other. If you are making a PDF, you will want to use `type="latex"` and if you are making an HTML report, you will want to use `type="html"`.

## 8.3 The `xtable` package

The `xtable` package is another great package for making tables. It is particularly good for LaTeX documents. It is a bit more complicated to use than `stargazer`, but it is very powerful. Here is an example of how to use it:

```
library(xtable)
xtable(sdat, caption = "A table of fake data")
```

Here you would again use the “results=‘asis’” in the chunk header to get the table to render properly in your R Markdown document.

# **Part III**

## **On Plotting**



## 9 Intro to ggplot

This chapter demonstrates some powerful features of `ggplot2`, the plotting package that we use most in this course.

`ggplot` can initially seem like a nightmare to some, but once you wrestle it to the ground it is one of the most powerful visualization tools you might have in your toolbox. Happily, it is fairly easy to get some basics up and running once you start looking at the world the way it does. Let's start doing that.

First, `ggplot` thinks of a plot as a collection of layers stacked on top of each other. The way this looks in code is a bunch of weird function calls connected together with `+`. You read this series of calls left to right. The first call is always a statement saying what data you are plotting and what variables you care about. So before you can even plot, you need to make sure your data are in a nice, tidy data frame.

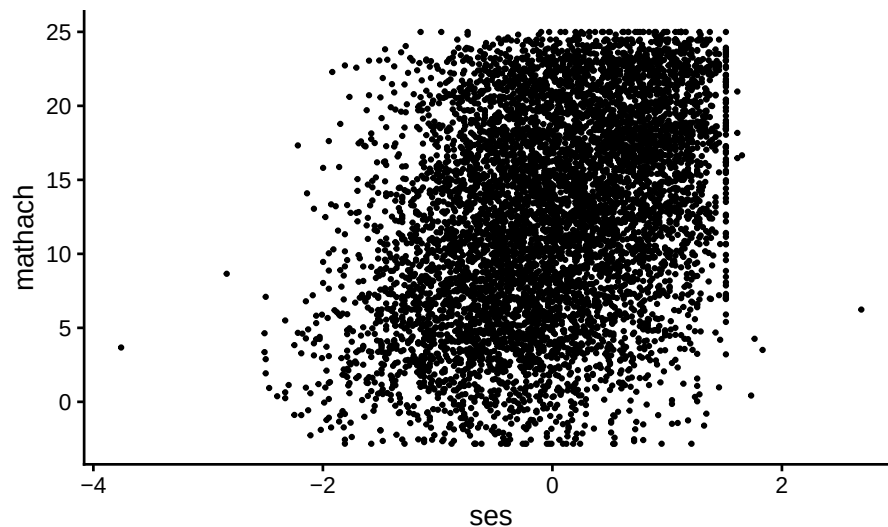
Happily, when you load data, it usually is. For example:

```
dat <- read_dta("data/hsb.dta")
head(dat)

A tibble: 6 x 26
 minority female ses mathach size sector pracad disclim himinty schoolid
 <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
1 0 1 -1.53 5.88 842 0 0.350 1.60 0 1224
2 0 1 -0.588 19.7 842 0 0.350 1.60 0 1224
3 0 0 -0.528 20.3 842 0 0.350 1.60 0 1224
4 0 0 -0.668 8.78 842 0 0.350 1.60 0 1224
5 0 0 -0.158 17.9 842 0 0.350 1.60 0 1224
6 0 0 0.0220 4.58 842 0 0.350 1.60 0 1224
i 16 more variables: mean <dbl>, sd <dbl>, sdalt <dbl>, junk <dbl>,
sdalt2 <dbl>, num <dbl>, se <dbl>, sealt <dbl>, sealt2 <dbl>, t2 <dbl>,
t2alt <dbl>, pickone <dbl>, mmses <dbl>, mnses <dbl>, xb <dbl>, resid <dbl>
```

The easiest full plot to make has two elements. The first gives what your variables are, and the second says how to plot them:

```
ggplot(dat, aes(y = mathach, x = ses)) +
 geom_point(cex=0.5)
```

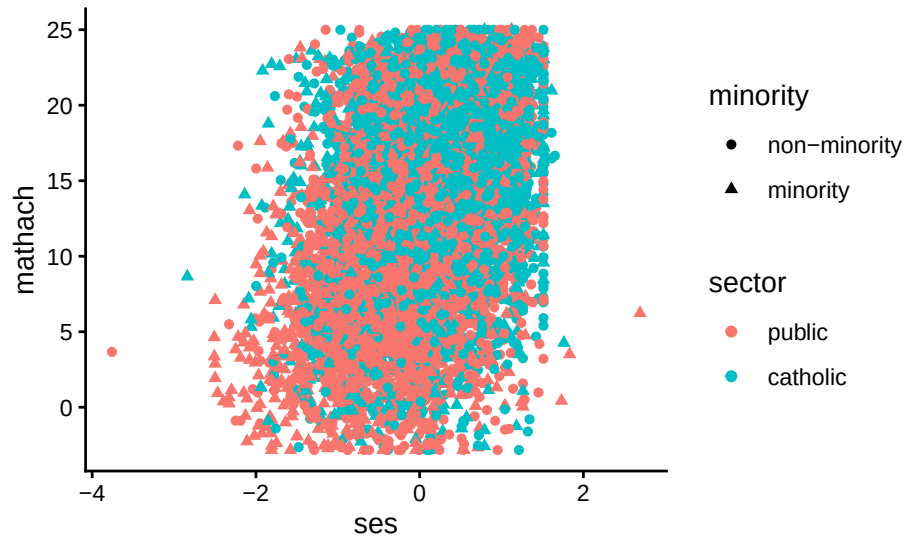


So far, nothing too scary, right? The `ggplot( dat, aes(x=mathach, y=ses) )` says “My plot is going to use `dat` for my data, and my *y*-axis is the `mathach` variable and my *x*-axis is `ses`.” The `aes()` bit is “aesthetics”—it is a way of tying variables to different kinds of things you could have on your plot: *x* location, *y* location, color, plotting symbol, and a few other things.

For example:

```
dat <- dat |>
 mutate(sector = factor(sector, levels=c(0,1), labels=c("public","catholic")),
 minority = factor(minority, labels=c("non-minority","minority")))

ggplot(dat, aes(y=mathach, x=ses, col=sector, pch=minority)) +
 geom_point()
```

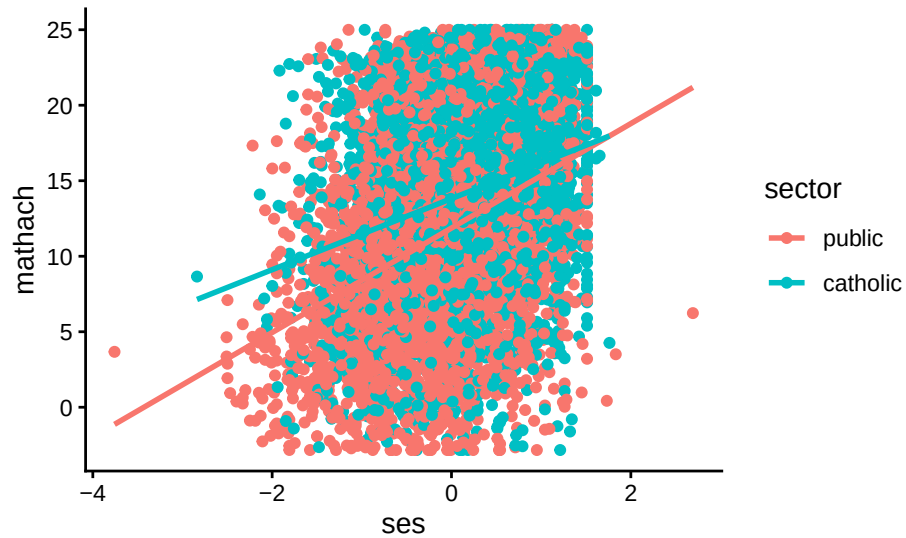


Note that `ggplot` wants the data frame to be neatly put together, including that categorical variables are listed as factors. This is why we convert the dummy `sector` to a factor above. Once you do this, however, it will label things in a nice way.

## 9.1 Summarizing

You can also automatically add various statistical summaries, such as simple regression lines:

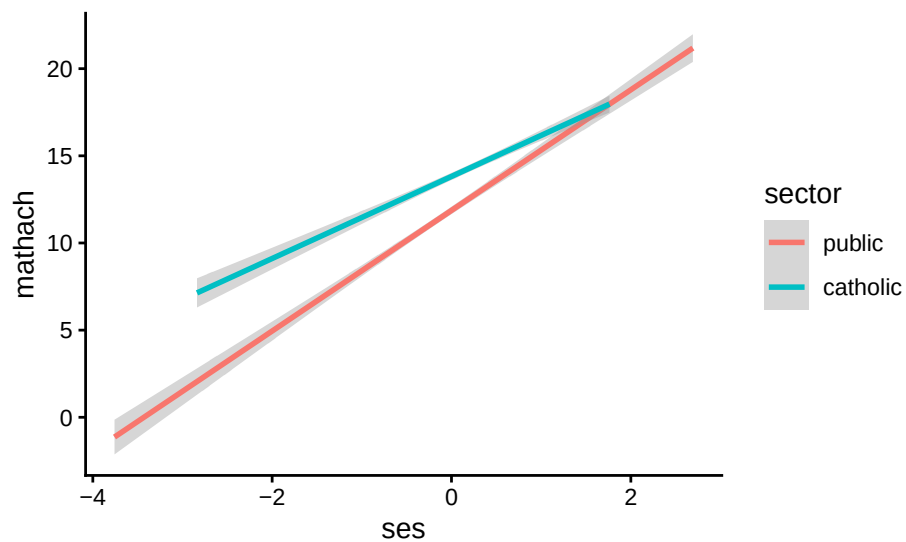
```
ggplot(dat, aes(y=mathach, x=ses, col=sector)) +
 geom_point() +
 stat_smooth(method="lm", se = FALSE)
```



Notice how it automatically realized you have two subgroups of data defined by sector. It gives you a regression line for each group.

The elements of the plot are stacked, and if you remove one of the elements, it will not appear:

```
ggplot(dat, aes(y=mathach, x=ses, col=sector)) +
 stat_smooth(method="lm")
```



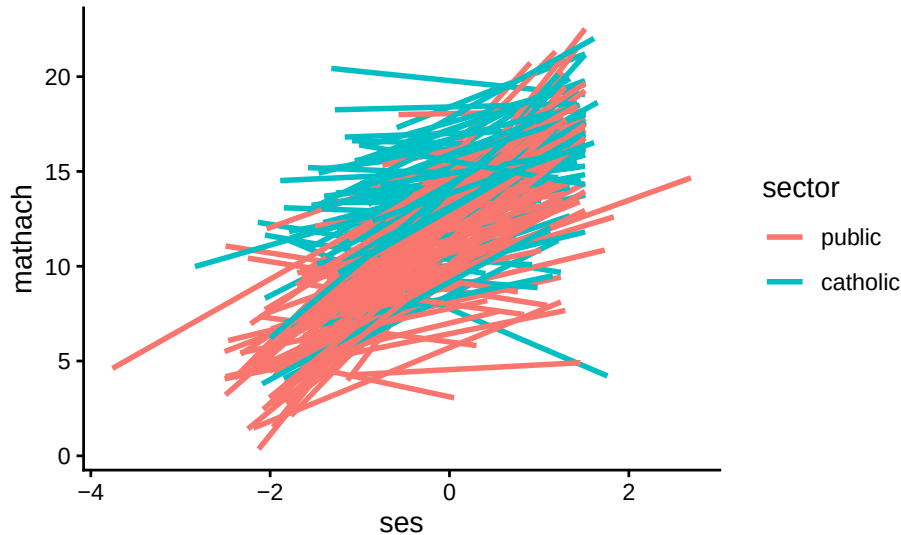
Here we also added some uncertainty bars around the regression lines by not saying `se = FALSE`. (Including uncertainty is the default; this uncertainty is not to be trusted, especially in this course, as it is not taking clustering into account.)

## 9.2 Grouping

Combining these ideas we can make a trend line for each school:

```
my.plot = ggplot(dat, aes(y=mathach, x=ses, col=sector, group=schoolid)) +
 stat_smooth(method="lm", alpha=0.5, se = FALSE)
```

```
my.plot
```



The trendlines automatically extend to the limits of the data they are run on, hence the different lengths.

Also, notice we “saved” the plot in the variable `my.plot`. Only when we “print” the plot will the plot appear on your display. When we type the name of a variable, it prints. Once you have a plot stored in a variable you can augment it very easily.

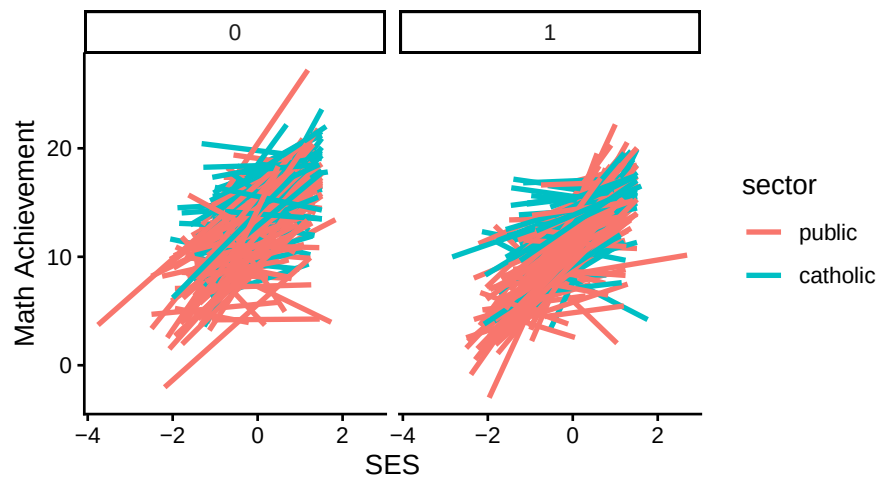
As you may now realize, `ggplot2` is very, very powerful.

## 9.3 Customization

We next show some other things you can do. For example, you can make lots of little plots:

```
my.plot +
 facet_grid(~ female) +
 ggtitle("School-level trend lines for their male and female students") +
 labs(x="SES",y="Math Achievement")
```

School-level trend lines for their male and female studen



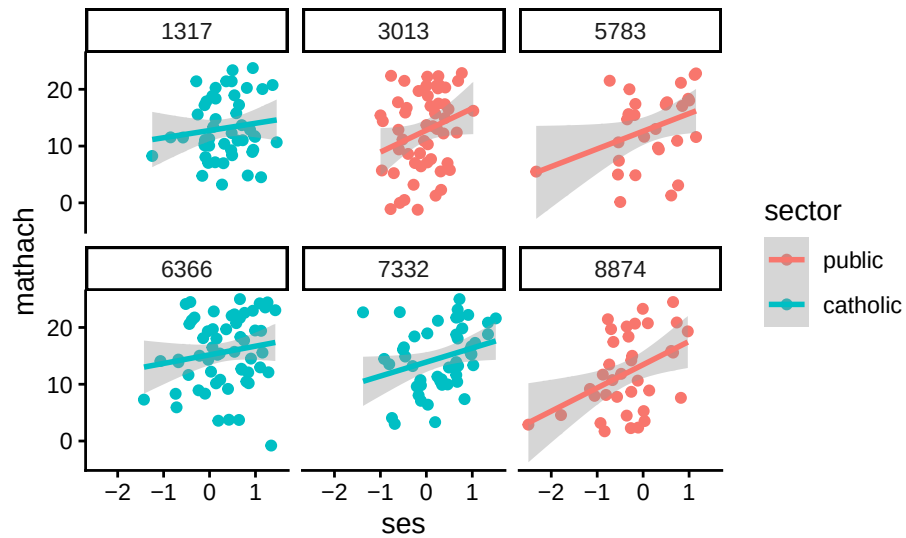
Or,

```
random subset of schoolid
sch <- sample(unique(dat$schoolid), 6)

pipe into ggplot
sch.six <- dat |>
 filter(schoolid %in% sch)

my.six.plot <- ggplot(sch.six, aes(y=mathach, x=ses, col=sector)) +
 facet_wrap(~ schoolid, ncol=3) +
 geom_point() + stat_smooth(method="lm")

my.six.plot
```



Also shown in the above are adding titles.

## 9.4 Themes

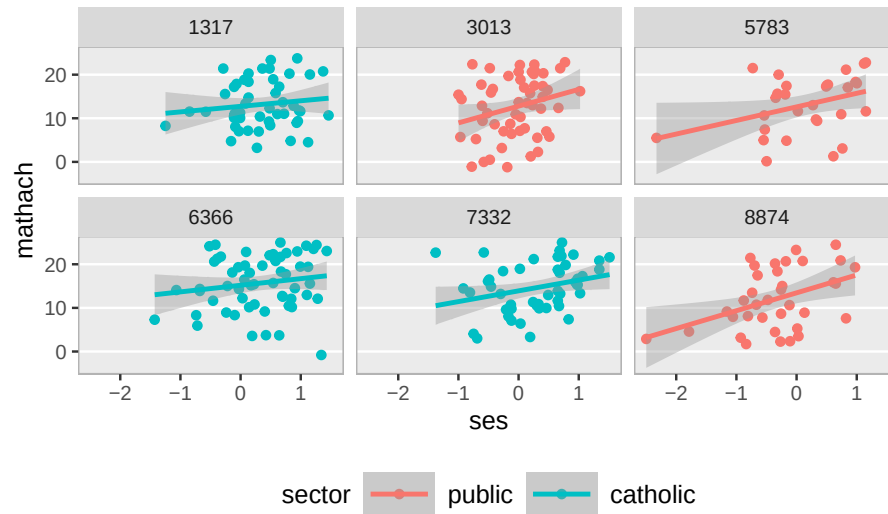
You can very quickly change the entire presentation of your plot using **themes**. There are pre-packaged ones, and you can make your own that you use over and over. Here we set up a theme to be used moving forward

```
library(ggthemes)
my_t = theme_calc() + theme(legend.position="bottom",
 legend.direction="horizontal",
 legend.key.width=unit(1,"cm"))

theme_set(my_t)
```

Compare the same plot from above, now with a new theme.

```
my.six.plot
```



Cool, no?

## 9.5 Next steps

There is a lot of information out there on `ggplot` and my best advice is to find code examples, and then modify them as needed. There are tutorials and blogs that walk through building plots (search for “ggplot tutorial” for example), but seeing examples seems to be the best way to learn the stuff. For example, you could use the above code for your project one quite readily. And don’t be afraid to ask how to modify plots on Piazza!

In particular, check out the excellent “[R for Data Science](#)” textbook. It extensively uses ggplot, starting [here](#).



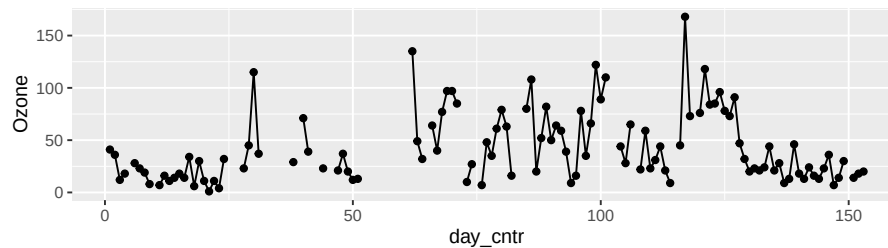
## 10 Some tips for making plots

This document provides several simple plotting tips for using RStudio to make plots (primarily using the `ggplot2` package in `tidyverse`). These are all very simple tricks that can radically enhance the legibility of a plot. This document also covers how to save your plots so you can use them in presentations and other documents, and how to control the size of your plot in a R Markdown document.

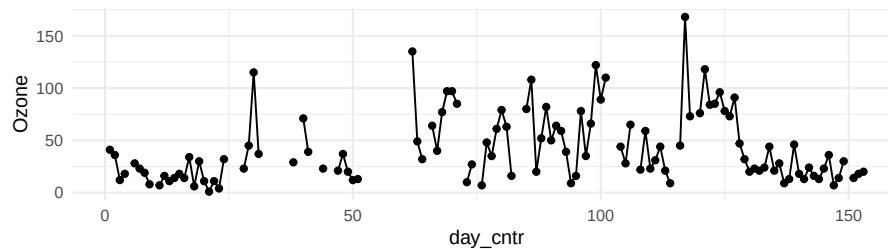
# 11 Setting a theme

Themes in ggplot give you a bunch of default options for plotting. For example, you can get rid of the grey background of plots with the `theme_minimal()` command.

```
airquality$day_cntr = 1:nrow(airquality)
ggplot(airquality, aes(day_cntr, Ozone)) +
 geom_point(na.rm=TRUE) + geom_line()
```

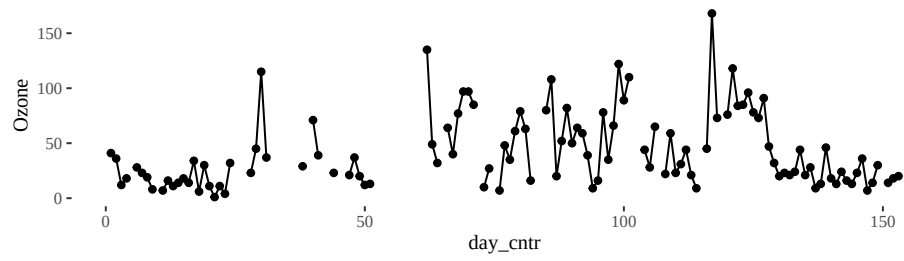


```
ggplot(airquality, aes(day_cntr, Ozone)) +
 geom_point(na.rm=TRUE) + geom_line() +
 theme_minimal()
```



I also quite like Tufte theme in the `ggthemes` bonus library (this library is a library of various themes with styles good and bad):

```
ggplot(airquality, aes(day_cntr, Ozone)) +
 geom_point(na.rm=TRUE) + geom_line() +
 ggthemes::theme_tufte()
```



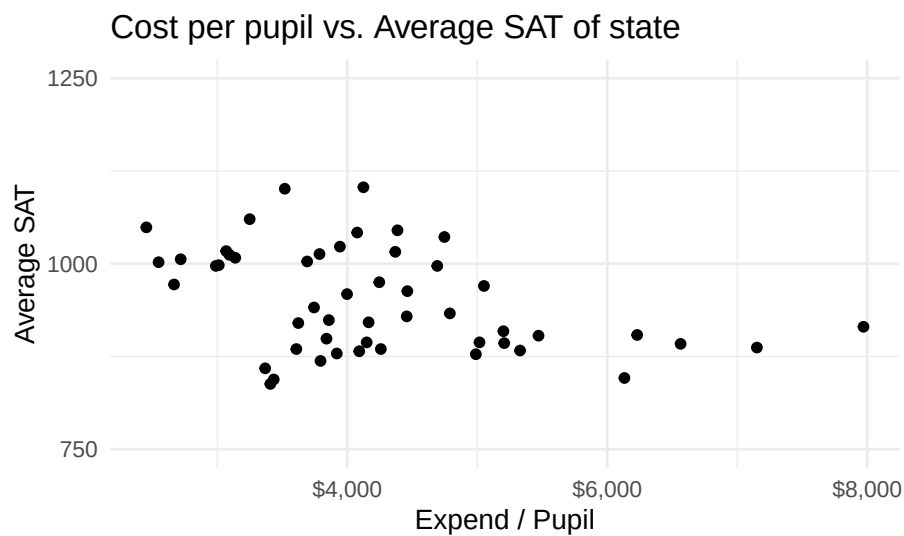
You can also set the theme globally:

```
theme_set(theme_minimal())
```

## 12 Formatting axes and making labels

The `labs()` command is critical for getting labels for your plot. The `scale_x_continuous()` (or `y`) allow you to control the scales on each axis. Also, there is a nice package, `scales` that will format your x and y-axes. Witness!

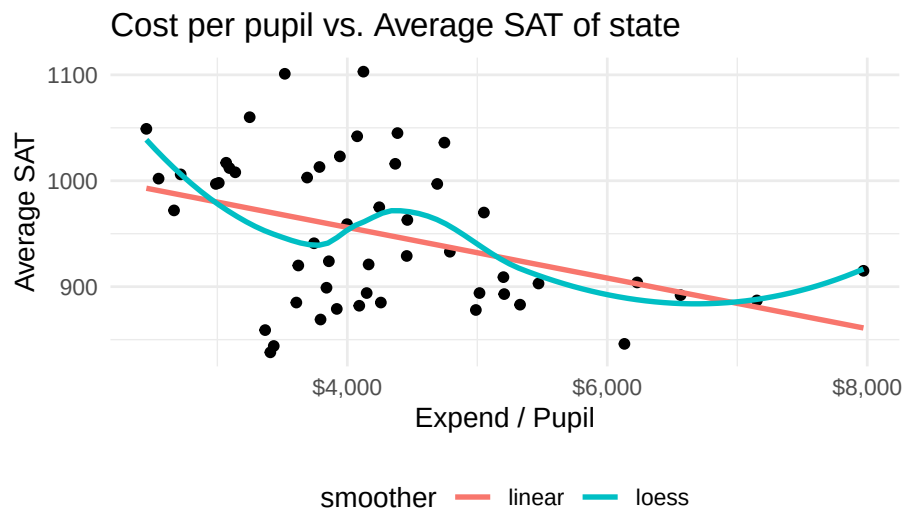
```
library(scales) # for label_dollar(), below
ggplot(sat93, aes(expend, tot_sat)) +
 geom_point() +
 scale_x_continuous(labels = label_dollar()) +
 labs(title = "Cost per pupil vs. Average SAT of state",
 y = "Average SAT",
 x = "Expend / Pupil") +
 theme_minimal() +
 scale_y_continuous(limits = c(750, 1250),
 breaks = c(750, 1000, 1250))
```



## 13 Making legends using aes()

If you are making a bunch of lines and want to color them, you can do so by giving them all names and then letting R pick the colors for you. This also lets you have a nice legend telling you which color is which. Also note how you can put a name for your legend inside `labs()`. Witness!

```
library(scales) # for label_dollar(), below
ggplot(sat93, aes(expend, tot_sat)) +
 geom_point() +
 scale_x_continuous(labels = label_dollar()) +
 labs(title = "Cost per pupil vs. Average SAT of state",
 y = "Average SAT",
 x = "Expend / Pupil",
 color = "smoother") +
 geom_smooth(aes(col = "linear"), method="lm", se=FALSE) +
 geom_smooth(aes(col = "loess"), se=FALSE) +
 theme_minimal() +
 theme(legend.position = "bottom")
```

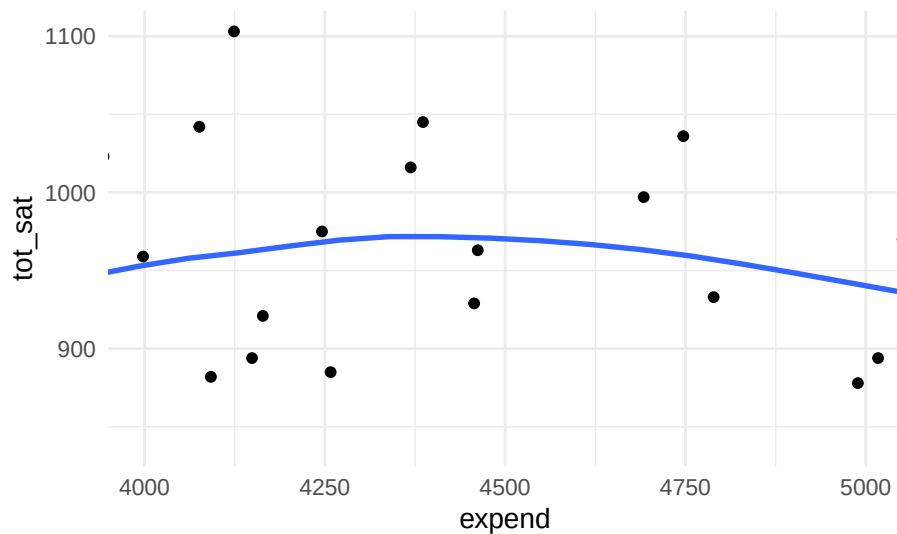


## 14 Cropping vs. zooming in

If you set the scale of your plot via `scale_x_continuous()` you will drop data from your analysis. If you use `coord_cartesian()` you will zoom in on the specified window, but it will be using all the data for smoothers, etc. This can be an important difference:

```
ggplot(sat93, aes(expend, tot_sat)) +
 geom_point() +
 geom_smooth(se=FALSE) +
 coord_cartesian(xlim=c(4000, 5000))
```

``geom_smooth()`` using method = 'loess' and formula = 'y ~ x'

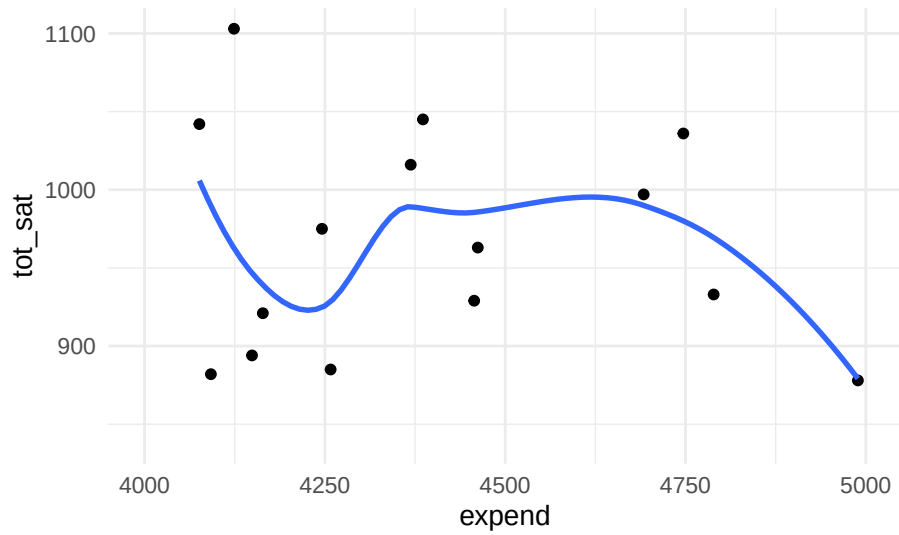


```
ggplot(sat93, aes(expend, tot_sat)) +
 geom_point() +
 geom_smooth(se=FALSE) +
 scale_x_continuous(limits = c(4000, 5000))
```

``geom_smooth()`` using method = 'loess' and formula = 'y ~ x'

Warning: Removed 36 rows containing non-finite outside the scale range  
(`stat\_smooth()`).

Warning: Removed 36 rows containing missing values or values outside the scale range  
(`geom\_point()`).



## 15 Controlling size in markdown files

You can add `fig.height=2`, `fig.width=7` into the header of a chunk. For example, the chunk above had:

```
{r,r my_figure, fig.height=2, fig.width=7}
```

You can also set overall size in your setup chunk via

```
knitr::opts_chunk$set(echo = TRUE,
 fig.width = 5,
 fig.height = 3,
 out.width = "75%",
 fig.align = "center")
```

The `fig.width` controls how big the figure is when plotting. The figure is then rescaled to fit into the area dictated by `out.width`. The “75%” can be replaced with, e.g., “4in” for 4 inches. The percent is the percent of text width of the page.

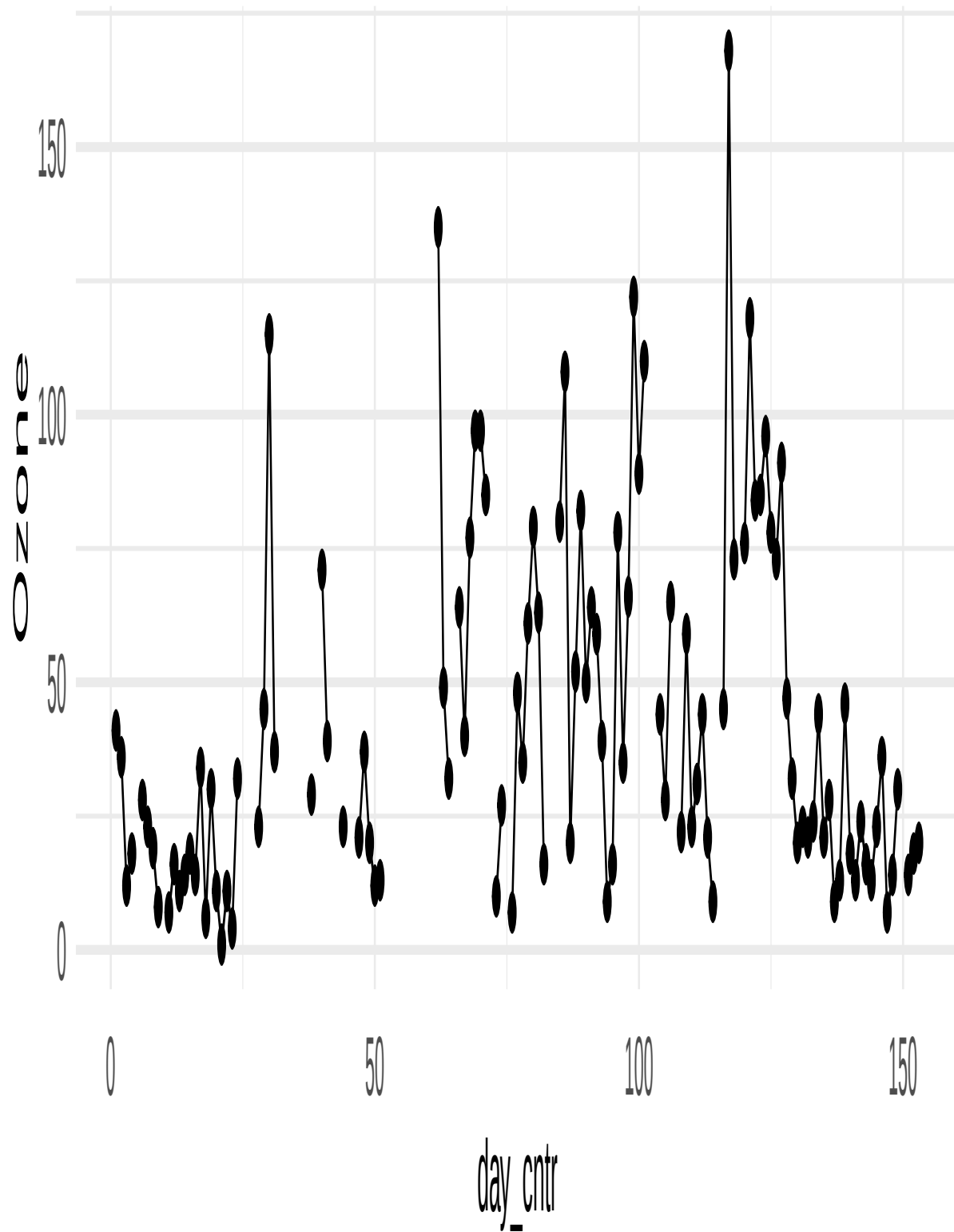


## 16 Making the fonts of labels, etc., larger

If you make the figure smaller, the axes labels and line thicknesses and everything get larger, relatively speaking. Compare the following, with the header chunks printed out for clarity:

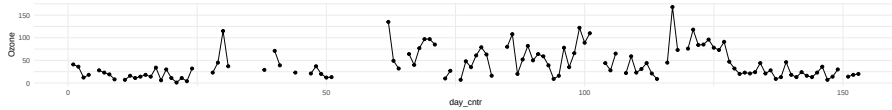
```
{r,r, fig.height=2, fig.width=7, out.width="100%", out.height="100%"}
```

```
ggplot(airquality, aes(day_cntr, Ozone)) +
 geom_point(na.rm=TRUE) + geom_line(na.rm=TRUE)
```



```
{r,r, fig.height=2, fig.width=16}
```

```
ggplot(airquality, aes(day_cntr, Ozone)) +
 geom_point(na.rm=TRUE) + geom_line()
```

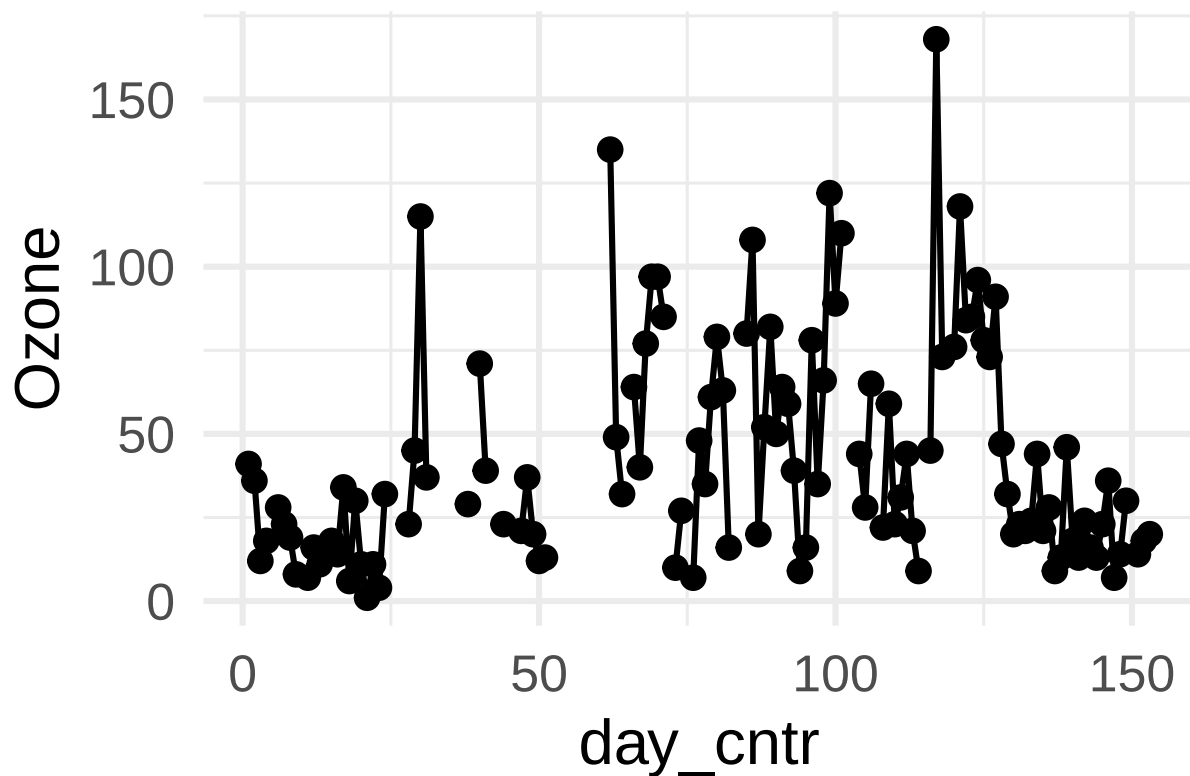


See how the second figure is a bigger figure rescaled to fit into a smaller space (width-wise). Everything thus looks tiny. The other plots in this document are `fig.height=2`, `fig.width=7`.

If the plot is small, but `out.width` is large, it will not scale up but instead keep to the desired size. The `out.width` only ensures plots are no larger than given. E.g. `fig.width=3`, `fig.height=2`, `out.width="100%"` gives:

```
{r,r, warning = FALSE, fig.width=3, fig.height=2, out.width="100%"}
```

```
ggplot(airquality, aes(day_cntr, Ozone)) +
 geom_point(na.rm=TRUE) + geom_line()
```



## 17 Saving high resolution plots

There are three ways to export plots from RStudio. You can click on the export button on RStudio. If you say “Copy to Clipboard” you will get a bitmap plot (basically a digital photo of your plot), which will be fuzzy if you blow it up in your report. One way around that is specify a larger width and height in the dialog box before you save. “Save as Image” has the same concern.

The “Save to PDF” will save a vector image of your plot. Vector images remember the dots and lines used for your plot, and will be crisp if you make them large or small.

You can also use `ggsave` as follows:

```
ggplot(airquality, aes(day_cntr, Ozone)) +
 geom_point(na.rm=TRUE) + geom_line()
ggsave("my_ac_plot.pdf", width = 8, height=2)
```

This allows you to specify the width and height and then, if you want to re-make your plot, it will save the exact same size. This is also good if you have a series of plots you want to make the same size.

I recommend saving to PDF. The image bitmap plots always look a bit lousy.

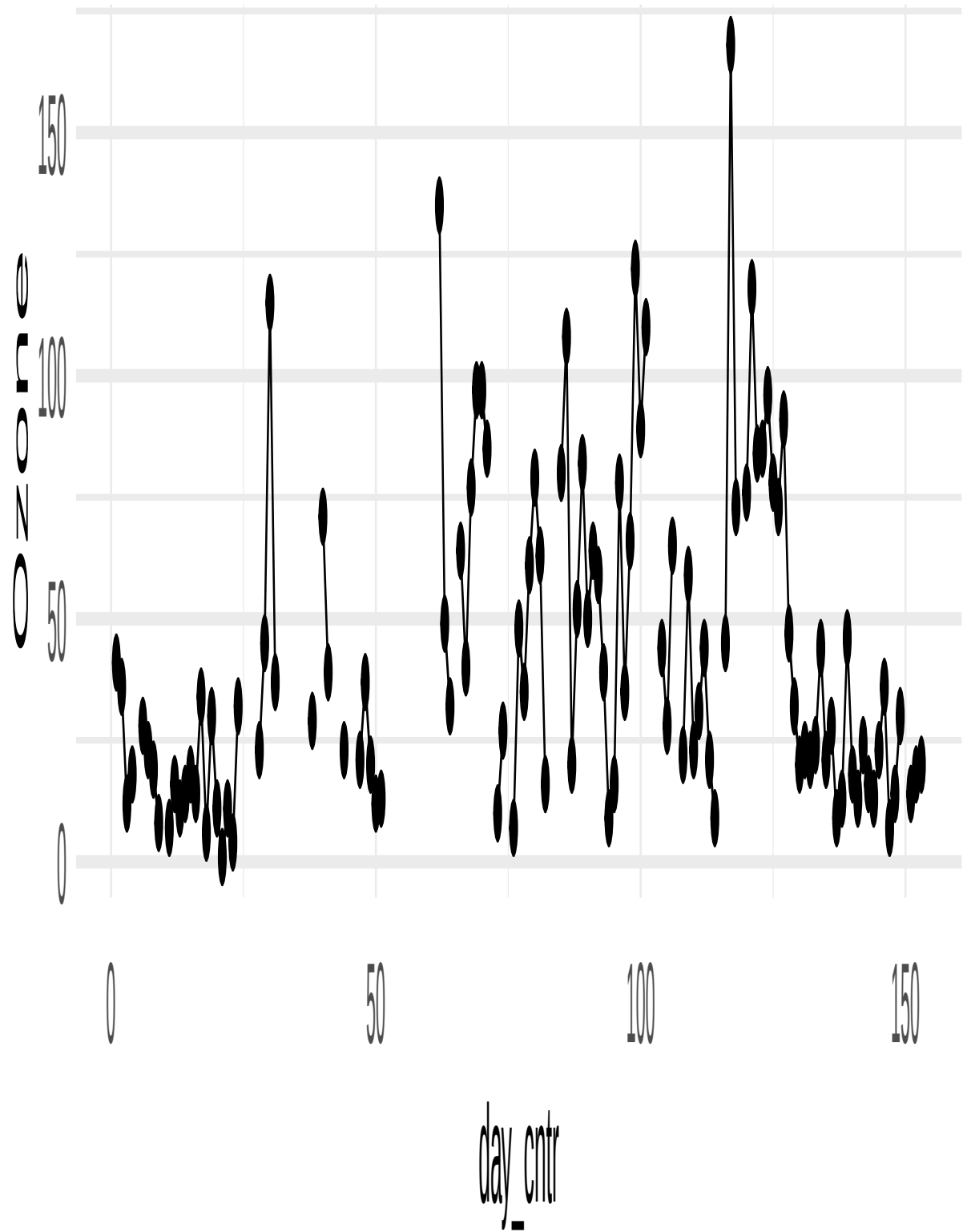
## 18 Think about the height of a plot.

Very often, making your plot *shorter* will make it better. Big vertical distance amplifies variation and makes it hard to follow trends.

Compare

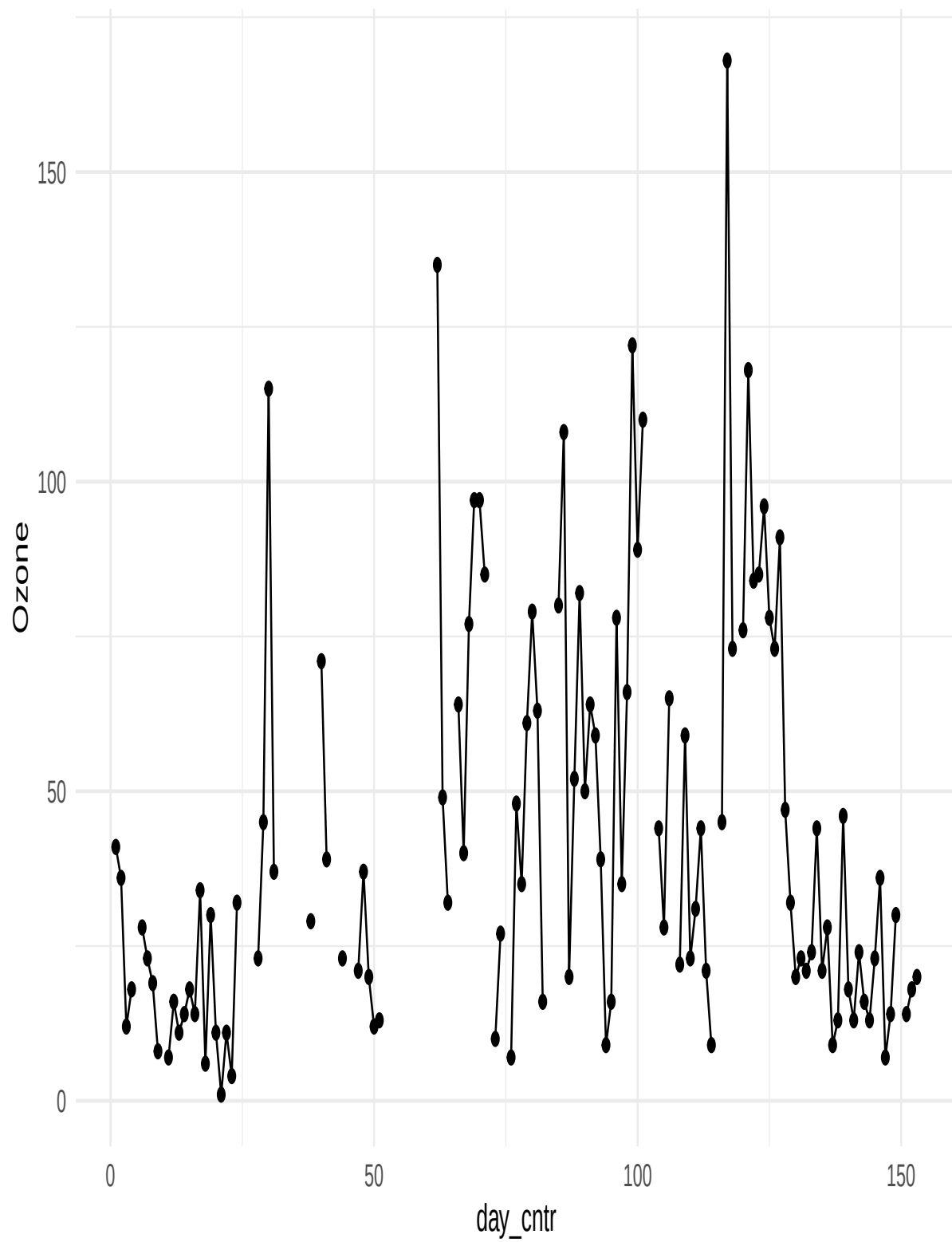
```
{r,r, fig.height=1.5, fig.width=7, out.width="100%", out.height="100%"}
```

```
ggplot(airquality, aes(day_cntr, Ozone)) +
 geom_point(na.rm=TRUE) + geom_line(na.rm=TRUE)
```



```
{r,r, fig.height=5, fig.width=7, out.width="100%", out.height="100%"}

ggplot(airquality, aes(day_cntr, Ozone)) +
 geom_point(na.rm=TRUE) + geom_line(na.rm=TRUE)
```



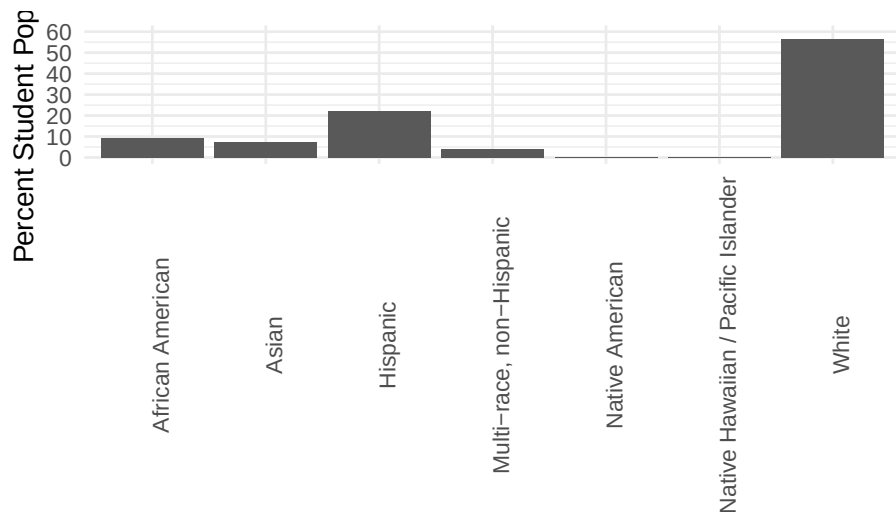


## 19 Think about the x and y axis of your plot

Similarly, swapping the x and y axes of a plot can make labels *a lot* easier to read.

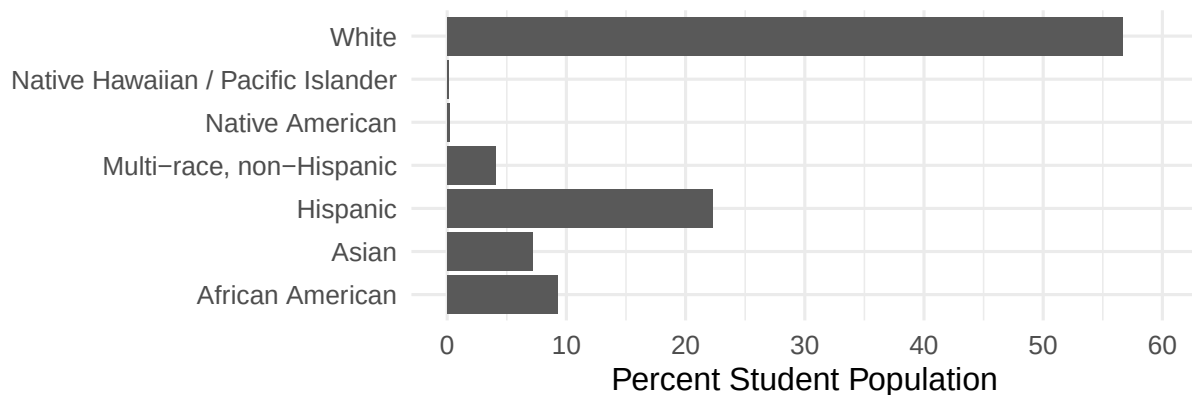
Consider the following two plots:

```
ggplot(dat, aes(x = Category, y=per_students)) +
 geom_bar(stat="identity") +
 theme(axis.text.x = element_text(angle = 90)) +
 scale_y_continuous(limits = c(0,60), breaks = c(0,10,20,30,40,50,60)) +
 labs(y = "Percent Student Population", x="")
```



```
{r,r, fig.height=2, fig.width = 6, out.width="100%"}
```

```
ggplot(dat, aes(x = Category, y=per_students)) +
 geom_bar(stat="identity") +
 theme(axis.text.x = element_text(angle = 90)) +
 labs(y = "Percent Student Population", x="") +
 scale_y_continuous(limits = c(0,60), breaks = c(0,10,20,30,40,50,60)) +
 coord_flip() +
 theme(axis.text.x = element_text(angle = 0))
```



Flipping is easy to do. Just add `coord_flip()` to your `ggplot` command! The choice of vertical height of the plot also matters here. Make your bars not too thick, but still a nice amount thick so your labels are well spaced.

For the first plot, we had to rotate the labels so they didn't overlap. Other choices are likely possible.

Also notice the rotating of the axes labels. With `coord_flip` be careful as to what element you are controlling. Usually fiddling around will get it right in no time!

Note: these data are from the Massachusetts Department of Elementary and Secondary Education, school year 2019-20, for the state.

## 20 Controlling colors, etc

For this, read the “Graphics for Communication” chapter of R for DS: <https://r4ds.had.co.nz/graphics-for-communication.html>

## 21 Tennessee Star Handout

This handout demonstrates a step-by-step worked example of creating a publication-ready plot, along with improvements and stylistic enhancements. Additionally, you will find detailed explanations and use cases for several useful functions, such as `grid.arrange()`, `annotate()`, and `geom_text()`, to help you refine your visualizations effectively.

```
Load in required libraries
library(tidyverse) #this automatically loads ggplot2

Note: you will likely need to install the Ecdat package to load the
data we use via:
#install.packages("Ecdat")
library(Ecdat)

library(gridExtra) # Required for using grid.arrange()
```

### 21.1 Re-creating the Tennessee STAR Boxplot

This handout provides the solution to the in-class plotting exercise where we recreated a plot of the Tennessee Star dataset. In this handout we start with a simple boxplot and work our way up to the final visualization. When then look at some extensions.

In our target plot, we want to visualize reading test scores (`treadssk`) by the type of classroom randomly assigned (`classsk`) along with variables showing whether the student was eligible for free or reduced price lunch in kindergarten (`freelunk`) and race (`race`). The plot we are trying to mimic looks like this:

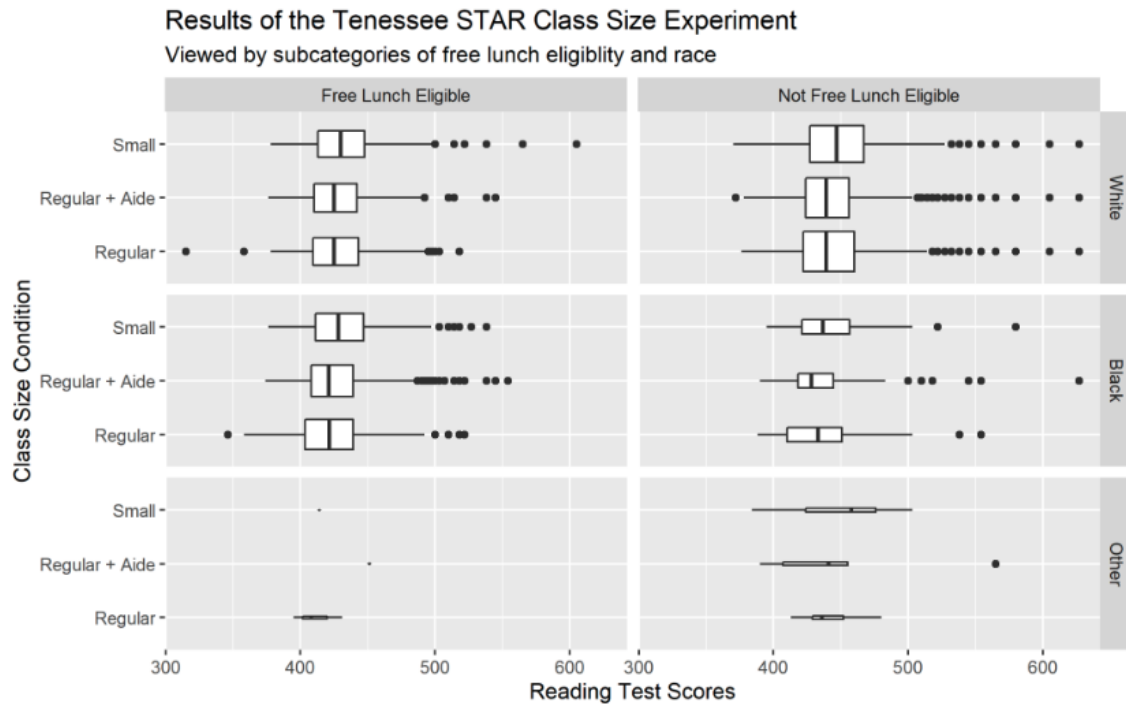
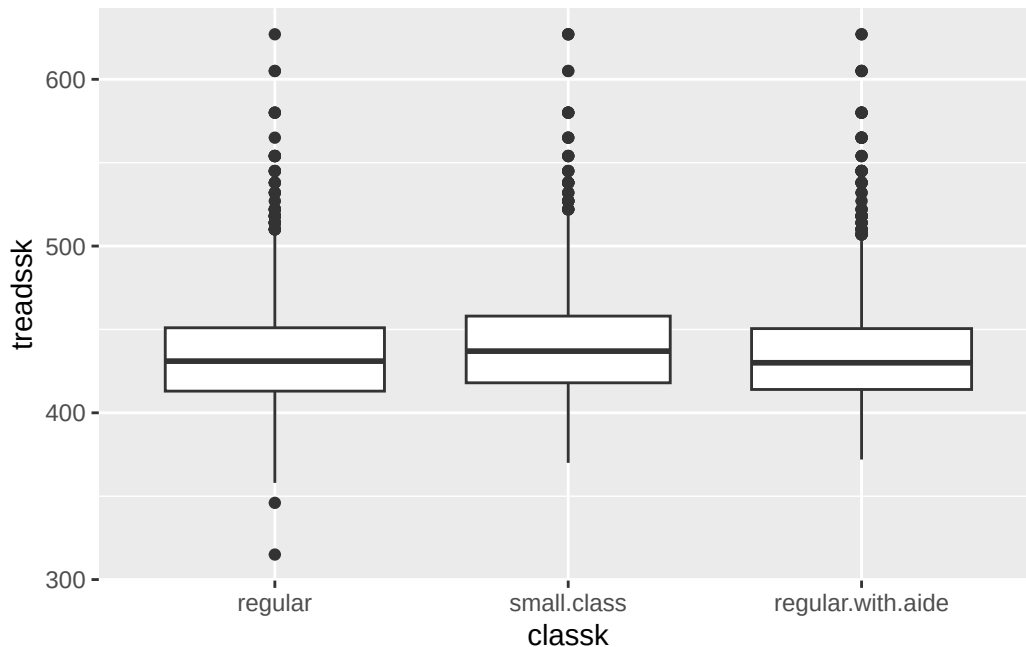


Figure 21.1: The target plot we are trying to recreate

It's useful to start small, and then build up iteratively to more complex plots. Start with a simple `geom_boxplot` of test scores, and then gradually make it more complex to try and emulate the original. Some manipulations are easier done in the `ggplot` itself, whereas other things might be easier if you manipulate the data *before* you give it to `ggplot`.

Let's start by a single boxplot that separate students only by class sizes:

```
star <- Ecdat::Star
star %>%
 ggplot(aes(x = classsk, y = treadssk)) +
 geom_boxplot(varwidth = TRUE)
```

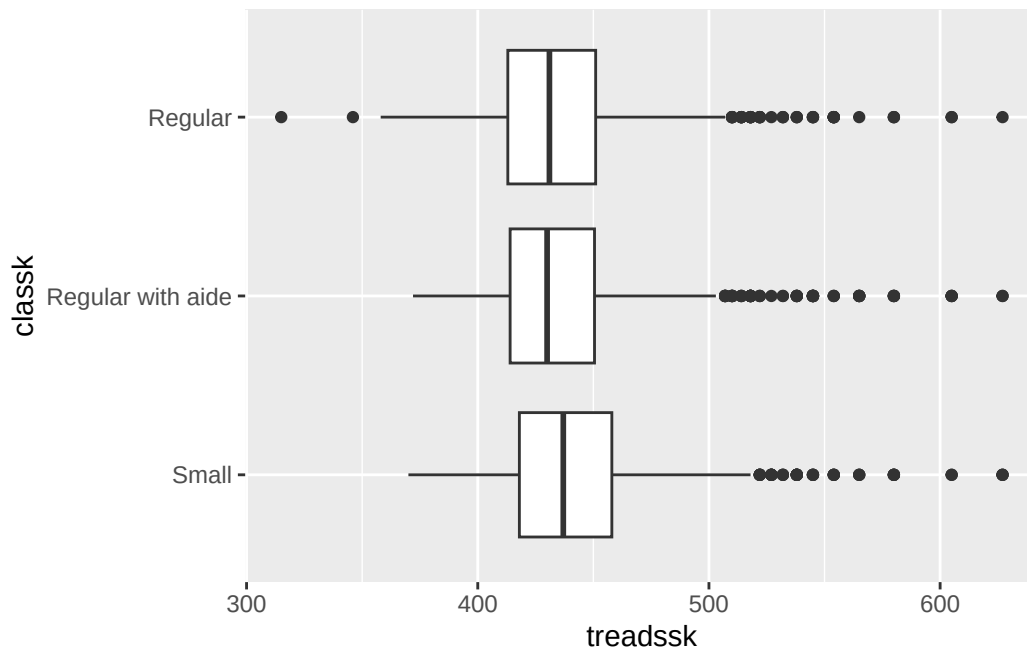


If we want to change the labels of each group, we can convert our `classk` to a factor. The order of the levels will determine the order of the boxes in the plot. Also, the boxes are vertical instead of horizontal, as in the example. We can fix this with `coord_flip()`. Here we go:

```
some pre-processing to make the class sizes a proper factor
star <- mutate(star, classk = factor(classk,
 levels = c("small.class",
 "regular.with.aide",
 "regular"),
 labels = c("Small",
 "Regular with aide",
 "Regular"))))

build simple boxplot
star %>%
 ggplot(aes(x = classk, y = treadssk)) +
 # When varwidth = TRUE,
 # the width of each boxplot is proportional to the
 # sample size of that category. This means that categories with
 # more data points will have wider boxplots, while categories with
 # fewer data points** will have narrower boxplots.
 # If varwidth = FALSE (default), all boxplots have the
 # same width regardless of sample size.
 geom_boxplot(varwidth = TRUE) +
```

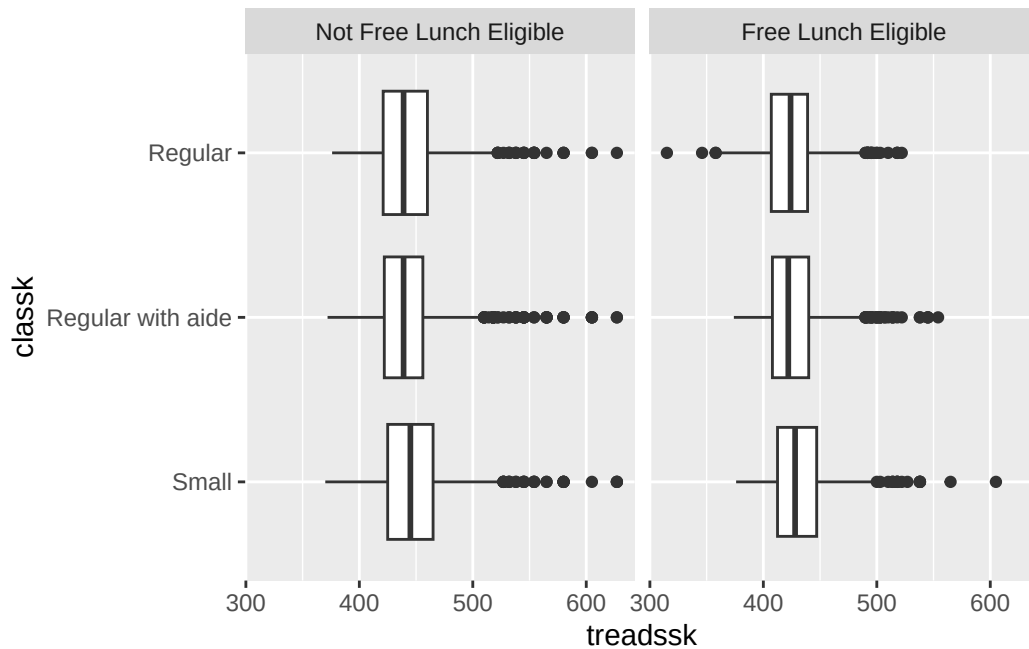
```
coord_flip() # Flip axes for better readability
```



Now let's add in just free lunch as an additional categorical variable.

```
again, pre-process to make sure that free lunch is properly labeled factor
star <- mutate(star, freelunk = recode(freelunk,
 no = "Not Free Lunch Eligible",
 yes = "Free Lunch Eligible"))
```

```
star %>%
 ggplot(aes(x = classk, y = treadssk)) +
 facet_wrap(~freelunk) +
 geom_boxplot(varwidth = TRUE) +
 coord_flip()
```

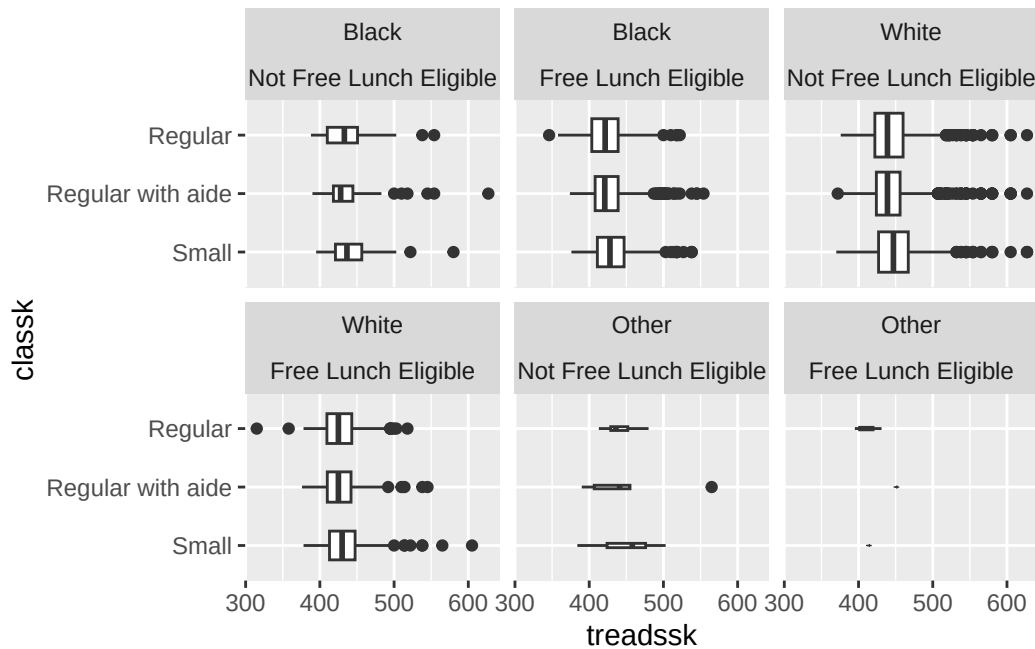


Add in race as another dimension of variability:

```
making sure race is also a properly labeled factor
star <- mutate(star, race = factor(race,
 levels = c("black", "white",
 "other"),
 labels = c("Black", "White",
 "Other"))))

star %>%
 ggplot(aes(x = classk, y = treadssk)) +
 facet_wrap(race~freelunk) +
 geom_boxplot(varwidth = TRUE) +
 coord_flip()
```

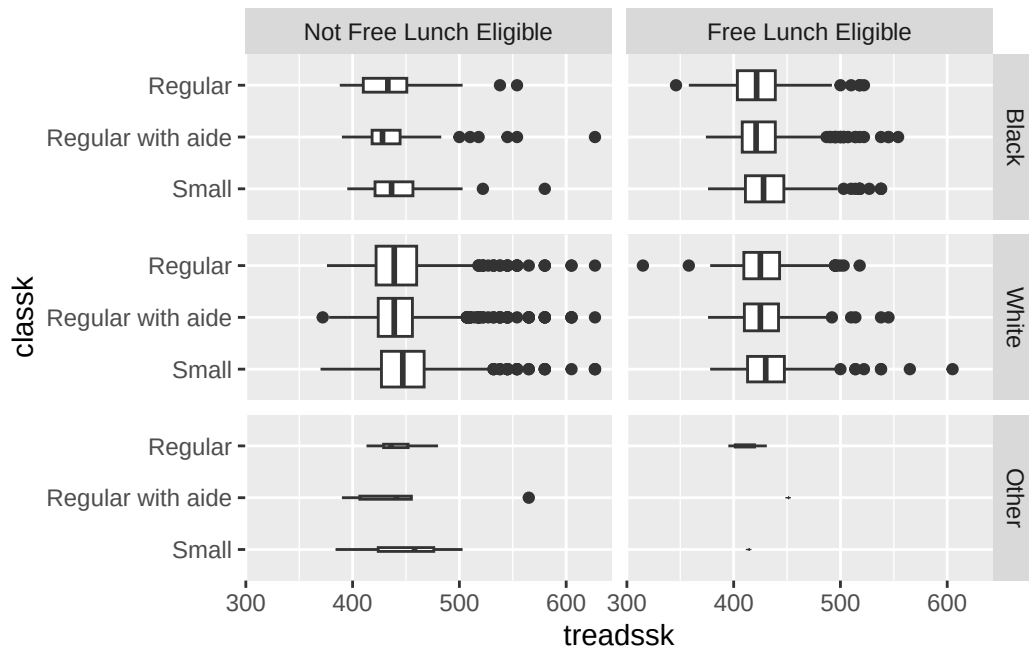




It looks almost there right now. But `facet_wrap` seems to create six completely separate panels. Let's try `facet_grid` instead.

*Side note:* if our goal is simply to create a “good” plot instead of replicating an existing one, is `facet_wrap` or `facet_grid` the better choice here? Why?.

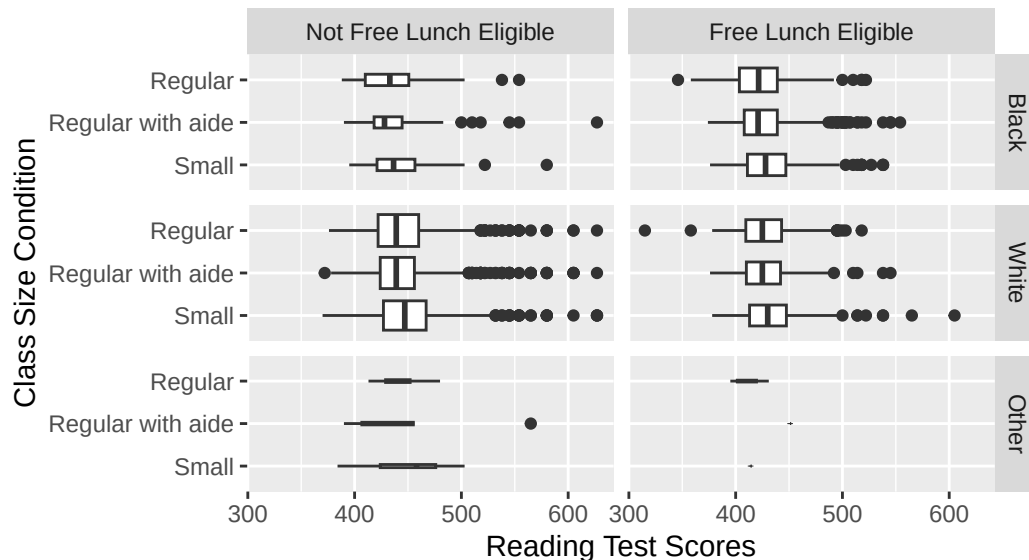
```
star %>%
 ggplot(aes(x = classk, y = treadssk)) +
 facet_grid(race~freelunk) +
 geom_boxplot(varwidth = TRUE) +
 coord_flip()
```



Last but not least, let's add in titles and axes:

```
star %>%
 ggplot(aes(x = classk, y = treadssk)) +
 facet_grid(race~freelunk) +
 geom_boxplot(varwidth = TRUE) +
 coord_flip() +
 labs(title = "Results of the Tennessee STAR Class Size Experiment",
 subtitle = "Viewed by subcategories of free lunch eligibility and race",
 y = "Reading Test Scores",
 x = "Class Size Condition")
```

## Results of the Tennessee STAR Class Size Experiment Viewed by subcategories of free lunch eligibility and race



In this iterative process, we managed to build a clean (i.e., not much chart junk) and accessible (e.g., relatively low cognitive load to process, friendly to people with color vision deficiencies) plot to represent the data. Viewers are able to see that (visually, as we haven't done any statistical tests yet) class size appears to impact reading test scores differently for students with different races and family incomes.

### 21.1.1 (Optional) Recreating the boxplot without manipulating the original dataset

We can label things on the fly rather than making the factors in our dataset different, like so:

```
Loading the Tennessee Star dataset in a new data frame
star_new <- Ecdat::Star

The ggplot code below shows us how we can modify the facet labels
without manipulating the dataset. We pass the new dataset star_new
to ggplot, and use the labeller function inside facet_grid to rename
facet labels. In the .cols argument, we specify the data label we want
to see in the plot, and set it equal to the value the label corresponds to.

ggplot(data=star_new, aes(x=treadssk, y=classk)) +
 geom_boxplot() +
 facet_grid(race~freelunk,
 # the facet columns correspond to the freelunk variable
```

```

here, we say where you see "yes", label it "Free lunch eligible"
Similarly, we label the remaining facet columns and rows
labeller = labeller(.cols = c("yes" = "Free lunch eligible",
 "no" = "Not Free Lunch Eligible"),
 .rows = c("white" = "White",
 "black" = "Black",
 "other" = "Other")) +

labs(x = "Reading Test Scores",
 y = "Class Size Condition",
 title = "Results of the Tennessee STAR Class Size Experiment",
 subtitle = "Viewed by subcategories of free lunch eligibility and race") + # we use t
to be tilted at a 45-degree angle for a cleaner look
theme(axis.text.y = element_text(angle = 45))

```



Which you do is a matter of coding taste, for the most part.

## 21.2 Bundling plots with grid.arrange()

If we want to place two completely separate plots side by side without faceting, we can use the `grid.arrange()` function. `grid.arrange()` allows you to arrange multiple ggplot2 plots in a grid layout. This is useful when you want to display multiple visualizations side by side in a single output.

## How is `grid.arrange()` different from faceting?

- `grid.arrange()` is used to arrange **completely separate ggplot objects** together in a grid
- Each plot can have **different aesthetics, geoms, themes, or datasets**
- Faceting (`facet_wrap()` or `facet_grid()`) is used **within a single ggplot object** to split data into multiple subplots based on a categorical variable
- Faceting is ideal when **all plots share the same x/y variables**, while `grid.arrange()` is better for combining **independent plots** that may have different structures

The code chunk below demonstrates use of this function.

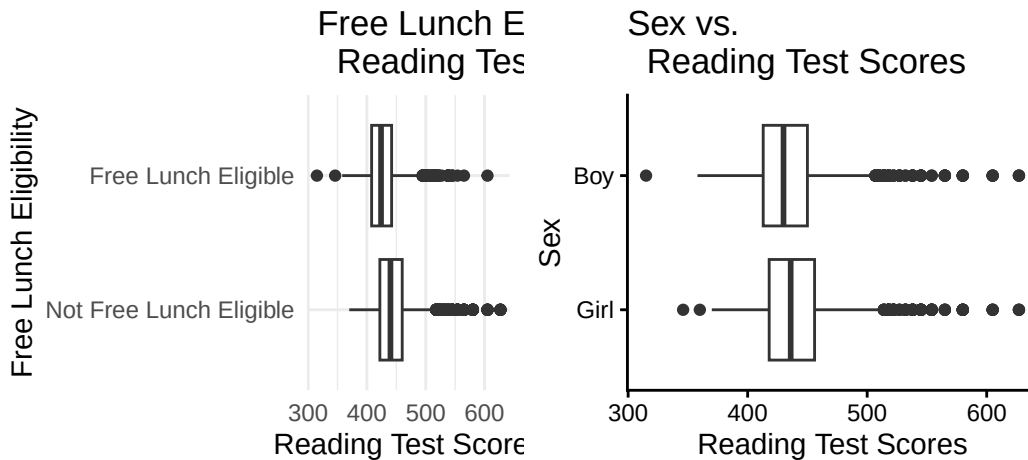
```
Let's go back to the Tennessee Star dataset,
and create two boxplots, one between reading test scores
and free lunch eligibility, and the second between
reading test scores and sex.

Assign the first plot to a variable called plot_1
plot_1 <- star %>%
 ggplot(aes(x=treadssk, y=freelunk)) +
 geom_boxplot() +
 theme_minimal() +
 labs(#Adding some labels here for x-axis, y-axis and the plot title
 title = " Free Lunch Eligibility vs.\n Reading Test Scores",
 x = "Reading Test Scores",
 y = "Free Lunch Eligibility"
)

Assign the first plot to a different variable called plot_2
plot_2 <- star %>%
 ggplot(aes(x=treadssk, y=sex)) +
 geom_boxplot() +
 theme_classic() +
 labs(#Adding some labels here for x-axis, y-axis and the plot title
 title = "Sex vs.\n Reading Test Scores",
 x = "Reading Test Scores",
 y = "Sex"
) + # Below, we use the scale_y_discrete function to rename labels on y-axis
 # to capitalize the first letter in the label
 scale_y_discrete(labels = c("boy" = "Boy", "girl" = "Girl"))

Using grid.arrange(), we arrange plot_1 and plot_2 in a single row
```

```
with two columns
grid.arrange(plot_1, plot_2, ncol = 2, nrow = 1)
```



## 21.3 Improvements on the Tennessee STAR Boxplot

### 21.3.1 What if we want to add a different color for the small class sizes to highlight the distributions in that category?

You can set colors quite easily. You may find the [documentation page](#) for `scale_manual` helpful. For a list of color names, this [link](#) will be handy. Here is a demonstration:

```
star %>%
 ggplot(aes(x = classk, y = treadssk, col=classk)) +
 facet_grid(race~freelunk) +
 geom_boxplot(varwidth = TRUE) +
 coord_flip() +
 labs(title = "Results of the Tennessee STAR Class Size Experiment",
 subtitle = "Viewed by subcategories of free lunch eligibility and race",
 y = "Reading Test Scores",
 x = "Class Size Condition") +
 # The scale_color_manual function below manually sets colors for the classk variable.
 # The first category (small class size) is highlighted in orange,
 # while the other class size categories are set to gray.
 # This helps emphasize the small class size condition in the visualization.
 scale_color_manual(values=c("orange", "gray", "gray"))
```

## Results of the Tennessee STAR Class Size Experiment Viewed by subcategories of free lunch eligibility and race

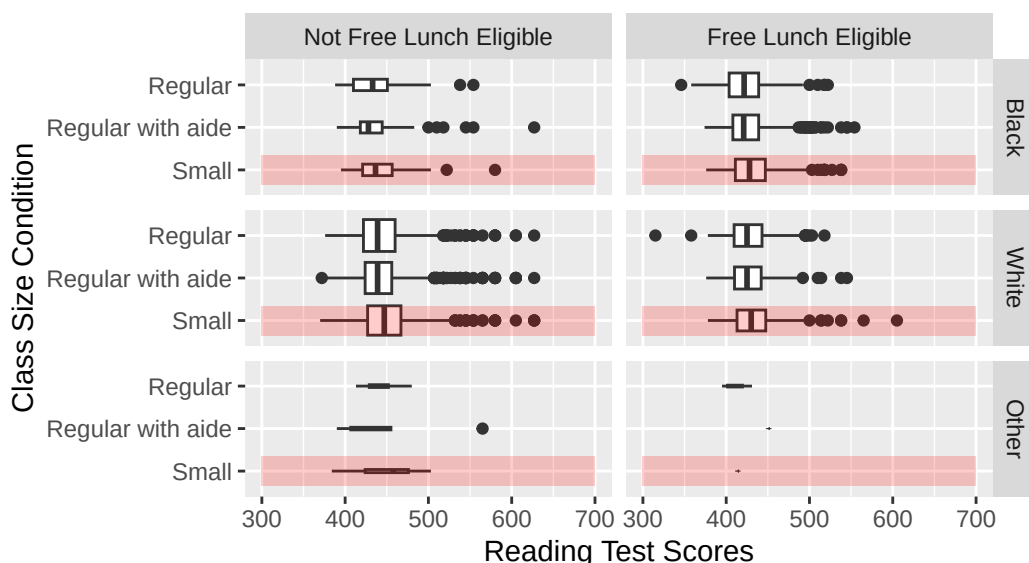


### 21.3.2 What if we want to annotate all the boxplots corresponding to the the small class size category?

You can also annotate your plot. You may find the [documentation page](#) for `annotate` helpful.

```
star %>%
 ggplot(aes(x = classk, y = treadssk)) +
 facet_grid(race~freelunk) +
 geom_boxplot(varwidth = TRUE) +
 coord_flip() +
 labs(title = "Results of the Tennessee STAR Class Size Experiment",
 subtitle = "Viewed by subcategories of free lunch eligibility and race",
 y = "Reading Test Scores",
 x = "Class Size Condition") +
 annotate("rect", xmin = 0.65, xmax = 1.35, ymin = 300, ymax = 700,
 alpha = .2, fill="red")
```

## Results of the Tennessee STAR Class Size Experiment Viewed by subcategories of free lunch eligibility and race



### Explanation of the `annotate()` function

The `annotate()` function is used to **add custom shapes or text** to a ggplot. Here, “rect” (rectangle) is used to **highlight** a specific region in the plot.

- **`xmin = 0.65`, `xmax = 1.35`**: `classk` is a **factor**, so ggplot assigns it numeric positions (e.g., 1 = small class, 2 = regular with aide, 3 = regular). The values 0.65 and 1.35 **define the “height” of the rectangle**, but because of `coord_flip()`, this actually affects the **y-axis** instead of the x-axis.
- **`ymin = 300`, `ymax = 700`**: Defines the **horizontal** range of the rectangle (test scores from 300 to 700), since `coord_flip()` swaps the x- and y-axes.
- **If `coord_flip()` was not used** and we instead used `x = treadssk`, `y = classk` in the ggplot aesthetics, we would need to swap `xmin` with `ymin` and `xmax` with `ymax`: `xmin = 300`, `xmax = 700` (test scores would be on the x-axis) and `ymin = 0.65`, `ymax = 1.35` (class size categories would be on the y-axis). This is because, without `coord_flip()`, the **x-axis stays horizontal and the y-axis stays vertical**.
- **`alpha = 0.2`**: This controls transparency (0 = fully transparent, 1 = fully opaque). A value of 0.2 makes the rectangle **semi-transparent**, so the data underneath is still visible.
- **`fill = “red”`**: The rectangle is filled with red, drawing attention to the selected region.



## 21.4 What about adding text to the annotated Tennessee Star Boxplot?

We next demonstrate adding some text to the annotated plot above. In particular we add the useful eye catching “Here!” in red.

```
p <- star %>%
 ggplot(aes(x = classsk, y = treadssk)) +
 facet_grid(race~freelunk) +
 geom_boxplot(varwidth = TRUE) +
 coord_flip() +
 labs(title = "Results of the Tennessee STAR Class Size Experiment",
 subtitle = "Viewed by subcategories of free lunch eligibility and race",
 y = "Reading Test Scores",
 x = "Class Size Condition") +
 annotate("rect", xmin = 0.65, xmax = 1.35, ymin = 300, ymax = 700,
 alpha = .2, fill="red")

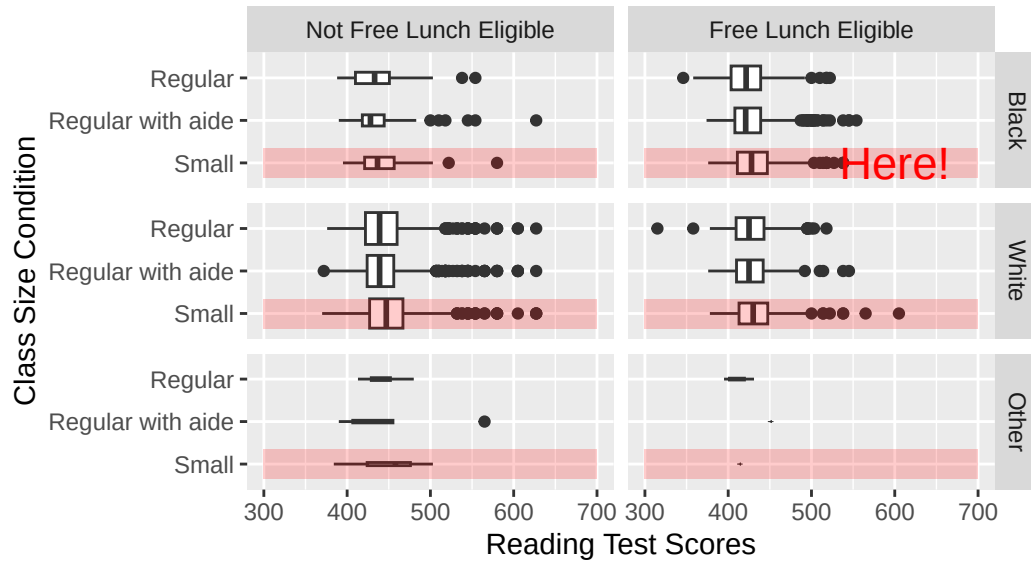
----- Creating a Data Frame for Text Annotation -----

text <- data.frame(
 classsk = 1, # To position the text at class size condition = 1 (small class)
 treadssk = 600, # To position the text at treadssk = 600 (as seen in the figure)
 # Below, we specify the free lunch eligibility category wherein to place the text
 freelunk = factor("Free Lunch Eligible",
 levels = c("Not Free Lunch Eligible", "Free Lunch Eligible")),
 # Below, we specify the race category wherein to place the text
 race = factor("Black", levels=c("Black", "White", "Other")))

----- Adding Text Annotation to the Plot -----
- `geom_text()` adds a text label to the plot using data from the `text` data frame.
- `data = text` tells ggplot where to position the text.
- `label = "Here!"` is the actual text that will appear on the plot.
- `col = "red"` sets the text color to red.
- `size = 6` controls the font size.
- Since `coord_flip()` is used, `classsk = 1` (normally an x value) now determines
the vertical position, and `treadssk = 600` (normally a y value) determines
the horizontal position.

p + geom_text(data = text, label = "Here!", col="red", size=6)
```

## Results of the Tennessee STAR Class Size Experiment Viewed by subcategories of free lunch eligibility and race



Voila!

## 22 Plotting Two Datasets at Once

It's easy (though not always advisable) to plot two data sets at once with `ggplot`. First, we load `tidyverse` and our HSB data. We then create a school-level aggregate data set of just the mean SES values.

```
library(tidyverse)
library(haven)

clear memory
rm(list = ls())

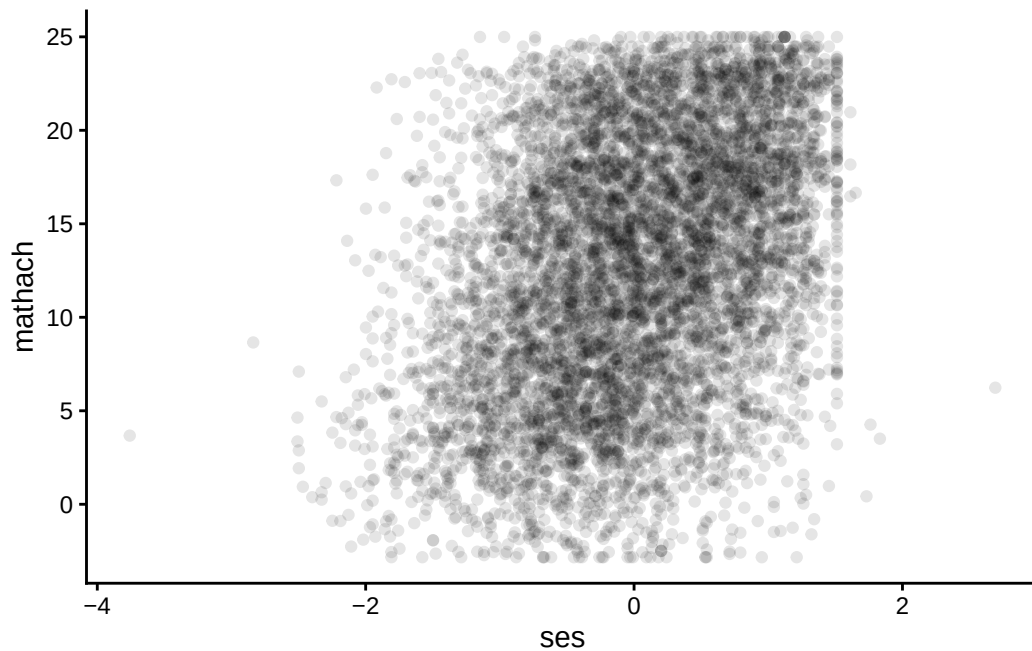
theme_set(theme_classic())

load HSB data
hsb <- read_dta("data/hsb.dta") |>
 select(mathach, ses, schoolid)

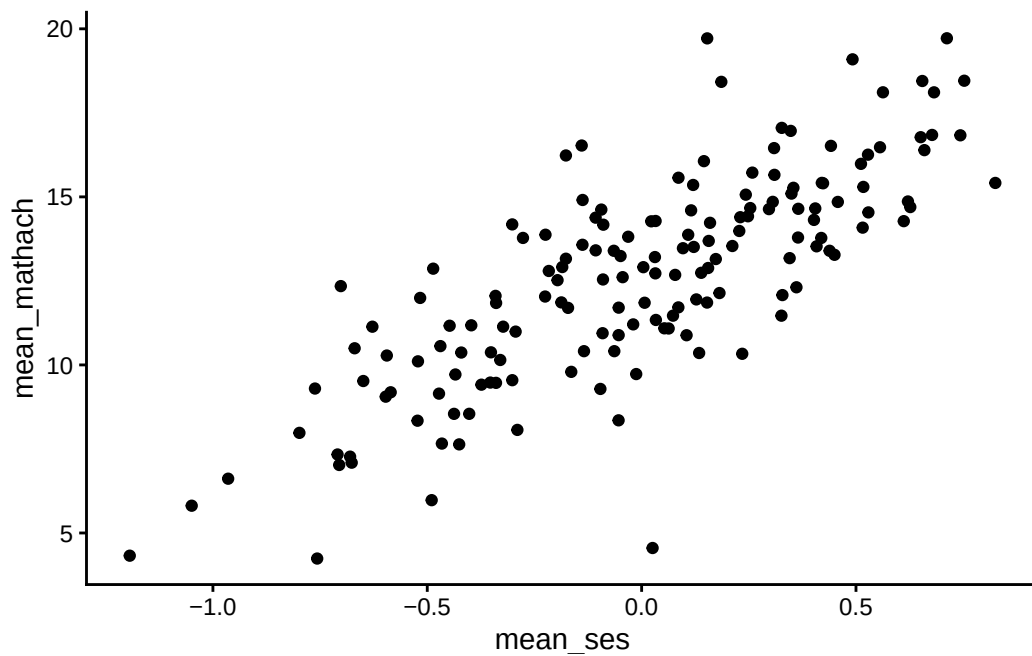
sch <- hsb |>
 group_by(schoolid) |>
 summarise(mean_ses = mean(ses),
 mean_mathach = mean(mathach))
```

Let's say we wanted to plot *both* the individual students *and* the school means. This is easy enough to do separately:

```
ggplot(hsb, aes(x = ses, y = mathach)) +
 geom_point(alpha = 0.1)
```



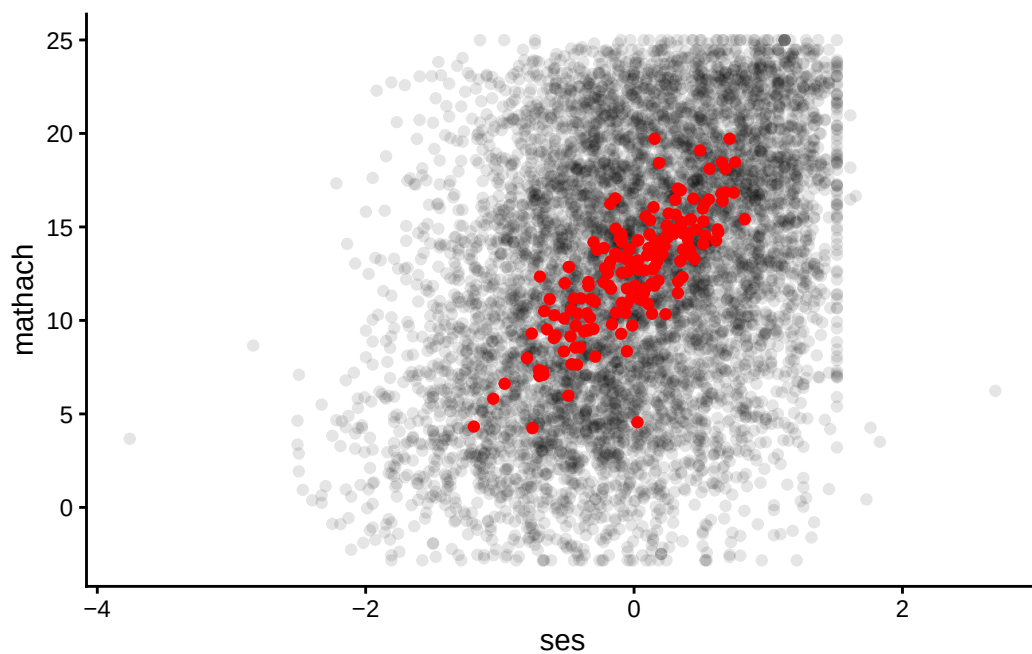
```
ggplot(sch, aes(x = mean_ses, y = mean_mathach)) +
 geom_point()
```



We can superimpose both plots as follows. Essentially, the first argument in `ggplot` provides the data, and by default, this is passed to all subsequent layers of the plot. We can override

this behavior by specifying a different data set (and aesthetic mappings, if desired) *within an individual layer* of `ggplot`, such as `geom_point`.

```
ggplot(hsb, aes(x = ses, y = mathach)) +
 geom_point(alpha = 0.1) +
 geom_point(data = sch, aes(x = mean_ses, y = mean_mathach), color = "red")
```



## 23 Plotting aggregate data with ggPlot

This handout covers how to plot univariate distributions of data (i.e. histograms and bar charts) if your data have already been aggregated (e.g. you have the counts for each value of a variable but not the individual observations). In particular, this handout demonstrates how to create histograms with ggplot when the data have already been aggregated or tallied.

### 23.0.1 Preface

Before we get started, let's load the packages we'll need.

```
library(tidyverse)
library(ggplot2)
```

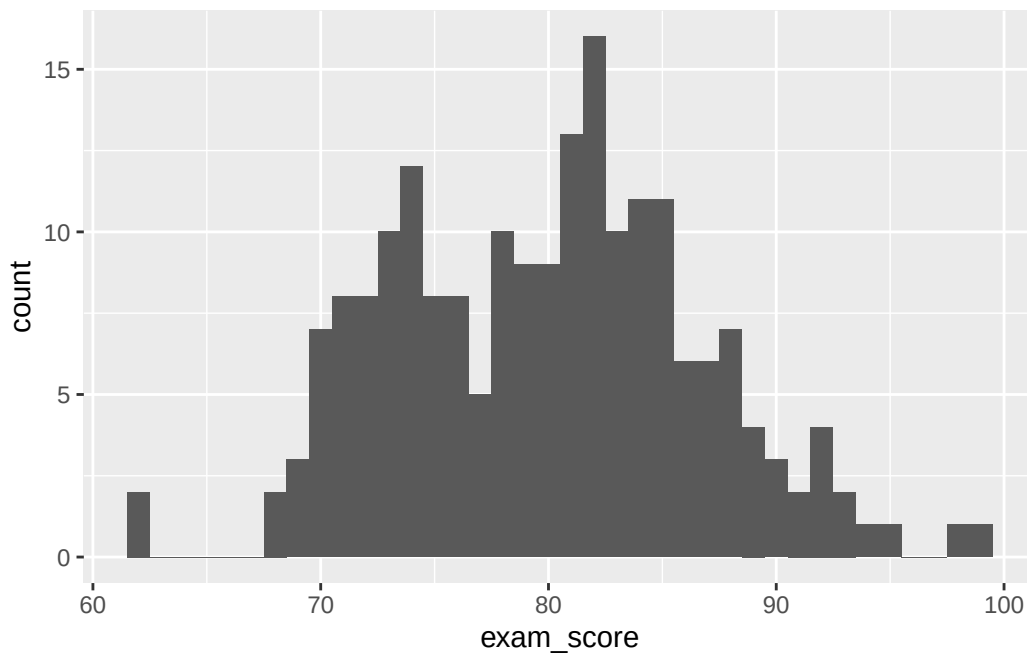
## 24 The Situation

Typically, we create histograms from individual data points. Suppose we have data on students. The data include a student ID number, a course identifier, the grade the student got in the course, and their final exam score (out of 100 points). Here's some example data.

```
num_students <- 200
df <- data.frame(sid=rep(1:num_students, 2),
 course=c(rep('math', num_students), rep('ela', num_students)),
 grade=sample(c('A', 'B', 'C', 'D', 'F'),
 2*num_students,
 replace=TRUE),
 exam_score=sample(0:100, 2*num_students, replace=TRUE,
 prob=dnorm(0:100, mean=80, sd=7)))
```

Let's make a histogram of the exam score data for math. This should be familiar.

```
ggplot(df %>% filter(course=='math'), aes(x=exam_score)) +
 geom_histogram(binwidth=1)
```



There's a lot we could do to make this a publication-ready graphic - label the axes, add a title, etc. - but our focus here is on the core plotting, so we'll skip it.

But what if the data were already aggregated by exam score? Let me show you what I mean in code.

```
exam_agg <- df %>%
 filter(course=='math') %>%
 group_by(exam_score) %>%
 summarize(n=n())
exam_agg
```

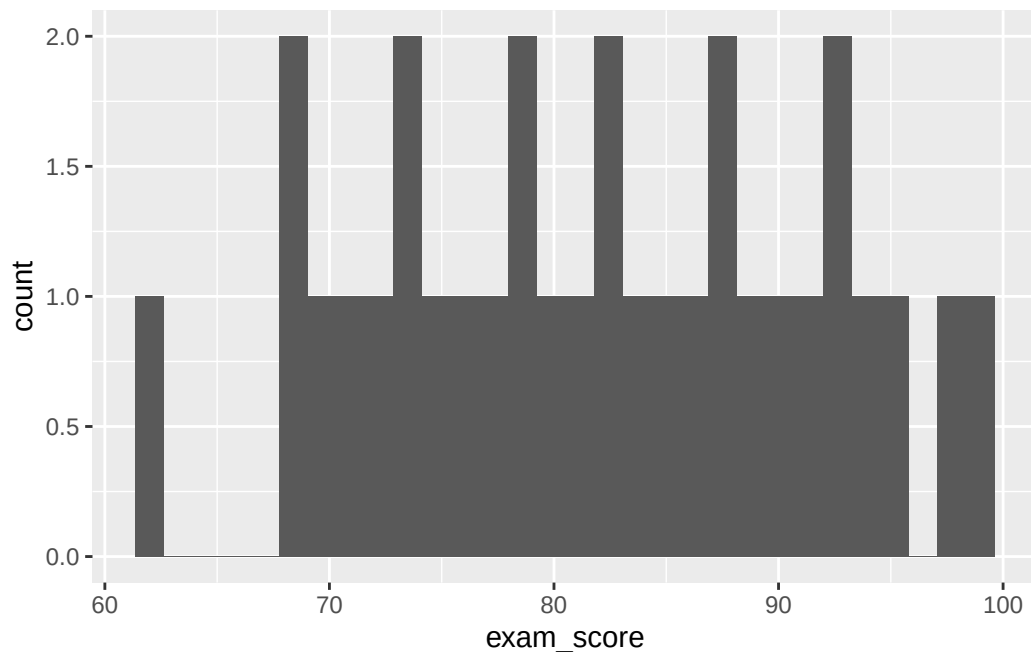
```
A tibble: 31 x 2
 exam_score n
 <int> <int>
1 62 2
2 68 2
3 69 3
4 70 7
5 71 8
6 72 8
7 73 10
8 74 12
9 75 8
10 76 8
i 21 more rows
```

The dataset `exam_agg` contains only two variables: the exam score and the number of students who received that score. Sometimes, this is the way we receive data. How can we make a histogram out of this? Let's try it the old way:

```
ggplot(exam_agg, aes(x=exam_score)) +
 geom_histogram()
```

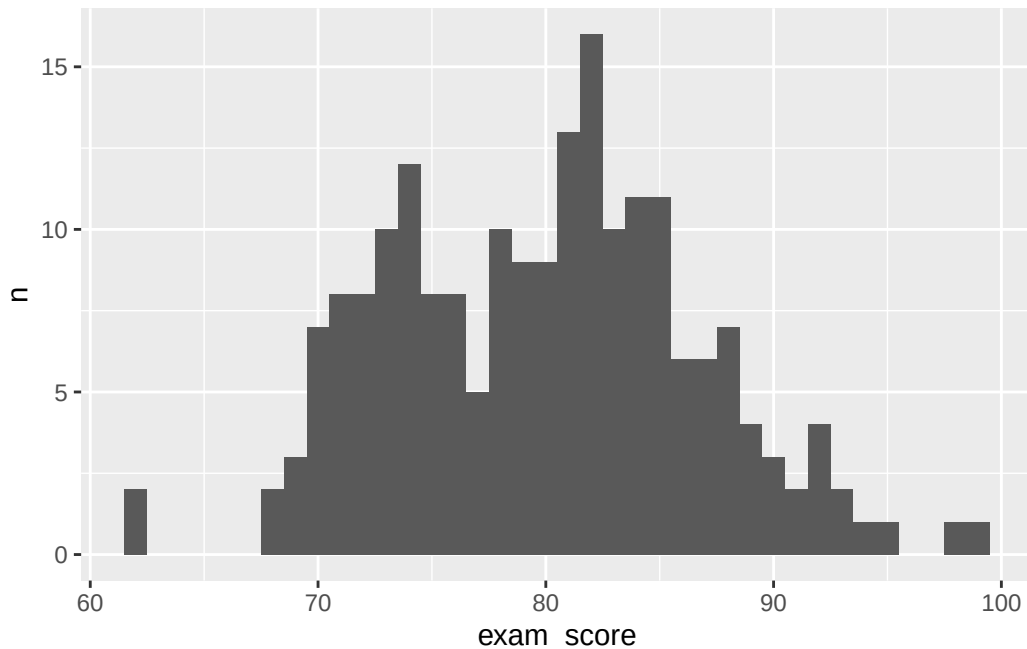
``stat_bin()` using `bins = 30`. Pick better value `binwidth`.`





Whoa! That doesn't look right. The problem is that `geom_histogram()` counts how many times each score appears in the data, but after we aggregated, each score only appears once. We need to tell R that the counts are stored in the `n` column of the dataset. Here's how we do it:

```
ggplot(exam_agg, aes(x=exam_score, y=n)) +
 geom_bar(stat='identity', width=1)
```

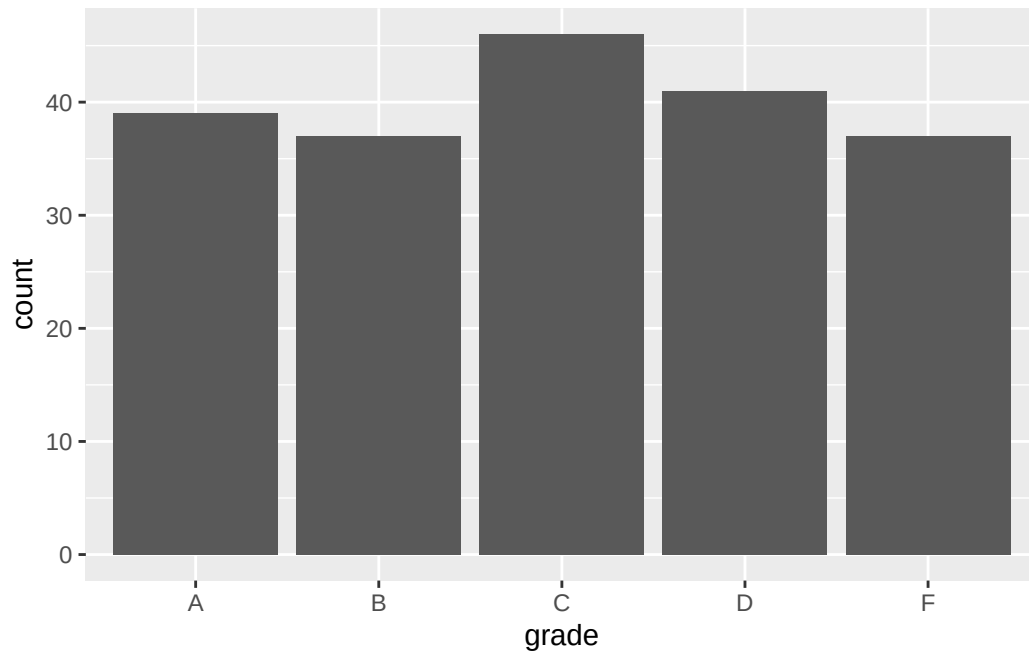


We've switched from `geom_histogram()` to `geom_bar()`. This allows us to specify a separate variable for the y-axis. Inside the `geom_bar()` call, we need to specify that `stat='identity'`. This tells ggplot that it should just use the raw value of the y-variable we gave it. If we don't make this explicit, R doesn't know what to do with the second variable.

## 24.1 Categorical data and bar charts

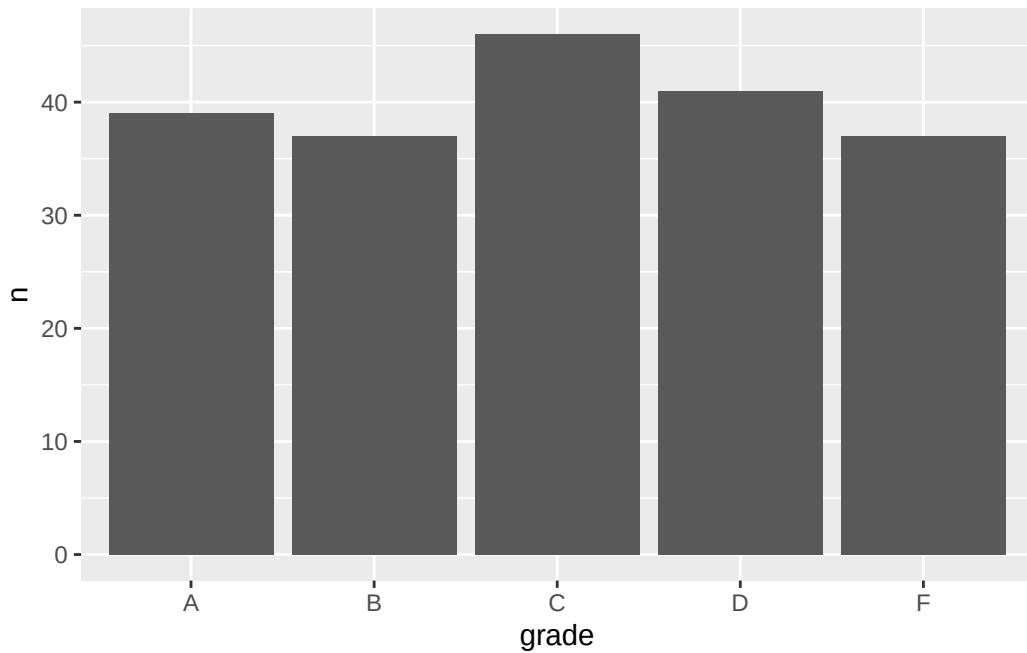
We can do the same thing with a categorical variable, course grades. First we'll plot using the raw data, and then we'll switch to the aggregated version. For the aggregated case, we'll again rely on the `stat` argument of the `geom_bar()` function.

```
Plot from raw data
For a categorical variable, we use geom_bar
df %>% filter(course=='ela') %>%
 ggplot(aes(x=grade)) +
 geom_bar()
```



```
Now let's aggregate the data
grade_agg <- df %>% filter(course=='ela') %>%
 group_by(grade) %>%
 summarize(n=n())

Now reproduce the bar plot from the aggregated data
ggplot(grade_agg, aes(x=grade, y=n)) + geom_bar(stat='identity')
```



Looks exactly the same! (Except for the y-axis label.)

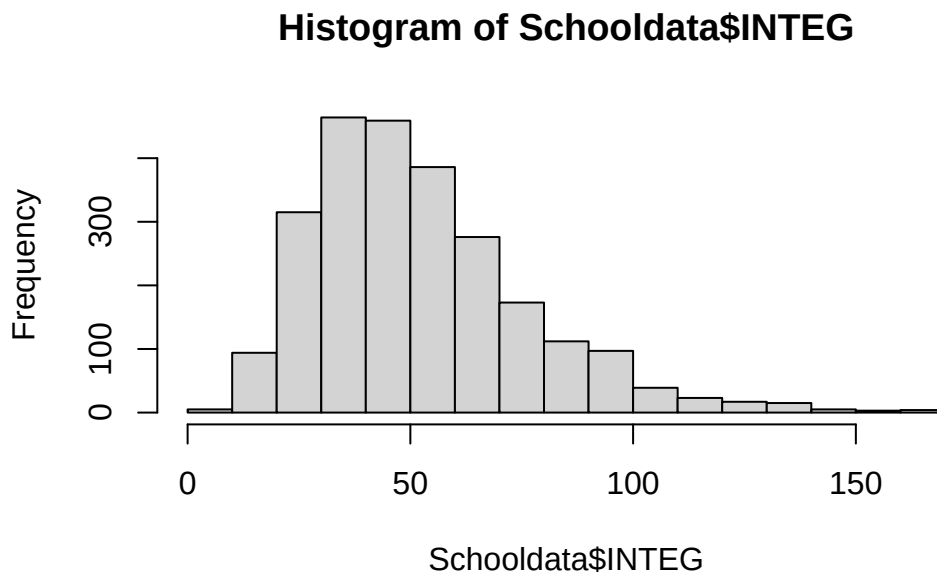
## 24.2 A Closing Thought

Sometimes the data we get have already been aggregated. In this case, we might still want to visualize distributions of the variables that have been grouped. The above guide introduces the `stat` argument of `geom_bar()` as a way to get this done.

## 25 Jitter and heatmap examples

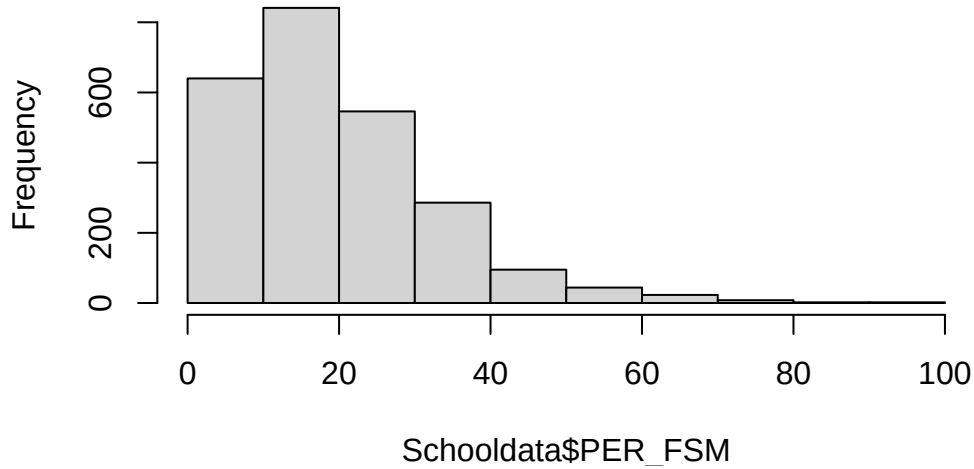
## 26 Jitter examples

```
Histogram of ouput variable (INTEG)
hist(Schooldata$INTEG)
```

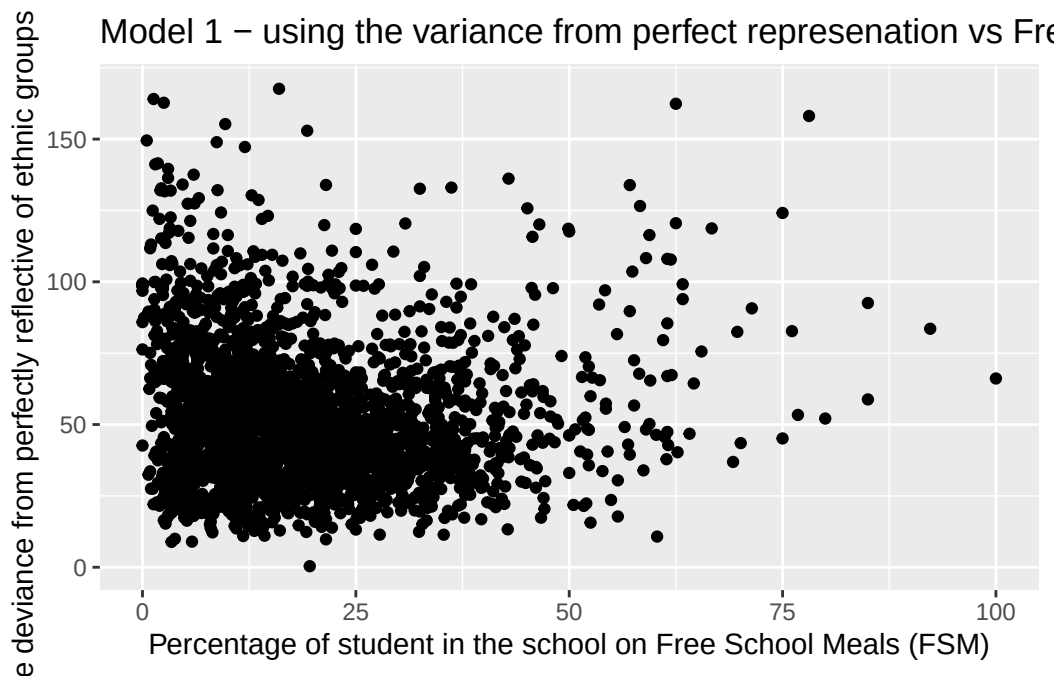


```
Histogram of predictor variable (PER_FSM)
hist(Schooldata$PER_FSM)
```

### Histogram of Schooldata\$PER\_FSM



```
Scatterplot of output variable (INTEG) on predictor (FSM)
ggplot(Schooldata, aes(x = PER_FSM, y = INTEG)) +
 geom_point() +
 labs(title= "Model 1 - using the variance from perfect representation vs Free School Meals",
 x = "Percentage of student in the school on Free School Meals (FSM)",
 y = "Percentage deviance from perfectly reflective of ethnic groups in local area")
```



```
fit1 <- lm(INTEG ~ PER_FSM, data=Schooldata)
summary(fit1)
```

Call:

```
lm(formula = INTEG ~ PER_FSM, data = Schooldata)
```

Residuals:

Min	1Q	Median	3Q	Max
-51.901	-17.485	-3.918	12.352	115.029

Coefficients:

	Estimate	Std. Error	t value	Pr(> t )
(Intercept)	54.10637	0.86768	62.358	<2e-16 ***
PER_FSM	-0.09395	0.03680	-2.553	0.0107 *

---

Signif. codes: 0 '\*\*\*' 0.001 '\*\*' 0.01 '\*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 24.4 on 2485 degrees of freedom

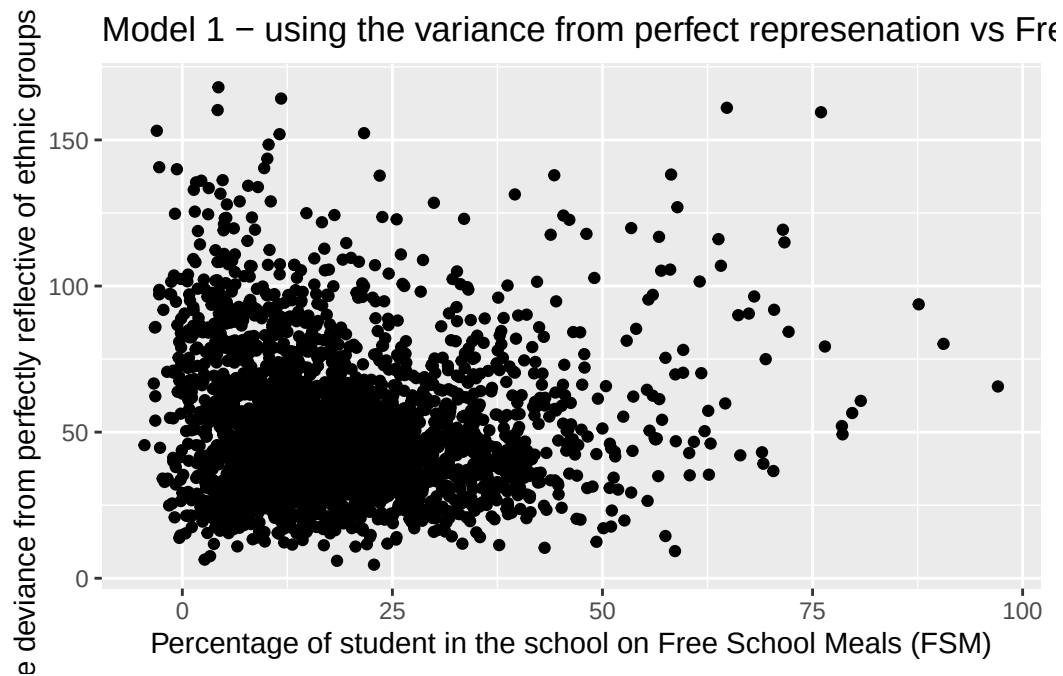
Multiple R-squared: 0.002616, Adjusted R-squared: 0.002215

F-statistic: 6.518 on 1 and 2485 DF, p-value: 0.01074

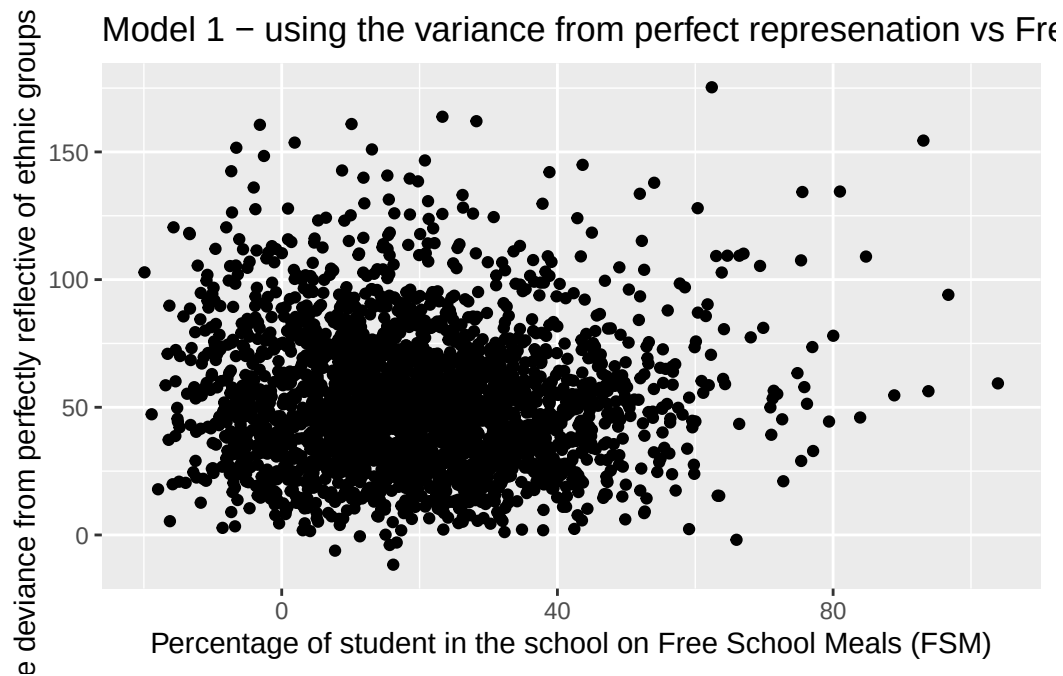
We can try jittering more (jittering here is not a good choice, by the way). The width and height control how much jitter to do both left-right and up-down.

```
ggplot(Schooldata, aes(x = PER_FSM, y = INTEG)) +
 geom_jitter(width = 5, height=5) +
 labs(title= "Model 1 - using the variance from perfect represenation vs Free School Meals"
 x = "Percentage of student in the school on Free School Meals (FSM)",
 y = "Percentage deviance from perfectly reflective of ethnic groups in local area")
```

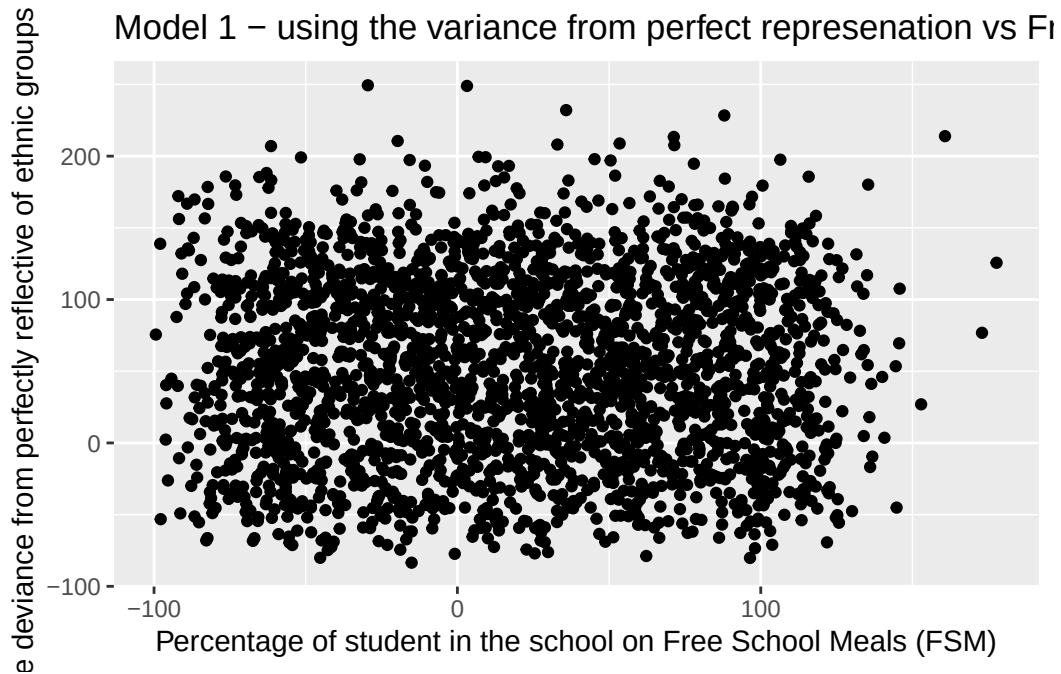




```
ggplot(Schooldata, aes(x = PER_FSM, y = INTEG)) +
 geom_jitter(width = 20, height=20) +
 labs(title= "Model 1 - using the variance from perfect representation vs Free School Meals"
 x = "Percentage of student in the school on Free School Meals (FSM)",
 y = "Percentage deviance from perfectly reflective of ethnic groups in local area")
```



```
ggplot(Schooldata, aes(x = PER_FSM, y = INTEG)) +
 geom_jitter(width = 100, height=100) +
 labs(title= "Model 1 - using the variance from perfect representation vs Free School Meals"
 x = "Percentage of student in the school on Free School Meals (FSM)",
 y = "Percentage deviance from perfectly reflective of ethnic groups in local area")
```



You can also generate jittered data with `jitter()`:

```
a = 1:10
```

```
a
```

```
[1] 1 2 3 4 5 6 7 8 9 10
```

```
jitter(a)
```

```
[1] 0.9856538 2.0160063 3.1348756 3.9974562 5.0137759 5.8531621
```

```
[7] 7.0520063 8.0084738 9.0013169 10.1339664
```

```
jitter(a, factor = 0.5)
```

```
[1] 0.9653208 2.0163983 3.0623294 4.0469996 4.9308730 6.0717723
```

```
[7] 7.0247136 8.0687477 8.9886224 10.0651493
```

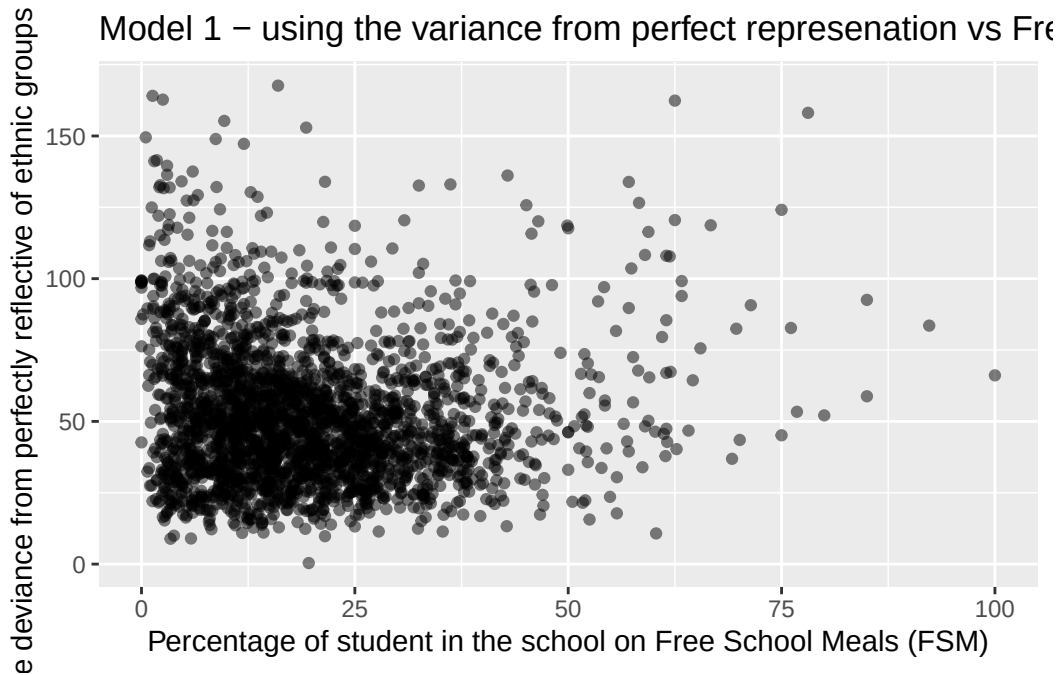
```
jitter(a, factor = 0.0005)
```

```
[1] 0.9999243 1.9999117 2.9999316 3.9999349 5.0000481 6.0000563
```

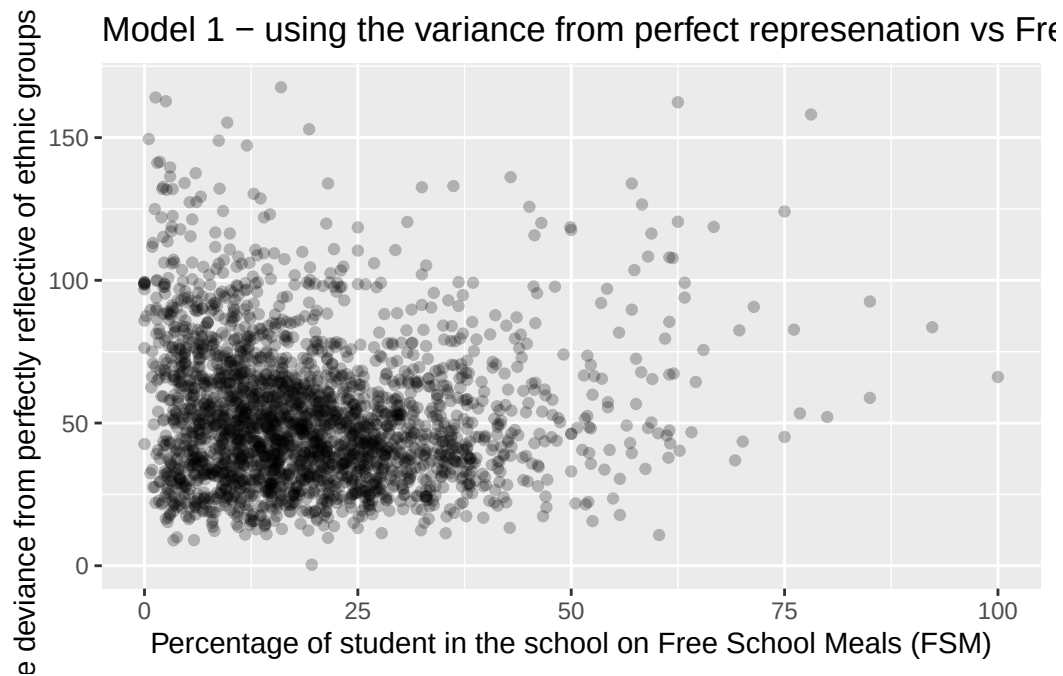
```
[7] 6.9999858 8.0000367 8.9999318 10.0000363
```

```
Try alpha
```

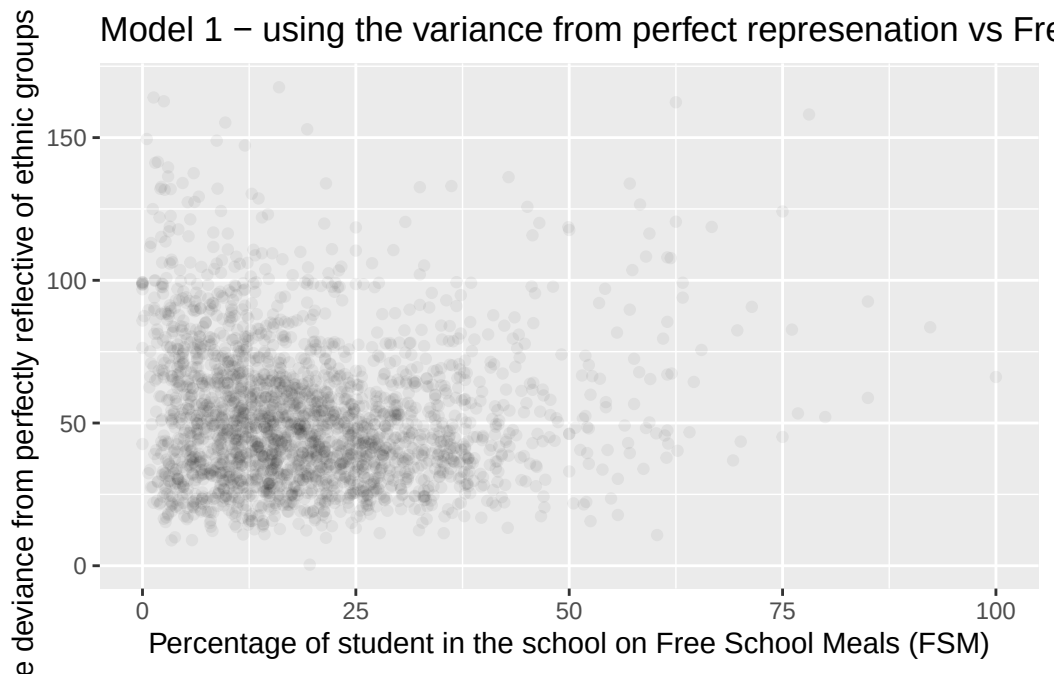
```
ggplot(Schooldata, aes(x = PER_FSM, y = INTEG)) +
 geom_point(alpha = 0.5) +
 labs(title= "Model 1 - using the variance from perfect representation vs Free School Meals",
 x = "Percentage of student in the school on Free School Meals (FSM)",
 y = "Percentage deviance from perfectly reflective of ethnic groups in local area")
```



```
ggplot(Schooldata, aes(x = PER_FSM, y = INTEG)) +
 geom_point(alpha = 0.25) +
 labs(title= "Model 1 - using the variance from perfect representation vs Free School Meals",
 x = "Percentage of student in the school on Free School Meals (FSM)",
 y = "Percentage deviance from perfectly reflective of ethnic groups in local area")
```



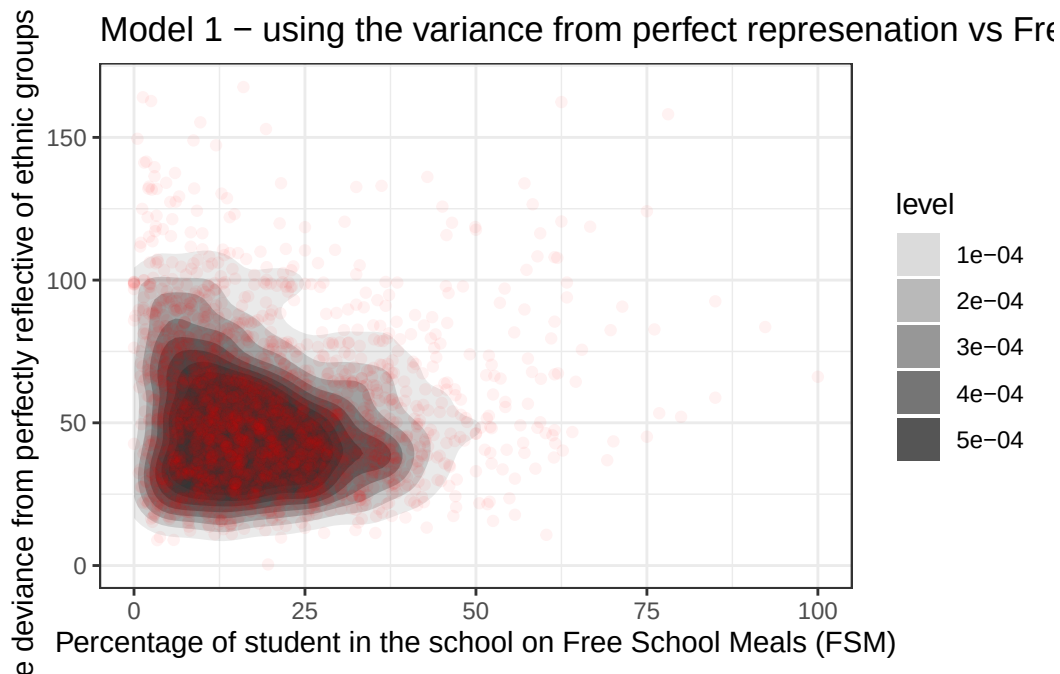
```
ggplot(Schooldata, aes(x = PER_FSM, y = INTEG)) +
 geom_point(alpha = 0.05) +
 labs(title= "Model 1 - using the variance from perfect representation vs Free School Meals"
 x = "Percentage of student in the school on Free School Meals (FSM)",
 y = "Percentage deviance from perfectly reflective of ethnic groups in local area")
```



## 27 Making a heat map

```
ggplot(Schooldata, aes(x = PER_FSM, y = INTEG)) +
 stat_density2d(aes(alpha=..level..), geom="polygon") +
 labs(title= "Model 1 - using the variance from perfect representation vs Free School Meals",
 x = "Percentage of student in the school on Free School Meals (FSM)",
 y = "Percentage deviance from perfectly reflective of ethnic groups in local area") +
 geom_point(colour="red", alpha=0.05) +
 theme_bw()
```

Warning: The dot-dot notation (`..level..`) was deprecated in ggplot2 3.4.0.  
i Please use `after\_stat(level)` instead.



## 28 Using pivot methods to make line graphs

### 28.1 Overview

In this short example I demonstrate how to aggregate data and make a line graph of the results. In particular, the setting is we have a bunch of data on church donations, along with the age of the donor.

We are going to look at what proportion of the donations are above a specific size as a function of age. We are also going to look at whether the distribution of donations from those belonging to a strict evangelical doctrine are different.

The key idea is if we have multiple columns, and we want a line for each column, we need to pivot our data long, so we can make each column a distinct group, and plot all the lines using a single `geom_line()` command.

### 28.2 Our data

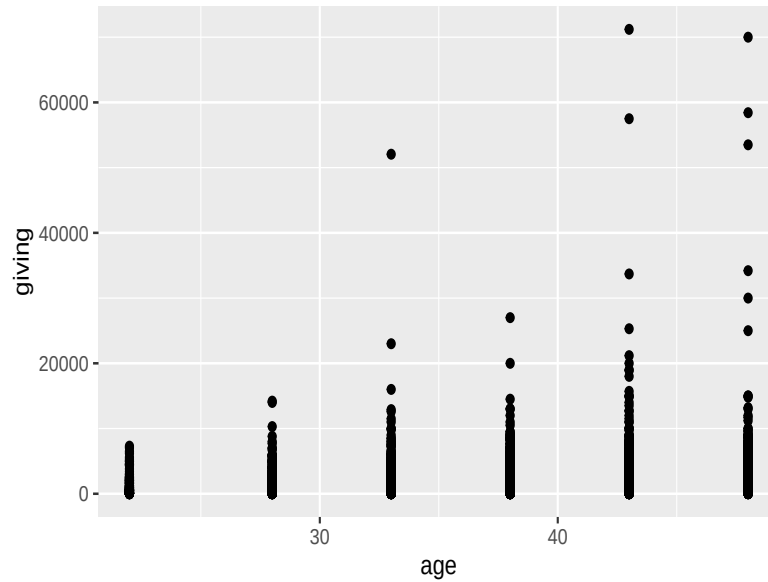
We load the data and look at the donations. It is hard to see trends given the heavy skew of donation size.

```
library(tidyverse)

church = read.table("data/CHURCH.txt")
names(church) = c("church", "memberID", "giving", "age", "strict")

ggplot(church, aes(age, giving)) +
 geom_point()
```





## 28.3 Aggregating

We group by age category and church type, and calculate the proportion of donations above \$5000 and above \$1000.

```
schurch = church %>% group_by(age, strict) %>%
 summarise(prop_large = mean(giving > 5000),
 prop_medium = mean(giving > 1000))
```

`summarise()` has regrouped the output.

i Summaries were computed grouped by age and strict.

i Output is grouped by age.

i Use `summarise(.groups = "drop\_last")` to silence this message.

i Use `summarise(.by = c(age, strict))` for per-operation grouping  
(`?dplyr::dplyr\_by`) instead.

schurch

# A tibble: 12 x 4

# Groups: age [6]

	age	strict	prop_large	prop_medium
	<int>	<int>	<dbl>	<dbl>
1	22	0	0.0141	0.141
2	22	1	0.0526	0.4

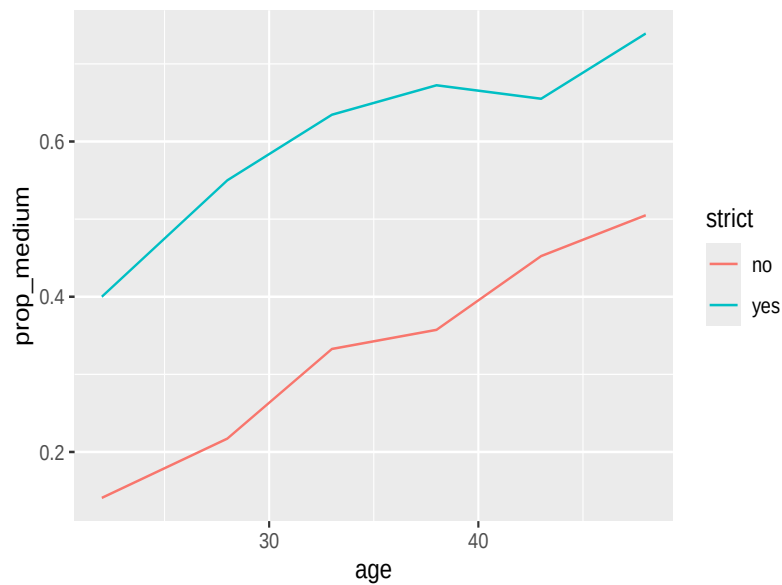
3	28	0	0.0287	0.217
4	28	1	0.0667	0.55
5	33	0	0.0253	0.333
6	33	1	0.0848	0.635
7	38	0	0.0353	0.357
8	38	1	0.161	0.672
9	43	0	0.0812	0.452
10	43	1	0.131	0.655
11	48	0	0.0532	0.505
12	48	1	0.186	0.739

## 28.4 Graphing (take 1)

We can make a graph of the proportion of medium donations vs age relatively easily:

```
schurch$strict = factor(schurch$strict, levels = c(0,1),
 labels=c("no","yes"))

ggplot(schurch, aes(age, prop_medium, group=strict, col=strict)) +
 geom_line()
```



## 28.5 Graphing (take 2)

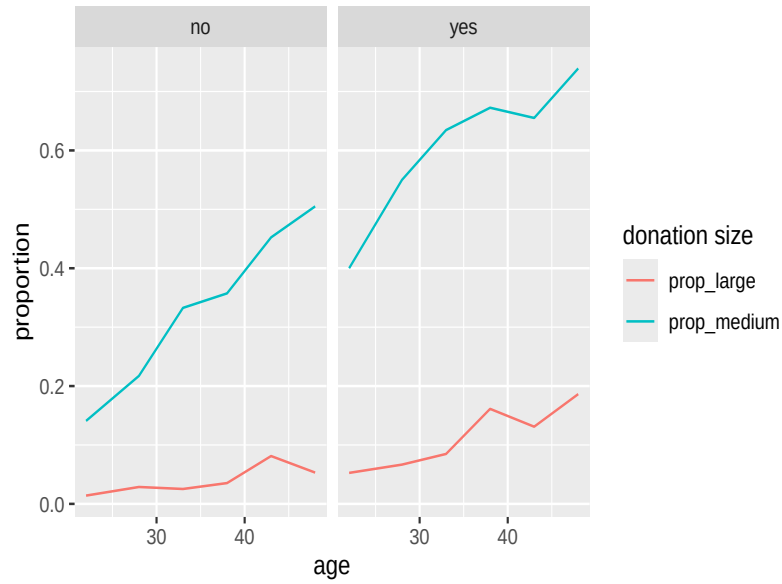
To get a line for both medium and large donations, we need to reshape our data. In particular, we want to have two groups of observations, one for each type:

```
schurchL <- schurch %>% pivot_longer(cols=c(prop_large, prop_medium),
 names_to = "size", values_to="proportion")
schurchL
```

```
A tibble: 24 x 4
Groups: age [6]
 age strict size proportion
 <int> <fct> <chr> <dbl>
1 22 no prop_large 0.0141
2 22 no prop_medium 0.141
3 22 yes prop_large 0.0526
4 22 yes prop_medium 0.4
5 28 no prop_large 0.0287
6 28 no prop_medium 0.217
7 28 yes prop_large 0.0667
8 28 yes prop_medium 0.55
9 33 no prop_large 0.0253
10 33 no prop_medium 0.333
i 14 more rows
```

See how now each row is a specific age, donation size and proportion. This sets us up nicely for ggplot since each variable for plotting is ready to go:

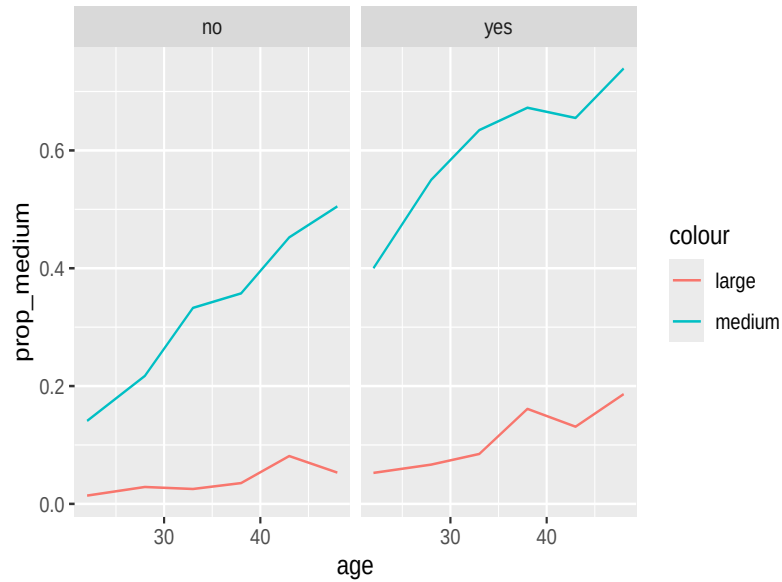
```
ggplot(schurchL, aes(age, proportion, col=size)) +
 facet_wrap(~ strict) +
 geom_line() +
 labs(col="donation size")
```



## 28.6 Graphing (another road)

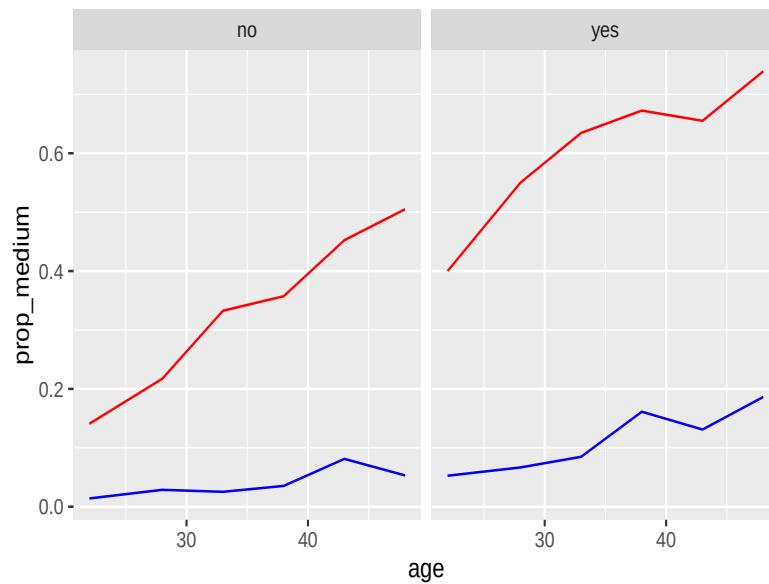
We can also plot individual lines without doing a `pivot_longer` by specifying different outcomes in the `ggplot` command:

```
ggplot(schurch, aes(age)) +
 facet_wrap(~strict) +
 geom_line(aes(y = prop_medium, col="medium")) +
 geom_line(aes(y = prop_large, col="large"))
```



Notice we are putting color to specific values inside each `geom_line`: this tells ggplot to make a legend of those different values. If we put these *outside* the `aes()` we can specify the color but we will not get an automatic legend:

```
ggplot(schurch, aes(age)) +
 facet_wrap(~strict) +
 geom_line(aes(y = prop_medium), col="red") +
 geom_line(aes(y = prop_large), col="blue")
```



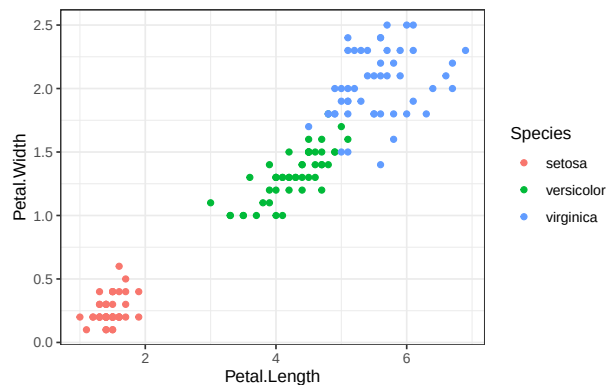
## 29 Prediction and Visualization with Models

To make nice plots of the predictions from models that have complex curves and relationships, the easiest way is to use the model to predict values for different covariate combinations, store all of this in a dataframe, and then use `ggplot` to plot your results using `geom_line` which will give you nice smoothed curves through the points. This document highlights some useful methods for doing this. We need the `tidyverse` and the `modelr` package

Prediction, and making plots from predictions, are foundational skills for data analysis, especially as models become more complex and interactions or non-linear terms are introduced.

We will use the `iris` dataset, as it has some strong relationships that make the ideas behind these tools more salient. Here is the raw data:

```
ggplot(iris, aes(x=Petal.Length, y=Petal.Width, col=Species)) +
 geom_point()
```



To visualize we go through a series of steps:

- 1) Fit a model to describe the relationship between variables
- 2) Generate a grid of values to predict over
- 3) Add predictions to the grid
- 4) Make the visualization, plotting our predictions vs. our grid values.

Related reading: R for Data Science (1st Edition), Section 23.3 (*Sections 23.1 and 23.2 are optional but helpful*)

## 29.1 Fit a model

We first fit a a model to describe how petal length can be predicted by petal width and species. Our model allows the length-width relationship to be different for the different species.

```
M1 <- lm(Petal.Length ~ Petal.Width * Species, data=iris)
summary(M1)
```

Call:

```
lm(formula = Petal.Length ~ Petal.Width * Species, data = iris)
```

Residuals:

Min	1Q	Median	3Q	Max
-0.84099	-0.19343	-0.03686	0.16314	1.17065

Coefficients:

	Estimate	Std. Error	t value	Pr(> t )
(Intercept)	1.3276	0.1309	10.139	< 2e-16 ***
Petal.Width	0.5465	0.4900	1.115	0.2666
Speciesversicolor	0.4537	0.3737	1.214	0.2267
Speciesvirginica	2.9131	0.4060	7.175	3.53e-11 ***
Petal.Width:Speciesversicolor	1.3228	0.5552	2.382	0.0185 *
Petal.Width:Speciesvirginica	0.1008	0.5248	0.192	0.8480

---

Signif. codes: 0 '\*\*\*' 0.001 '\*\*' 0.01 '\*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.3615 on 144 degrees of freedom

Multiple R-squared: 0.9595, Adjusted R-squared: 0.9581

F-statistic: 681.9 on 5 and 144 DF, p-value: < 2.2e-16

## 29.2 Generate a Prediction Grid

We often want to understand what our model is saying by predicting outcomes over combinations of input variables. We do this by first making a new dataset of the combinations of inputs we want to explore, and then we use our fit model to predict for all these combinations.

`data_grid()`, in the `modelr` package, is the tool for this. It will generate a simple dataframe with one row for each possible combination of variables you want. We will then predict for each of these rows.

Here is a toy example: see how duplicates are removed:

```
library(modelr)
my.df = data.frame(A=c(1,2,1,2), B=c(10,20,20,20), C=c(10,20,30,40))
data_grid(my.df, A, B)
```

```
A tibble: 4 x 2
```

	A	B
	<dbl>	<dbl>
1	1	10
2	1	20
3	2	10
4	2	20

For our actual data we have:

```
grid = iris %>%
 data_grid(Petal.Width, Species)
grid
```

```
A tibble: 66 x 2
```

	Petal.Width	Species
	<dbl>	<fct>
1	0.1	setosa
2	0.1	versicolor
3	0.1	virginica
4	0.2	setosa
5	0.2	versicolor
6	0.2	virginica
7	0.3	setosa
8	0.3	versicolor
9	0.3	virginica
10	0.4	setosa

# i 56 more rows

You can also get “typical values” (usually the median or most commonly occurring) for things not on your grid using a modeling argument:

```
mod <- lm(Petal.Length ~ Petal.Width * Species, data=iris)
data_grid(iris, .model = mod)
```

```
A tibble: 3 x 2
```

	Petal.Width	Species
	<dbl>	<chr>



```

1 1.3 setosa
2 1.3 versicolor
3 1.3 virginica

```

```
data_grid(iris, Petal.Width = seq_range(Petal.Width, 9), .model = mod)
```

```

A tibble: 27 x 2
 Petal.Width Species
 <dbl> <chr>
1 0.1 setosa
2 0.1 versicolor
3 0.1 virginica
4 0.4 setosa
5 0.4 versicolor
6 0.4 virginica
7 0.7 setosa
8 0.7 versicolor
9 0.7 virginica
10 1 setosa
i 17 more rows

```

A tool that is also good is `seq_range()` that gives you a nice sequence of numbers over a range of values

```

X = rnorm(10000)
seq_range(X, 5)

```

```
[1] -4.4183000 -2.3517051 -0.2851103 1.7814846 3.8480794
```

```

drop 5% of data on tails
seq_range(X, 5, trim=0.05)

```

```
[1] -1.985483160 -0.995851324 -0.006219487 0.983412350 1.973044187
```

```

use pretty numbers
seq_range(X, 5, trim=0.05, pretty=TRUE)

```

```
[1] -2 -1 0 1 2
```

You can combine with `data_grid` to get a nice data frame of points to use for your “canonical points” for prediction or whatnot:

```
my.df = data.frame(A=rnorm(1000), B=rexp(1000), C=rnorm(1000,mean=5))
data_grid(my.df,
 A=seq_range(A,n=3),
 B=seq_range(B,n=3))
```

```
A tibble: 9 x 2
 A B
 <dbl> <dbl>
1 -3.02 0.00460
2 -3.02 3.20
3 -3.02 6.39
4 0.190 0.00460
5 0.190 3.20
6 0.190 6.39
7 3.40 0.00460
8 3.40 3.20
9 3.40 6.39
```

## 29.3 Add Predictions to the Grid

This is easy, like so:

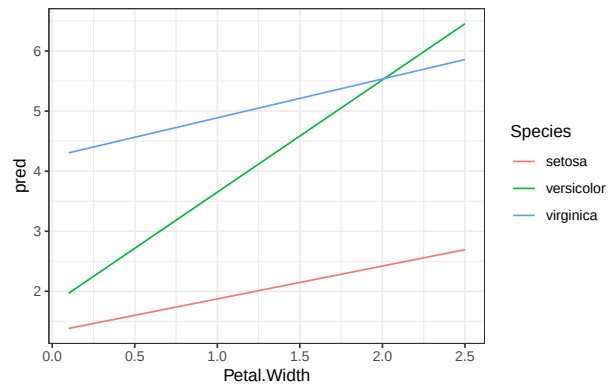
```
grid <- grid %>%
 add_predictions(M1)
head(grid)
```

```
A tibble: 6 x 3
 Petal.Width Species pred
 <dbl> <fct> <dbl>
1 0.1 setosa 1.38
2 0.1 versicolor 1.97
3 0.1 virginica 4.31
4 0.2 setosa 1.44
5 0.2 versicolor 2.16
6 0.2 virginica 4.37
```

## 29.4 Plot Predicted Trends

We can then plot our predictions for each of our species:

```
ggplot(grid, aes(x=Petal.Width, y=pred,
 col=Species, group=Species)) +
 geom_line()
```

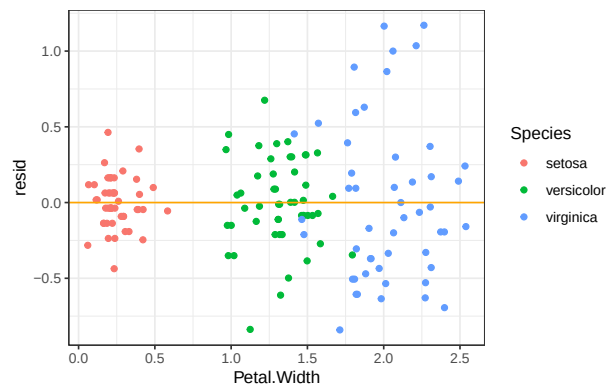


## 29.5 Extensions

### 29.5.1 Residuals and Diagnostics

We can add residuals to our data for model diagnostics:

```
iris <- add_residuals(iris, M1)
ggplot(iris, aes(x=Petal.Width, y=resid, col=Species)) +
 geom_jitter() +
 geom_ref_line(h=0, col="orange", size=0.5)
```



## 29.5.2 Multiple Models

Let's try multiple models and compare their predictions. We will have a simple model with no interactions, and a more complex one that allows curved relationships within each species.

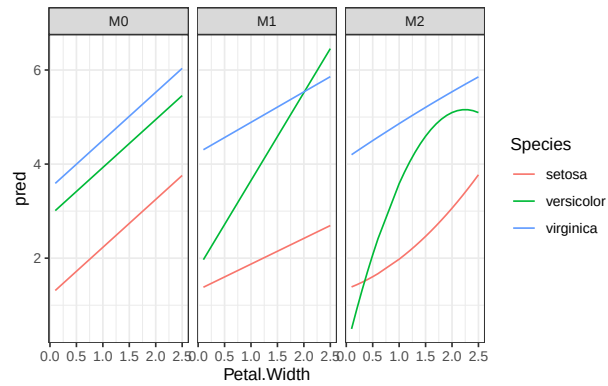
```
M0 <- lm(Petal.Length ~ Petal.Width + Species, data=iris)
M2 <- lm(Petal.Length ~ Species *(Petal.Width + I(Petal.Width^2)), data=iris)
```

```
grid = iris %>%
 data_grid(Petal.Width, Species) %>%
 gather_predictions(M1, M0, M2)
grid
```

```
A tibble: 198 x 4
 model Petal.Width Species pred
 <chr> <dbl> <fct> <dbl>
1 M1 0.1 setosa 1.38
2 M1 0.1 versicolor 1.97
3 M1 0.1 virginica 4.31
4 M1 0.2 setosa 1.44
5 M1 0.2 versicolor 2.16
6 M1 0.2 virginica 4.37
7 M1 0.3 setosa 1.49
8 M1 0.3 versicolor 2.34
9 M1 0.3 virginica 4.43
10 M1 0.4 setosa 1.55
i 188 more rows
```

And now plot:

```
ggplot(grid, aes(x=Petal.Width, y=pred, col=Species)) +
 facet_wrap(~ model) +
 geom_line()
```



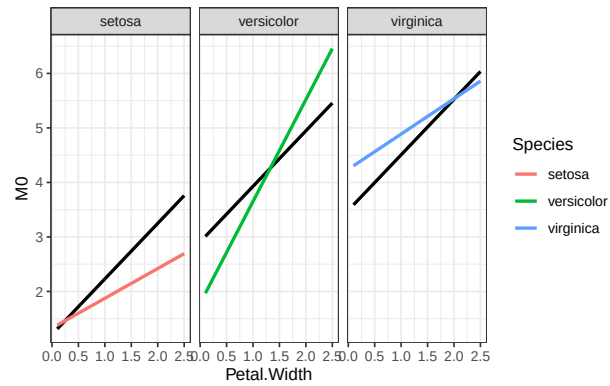
### 29.5.3 Fancy Plotting with spread\_predictions

The `spread_predictions()` function can be used to make each model a separate column:

```
grid <- iris %>%
 data_grid(Petal.Width, Species)
grid_spread = spread_predictions(grid, M0, M1)
grid_spread
```

```
A tibble: 66 x 4
 Petal.Width Species M0 M1
 <dbl> <fct> <dbl> <dbl>
1 0.1 setosa 1.31 1.38
2 0.1 versicolor 3.01 1.97
3 0.1 virginica 3.59 4.31
4 0.2 setosa 1.42 1.44
5 0.2 versicolor 3.11 2.16
6 0.2 virginica 3.69 4.37
7 0.3 setosa 1.52 1.49
8 0.3 versicolor 3.21 2.34
9 0.3 virginica 3.79 4.43
10 0.4 setosa 1.62 1.55
i 56 more rows
```

```
ggplot(grid_spread, aes(x=Petal.Width)) +
 facet_wrap(~ Species) +
 geom_line(aes(y=M0), col="black", lwd=1) +
 geom_line(aes(y=M1, col=Species), lwd=1, lty=1)
```



## 29.6 Handy Functions Recap

Function	Use
<code>data_grid()</code>	Create a grid of input values for prediction
<code>add_predictions()</code>	Add predicted values from a model to a dataset
<code>add_residuals()</code>	Add model residuals to original data
<code>gather_predictions()</code>	Combine predictions from multiple models for plotting
<code>spread_predictions()</code>	Spread predictions into separate columns
<code>seq_range()</code>	Create evenly spaced values over a variables range

## 29.7 Final Thoughts

These techniques make model interpretation and communication much easier. Being able to **visualize predictions and model trends** can bring clarity, especially when dealing with interactions or nonlinearities.

Practice using these tools and adapt them to your own datasets!

# **Part IV**

## **On Coding**

## 30 Basic Data Manipulation with tidyverse

Let's use the following simple table and run through various tidyverse commands. This is not a complete reference! See the text of the textbook for further details.

```
library(tidyverse)
table1

A tibble: 6 x 4
 country year cases population
 <chr> <dbl> <dbl> <dbl>
1 Afghanistan 1999 745 19987071
2 Afghanistan 2000 2666 20595360
3 Brazil 1999 37737 172006362
4 Brazil 2000 80488 174504898
5 China 1999 212258 1272915272
6 China 2000 213766 1280428583
```

### 30.1 filter() (Grab the rows you want, 5.1)

```
filter(table1, year > 1999)

A tibble: 3 x 4
 country year cases population
 <chr> <dbl> <dbl> <dbl>
1 Afghanistan 2000 2666 20595360
2 Brazil 2000 80488 174504898
3 China 2000 213766 1280428583
```

Remember, if you want to save the results of your command, you need to put it in a new variable, like so:

```
my.table <- filter(table1, year > 1999)
my.table
```



```
A tibble: 3 x 4
 country year cases population
 <chr> <dbl> <dbl> <dbl>
1 Afghanistan 2000 2666 20595360
2 Brazil 2000 80488 174504898
3 China 2000 213766 1280428583
```

When you do something like this, you should see it appear in your workplace.

## 30.2 arrange() (Sort your rows the way you want, 5.2)

```
arrange(table1, cases)
```

```
A tibble: 6 x 4
 country year cases population
 <chr> <dbl> <dbl> <dbl>
1 Afghanistan 1999 745 19987071
2 Afghanistan 2000 2666 20595360
3 Brazil 1999 37737 172006362
4 Brazil 2000 80488 174504898
5 China 1999 212258 1272915272
6 China 2000 213766 1280428583
```

```
arrange(table1, desc(country), population)
```

```
A tibble: 6 x 4
 country year cases population
 <chr> <dbl> <dbl> <dbl>
1 China 1999 212258 1272915272
2 China 2000 213766 1280428583
3 Brazil 1999 37737 172006362
4 Brazil 2000 80488 174504898
5 Afghanistan 1999 745 19987071
6 Afghanistan 2000 2666 20595360
```

## 30.3 select() (Grab the columns you want, 5.3)

```
select(table1, country, population)
```

```
A tibble: 6 x 2
 country population
 <chr> <dbl>
1 Afghanistan 19987071
2 Afghanistan 20595360
3 Brazil 172006362
4 Brazil 174504898
5 China 1272915272
6 China 1280428583
```

### 30.4 mutate() (Make new variables out of your old ones, 5.4)

```
table1 <- mutate(table1, case.per.1000 = 1000 * cases / population)
table1
```

```
A tibble: 6 x 5
 country year cases population case.per.1000
 <chr> <dbl> <dbl> <dbl> <dbl>
1 Afghanistan 1999 745 19987071 0.0373
2 Afghanistan 2000 2666 20595360 0.129
3 Brazil 1999 37737 172006362 0.219
4 Brazil 2000 80488 174504898 0.461
5 China 1999 212258 1272915272 0.167
6 China 2000 213766 1280428583 0.167
```

(We will use this new variable later, so I am saving it in our table)

### 30.5 group\_by() and summarize (Summarize your data by subgroup, 5.6)

```
tbl <- group_by(table1, country)
summarize(tbl, av.pop = mean(population), av.cases = mean(cases))
```

```
A tibble: 3 x 3
 country av.pop av.cases
 <chr> <dbl> <dbl>
1 Afghanistan 20291216. 1706.
2 Brazil 173255630 59112.
3 China 1276671928. 213012
```

Same thing, with the pipe!

```
table1 %>% group_by(country) %>%
 summarize(av.pop = mean(population), av.cases = mean(cases))
```

```
A tibble: 3 x 3
 country av.pop av.cases
 <chr> <dbl> <dbl>
1 Afghanistan 20291216. 1706.
2 Brazil 173255630 59112.
3 China 1276671928. 213012
```

## 30.6 Special: grouped mutates (Making new variables within subgroups, 5.6)

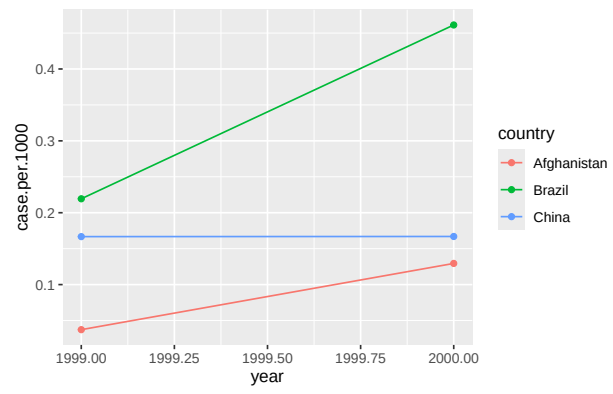
This combo is for doing things like group mean centering your data:

```
table1 %>% group_by(year) %>% mutate(case.per.1000.cent = case.per.1000 - mean(case.per.1000))
```

```
A tibble: 6 x 6
Groups: year [2]
 country year cases population case.per.1000 case.per.1000.cent
 <chr> <dbl> <dbl> <dbl> <dbl> <dbl>
1 Afghanistan 1999 745 19987071 0.0373 -0.104
2 Afghanistan 2000 2666 20595360 0.129 -0.123
3 Brazil 1999 37737 172006362 0.219 0.0783
4 Brazil 2000 80488 174504898 0.461 0.209
5 China 1999 212258 1272915272 0.167 0.0256
6 China 2000 213766 1280428583 0.167 -0.0856
```

And a plot

```
ggplot(table1, aes(x=year, y=case.per.1000, col=country)) + geom_line() + geom_point()
```



## 31 A Deep Dive on `group_by()`

The `group_by()` functionality in the Tidyverse is a powerhouse function, with many applications for summarizing and cleaning data. This handout walks through several common use cases for the `groupby_()` function.

## 32 Importing the ‘dplyr’ package

The `group_by()` function technically belongs to the `dplyr` package, but we can also load it by importing the `tidyverse` shortcut.

```
library(dplyr)
library(tidyverse)
```

For this walkthrough, we will be using a dataset of information on NFL players. We will use the `group_by()` function to analyze information across NFL teams. Our dataset consists of 16 columns and 1,822 players (one player per row).

```
nfl
```

```
A tibble: 1,822 x 16
```

	`Player Name` <chr>	Team <chr>	Number <dbl>	Position <chr>	Salary <dbl>	Height <chr>	Weight <dbl>	Birthdate <chr>	Age <dbl>
1	Abdullah, Husain	KC	39	S	715000	"6'0\"~	204	7/27/85	29
2	Abdul-Quddus, Isa	NO	42	S	465000	"6'1\"~	205	8/3/89	25
3	Aboushi, Oday	NYJ	75	OL	589750	"6'5\"~	308	6/5/91	24
4	Abraham, John	ARI	55	DE	2300000	"6'4\"~	263	5/6/78	37
5	Acho, Emmanuel	PHI	53	LB	525920	"6'2\"~	240	11/10/90	24
6	Acho, Sam	ARI	94	LB	628607	"6'3\"~	257	9/6/88	26
7	Adams, Joe	CAR	15	WR	641888	"5'11\"~	185	11/22/89	25
8	Adams, Kris	NYG	89	WR	510000	"6'3\"~	194	9/4/87	27
9	Adams, Kyle	CHI	86	TE	465000	"6'4\"~	255	1/19/88	27
10	Adams, Michael	TB	27	CB	NA	"5'8\"~	181	6/17/85	30

```
i 1,812 more rows
i 7 more variables: Experience <dbl>, College <chr>, City <chr>, State <chr>,
Race <chr>, birth_date <date>, birth_month <dbl>
```

### 32.1 What does `group_by()` do?

When you call `group_by()` on a dataframe, you are asking R to “organize” your data according to the grouping variable. For example, let’s group our data on NFL players by team. If you look carefully at the output below, you will see that our data has 1,822 rows and 16 columns, just like it did before. However, it now also has 32 groups (one for each NFL team).

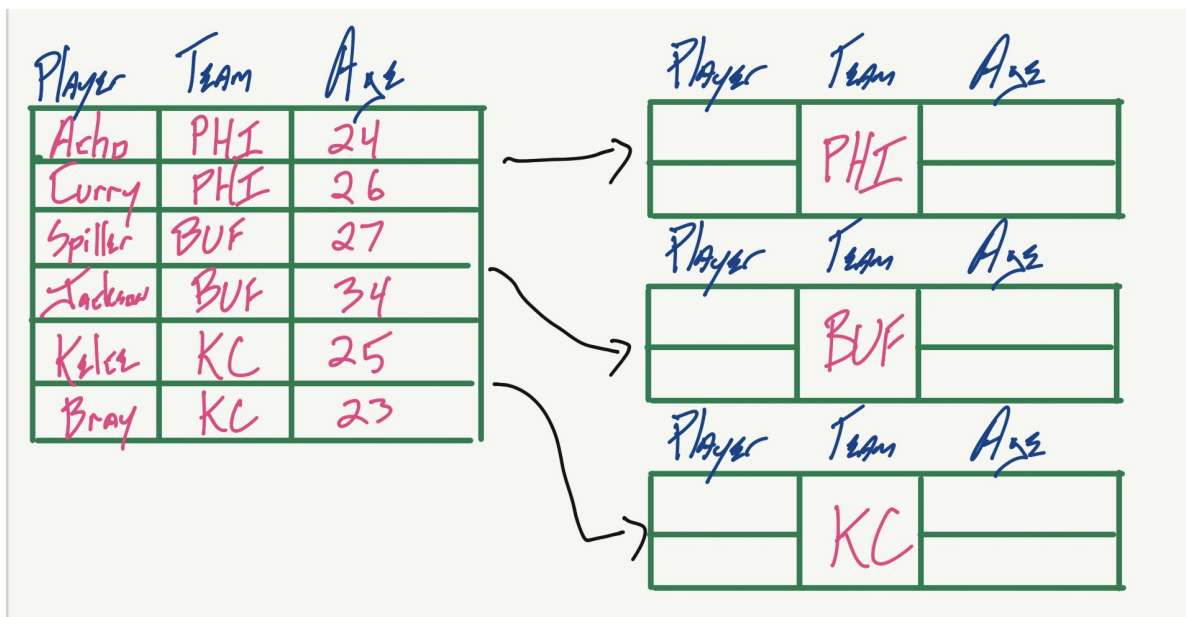
```
nfl %>% group_by(Team)
```

```
A tibble: 1,822 x 16
```

```
Groups: Team [32]
```

```
 `Player Name` Team Number Position Salary Height Weight Birthdate Age
 <chr> <chr> <dbl> <chr> <dbl> <chr> <dbl> <chr> <dbl>
1 Abdullah, Husain KC 39 S 715000 "6'0\"~ 204 7/27/85 29
2 Abdul-Quddus, Isa NO 42 S 465000 "6'1\"~ 205 8/3/89 25
3 Aboushi, Oday NYJ 75 OL 589750 "6'5\"~ 308 6/5/91 24
4 Abraham, John ARI 55 DE 2300000 "6'4\"~ 263 5/6/78 37
5 Acho, Emmanuel PHI 53 LB 525920 "6'2\"~ 240 11/10/90 24
6 Acho, Sam ARI 94 LB 628607 "6'3\"~ 257 9/6/88 26
7 Adams, Joe CAR 15 WR 641888 "5'11\"~ 185 11/22/89 25
8 Adams, Kris NYG 89 WR 510000 "6'3\"~ 194 9/4/87 27
9 Adams, Kyle CHI 86 TE 465000 "6'4\"~ 255 1/19/88 27
10 Adams, Michael TB 27 CB NA "5'8\"~ 181 6/17/85 30
i 1,812 more rows
i 7 more variables: Experience <dbl>, College <chr>, City <chr>, State <chr>,
Race <chr>, birth_date <date>, birth_month <dbl>
```

Conceptually, here is a visualization of what R does when you `group_by(Team)`. Behind the scenes, your data is partitioned according to values of your grouping variables. This becomes extremely useful if we want to calculate group-specific variables or create group-specific summary variables.



## 32.2 Introducing `summarize()`

The `summarize()` function is extremely useful for creating summary statistics on aggregated data, such as calculating the number of observations, the mean of a particular variable, etc. First, let's look at what happens when we use `summarize()` on its own. In the example below, we use `summarize()` on the *ungrouped* `nfl` data to find the overall mean salary of players (\$2,142,272).

```
nfl %>% summarize(mean_salary = mean(Salary, na.rm = TRUE))

A tibble: 1 x 1
 mean_salary
 <dbl>
1 2142272.
```

We can also use `summarize()` to get a variety of summary features using different functions to suit our purposes. We can also name these summary variables whatever we want (“mean\_salary”, “n\_obs”, “number\_of\_observations”, etc.). The code below calculates the average salary, the standard deviation of salaries, average age of players, and the total number of observations.

Below, you might also notice the `n()` function. This “short and sweet” function tells us the number of observations that we are summarizing across. In this case, `n()` returns 1,822—the same number of rows in our NFL data frame!

```
nfl %>%
 summarize(mean_salary = mean(Salary, na.rm = TRUE), # gets the overall average salary
 sd_salary = sd(Salary, na.rm = TRUE), # gets the standard deviation of salaries
 mean_age = mean(Age, na.rm = TRUE), # we can also look at other variables
 n_obs = n() # gets the number of observations
)

A tibble: 1 x 4
 mean_salary sd_salary mean_age n_obs
 <dbl> <dbl> <dbl> <int>
1 2142272. 2776358. 27.8 1822
```

## 32.3 Combining `group_by()` with `summarize()`

Now, let's add in `group_by()` to find the average (mean) and maximum salary *within each team*. Let's also get a count of the number of players on each team. Our output below has 32 rows—one for each value of the `Team` grouping variable.



Looking at the output, we can see that the Cleveland Browns (CLE) have an average salary of \$1,924,681 and that the Baltimore Ravens (BAL) have an average salary of \$2,145,966. Note that we set `mean(..., na.rm = TRUE)`, so we are ignoring any players with missing salary data when calculating our team averages.

```
nfl %>%
 group_by(Team) %>%
 summarize(num_players = n(),
 mean_sal = mean(Salary, na.rm = TRUE),
 max_sal = max(Salary, na.rm = TRUE))

A tibble: 32 x 4
 Team num_players mean_sal max_sal
 <chr> <int> <dbl> <dbl>
1 ARI 58 2110725. 15750000
2 ATL 55 2278403. 18958333
3 BAL 57 2145966. 20100000
4 BUF 55 1992520. 16000000
5 CAR 61 2228745 12666667
6 CHI 54 2369191. 14000000
7 CIN 59 1955534. 11175000
8 CLE 55 1924681. 10062500
9 DAL 56 2491651. 17071429
10 DEN 60 2145368. 19200000
i 22 more rows
```

### 32.3.1 Getting fancy: Grouping with more than one variable

However, we can also group by more than one variable! For example, we can extend our code to get the average salary within each player position on each team. Let's extend the code above to find the average and maximum salary within each team and position on that team. In this case, the `n()` summary variable tells us how many observations are within each grouping cell (i.e. the number of players in each position on each team).

What do you notice that is similar or different from the output above with a single grouping variable?

```
nfl %>%
 group_by(Team, Position) %>%
 summarize(num_players = n(),
 min_sal = min(Salary, na.rm = TRUE),
 mean_sal = mean(Salary, na.rm = TRUE),
 max_sal = max(Salary, na.rm = TRUE))
```

```
A tibble: 471 x 6
Groups: Team [32]
 Team Position num_players min_sal mean_sal max_sal
<chr> <chr> <int> <dbl> <dbl> <dbl>
1 ARI C 1 3250000 3250000 3250000
2 ARI CB 8 498367 1615209. 4607375
3 ARI DE 5 555000 3152000 11000000
4 ARI DL 2 405000 4869166. 9333333
5 ARI K 1 1500000 1500000 1500000
6 ARI LB 8 495000 1317682. 3166667
7 ARI LS 1 1090667 1090667 1090667
8 ARI NT 1 2067000 2067000 2067000
9 ARI OL 9 539856 2250741. 6000000
10 ARI P 1 1150000 1150000 1150000
i 461 more rows
```

## 32.4 Using mutate() with group\_by()

On its own, the `mutate()` function allows us to create and modify variables in a dataframe (see Chapter 30 for more information about `mutate()`). For example, we can use `mutate` to convert players' weight from pounds to kilograms.

In this basic example, note that we have not grouped the `nfl` dataframe.

```
nfl %>%
 select(`Player Name`, Weight) %>%
 mutate(weight_kg = Weight / 2.2) %>%
 head()
```

```
A tibble: 6 x 3
 `Player Name` Weight weight_kg
 <chr> <dbl> <dbl>
1 Abdullah, Husain 204 92.7
2 Abdul-Quddus, Isa 205 93.2
3 Aboushi, Oday 308 140
4 Abraham, John 263 120.
5 Acho, Emmanuel 240 109.
6 Acho, Sam 257 117.
```

However, when we combine `group_by()` and `mutate()` we can create new variables for individual players that reflect group-level attributes. For example, let's look at how we could find how much younger each player is than the oldest player on their team.

In this case, the grouping variable essentially allows us to create a new column for each individual in the NFL using summary variables within each NFL team.

```
let's create a new dataframe to hold our new data
nfl_1 <- nfl %>%
 select(`Player Name`, Team, Age) %>%
 group_by(Team) %>%
 mutate(years_younger = max(Age, na.rm = TRUE) - Age,
 team_max_age = max(Age, na.rm = TRUE) # we don't need this, but you can include it
) %>%
 arrange(Team) # re-order our data to help us understand how grouping is occurring at the t

nfl_1 %>% head()
```

```
A tibble: 6 x 5
Groups: Team [1]
 `Player Name` Team Age years_younger team_max_age
 <chr> <chr> <dbl> <dbl> <dbl>
1 Abraham, John ARI 37 2 39
2 Acho, Sam ARI 26 13 39
3 Alexander, Lorenzo ARI 32 7 39
4 Arenas, Javier ARI 27 12 39
5 Bell, Jeremiah ARI 37 2 39
6 Bethel, Justin ARI 25 14 39
```

If you look at the output above, you will notice that our new `nfl_1` dataframe is still grouped by Team. We might not want this, especially if we want to great other variables based on other grouping factors later. We can remove the groups from the `nfl_1` dataset like this:

```
nfl_1 <- nfl_1 %>% ungroup()
```

## 32.5 Learning Check: What is the difference between pairing `group_by()` with `summarize()` vs `group_by()` with `mutate()`?

Take a moment to review the output of the code above. How many rows do we have in our resulting dataframes after calling `group_by(Team)` with `summarize()` vs `group_by(Team)` with `mutate()`?

You might notice that there are far fewer rows in the output from commands using `summarize()`. This is because the `summarize()` function collapses data down to return a single value per group level (ex. one row for each of the 32 teams in the NFL). By contrast, using `group_by()`

with `mutate()` preserves all rows in the data while allowing the user to synthesize and make use of group-level information.

## 32.6 Practical Application: Centering Data

Centering data is a common practice in social science and education research. For example, we might often be less interested in a student's absolute test score and more interested in how they performed relative to an average value.

For example, we established above that the average salary in the entire NFL is \$2,142,272. We can use centering to calculate the amount each player earns above or below this overall average salary using the code below. This is called “grand mean” centering (i.e. we are comparing each player's salary to the overall average).

Note that the code below tabulates the average salary `mean(Salary)` without first calling `group_by()`.

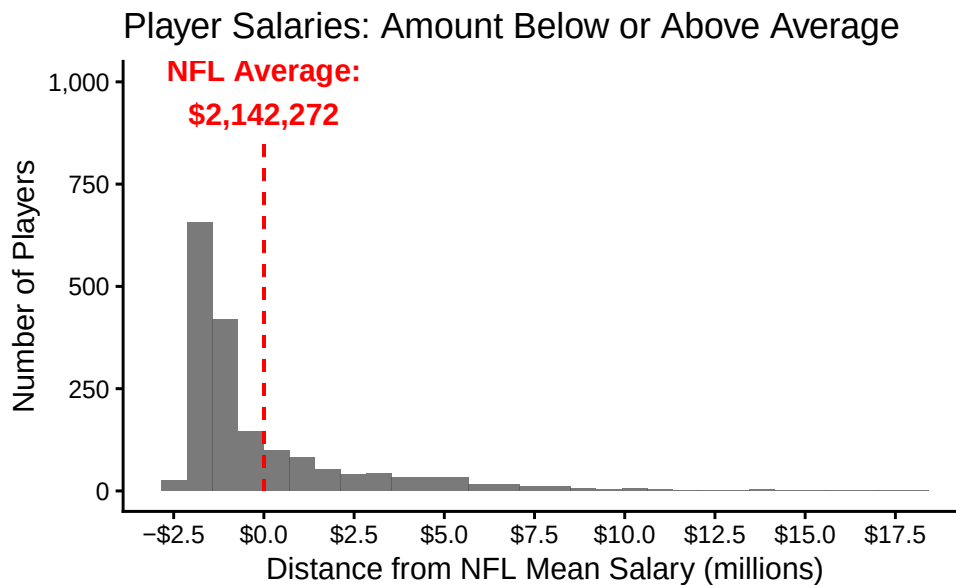
```
nfl <- nfl %>%
 group_by(Team) %>%
 mutate(grand_cent_sal = (Salary - mean(Salary, na.rm = TRUE))) %>%
 ungroup() # ungroup our data once we have what we need
```

```
nfl %>% select(Team, Salary, grand_cent_sal)
```

```
A tibble: 1,822 x 3
 Team Salary grand_cent_sal
 <chr> <dbl> <dbl>
1 KC 715000 -2159274.
2 NO 465000 -1468431.
3 NYJ 589750 -1284454.
4 ARI 2300000 189275.
5 PHI 525920 -1932906.
6 ARI 628607 -1482118.
7 CAR 641888 -1586857.
8 NYG 510000 -1578188.
9 CHI 465000 -1904191.
10 TB NA NA
i 1,812 more rows
```

We can visualize the resulting `grand_cent_sal` data series using `ggplot()`.

```
ggplot(nfl, aes(x = (grand_cent_sal) / 1000000)) +
 geom_histogram(alpha = 0.8) +
 geom_segment(x = 0, xend = 0, y = 0, yend = 875, linetype = 'dashed', color = 'red',) +
 annotate(geom = 'text', x = 0, y = 975, label = str_wrap("NFL Average: $2,142,272", 12),
 color = 'red', fontface = 'bold') +
 scale_x_continuous(breaks = seq(-5, 18.5, 2.5), labels = scales::dollar_format(accuracy = 0),
 scale_y_continuous(breaks = seq(0, 1000, 250), limits = c(0, 1000), labels = scales::comma),
 labs(title = "Player Salaries: Amount Below or Above Average",
 x = "Distance from NFL Mean Salary (millions)",
 y = "Number of Players")
```



### 32.6.1 Group Mean Centering

Now let's assume that it actually might be more informative to compare players' salaries to those of other players *on the same team*. In this case, we want to group our data by team before calculating our centered salary variable. This is called group mean centering.

```
nfl <- nfl %>%
 group_by(Team) %>%
 mutate(group_cent_sal = (Salary - mean(Salary, na.rm = TRUE)),
 team_average_salary = mean(Salary, na.rm = TRUE))

nfl %>% select(Team, Salary, group_cent_sal, team_average_salary) %>% head()
```

```

A tibble: 6 x 4
Groups: Team [5]
 Team Salary group_cent_sal team_average_salary
 <chr> <dbl> <dbl> <dbl>
1 KC 715000 -2159274. 2874274.
2 NO 465000 -1468431. 1933431.
3 NYJ 589750 -1284454. 1874204.
4 ARI 2300000 189275. 2110725.
5 PHI 525920 -1932906. 2458826.
6 ARI 628607 -1482118. 2110725.

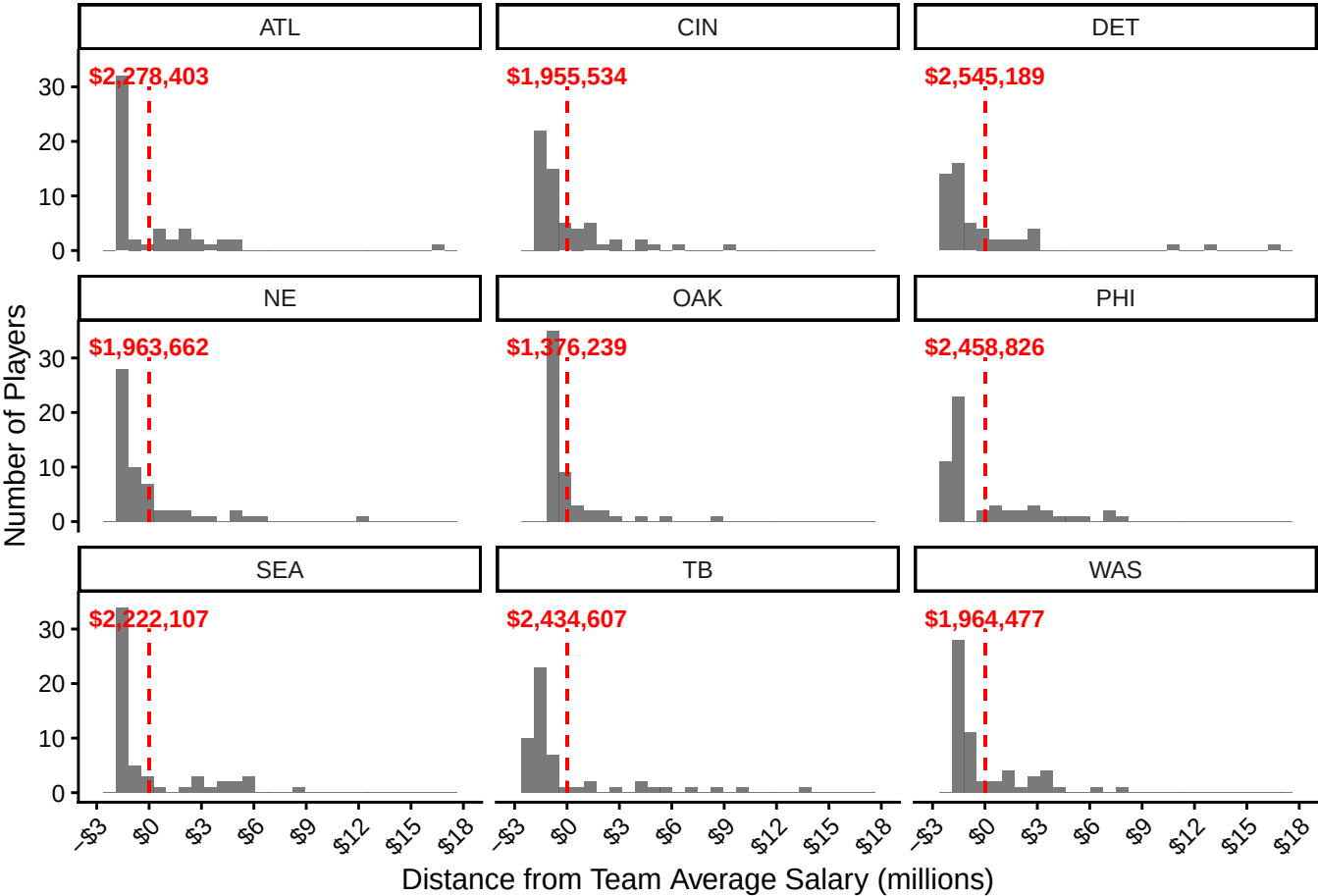
choose some teams to plot
teams <- sample(unique(nfl$Team), 9)

get the average salary for each team
team_average_sal_df <- nfl %>%
 filter(Team %in% teams) %>%
 group_by(Team) %>%
 summarize(team_avg_sal = mean(Salary, na.rm = T))

ggplot(nfl %>% filter(Team %in% teams),
 aes(x = (group_cent_sal) / 1000000)) +
 geom_histogram(alpha = 0.8) +
 facet_wrap(~ Team) +
 geom_segment(x = 0, xend = 0, y = 0, yend = 30, linetype = 'dashed', color = 'red',) +
 geom_text(data = team_average_sal_df, aes(x = 0, y = 32, label = scales::dollar(team_avg_sal)),
 color = 'red', size = 3,
 fontface = 'bold', inherit.aes = F) +
 scale_x_continuous(breaks = seq(-6, 18.5, 3), limits = c(-3, 18),
 labels = scales::dollar_format(accuracy = 1)) +
 scale_y_continuous(breaks = seq(0, 30, 10), limits = c(0, 35)) +
 labs(title = "Player Salaries: Amount Below or Above Team Average",
 x = "Distance from Team Average Salary (millions)",
 y = "Number of Players") +
 theme(axis.text.x = element_text(angle = 45, hjust = 1))

```

Player Salaries: Amount Below or Above Team Average



## 33 Intro to Regression

In S022 we assume a general understanding of linear regression, but sometimes one might like to review those concepts! In the following we first give some readings about regression, and then give a code walkthrough of how to do it in R.

The suggested readings (which can be accessed via the Library Reserves Tool on the Canvas site) are:

- **Gelman, A. & Hill, J. Chapter 4. In Data analysis using regression and multi-level/hierarchical models. Cambridge University Press, pp 53-74.** This chapter talks a bit more about regression, in particular transforming and centering variables, both being important tools for making your regression models more interpretable.
- **Kaplan, D.T. (2012). Chapter 7. In Statistical modeling: A fresh approach. (n.p.): Project Mosaic, pp. 129-151.** This reading reviews what regression is, and how to think about regression coefficients. An excellent review that points out that we can view regression as a summary of trends in our data. (i.e., use regression without believing in the model being correct). This is more in line with how data scientists think about models.
- **Gerber, A. S. & Green, D. (2012). Chapter 3. In Field experiments: Design, analysis, and interpretation. New York, NY: W. W. Norton & Company, pp. 51-86.** This chapter is a nice overview of what a standard error is, and how we should think about statistical uncertainty. It will deepen your understanding of the S040 material.

### 33.1 Simple Regression

We now give a walk-through on how to fit simple linear regression models in R. Linear regression is the main way researchers tend to examine the relationships between multiple variables. This document runs through some code without too much discussion, with the assumption that you are already familiar with interpretation of such models. If you want some help with the interpretation, see the suggested readings above!

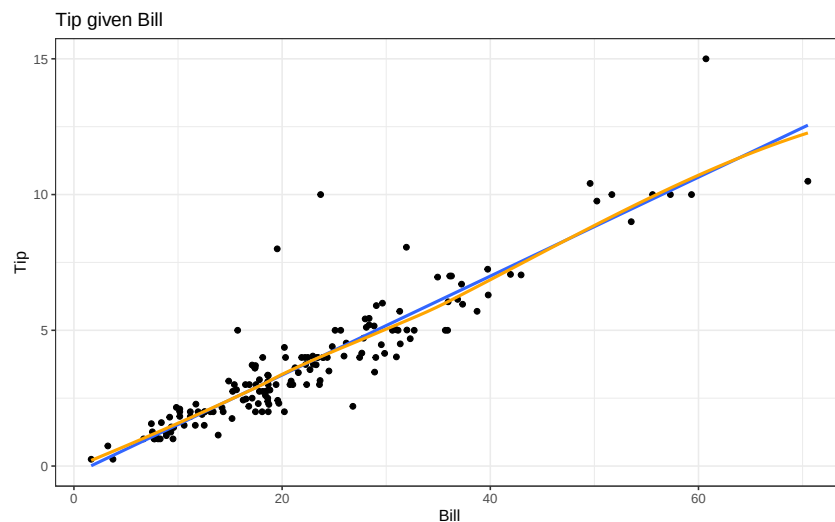
For our walk-through, we are going to use an example dataset, `RestaurantTips`, that records tip amounts for a series of bills. Let's first regress `Tip` on `Bill`. Before doing regression, we should plot the data to make sure using simple linear regression is reasonable. For kicks, we



add in an automatic regression line as well by taking advantage of ggplot's `geom_smooth()` method:

```
load the data into memory
data(RestaurantTips)

plot Tip on Bill
ggplot(RestaurantTips, aes(x = Bill, y = Tip)) +
 geom_point() +
 geom_smooth(method="lm", se=FALSE) +
 geom_smooth(method="loess", se=FALSE, col="orange") +
 labs(title = "Tip given Bill")
```



That looks pretty darn linear! There are a few unusually large tips, but no extreme outliers, and variability appears to be constant at all levels of Bill , so we proceed:

```
fit the linear model
mod <- lm(Tip ~ Bill, data = RestaurantTips)
summary(mod)
```

Call:

```
lm(formula = Tip ~ Bill, data = RestaurantTips)
```

Residuals:

Min	1Q	Median	3Q	Max
-2.391	-0.489	-0.111	0.284	5.974

```

Coefficients:
 Estimate Std. Error t value Pr(>|t|)
(Intercept) -0.29227 0.16616 -1.76 0.081 .
Bill 0.18221 0.00645 28.25 <2e-16 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.98 on 155 degrees of freedom
Multiple R-squared: 0.837, Adjusted R-squared: 0.836
F-statistic: 798 on 1 and 155 DF, p-value: <2e-16

```

The first line tells R to fit the regression. The thing on the left of the `~` is our outcome, the things on the right are our covariates or predictors. R then saves the results of all that work under the name `mod` (short for model - you can call it anything you want). Once we fit the model, we used `summary()` command to print the output to the screen.

Results relevant to the intercept are in the **(Intercept)** row and results relevant to the slope are in the **Bill** row (**Bill** is the explanatory variable). The **Estimate** column gives the estimated coefficients, the **Std. Error** column gives the standard error for these estimates, the **t value** is simply estimate/SE, and the p-value is the result of a hypothesis test testing whether that coefficient is significantly different from 0.

We also see the RMSE as **Residual standard error** and  $R^2$  as **Multiple R-squared**. The last line of the regression output gives details relevant to an ANOVA table for testing our model against no model. It has the F-statistic, degrees of freedom, and p-value.

You can pull the coefficients of your model out with the `coef()` command:

```

coef(mod)

(Intercept) Bill
 -0.292 0.182

coef(mod)[1] # intercept

(Intercept)
 -0.292

coef(mod)[2] # slope

Bill
0.182

```

```
coef(mod)["Bill"] # alternate way.
```

```
Bill
0.182
```

Alternatively, you can use the `tidy()` function from **broom** to turn the regression results into a tidy data frame, which makes it easier to work with:

```
tidy(mod)
```

```
A tibble: 2 x 5
```

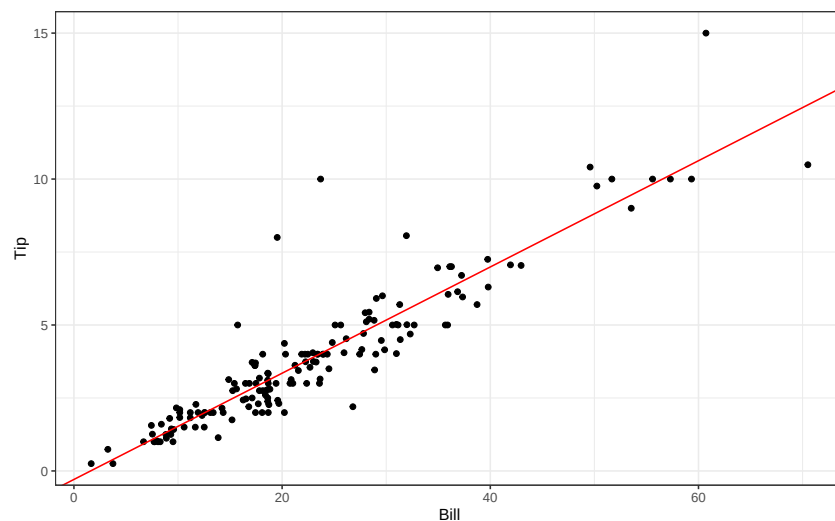
term	estimate	std.error	statistic	p.value
<chr>	<dbl>	<dbl>	<dbl>	<dbl>
1 (Intercept)	-0.292	0.166	-1.76	8.06e- 2
2 Bill	0.182	0.00645	28.2	5.24e-63

```
tidy(mod)[[2,2]] # slope
```

```
[1] 0.182
```

We can plot our regression line on top of the scatterplot manually using the `geom_abline()` layer in `ggplot`:

```
ggplot(RestaurantTips, aes(Bill, Tip)) +
 geom_point() +
 geom_abline(intercept = -0.292, slope = 0.182, col="red")
```



## 33.2 Multiple Regression

We now include the additional explanatory variables of number in party (**Guests**) and whether or not they pay with a credit card (**Credit**):

```
tip.mod <- lm(Tip ~ Bill + Guests + Credit, data=RestaurantTips)
summary(tip.mod)
```

Call:

```
lm(formula = Tip ~ Bill + Guests + Credit, data = RestaurantTips)
```

Residuals:

Min	1Q	Median	3Q	Max
-2.384	-0.478	-0.108	0.272	5.984

Coefficients:

	Estimate	Std. Error	t value	Pr(> t )
(Intercept)	-0.25468	0.20273	-1.26	0.21
Bill	0.18302	0.00846	21.64	<2e-16 ***
Guests	-0.03319	0.10282	-0.32	0.75
Credit	0.04217	0.18282	0.23	0.82

---

Signif. codes: 0 '\*\*\*' 0.001 '\*\*' 0.01 '\*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.985 on 153 degrees of freedom

Multiple R-squared: 0.838, Adjusted R-squared: 0.834

F-statistic: 263 on 3 and 153 DF, p-value: <2e-16

This output should look very similar to the output for one variable, except now there is a row corresponding to each explanatory variable. Our two-category (y, n) **Credit** variable was automatically converted to a 0-1 dummy variable (with “y” being 1 and “n” our baseline).

You can make plots and tables of your fit models. For one easy kind of regression graph, try **ggeffects**:

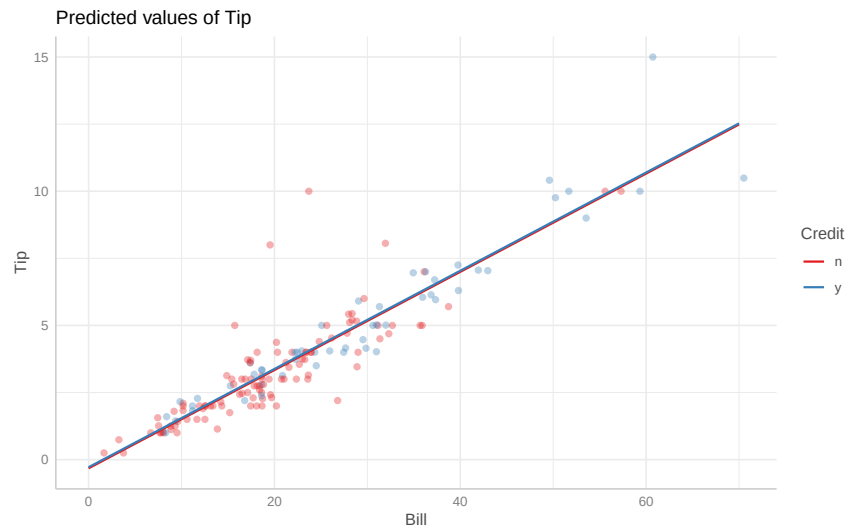
```
graph model 2, with Bill on X, Credit as color, and Guests held constant at the mean
ggeffect(tip.mod, terms = c("Bill", "Credit")) |>
 plot(show_data = TRUE, show_ci = FALSE)
```

Data points may overlap. Use the ``jitter`` argument to add some amount of random variation to the location of data points and avoid overplotting.

Ignoring unknown labels:

```
* linetype : "Credit"
```

```
* shape : "Credit"
```



For making tables, `?@sec-make-regression-tables`.

## 33.3 Categorical Variables (and Factors)

You can include any explanatory categorical variable in a multiple regression model, and R will automatically create corresponding 0/1 variables. For example, if you were to include gender coded as male/female, R would create a variable `GenderMale` that is 1 for males and 0 for females.

### 33.3.1 Numbers Coding Categories.

If you have multiple levels of a category, but your levels are coded with numbers you have to be a bit careful because R can treat this as a quantitative (continuous) variable by mistake in some cases. You will know it did this if you only see the single variable on one line of your output. For categorical variables with  $k$  categories, you should see  $k - 1$  lines.

To make a variable categorical, even if the levels are numbers, convert the variable to a factor with `as.factor` or `factor`:

```
load the US states data
data(USStates)

convert Region to a factor
USStates <- USStates |>
 mutate(Region = factor(Region))
```

### 33.3.2 Setting new baselines.

We can reorder the levels if desired (the first is our baseline).

```
levels(USStates$Region)

[1] "MW" "NE" "S" "W"

USStates$Region = relevel(USStates$Region, "S")
levels(USStates$Region)

[1] "S" "MW" "NE" "W"
```

Now any regression will use the south as baseline.

### 33.3.3 Testing for significance of a categorical variable.

When deciding whether to keep a categorical variable, we need to test how important all the dummy variables for that category are to the model all at once. We do this with ANOVA. Here we examine whether region is useful for predicting the percent vote for Clinton in 2016:

```
mlm = lm(ClintonVote ~ Region, data=USStates)
anova(mlm)
```

Analysis of Variance Table

```
Response: ClintonVote
 Df Sum Sq Mean Sq F value Pr(>F)
Region 3 1643 548 6.99 0.00057 ***
Residuals 46 3603 78
```

---

```
Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

It is quite important.

We can also compare for region beyond some other variable:

```
mlm2 = lm(ClintonVote ~ HouseholdIncome + HouseholdIncome + HighSchool +
 EighthGradeMath, data=USStates)

mlm3 = lm(ClintonVote ~ HouseholdIncome + HouseholdIncome + HighSchool +
 EighthGradeMath + Region, data=USStates)
anova(mlm2, mlm3)
```

Analysis of Variance Table

Model 1: ClintonVote ~ HouseholdIncome + HouseholdIncome + HighSchool +  
EighthGradeMath

Model 2: ClintonVote ~ HouseholdIncome + HouseholdIncome + HighSchool +  
EighthGradeMath + Region

	Res.Df	RSS	Df	Sum of Sq	F	Pr(>F)
1	46	3287				
2	43	2649	3	638	3.45	0.025 *

---

Signif. codes: 0 '\*\*\*' 0.001 '\*\*' 0.01 '\*' 0.05 '.' 0.1 ' ' 1

Region is still important, beyond including some further controls. Interpreting this mess of a regression is not part of this document; this document shows you how to run regressions but it doesn't discuss whether you should or not.

### 33.3.4 Missing levels in a factor

R often treats categorical variables as factors. This is often useful, but sometimes annoying. A factor has different **levels** which are the different values it can be. For example:

```
data(FishGills3)
levels(FishGills3$Calcium)
```

```
[1] "" "High" "Low" "Medium"
```

```
table(FishGills3$Calcium)
```

	High	Low	Medium
0	30	30	30

Note the weird nameless level; it also has no actual observations in it. Nevertheless, if you make a boxplot, you will get an empty plot in addition to the other three. This error was likely due to some past data entry issue. You can drop the unused level:

```
FishGills3$Calcium = droplevels(FishGills3$Calcium)
```

You can also turn a categorical variable into a numeric one like so:

```
summary(FishGills3$Calcium)
```

```
High Low Medium
 30 30 30
```

```
asnum = as.numeric(FishGills3$Calcium)
asnum
```

```
[1] 2 3 3 3 3 3 3 3 3
[39] 3 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
[77] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
```

Regression on only a categorical variable is fine:

```
mylm = lm(GillRate ~ Calcium, data=FishGills3)
mylm
```

Call:

```
lm(formula = GillRate ~ Calcium, data = FishGills3)
```

Coefficients:

```
(Intercept) CalciumLow CalciumMedium
 58.2 10.3 0.5
```

R has made you a bunch of dummy variables automatically. Here “high” is the baseline, selected automatically. We can also force it so there is no baseline by removing the intercept, in which case the coefficients are the means of each group.

```
myymm = lm(GillRate ~ 0 + Calcium, data=FishGills3)
myymm
```



```
Call:
lm(formula = GillRate ~ 0 + Calcium, data = FishGills3)
```

```
Coefficients:
 CalciumHigh CalciumLow CalciumMedium
 58.2 68.5 58.7
```

## 33.4 Some extensions (optional)

### 33.4.1 Confidence Intervals

To get confidence intervals around each parameter in your model, try this:

```
confint(tip.mod)
```

```
 2.5 % 97.5 %
(Intercept) -0.655 0.146
Bill 0.166 0.200
Guests -0.236 0.170
Credity -0.319 0.403
```

You can also create them easily using `tidy` and `mutate`:

```
tip.mod |>
 tidy() |>
 mutate(upper = estimate + 1.96*std.error,
 lower = estimate - 1.96*std.error)
```

```
A tibble: 4 x 7
 term estimate std.error statistic p.value upper lower
 <chr> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
1 (Intercept) -0.255 0.203 -1.26 2.11e- 1 0.143 -0.652
2 Bill 0.183 0.00846 21.6 2.07e-48 0.200 0.166
3 Guests -0.0332 0.103 -0.323 7.47e- 1 0.168 -0.235
4 Credidity 0.0422 0.183 0.231 8.18e- 1 0.400 -0.316
```

### 33.4.2 Prediction

Suppose a server at this bistro is about to deliver a \$20 bill, and wants to predict their tip. They can get a predicted value and 95% (this is the default level, change with level) prediction interval with

```
new.dat = data.frame(Bill = c(20))
predict(mod,new.dat, interval = "prediction")
```

```
 fit lwr upr
1 3.35 1.41 5.29
```

They should expect a tip somewhere between \$1.41 and \$5.30.

If we know a bit more we can use our more complex model called `tip.mod` from above:

```
new.dat = data.frame(Bill = c(20), Guests=c(1), Credit=c("n"))
predict(tip.mod,new.dat, interval = "prediction")
```

```
 fit lwr upr
1 3.37 1.41 5.34
```

This is the predicted tip for one guest paying with cash for a \$20 tip. It is wider than our original interval because our model is a bit more unstable (it turns out guest number and credit card aren't that relevant or helpful).

Compare the prediction interval to the confidence interval

```
new.dat = data.frame(Bill = c(20), Guests=c(1), Credit=c("n"))
predict(tip.mod, new.dat, interval = "confidence")
```

```
 fit lwr upr
1 3.37 3.09 3.65
```

This predicts the mean tip for all single guests who pay a \$20 bill with cash. Our interval is smaller because we are generating a confidence interval for where the mean is, and are ignoring that individuals will vary around that mean. Confidence intervals are different from prediction intervals.

### 33.4.3 Removing Outliers

If you can identify which rows the outliers are on, you can do this by hand (say the rows are 5, 10, 12).

```
new.data = old.data[-c(5,10,12),]
lm(Y ~ X, data=new.data)
```

Some technical details: The `c(5,10,12)` is a list of 3 numbers. The `c()` is the concatenation function that takes things makes lists out of them. The “-list” notation means give me my old data, but without rows 5, 10, and 12. Note the comma after the list. This is because we identify elements in a dataframe with row, column notation. So `old.data[1,3]` would be row 1, column 3.

If you notice your points all have X bigger than some value, say 20.5, you could use filtering to keep everything less than some value:

```
new.data = filter(old.data, X <= 20.5)
```

### 33.4.4 Missing data

If you have missing data, `lm` will automatically drop those cases because it doesn't know what else to do. It will tell you this, however, with the `summary` command.

```
data(AllCountries)
dev.lm = lm(BirthRate ~ Rural + Health + ElderlyPop, data=AllCountries)
summary(dev.lm)
```

Call:

```
lm(formula = BirthRate ~ Rural + Health + ElderlyPop, data = AllCountries)
```

Residuals:

Min	1Q	Median	3Q	Max
-16.592	-3.728	-0.791	3.909	16.218

Coefficients:

	Estimate	Std. Error	t value	Pr(> t )
(Intercept)	26.5763	1.6795	15.82	< 2e-16 ***
Rural	0.0985	0.0224	4.40	1.9e-05 ***
Health	-0.0995	0.0930	-1.07	0.29
ElderlyPop	-1.0249	0.0881	-11.64	< 2e-16 ***

---

Signif. codes: 0 '\*\*\*' 0.001 '\*\*' 0.01 '\*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 5.83 on 174 degrees of freedom

(39 observations deleted due to missingness)

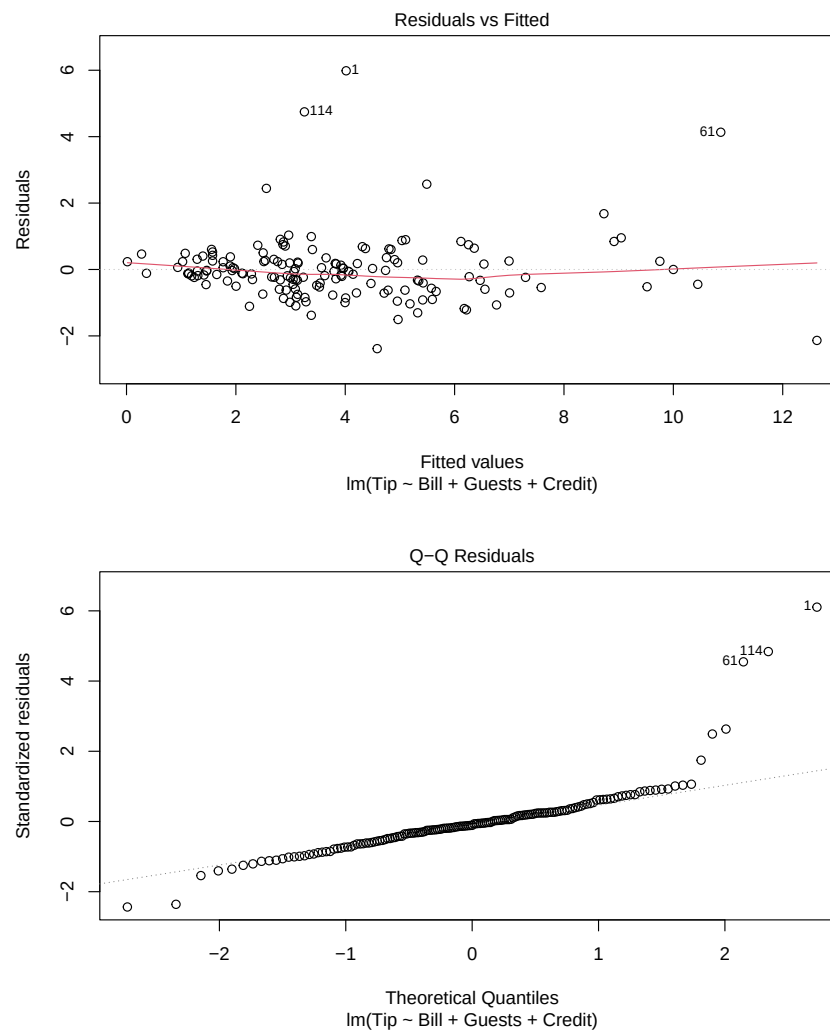
Multiple R-squared: 0.663, Adjusted R-squared: 0.657

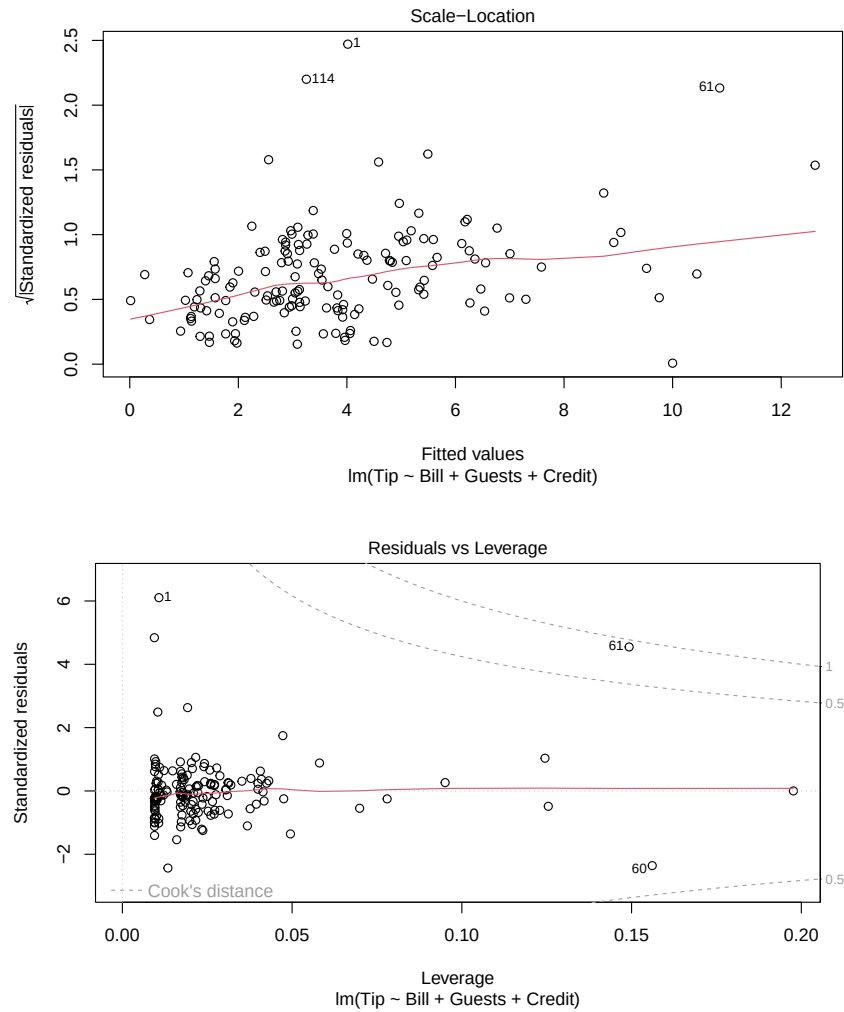
F-statistic: 114 on 3 and 174 DF, p-value: <2e-16

### 33.4.5 Residual plots and model fit

If we throw out model into the `plot` function, we get some nice regression diagnostics.

```
plot(tip.mod)
```

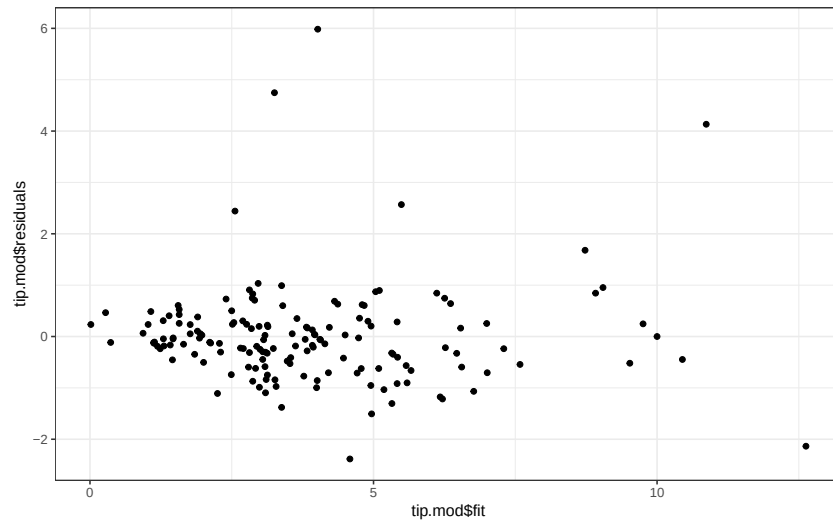




To generate classic model fit diagnostics with more control, we need to calculate residuals, make a residual versus fitted values plot, and make a histogram of the residuals. We can make some quick and dirty plots with `qplot` (standing for “quick plot”) like so:

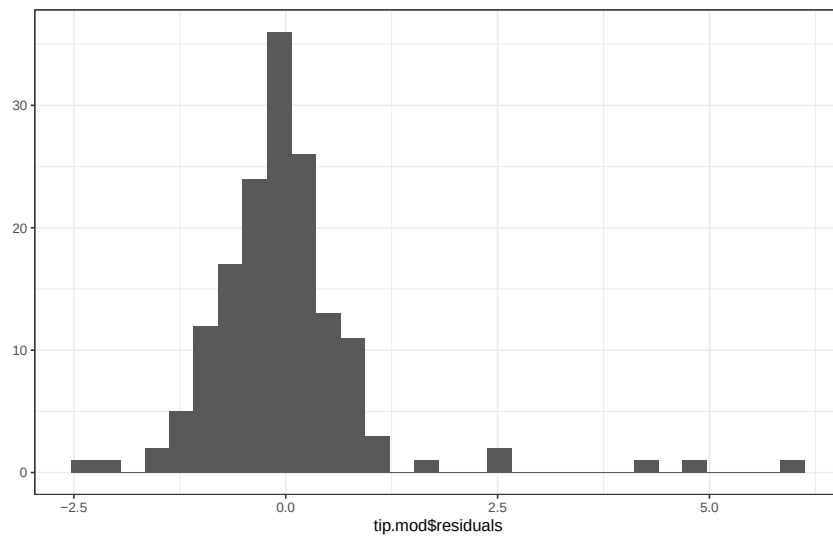
```
qplot(tip.mod$fit, tip.mod$residuals)
```

Warning: ``qplot()`` was deprecated in `ggplot2 3.4.0`.



and

```
qplot(tip.mod$residuals, bins=30)
```



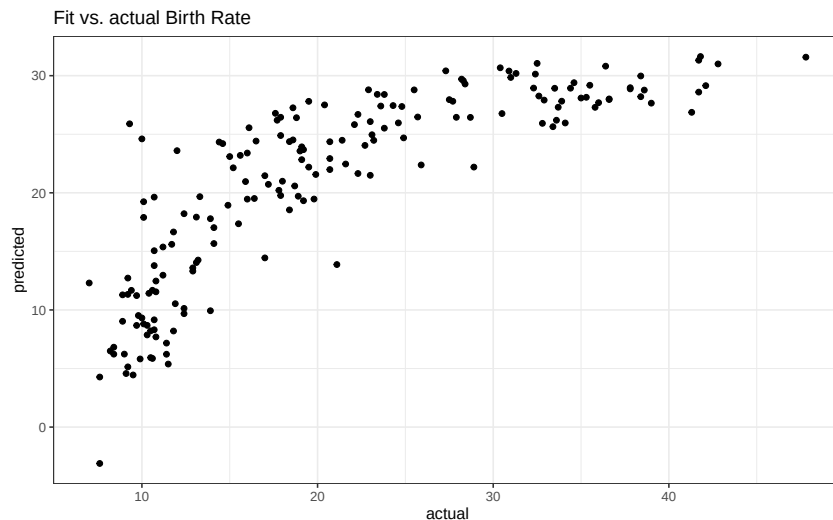
We see no real pattern other than some extreme outliers. The residual histogram suggests we are not really normally distributed, so we should treat our SEs and  $p$ -values with caution. These plots are the canonical “model-checking” plots you might use.

Another is the “fitted outcomes vs. actual outcomes” plot of:

```

predicted = predict(dev.lm)
actual = dev.lm$model$BirthRate
qplot(actual, predicted, main="Fit vs. actual Birth Rate")

```



Note the `dev.lm` variable has a `model` variable inside it. This is a data frame of the **used** data for the model (i.e., if cases were dropped due to missingness, they will not be in the model). We then grab the birth rates from this, and make a scatterplot. If we tried to skip this, and use the original data, we would get an error because our original data set has some observations that were dropped.

Note we can't just add our predictions to `AllCountries` since we would get an error due to this dropped data issue:

```

AllCountries$predicted = predict(dev.lm)

```

```

Error in `<-data.frame`(`*tmp*`, predicted, value = c(`1` = 31.630301617421, :
 replacement has 179 rows, data has 217

```

We can, however, predict like this:

```

AllCountries$predicted = predict(dev.lm, newdata=AllCountries)

```

The `newdata` tells `predict` to generate a prediction for each row in `AllCountries` rather than each row in the left over data after `lm` dropped cases with missing values.

## 34 Doing things over and over again

In this handout, we will look at several ways of doing things over and over again in R. This comes up all the time. Three main ones we have seen are the following:

- Fit a loess curve to my data for each of a series of smoothing parameters.
- Repeatedly bootstrap my dataset and analyze it.
- Run a simulation where we generate data and analyze it over and over.
- Scrape a series of web pages

Coding wise, there are a few ways of repeating yourself. We are going to walk through them and compare these tools to each other.



## 35 The replicate() command

This simple command repeats a line of code a given number of times. If the line of code gives you a number back each time it is run, then you will end up with a list of numbers.

To illustrate, say we want to look at rolling dice. Here is some code that provides a function that will roll some number of 6-sided dice:

```
roll_dice = function(ndice) {
 rolls = sample(1:6, ndice, replace=TRUE)
 sum(rolls)
}
```

```
Roll a single die
roll_dice(1)
```

```
[1] 5
```

```
Roll 3 dice and add them up.
roll_dice(3)
```

```
[1] 8
```

If we want to get the sum of three dice over and over, we can replicate:

```
rolls = replicate(10, roll_dice(3))
rolls
```

```
[1] 11 15 6 11 10 13 10 11 16 8
```

Note how the `rolls` variable is a nice numeric vector, easy to work with. It is easy to do calculations with it, like take the average:

```
mean(rolls)
```

```
[1] 11.1
```

Here we use this to see how often we roll above a 15:

```
rolls = replicate(10000, roll_dice(3))
mean(rolls > 15)
```

```
[1] 0.0474
```

## 36 The rerun() command

The `rerun()` command, from the tidyverse, is almost exactly like `replicate`, but instead of giving a numeric vector of numbers back, it gives an R list. For brevity, I rerun twice in the following:

```
rolls = rerun(2, roll_dice(3))
```

```
Warning: `rerun()` was deprecated in purrr 1.0.0.
```

```
i Please use `map()` instead.
```

```
Previously
```

```
rerun(2, roll_dice(3))
```

```
Now
```

```
map(1:2, ~ roll_dice(3))
```

```
rolls
```

```
[[1]]
```

```
[1] 14
```

```
[[2]]
```

```
[1] 9
```

Each element of the list is itself a list of numbers! Why would someone make such an annoying command like that? This is useful if the function we are rerunning gives us back multiple things. For example:

```
roll_dice_extended = function(ndice) {
 rolls = sample(1:6, ndice, replace=TRUE)
 c(mean = mean(rolls), median = median(rolls), max = max(rolls))
}
roll_dice_extended()
```

```
 mean median max
3.833333 3.500000 6.000000
```

Now when we rerun we get this:

```
rolls = rerun(2, roll_dice_extended(3))

Warning: `rerun()` was deprecated in purrr 1.0.0.
i Please use `map()` instead.
Previously
rerun(2, roll_dice_extended(3))

Now
map(1:2, ~ roll_dice_extended(3))
```

```
rolls
```

```
[[1]]
 mean median max
5.333333 6.000000 6.000000

[[2]]
 mean median max
4.333333 5.000000 6.000000
```

But this is hard to work with. All our numbers are nested and weird. But there is a solution, which is the “rerun + bind\_rows combo”. Once we have a list of our answers, we can “stack” them with `bind_rows`:

```
bind_rows(rolls)

A tibble: 2 x 3
 mean median max
<dbl> <dbl> <dbl>
1 5.33 6 6
2 4.33 5 6
```

We can also give each row a name:

```
bind_rows(rolls, .id="runID")

A tibble: 2 x 4
 runID mean median max
<chr> <dbl> <dbl> <dbl>
1 1 5.33 6 6
2 2 4.33 5 6
```

**Side Note:** I recommend making your function give back a simple little dataframe of all your stuff. It is less prone to having weird errors. The `bind_rows()` method does really well with data.frames or tibbles. Here is an updated version of the above:

```
roll_dice_extended = function(ndice) {
 rolls = sample(1:6, ndice, replace=TRUE)
 data.frame(mean = mean(rolls), median = median(rolls), max = max(rolls))
}
roll_dice_extended()
```

```
 mean median max
1 3.666667 3.5 6
```

```
rolls = rerun(6, roll_dice_extended(3))
```

Warning: `rerun()` was deprecated in purrr 1.0.0.

i Please use `map()` instead.

# Previously

```
rerun(6, roll_dice_extended(3))
```

# Now

```
map(1:6, ~ roll_dice_extended(3))
```

```
rolls = bind_rows(rolls, .id="runID")
```

```
rolls
```

```
 runID mean median max
1 1 3.666667 3 6
2 2 3.333333 3 6
3 3 3.333333 3 4
4 4 3.333333 4 5
5 5 3.000000 2 6
6 6 3.333333 3 6
```

## 36.1 Warning: replicate() doesn't do well with fancy functions

The `replicate()` command doesn't act nice in the following:

```
rolls = replicate(2, roll_dice_extended(3))
rolls
```

```
, , 1
```

```
 mean median max
1 2 2 3
```

```
, , 2
```

```
 mean median max
1 2.333333 2 4
```

That doesn't look like a fun thing to work with. (It is a 3 dimensional array of output, in case you are wondering.) Use `rerun + bind_rows`; it is easier to control and understand.

## 36.2 Take-away

If your function that you want to repeat returns a single number with each call, use `replicate()`. If it returns more than one thing, use the `rerun() + bind_rows()` combination.

## 37 map(), another way of repeating yourself

The above runs the exact same code over and over. Sometimes you want to run the same function on a collection of different things (e.g., fit a loess line for each of a series of bandwidths). This is done with the `map()` command which takes a list of things, and then calls a function for each thing on that list.

To illustrate, we will see how the largest number rolled changes as a function of the number of dice rolled. To start, let's roll 1 die, then 2 dice, then 3 dice, and each time calculate the average, median, and max:

```
dice = 1:3
result = map(dice, roll_dice_extended)
result
```

```
[[1]]
 mean median max
1 5 5 5

[[2]]
 mean median max
1 4 4 6

[[3]]
 mean median max
1 3.333333 3 5
```

Note unlike `rerun` or `replicate` we are not **calling** our `roll_dice_extended` function, we are just giving the name of it. You can tell since we do not have the `()` after `roll_dice_extended`, we just pass the name of the function we want to call. We are asking `map` to call `roll_dice_extended` over and over. For `rerun` or `replicate`, by comparison, we provide a stand-alone complete line of code that would run by itself. For `map` we just give the name of a function to run.

We can make our output nicer with the same `bind_rows` trick from `rerun()`:

```
bind_rows(result)
```

	mean	median	max
1	5.000000	5	5
2	4.000000	4	6
3	3.333333	3	5

Even better is using the `map_df()` method, which works nicely **provided your function returns a dataframe**:

```
result = map_df(dice, roll_dice_extended)
result$ndice = dice
result
```

	mean	median	max	ndice
1	3.0	3.0	3	1
2	5.5	5.5	6	2
3	4.0	5.0	6	3



## 38 Repeating yourself when you are repeating yourself

Our little simulation is a bit sad in that we only have a single trial for each of our number of dice scenarios. What we really want is to know the distribution of the maximum roll for each number of dice. We do this by repeating ourself and then repeating this repeating ourself!

Study this:

```
one_trial = function(ndice) {
 rolls = rerun(1000, roll_dice_extended(ndice))
 rolls = bind_rows(rolls)
 tbl = table(rolls$max)
 tbl
}
one_trial(1)
```

Warning: `rerun()` was deprecated in purrr 1.0.0.

i Please use `map()` instead.

# Previously

```
rerun(1000, roll_dice_extended(ndice))
```

# Now

```
map(1:1000, ~ roll_dice_extended(ndice))
```

```
 1 2 3 4 5 6
168 155 169 158 166 184
```

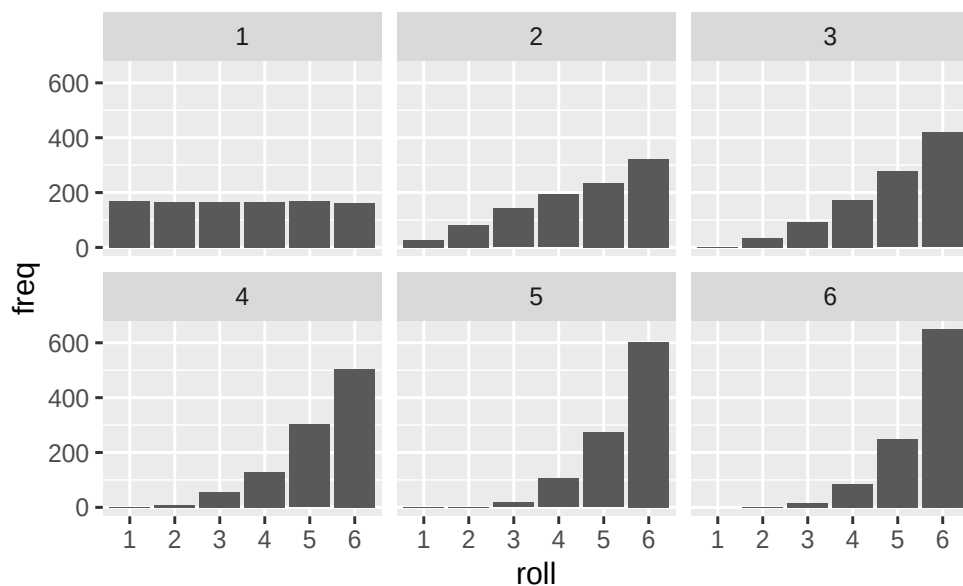
Then this:

```
result = map(1:6, one_trial)
result = bind_rows(result)
result$ndice = 1:6
result
```

```
A tibble: 6 x 7
 `1` `2` `3` `4` `5` `6` ndice
 <table[1d]> <table[1d]> <table[1d]> <table[1d]> <table[1d]> <table[1d]> <int>
1 170 167 165 167 170 161 1
2 26 81 142 193 235 323 2
3 2 34 94 174 277 419 3
4 1 8 57 128 302 504 4
5 1 1 18 105 273 602 5
6 NA 1 17 85 248 649 6
```

We can then make a nice plot:

```
resL = pivot_longer(result, `1`:`6`,
 names_to="roll", values_to="freq") %>%
 filter(!is.na(freq))
resL$freq = as.numeric(resL$freq)
ggplot(resL, aes(roll, freq)) +
 facet_wrap(~ndice) +
 geom_col()
```



**Note.** The `as.numeric` line is because the `table` command makes table objects which are really just lists of numbers, but it seems cleaner to tell R to knock it off and just let it be a number. Not doing this still works fine, it just gives a warning in `ggplot()`

## 39 Advanced stuff

In the following we look at a few cool functions that help with mapping.

### 39.1 relocate()

Not really mapping specific, but still a nice way to move a variable to the start of a data frame.

```
relocate(resL, freq)
```

```
A tibble: 35 x 3
 freq ndice roll
 <dbl> <int> <chr>
1 170 1 1
2 167 1 2
3 165 1 3
4 167 1 4
5 170 1 5
6 161 1 6
7 26 2 1
8 81 2 2
9 142 2 3
10 193 2 4
i 25 more rows
```

### 39.2 pull()

This will grab a column from a data frame in a list of pipe commands, which can make it easier to plug into some other tools.

Here we pull the roll column and then hand it to the `table()` command to count the number of instances of each roll.

```
resL %>% pull(roll) %>%
 table()
```

```
.
1 2 3 4 5 6
5 6 6 6 6 6
```

### 39.3 set\_names( list, names )

On the fly name a list before handing to map!

```
dice = 2:6
dice %>% set_names(paste0(dice, " dice")) %>%
 map_df(roll_dice_extended, .id = "scenario")
```

	scenario	mean	median	max
1	2 dice	3.000000	3.0	5
2	3 dice	3.333333	3.0	4
3	4 dice	2.500000	1.5	6
4	5 dice	3.400000	3.0	4
5	6 dice	3.833333	4.0	6

### 39.4 unpack() and pack()

These will translate a data frame column into individual columns. You can end up with a data frame column if you use map to make a new column in your data:

```
scenarios = tibble(n_dice = 1:6)
scenarios = scenarios %>%
 mutate(result = map_df(n_dice, roll_dice_extended))
scenarios
```

```
A tibble: 6 x 2
 n_dice result$mean $median $max
 <int> <dbl> <dbl> <int>
1 1 4 4 4
2 2 3.5 3.5 4
3 3 1.67 2 2
4 4 4.25 5 6
5 5 3.8 4 5
6 6 3.17 4 4
```

Notice the weird \$ in the printout? This is because the three columns are inside the dataframe of `result`. You can unpack it like so:

```
unpack(scenarios, result)
```

```
A tibble: 6 x 4
 n_dice mean median max
 <int> <dbl> <dbl> <int>
1 1 4 4 4
2 2 3.5 3.5 4
3 3 1.67 2 2
4 4 4.25 5 6
5 5 3.8 4 5
6 6 3.17 4 4
```

to get your nice, normal dataframe.

## 40 Exploring map and friends (tidyverse)

## 41 Overview

This document pulls out elements from R for DS Chapter 21, which is on ways of doing iteration. And then it jumps to the `map()` function and list columns extensively used in Chapter 25. It is all within the tidyverse:

```
library(tidyverse)
```

## 42 The map() function (R for DS 21.5)

The core idea of `map` is to take a list and call a function on each thing on that list. This list could be a list of lists (i.e. a dataframe), which can be very important, because then you can call a function on each column in your dataframe.

Generally what `map` gives back is another list, with each element being what we got back from the function from each call. If your first list has named items, your result will have those names as well. This can be quite useful.

To illustrate we make a silly list with a list inside it, and then call `map()`:

```
lst = list(1, B=3, inner_list = c(10, 20, 30))
```

```
lst
```

```
[[1]]
```

```
[1] 1
```

```
$B
```

```
[1] 3
```

```
$inner_list
```

```
[1] 10 20 30
```

```
map(lst, mean)
```

```
[[1]]
```

```
[1] 1
```

```
$B
```

```
[1] 3
```

```
$inner_list
```

```
[1] 20
```

See? We got the mean out of each element in our list.

By adding `_dbl` or similar, we can explicitly tell `map()` to turn this list into a vector of some type:



```
map_dbl(lst, mean)
```

	B	inner_list
1	3	20

We also have `map_int`, `map_chr`, `map_lgl`, and `map_df`. Be warned, if you call one of these and the things you get back are the wrong type, you get an error:

```
map_df(lst, mean)
```

```
Error in `dplyr::bind_rows()`:
! Argument 1 must be a data frame or a named atomic vector.
```

## 42.1 Making data frames

The `map_df()` is especially important. But you need something that gives back a dataframe for it to work! This dataframe can be a 1 row dataframe, however. For example:

```
my.func = function(numbers) {
 data.frame(mean=mean(numbers), sd=sd(numbers))
}
map_df(lst, my.func)
```

	mean	sd
1	1	NA
2	3	NA
3	20	10

## 42.2 The `unlist()` function

A useful function is `unlist` which takes a bunch of lists and just smashes it all together into one vector

```
unlist(lst)
```

	B	inner_list1	inner_list2	inner_list3
1	3	10	20	30

It even keeps the names, if there are any!

## 42.3 Extra arguments to map

If your function takes extra stuff, then you can pass it along like so:

```
log(10, base=3)
```

```
[1] 2.095903
```

```
map(lst, log, base=3)
```

```
[[1]]
```

```
[1] 0
```

```
$B
```

```
[1] 1
```

```
$inner_list
```

```
[1] 2.095903 2.726833 3.095903
```

## 43 Banging on Dataframes

If you have a dataframe that is a bunch of smaller dataframes, you can get a list of the smaller dataframes via `split()`. This will make a new dataframe with the rows corresponding to each unique value of a passed variable:

```
df = data.frame(A=1:6, B=c("A","A","A","B","B","C"))
df.list = split(df, df$B)
df.list
```

```
$A
```

```
 A B
1 1 A
2 2 A
3 3 A
```

```
$B
```

```
 A B
4 4 B
5 5 B
```

```
$C
```

```
 A B
6 6 C
```

```
class(df.list)
```

```
[1] "list"
```

```
map_dbl(df.list, nrow)
```

```
 A B C
3 2 1
```

See R for DS 25.5.1

## 43.1 Chaining map

You can chain maps together. Say we are going to fit a linear model for each group of a dataframe defined by different values of a categorical variable. For example, let's divide the famous “iris data” into groups by species, and then fit a linear model for each group, and then get the R squared from that model.

Before we dive into this, we need to note that you can use “.” as a pronoun for “the thing we are working on.” So, for example, the following two lines are equivalent:

```
iris.list = split(iris, iris$Species)
iris.list = iris %>% split(.$Species)
```

The “.” is what is “coming down the pipe.”

Now we call `lm()` on each part by writing a function to analyze our dataframe:

```
my.func = function(my.df) {
 lm(Sepal.Length ~ Sepal.Width, data=my.df)
}
```

```
mods = iris.list %>% map(my.func)
```

Our `mods` is a list of results. Here is the first result (a linear regression output):

```
mods[[1]]
```

Call:

```
lm(formula = Sepal.Length ~ Sepal.Width, data = my.df)
```

Coefficients:

```
(Intercept) Sepal.Width
 2.6390 0.6905
```

The above can be bundled as follows

```
mods = iris %>% split(.$Species) %>% map(my.func)
```

But writing the wrapper function for `lm` is annoying. We can tell our `map` that we are “writing a function on the fly” with an abuse of the “~” notation:

```
mods = iris %>% split(.$Species) %>%
 map(~lm(Sepal.Length ~ Sepal.Width, data=.))
```

Here we are saying, inside the `map`, that we want to run the `lm()` function, and that the stuff we got from the map should be the data argument (`data = .`).

Finally, we map the above again to `summary` to calculate our R squared, and then we extract it:

```
names(summary(mods[[1]]))

[1] "call" "terms" "residuals" "coefficients"
[5] "aliased" "sigma" "df" "r.squared"
[9] "adj.r.squared" "fstatistic" "cov.unscaled"
```

```
rs = iris %>% split(.$Species) %>%
 map(~lm(Sepal.Length ~ Sepal.Width, data=.)) %>%
 map(summary) %>%
 map_dbl(~.$r.squared)
rs

 setosa versicolor virginica
0.5513756 0.2765821 0.2090573
```

The `~.$r.squared` line is a bit weird. We are taking our pronoun, and grabbing the “r.squared” variable from inside it (if there is no such value inside it we will get NA). This is a common activity, and thus the tidyverse provides a shortcut: just put in the name as a string:

```
rs = iris %>% split(.$Species) %>%
 map(~lm(Sepal.Length ~ Sepal.Width, data=.)) %>%
 map(summary) %>%
 map_dbl("r.squared")
rs

 setosa versicolor virginica
0.5513756 0.2765821 0.2090573
```

## 43.2 Dataframes are lists of the columns

Let’s calculate the unique number of values in each column of iris:

```
analyze_column = function(col) {
 length(unique(col))
}
map_dbl(iris, analyze_column)
```

Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
35	23	43	22	3

Or, by map chaining:

```
iris %>% map(unique) %>% map_dbl(length)
```

Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
35	23	43	22	3

### 43.3 The transpose() function

We can do some funky list manipulation. Say our map gives us back a list of pairs (note cool `select_if()` to grab only numeric columns):

```
my.func = function(col) {
 c(mean=mean(col), sd=sd(col))
}
res <- iris %>% select_if(is.numeric) %>% map(my.func)
res
```

```
$Sepal.Length
 mean sd
5.843333 0.8280661
```

```
$Sepal.Width
 mean sd
3.057333 0.4358663
```

```
$Petal.Length
 mean sd
3.758000 1.765298
```

```
$Petal.Width
 mean sd
1.199333 0.7622377
```

But this is hard to use. `transpose()` to the rescue!

```
res2 <- transpose(res)
names(res2)
```

```
[1] "mean" "sd"
```

```
res2$mean
```

```
$Sepal.Length
```

```
[1] 5.843333
```

```
$Sepal.Width
```

```
[1] 3.057333
```

```
$Petal.Length
```

```
[1] 3.758
```

```
$Petal.Width
```

```
[1] 1.199333
```

We now have two lists, one for the means, one for the sds.

See R for DS 21.6 for more on this.

## 44 List columns and notes on R for DS 25 (25.2)

Chapter 25, while nominally about modeling, actually shows a lot of useful code for complex data wrangling. In particular it talks about list columns, which are ways of storing data in chunks and manipulating the data chunk-by-chunk. This connects heavily with the `map` concepts above.

### 44.1 List columns

A “list column” is a column of your dataframe where each entry in the column can be anything you want, including a list or entire dataframe. For example, we can make some on the fly using `enframe()` (See R for DS 25.4.4)

```
x <- list(a=c("a","b"),
 b = 3:7,
 c = iris
)
```

```
df <- enframe(x)
df
```

```
A tibble: 3 x 2
 name value
 <chr> <list>
1 a <chr [2]>
2 b <int [5]>
3 c <df [150 x 5]>
```

There are several ways to make list columns to be aware of:

- Using `nest()`
- Using `mutate()` where your call gives a list back (e.g., due to `map()`)
- Using `summarise()` with a function that gives back multiple results (such as `quantile()`).



## 44.2 nest() and unnest()

In particular, you can group a dataframe by something, and then nest it to make a small dataframe with each row corresponding to a single dataframe slice of your original data

```
dfs <- iris %>% group_by(Species) %>% nest()
dfs
```

```
A tibble: 3 x 2
Groups: Species [3]
 Species data
 <fct> <list>
1 setosa <tibble [50 x 4]>
2 versicolor <tibble [50 x 4]>
3 virginica <tibble [50 x 4]>
```

```
look, our 2nd value in our dataframe's data variable is a
dataframe!
head(dfs$data[[2]], 3)
```

```
A tibble: 3 x 4
 Sepal.Length Sepal.Width Petal.Length Petal.Width
 <dbl> <dbl> <dbl> <dbl>
1 7 3.2 4.7 1.4
2 6.4 3.2 4.5 1.5
3 6.9 3.1 4.9 1.5
```

Now you can use map to make new columns!

```
dfs %>% mutate(n = map_dbl(data, nrow))
```

```
A tibble: 3 x 3
Groups: Species [3]
 Species data n
 <fct> <list> <dbl>
1 setosa <tibble [50 x 4]> 50
2 versicolor <tibble [50 x 4]> 50
3 virginica <tibble [50 x 4]> 50
```

```
dfs2 <- dfs %>% mutate(mod = map(data, ~lm(Sepal.Length ~ Sepal.Width, data = .)))
dfs2
```

```
A tibble: 3 x 3
Groups: Species [3]
 Species data mod
 <fct> <list> <list>
1 setosa <tibble [50 x 4]> <lm>
2 versicolor <tibble [50 x 4]> <lm>
3 virginica <tibble [50 x 4]> <lm>
```

```
map(dfs2$mod, "coefficients")
```

```
[[1]]
(Intercept) Sepal.Width
 2.6390012 0.6904897
```

```
[[2]]
(Intercept) Sepal.Width
 3.5397347 0.8650777
```

```
[[3]]
(Intercept) Sepal.Width
 3.9068365 0.9015345
```

The `nest()` method has `unnest()` which will unstack an internal list, repeating the row the list came from over and over to make a full set. Witness

```
dfs3 <- dfs2 %>% mutate(coefs = map(mod, "coefficients"))
dfs3
```

```
A tibble: 3 x 4
Groups: Species [3]
 Species data mod coefs
 <fct> <list> <list> <list>
1 setosa <tibble [50 x 4]> <lm> <dbl [2]>
2 versicolor <tibble [50 x 4]> <lm> <dbl [2]>
3 virginica <tibble [50 x 4]> <lm> <dbl [2]>
```

```
unnest(dfs3, coefs)
```

```
A tibble: 6 x 4
Groups: Species [3]
 Species data mod coefs
 <fct> <list> <list> <dbl>
1 setosa <tibble [50 x 4]> <lm> <dbl>
2 setosa <tibble [50 x 4]> <lm> <dbl>
3 versicolor <tibble [50 x 4]> <lm> <dbl>
4 versicolor <tibble [50 x 4]> <lm> <dbl>
5 virginica <tibble [50 x 4]> <lm> <dbl>
6 virginica <tibble [50 x 4]> <lm> <dbl>
```

```

1 setosa <tibble [50 x 4]> <lm> 2.64
2 setosa <tibble [50 x 4]> <lm> 0.690
3 versicolor <tibble [50 x 4]> <lm> 3.54
4 versicolor <tibble [50 x 4]> <lm> 0.865
5 virginica <tibble [50 x 4]> <lm> 3.91
6 virginica <tibble [50 x 4]> <lm> 0.902

```

See Chapter 25 of R for DS for examples of getting `unnest()` to work, including the fiddly stuff to get the names right. It can be a frustrating pain.

### 44.3 Using `summarise()`

We can call `summarise()` with functions that give multiple things. We need to wrap the response in a list.

```

ir2 <- iris %>% group_by(Species) %>% summarise(probs = list(c(0, 25, 50, 75, 100)),
 quants = list(quantile(Sepal.Length)))
ir2

```

```

A tibble: 3 x 3
 Species probs quants
 <fct> <list> <list>
1 setosa <dbl [5]> <dbl [5]>
2 versicolor <dbl [5]> <dbl [5]>
3 virginica <dbl [5]> <dbl [5]>

```

Note the wrapping of everything in the list. Now each row has a list of the probabilities and a list of the results. We can get back a long frame with `unnest()`:

```
unnest(ir2)
```

Warning: `cols` is now required when using `unnest()`.  
 i Please use `cols = c(probs, quants)`.

```

A tibble: 15 x 3
 Species probs quants
 <fct> <dbl> <dbl>
1 setosa 0 4.3
2 setosa 25 4.8
3 setosa 50 5

```

4	setosa	75	5.2
5	setosa	100	5.8
6	versicolor	0	4.9
7	versicolor	25	5.6
8	versicolor	50	5.9
9	versicolor	75	6.3
10	versicolor	100	7
11	virginica	0	4.9
12	virginica	25	6.22
13	virginica	50	6.5
14	virginica	75	6.9
15	virginica	100	7.9

See R for DS Chapter 25.4 for lots of examples and discussion.

**Part V**

**On Missing Data**

# 45 An Introduction to Missing Data

## 45.1 Introduction

Handling missing data is the icky, unglamorous part of any statistical analysis. It is where the skeletons lie. There's a range of options available, which are, broadly speaking:

1. Delete the observations with missing covariates (this is a “complete case analysis”)
2. Plug in reasonable values for the missing values. This is called “imputation.” We discuss three ways of doing this that are increasingly sophisticated and layered on each other:
  - a. Mean imputation. Simply take the mean of all the observations where you know the value, and then use that for anything that is missing.
  - b. Regression imputation. Generate regression equations describing how all the variables are connected, and use those to predict any missing value.
  - c. Stochastic regression imputation. Use regression imputation, but also add some residual noise to all our imputed values so that our imputed values have as much variation as our actual values (otherwise our imputed values will tend to be all clumped together).
3. Multiply impute the missing data, by fully modeling the covariate and the missingness, and generating a range of complete datasets under this model. Here you end up with a collection of complete datasets that are all “reasonable guesses” as to what the full dataset might have been. You then analyze each one, and aggregate your findings across them to get a final answer.

The first two general approaches are imperfect, while the third is often more work than the original analysis that we were hoping to perform. For this course, doing a 2a, 2b, or 2c are all reasonable choices. If you have very little missing data you can often get away with #1. We have no expectations that people will take the plunge into #3 (multiple imputation). In real life, people will often analyze their data with a complete case analysis and some other strategy, and then compare the results. In Education, if missingness is below 10% people usually just do mean imputation, but regression imputation would probably be superior.

This handout provides an introduction to missing data, and includes a few commands to explore and deal with missing data. In this document we first talk about exploring missing data (in particular getting plots that show you if you have any notable patterns in how things are missing) and then we give a brief walk-through of the 3 methods listed above.

We will use the `mice` and `VIM` packages, which you can install using `install.packages()` if you have not yet done so. These are simple and powerful packages for visualizing and imputing missing data. At the end of this document we also describe the `Amelia` package.

```
library(tidyverse)
library(mice)
library(VIM)
```

Throughout we use a small built-in R dataset on air quality as a working example.

```
data(airquality)
nrow(airquality)
```

```
[1] 153
```

```
head(airquality)
```

	Ozone	Solar.R	Wind	Temp	Month	Day
1	41	190	7.4	67	5	1
2	36	118	8.0	72	5	2
3	12	149	12.6	74	5	3
4	18	313	11.5	62	5	4
5	NA	NA	14.3	56	5	5
6	28	NA	14.9	66	5	6

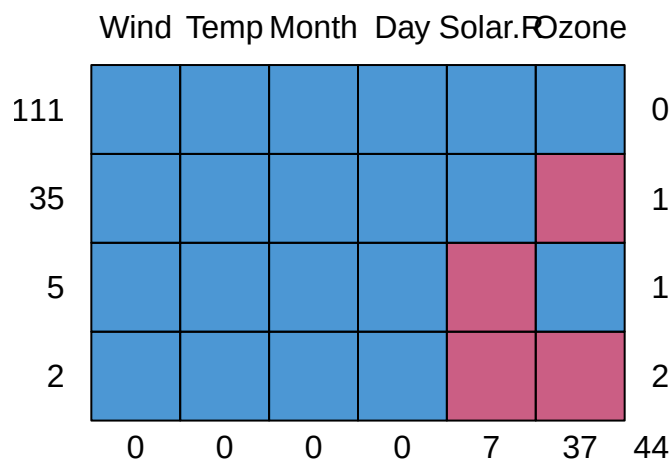
```
summary(airquality[1:4])
```

Ozone		Solar.R		Wind		Temp	
Min.	: 1.0	Min.	: 7	Min.	: 1.70	Min.	:56.0
1st Qu.:	18.0	1st Qu.:	116	1st Qu.:	7.40	1st Qu.:	72.0
Median :	31.5	Median :	205	Median :	9.70	Median :	79.0
Mean :	42.1	Mean :	186	Mean :	9.96	Mean :	77.9
3rd Qu.:	63.2	3rd Qu.:	259	3rd Qu.:	11.50	3rd Qu.:	85.0
Max.	:168.0	Max.	:334	Max.	:20.70	Max.	:97.0
NA's	:37	NA's	:7				

## 45.2 Visualizing missing data

Just like with anything in statistics, the first thing to do is to look at our data. We want to know which variables are often missing, and if some variables are often missing together. We also want to know how much data is missing. The `mice` package has a variety of plots to show us patterns of missingness:

```
md.pattern(airquality)
```



```

 Wind Temp Month Day Solar.R Ozone
111 1 1 1 1 1 1 0
35 1 1 1 1 1 0 1
5 1 1 1 1 0 1 1
2 1 1 1 1 0 0 2
 0 0 0 0 7 37 44

```

This plot gives us the different missing data patterns and the number of observations that have each missing data pattern. For example, the second row in the plot says there are 35 observations that have a missing data pattern where only Ozone is missing.

We can also just look at 10 observations to see everything that is going on. Here we take the first 10 rows of our dataset, but could also take a random 10 row with the tidyverse's `sample_n` method. We will use this little dataset to really dig into some of the methods.

```
airqualitysub = airquality[1:10,]
airqualitysub
```

```

 Ozone Solar.R Wind Temp Month Day
1 41 190 7.4 67 5 1
2 36 118 8.0 72 5 2
3 12 149 12.6 74 5 3
4 18 313 11.5 62 5 4
5 NA NA 14.3 56 5 5
6 28 NA 14.9 66 5 6
7 23 299 8.6 65 5 7

```



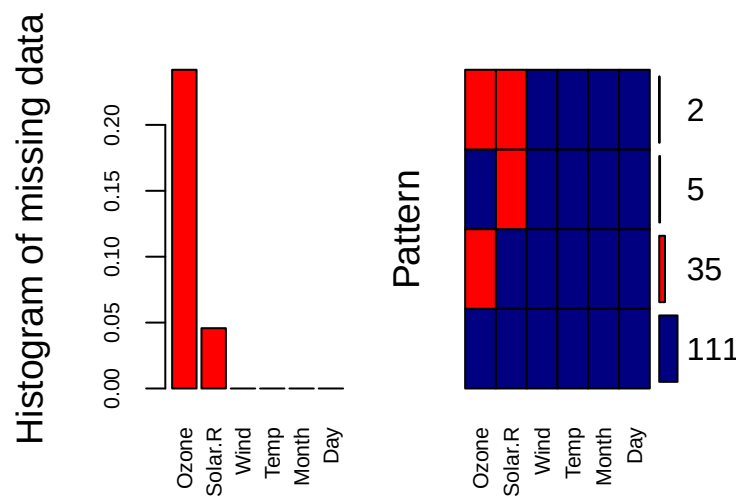
8	19	99	13.8	59	5	8
9	8	19	20.1	61	5	9
10	NA	194	8.6	69	5	10

In our tiny data, we have one observation missing two covariates and one each of missing `Ozone` only and `Solar.R` only.

### 45.2.1 Missing data plots with the VIM Package

The VIM package gives some alternate plots to explore missing data patterns. For example, `aggr()`:

```
aggr(airquality, col=c('navyblue','red'),
 numbers=TRUE, sortVars=TRUE, labels=names(data),
 cex.axis=.7, gap=3, prop=c(TRUE, FALSE),
 ylab=c("Histogram of missing data","Pattern"))
```

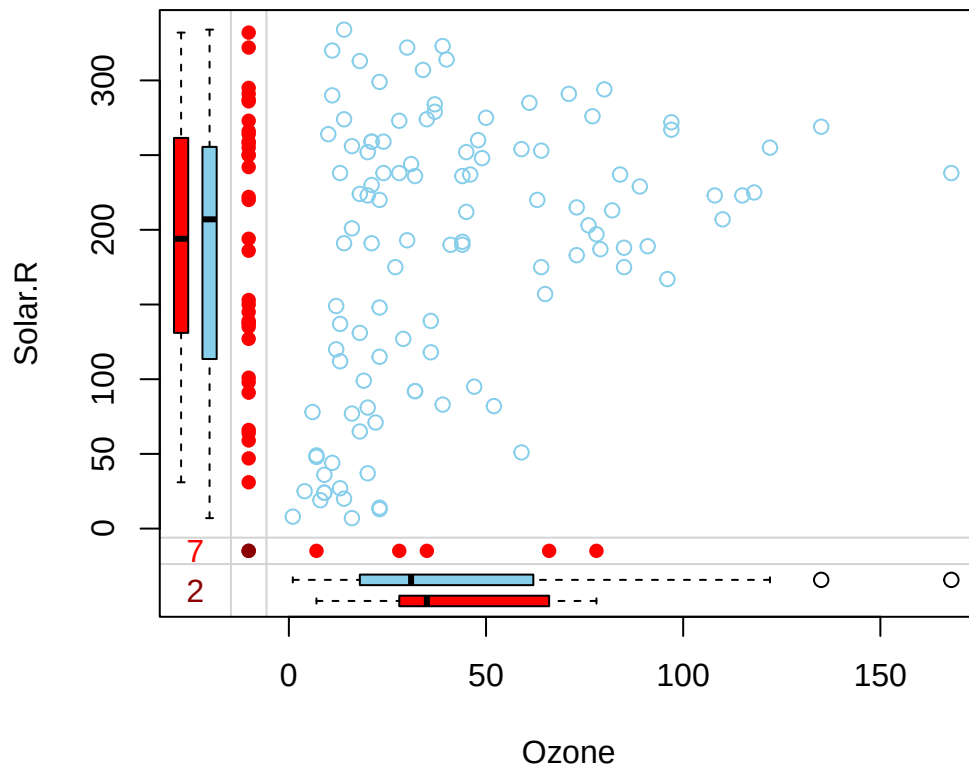


Variables sorted by number of missings:

Variable	Count
Ozone	0.2418
Solar.R	0.0458
Wind	0.0000
Temp	0.0000
Month	0.0000
Day	0.0000

On the left, we have the proportion of missing data for each variable in our dataset; We can see that `Ozone` and `Solar.R` have missing values, and `Ozone` is clearly missing at higher rates than `Solar.R`. On the right, we have the joint distribution of missingness. We can see that 111 observations have no missing values. From those with missing values, the majority have missing values for `Ozone`, some have missing values for `Solar.R` and only 2 observations have missing values for both `Ozone` and `Solar.R`.

```
marginplot(airquality[1:2])
```



Here we have a scatterplot for the first two variables in our dataset: `Ozone` and `Solar.R`. These are the variables that have missing data. In addition to the standard scatterplot we are familiar with, information about missingness is shown in the margins. The red dots indicate observations with one or both values missing (so there can be a bunch of dots stacked up in the bottom-left corner). The numbers (37, 7, and 2 tells us how many observations are missing either or both of these variables).

## 45.3 Complete case analysis

Working with complete cases (dropping observations with any missing data on our outcome and predictors) is always an option. We have been doing this in class and section. However, this can lead to substantial data loss, if we have a lot of missingness and it can heavily bias our results depending on why observations are missing.

Complete case analysis is the R default.

```
fit <- lm(Ozone ~ Wind, data = airquality)
summary(fit)
```

Call:

```
lm(formula = Ozone ~ Wind, data = airquality)
```

Residuals:

Min	1Q	Median	3Q	Max
-51.57	-18.85	-4.87	15.23	90.00

Coefficients:

	Estimate	Std. Error	t value	Pr(> t )
(Intercept)	96.87	7.24	13.38	< 2e-16 ***
Wind	-5.55	0.69	-8.04	9.3e-13 ***

---

Signif. codes: 0 '\*\*\*' 0.001 '\*\*' 0.01 '\*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 26.5 on 114 degrees of freedom

(37 observations deleted due to missingness)

Multiple R-squared: 0.362, Adjusted R-squared: 0.356

F-statistic: 64.6 on 1 and 114 DF, p-value: 9.27e-13

Note the listing in the summary of number of items deleted. You can find out which rows were deleted:

```
which rows/observations were deleted
```

```
deleted <- na.action(fit)
```

```
deleted
```

```
5 10 25 26 27 32 33 34 35 36 37 39 42 43 45 46 52 53 54 55
5 10 25 26 27 32 33 34 35 36 37 39 42 43 45 46 52 53 54 55
56 57 58 59 60 61 65 72 75 83 84 102 103 107 115 119 150
```

```

56 57 58 59 60 61 65 72 75 83 84 102 103 107 115 119 150
attr(,"class")
[1] "omit"

```

```

naprint(deleted)

```

```

[1] "37 observations deleted due to missingness"

```

We have more incomplete rows if we add Solar.R as predictor.

```

fit2 <- lm(Ozone ~ Wind+Solar.R, data=airquality)
naprint(na.action(fit2))

```

```

[1] "42 observations deleted due to missingness"

```

We can also drop observations with missing data ourselves instead of letting R do it for us. **Dropping data preemptively is generally a good idea, especially if you plan on using predict().**

```

complete cases on all variables in the data set
complete.v1 = filter(airquality, complete.cases(airquality))

drop observations with missing values, but ignoring a specific
variable
complete.v2 = filter(airquality,
 complete.cases(select(airquality, -Wind)))

drop observations with missing values on a specific variable
complete.v3 = filter(airquality, !is.na(Ozone))

```

Once you have subset your data, you just analyze what is left as normal. Easy as pie!

## 45.4 Mean imputation

Instead of dropping observations with missing values, we can plug in some kind of reasonable value for the missing value, e.g. the grand/global mean. While this can be statistically questionable, it does allow us to use the information provided by that unit's outcome and other covariates, without, we hope, unduly affecting the analysis of the missing covariate.

Generally, people will first plug in the mean value for anything missing, but then also make a dummy variable of whether that observation had a missing value there (or sometimes any missing value). You would then include both the original vector of covariates (with the means plugged in) along with the dummy variable in subsequent regressions and analyses.

### 45.4.1 Doing Mean Imputation manually

Manually, we can just replace missing values for a variable with the grand/global mean.

```
make a new copy of the data
data.mean.impute = airquality

select the observations with missing Ozone
miss.ozone = is.na(data.mean.impute$Ozone)

replace those NAs with mean(Ozone)
data.mean.impute[miss.ozone, "Ozone"] = mean(airquality$Ozone, na.rm=TRUE)
```

In a multi-level context, it might make more sense to impute using the group mean rather than the grand mean. Here's a generic function to do it. Here we group by month:

```
a function that replaces missing values in a vector
by the mean of the other values
mean.impute = function(y) {
 y[is.na(y)] = mean(y, na.rm=TRUE)
 return(y)
}

data.mean.impute = airquality %>% group_by(Month) %>%
 mutate(Ozone = mean.impute(Ozone),
 Solar.R = mean.impute(Solar.R))
```

We have mean imputed the Ozone column and the Solar.R column

### 45.4.2 Mean imputation with the Mice package

We can use the mice package to do mean imputation. The mice package is a package that can do some quite complex imputation, and so when you call `mice()` (which says “impute missing values please”) you get back a rather complex object telling you what mice imputed, for whom, etc. This object, which is a `mids` object (see `help(mids)`), contains the multiply imputed dataset (or in our case, so far, singly imputed). The mice package then provides a lot of nice functions allowing you to get your imputed information out of this object.

We first demonstrate this for the smaller 10 observations dataset we made above. Mice is generally going to be a two-step process: impute data, get completed dataset.

For step 1:

```
imp <- mice(airqualitysub, method="mean", m=1, maxit=1)
```

```

iter imp variable
1 1 Ozone Solar.R

```

Warning: Number of logged events: 1

```

imp

```

Class: mids

Number of multiple imputations: 1

Imputation methods:

Ozone	Solar.R	Wind	Temp	Month	Day
"mean"	"mean"	" "	" "	" "	" "

PredictorMatrix:

	Ozone	Solar.R	Wind	Temp	Month	Day
Ozone	0	1	1	1	0	1
Solar.R	1	0	1	1	0	1
Wind	1	1	0	1	0	1
Temp	1	1	1	0	0	1
Month	1	1	1	1	0	1
Day	1	1	1	1	0	0

Number of logged events: 1

it	im	dep	meth	out
1	0	0	constant	Month

For step 2:

```

cmp = complete(imp)
cmp

```

	Ozone	Solar.R	Wind	Temp	Month	Day
1	41.0	190	7.4	67	5	1
2	36.0	118	8.0	72	5	2
3	12.0	149	12.6	74	5	3
4	18.0	313	11.5	62	5	4
5	23.1	173	14.3	56	5	5
6	28.0	173	14.9	66	5	6
7	23.0	299	8.6	65	5	7
8	19.0	99	13.8	59	5	8
9	8.0	19	20.1	61	5	9
10	23.1	194	8.6	69	5	10

We see there are no missing values in `cmp`. They were all imputed with the mean of the other non-missing values. This is **mean imputation**.

Now let's impute the full dataset.

```
imp <- mice(airquality, method="mean", m=1, maxit=1)
```

```
iter imp variable
1 1 Ozone Solar.R
```

```
cmp = complete(imp)
```

We next make a dummy variable for each row of our data noting whether anything was imputed or not. We use the `ici` (Incomplete Case Indication) function to list all rows with any missing values.

```
head(ici(airquality))
```

```
[1] FALSE FALSE FALSE FALSE TRUE TRUE
```

Note how we have a TRUE or FALSE for each row of our data.

We then store our `ici` values as a covariate in our completed dataset:

```
cmp$imputed = ici(airquality)
head(cmp)
```

	Ozone	Solar.R	Wind	Temp	Month	Day	imputed
1	41.0	190	7.4	67	5	1	FALSE
2	36.0	118	8.0	72	5	2	FALSE
3	12.0	149	12.6	74	5	3	FALSE
4	18.0	313	11.5	62	5	4	FALSE
5	42.1	186	14.3	56	5	5	TRUE
6	28.0	186	14.9	66	5	6	TRUE

#### 45.4.2.1 How well did mean imputation work?

Mean imputation has problems. The imputed values will all be the same, and thus when we look at how much variation is in our variables after imputation, it will go down. Compare the SD of our completed dataset Ozone values to the SD of the Ozone values for our non-missing values.

```
sd(airquality$Ozone, na.rm=TRUE)
```

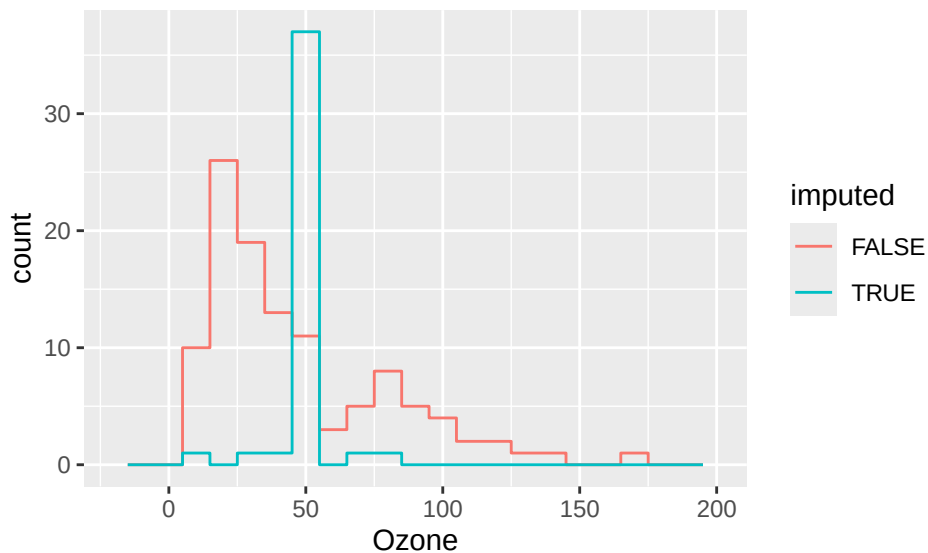
```
[1] 33
```

```
sd(cmp$Ozone)
```

```
[1] 28.7
```

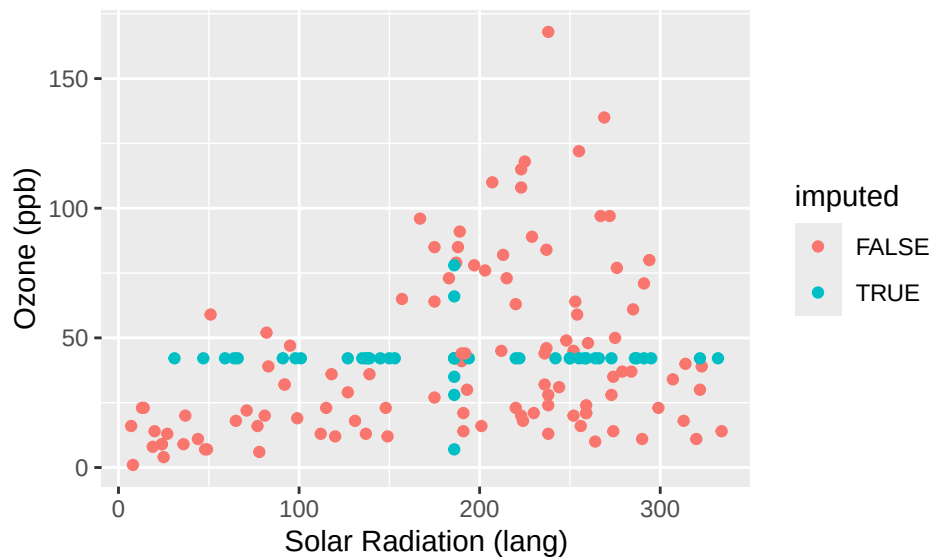
Next, let's look at some plots of our completed data, coloring the points by whether they were imputed.

```
library(ggplot2)
ggplot(cmp, aes(x=Ozone, col=imputed)) +
 stat_bin(geom="step", position="identity",
 breaks=seq(-20, 200, 10))
```



```
ggplot(cmp, aes(y=Ozone, x=Solar.R, col=imputed)) +
 geom_point() +
 labs(y="Ozone (ppb)", x="Solar Radiation (lang)")
```





What we see in the above plots is that our imputed observations do not look like the rest of our data because one (or both) of their values always is in the exact center. This creates the “+” shape. It also gives the big spike at the mean for the histogram.

#### 45.4.2.2 Important Aside: Namespaces and function collisions

We now need to discuss a sad aspect of R. The short story is, different packages have functions with the same names and so if you have both packages loaded you will need to specify which package to use when calling such a function. You can do this by giving the “surname” of the function at the beginning of the function call. Namespace collision comes up here because the method `complete()` exists both in the tidyverse and in mice. In tidyverse, `complete()` fills in rows of missing combinations of values. In mice, `complete()` gives us a completed dataset after we have made an imputation call.

It turns out that since we loaded tidyverse first and mice second, the mice’s `complete()` method is the default. But if we loaded the packages in the other order, we would get strange errors. To be clear, we can tell R to use mice’s `complete()` by writing

```
cmp = mice::complete(imp)
```

In general, you can detect such “namespace collisions” by noticing weird error messages all of a sudden when you don’t expect them. You can then type `help( complete )` in the console and the help index page will list all the different `completes` that have been loaded.

Also when you load a package it will write down what functions are getting mixed up for you. If you were looking at your R console output you should see something like this:

```
tidyr::complete() masks mice::complete()
```

## 45.5 Regression imputation

Regression imputation is half way between mean imputation and multiple imputation. In regression imputation we predict what values we expect for anything missing based on the other values of the observation. For example, if, in the nonmissing part of our data, urban/rural is correlated with race, we might impute a different value for race if we know an observation came from an urban environment vs. rural. We do this with regression: we fit a model predicting each variable using the others and then use that regression model to predict any missing values.

We can do this manually, but then it gets very hard when multiple variables are missing for a given observation. The mice package is more clever: it does variables one at a time, and the cycles around so everything can get imputed using the completed data from earlier iterations.

### 45.5.1 Manually

Here is how to use other variables to predict missing values.

```
ic(airqualitysub)
```

	Ozone	Solar.R	Wind	Temp	Month	Day
5	NA	NA	14.3	56	5	5
6	28	NA	14.9	66	5	6
10	NA	194	8.6	69	5	10

```
fit <- lm(Ozone ~ Solar.R, data=airqualitysub)
```

```
predict for missing ozone
need.pred = subset(airqualitysub, is.na(Ozone))
need.pred
```

	Ozone	Solar.R	Wind	Temp	Month	Day
5	NA	NA	14.3	56	5	5
10	NA	194	8.6	69	5	10

```
pred <- predict(fit, newdata=need.pred)
pred
```

5	10
NA	23.1

But now we have to merge our imputed data back in, and we still haven't solved for observation 5 because we are missing the variable we would use to predict the other missing variable! Ick. Multiple missing values on multiple variables is where missing data gets *really* hard. So let's quit now and turn to a package that will handle all of this for us.

## 45.5.2 Mice

To do regression imputation using mice, we simply call the `mice()` method:

```
imp <- mice(airquality[,1:2], method="norm.predict",
 m=1, maxit=3)
```

```
iter imp variable
1 1 Ozone Solar.R
2 1 Ozone Solar.R
3 1 Ozone Solar.R
```

The `m=1` means “make 1 imputed dataset.” The `maxit=3` means “Do a maximum of 3 cycles of imputation (so each variable could be imputed up to three times).”

Now `imp` is just like with mean imputation and so we have everything! How did it do it? By *chaining equations*. First `mice` starts with mean imputation. This gives an initial complete dataset. Then `mice` uses a prediction model to re-predict for the missing values for one covariate. We now have a new and improved (and still complete) dataset. We use this *new* dataset to run a prediction model on the next covariate, and so forth. We cycle around a few times and then everything is jointly predicting everything else.

The `complete()` method gives us the final complete dataset with everything imputed. Like so:

```
cdat = mice::complete(imp)
head(cdat)

 Ozone Solar.R
1 41.0 190
2 36.0 118
3 12.0 149
4 18.0 313
5 42.5 186
6 28.0 169

nrow(cdat)
```

```
[1] 153
```

```
nrow(airquality)
```

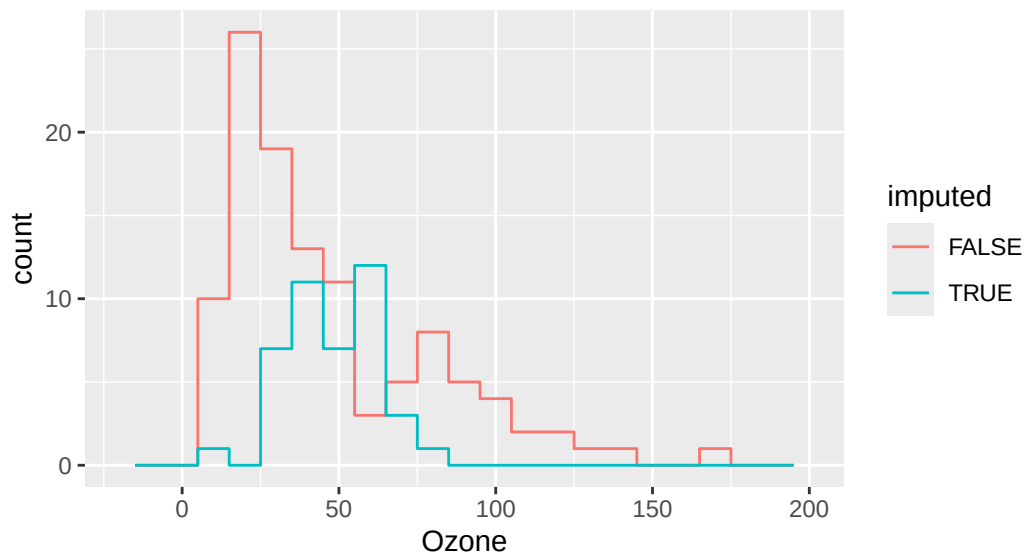
```
[1] 153
```

Next we make a variable of which cases have imputed values and not (any row with missing data must have been partially imputed.)

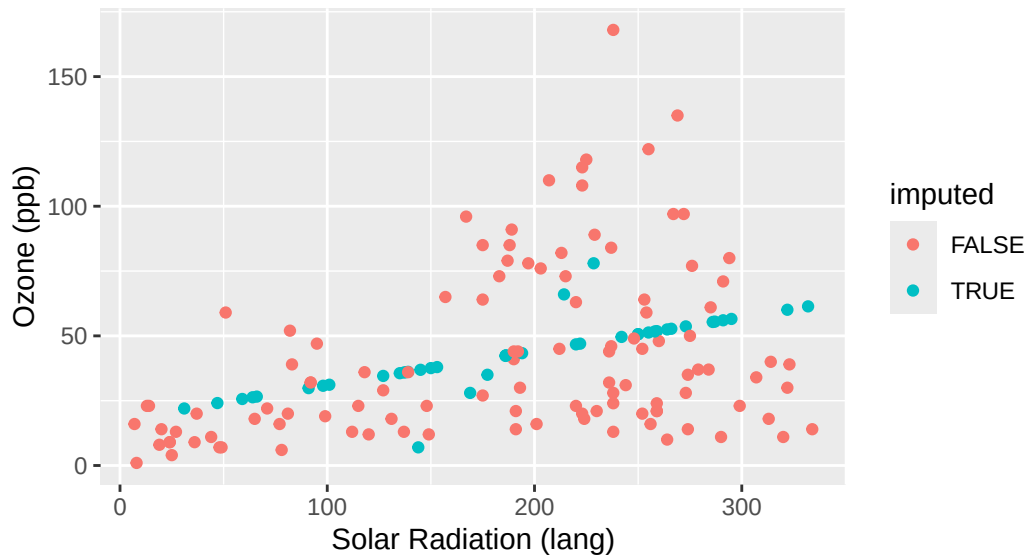
```
cdat$imputed = ici(airquality)
```

And see our results! Compare to mean imputation, above.

```
ggplot(cdat, aes(x=Ozone, col=imputed)) +
 stat_bin(geom="step", position="identity",
 breaks=seq(-20, 200, 10))
```



```
ggplot(cdat, aes(y=Ozone, x=Solar.R, col=imputed)) +
 geom_point() +
 labs(y="Ozone (ppb)", x="Solar Radiation (lang)")
```



Our regression imputation is better than mean imputation. See how we impute different Ozone for different Solar Radiation values, taking advantage of the information of knowing that they are correlated? But it still is obvious what is mean imputed and what is not. Also, the variance of our imputed values still does not contain the residual variation around the predicted values that we would get in real data. We can do one more enhancement to fix this.

### 45.5.3 Stochastic regression imputation

We extend regression imputation by randomly drawing observations that *look like* real ones. The easiest way to think of this is we first predict our missing value, and then we add some residual noise to account for the fact that any covariate value would not be *perfectly* predicted by the other values.

To illustrate, we first do it on our mini-dataset and look at the imputed values for our observations with missing values only:

```
imp <- mice(airqualitysub[,1:2],
 method="norm.nob",
 m=1, maxit=1, seed=1234)

iter imp variable
1 1 Ozone Solar.R

imp$imp
```

```

$Ozone
 1
5 36.67
10 -7.32

$Solar.R
 1
5 235
6 238

imp <- mice(airqualitysub[,1:2],
 method="norm.nob",
 m=1, maxit=1, seed=4234)

```

```

iter imp variable
 1 1 Ozone Solar.R

```

```
imp$imp
```

```

$Ozone
 1
5 30.8
10 57.6

```

```

$Solar.R
 1
5 277
6 498

```

Stochastic imputation is *random*. Because of this, we now have a new argument to `mice()`: the `seed = 1234` sets the random seed so that we can get the same results if we run this code again. We set two different seeds to get two different randomly generated imputations. Note in the two imputations above how we get slightly different values for our imputed data.

Now let's do it on the full data and look at the imputed values and compare to our plots above.

```

imp <- mice(airquality[,1:2], method="norm.nob",
 m=1,maxit=1,seed=13434)

```

```

iter imp variable
 1 1 Ozone Solar.R

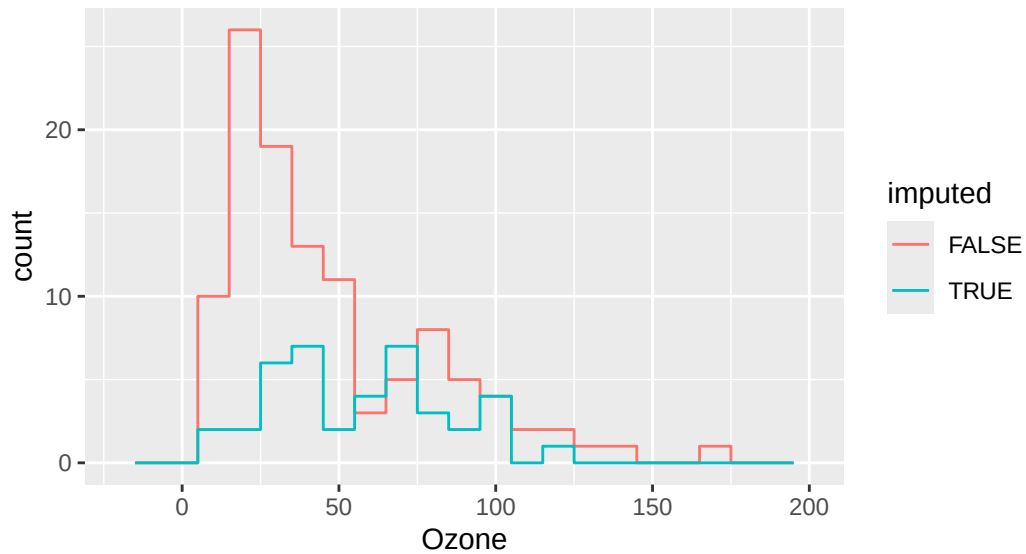
```

```

cdat = mice::complete(imp)
cdat$imputed = ici(airquality)

ggplot(cdat, aes(x=Ozone, col=imputed)) +
 stat_bin(geom="step", position="identity",
 breaks=seq(-20, 200, 10))

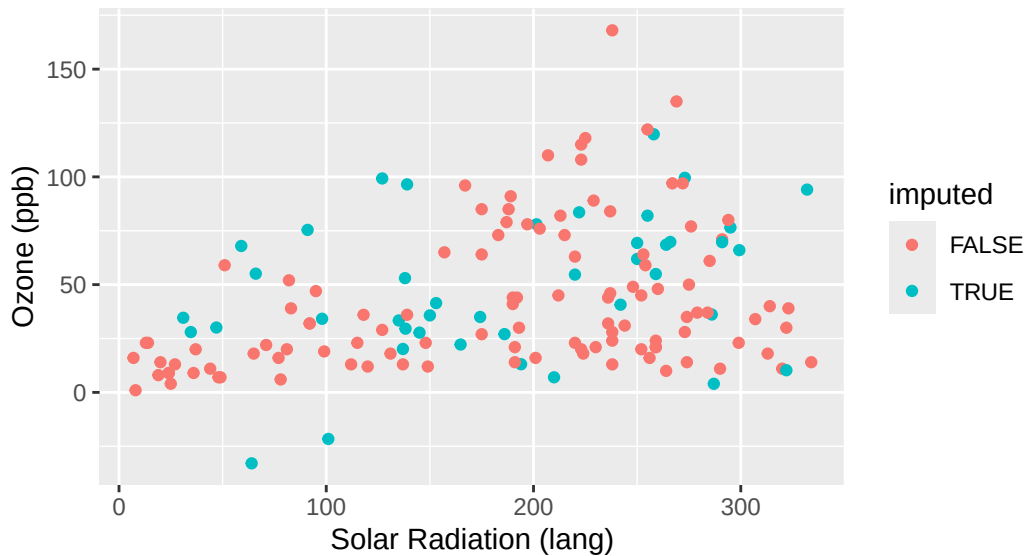
```



```

ggplot(cdat, aes(y=Ozone, x=Solar.R, col=imputed)) +
 geom_point() +
 labs(y="Ozone (ppb)", x="Solar Radiation (lang)")

```



Better, but not perfect. What is better? What is still not perfect?

**First aside:** All variables you'll be using for your final analysis model should be included in the imputation model. Notice we included the full dataset in `mice`, not just the variables with missing values. This way we can account for associations between all the outcome and the predictors in the model we'll be fitting. Your imputation model can even be more complicated than your model of interest. That is, you can include additional variables that predict missing values but will not be part of your final model of interest.

**Second aside:** All variables in your imputation model should be in the correct functional form! Quadratic, higher order polynomials and interaction terms are just another variable that we would need to impute. Although it may seem logical to impute your variables first and then calculate the interaction or non-linear term later, this can lead to bias.

**Third aside:** The ordering of the variables in the dataset you are feeding into `mice` can make a difference in results and model convergence. Generally, you want to order your variables from least missing to most missing. Here, we reorder the variables from least to most missing, and obtain different results.

```
test = airqualitysub[,c(6,5,4,3,2,1)]
head(test)
```

	Day	Month	Temp	Wind	Solar.R	Ozone
1	1	5	67	7.4	190	41
2	2	5	72	8.0	118	36
3	3	5	74	12.6	149	12
4	4	5	62	11.5	313	18
5	5	5	56	14.3	NA	NA
6	6	5	66	14.9	NA	28

```
test.imp <- mice(test, seed=2, m=1, print=FALSE)
```

Warning: Number of logged events: 1

## 45.6 Multiple imputation

If missing data is a significant issue in your dataset, then mean or regression imputation can be a bit too hacky and approximate. In these contexts, multiple imputation is the way to go.

We do this as follows:

```
imp <- mice(airqualitysub, seed=2, print=FALSE)
```



Warning: Number of logged events: 1

```
imp
```

Class: mids

Number of multiple imputations: 5

Imputation methods:

Ozone	Solar.R	Wind	Temp	Month	Day
"pmm"	"pmm"	" "	" "	" "	" "

PredictorMatrix:

	Ozone	Solar.R	Wind	Temp	Month	Day
Ozone	0	1	1	1	0	1
Solar.R	1	0	1	1	0	1
Wind	1	1	0	1	0	1
Temp	1	1	1	0	0	1
Month	1	1	1	1	0	1
Day	1	1	1	1	0	0

Number of logged events: 1

	it	im	dep	meth	out
1	0	0		constant	Month

We can look at our imputed solar values:

```
impimpSolar.R
```

	1	2	3	4	5
5	149	99	194	99	19
6	194	19	194	19	19

See multiple columns of imputed data? (We have 5 columns here, one for each imputation, with each row a single observation we are imputing for.)

**How to get each complete dataset?**

```
first complete dataset
```

```
mice::complete(imp, 1)
```

	Ozone	Solar.R	Wind	Temp	Month	Day
1	41	190	7.4	67	5	1
2	36	118	8.0	72	5	2
3	12	149	12.6	74	5	3
4	18	313	11.5	62	5	4

5	18	149	14.3	56	5	5
6	28	194	14.9	66	5	6
7	23	299	8.6	65	5	7
8	19	99	13.8	59	5	8
9	8	19	20.1	61	5	9
10	36	194	8.6	69	5	10

*## and our second complete dataset*  
`mice::complete(imp, 2)`

	Ozone	Solar.R	Wind	Temp	Month	Day
1	41	190	7.4	67	5	1
2	36	118	8.0	72	5	2
3	12	149	12.6	74	5	3
4	18	313	11.5	62	5	4
5	41	99	14.3	56	5	5
6	28	19	14.9	66	5	6
7	23	299	8.6	65	5	7
8	19	99	13.8	59	5	8
9	8	19	20.1	61	5	9
10	18	194	8.6	69	5	10

See how the two datasets are different? They were randomly imputed. We basically used the stochastic regression thing, above, multiple times.

```
mice::complete(imp, 1)[ici(airqualitysub),]
```

	Ozone	Solar.R	Wind	Temp	Month	Day
5	18	149	14.3	56	5	5
6	28	194	14.9	66	5	6
10	36	194	8.6	69	5	10

```
mice::complete(imp, 2)[ici(airqualitysub),]
```

	Ozone	Solar.R	Wind	Temp	Month	Day
5	41	99	14.3	56	5	5
6	28	19	14.9	66	5	6
10	18	194	8.6	69	5	10

On full data:

```
imp <- mice(airquality, seed=1, print=FALSE)
```

Once you have imputed, you can estimate for each imputed dataset using the `with()` method that, in `mice`, will do the regression or other specified analysis for each completed dataset. See `help with.mids` for more information.

```
fit <- with(imp, lm(Ozone ~ Wind + Temp + Solar.R))
class(fit)
```

```
[1] "mira"
```

We get back a `mira` object, which is essentially a list of results, one for each imputed dataset. The `with()` method can take *any* function call that takes a formula. So `glm`, `lm`, whatever...

The next step is to pool the estimates using the standard theory of combining multiply imputed datasets. The basic idea is to combine the variation/uncertainty of the multiple sets with the average uncertainty we would have for each set, if it were truly complete and not imputed.

Pooling looks like this:

```
pooled <- pool(fit)
pooled
```

```
Class: mipo m = 5
 term m estimate ubar b t dfcom df riv lambda
1 (Intercept) 5 -63.1480 3.66e+02 2.88e+02 7.12e+02 149 13.8 0.945 0.486
2 Wind 5 -3.0132 3.02e-01 1.37e-01 4.67e-01 149 24.0 0.546 0.353
3 Temp 5 1.5974 4.52e-02 2.50e-02 7.52e-02 149 19.6 0.663 0.399
4 Solar.R 5 0.0566 3.92e-04 8.44e-05 4.94e-04 149 52.4 0.258 0.205
 fmi
1 0.547
2 0.401
3 0.452
4 0.234
```

Parameter estimates generated by `pool` are the average of the parameter estimates from the model fit on each imputed dataset separately. The SEs are calculated by averaging the SEs and inflating them a bit based on how much the individual estimates changed—this accounts for the uncertainty from having missing values that we can only guess at, rather than see directly.

We can summarize our analysis to get p-values like this:

```
tab <- summary(pooled)
knitr::kable(tab)
```

term	estimate	std.error	statistic	df	p.value
(Intercept)	-63.148	26.677	-2.37	13.8	0.033
Wind	-3.013	0.683	-4.41	24.0	0.000
Temp	1.597	0.274	5.83	19.6	0.000
Solar.R	0.057	0.022	2.55	52.4	0.014

We can compare to our old, sad method of complete case analysis:

```
fit <- lm(Ozone~Wind+Temp+Solar.R,
 data=airquality,na.action=na.omit)
summary(fit)
```

Call:

```
lm(formula = Ozone ~ Wind + Temp + Solar.R, data = airquality,
 na.action = na.omit)
```

Residuals:

```
 Min 1Q Median 3Q Max
-40.48 -14.22 -3.55 10.10 95.62
```

Coefficients:

```
 Estimate Std. Error t value Pr(>|t|)
(Intercept) -64.3421 23.0547 -2.79 0.0062 **
Wind -3.3336 0.6544 -5.09 1.5e-06 ***
Temp 1.6521 0.2535 6.52 2.4e-09 ***
Solar.R 0.0598 0.0232 2.58 0.0112 *
```

---
Signif. codes: 0 '\*\*\*' 0.001 '\*\*' 0.01 '\*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 21.2 on 107 degrees of freedom
(42 observations deleted due to missingness)
Multiple R-squared: 0.606, Adjusted R-squared: 0.595
F-statistic: 54.8 on 3 and 107 DF, p-value: <2e-16

In this case, the missing data estimates are basically the same as the complete case analysis. We only had 5% missing data though; generally, with small amounts of missing data, nothing changes anything much.

**Aside:** You will notice that once we fit our model on the imputed data, `with()` returned an object of class `mira`. `Mira` objects can be pooled to get the pooled estimates, whereas objects of class `glm`, `lm`, `lmer`, etc. cannot be pooled.

You unfortunately cannot use `predict()` with a `mira` object. To use `predict`, one trick is to first stack the imputed datasets into one mega dataset, and then fit your model on this complete dataset.

## 45.7 Extensions

### 45.7.1 Multi-level data

Multilevel data gets more tricky: should we impute taking into account cluster? How do we do that?

For an initial pass, I recommend simply doing regression imputation *ignoring* cluster/grouping, and then adding in a dummy variable of whether a value is imputed.

### 45.7.2 Longitudinal data

With longitudinal data we can often use all our data even for individuals with missing data on the outcome, if we use longitudinal modeling and assume data are MAR (“Missing at Random”). MAR means that conditional on the observed data, missingness may depend on any observed data, but not on unobserved data.

We can explore our missing data on individuals over time and on outcomes as above to get a sense of whether MAR is a reasonable assumption or not. Then multilevel regression (using `lmer()` from the `lme4` package) basically handles the rest for us, as far as we have enough observations per individual, on average, to estimate the number of random effects we are trying to estimate. With respect to missing data on covariates or predictors, you would handle those with one of the imputation methods described above.

Here we show how to explore missing data in longitudinal analysis using data on toenail detachment. The data is from a RCT where patients were getting one of two different types of drug to prevent toenail detachment (a binary outcome).

Here we read and prep the data:

```
load data
toes = foreign::read.dta("data/toenail.dta")
```

First, let's look at how many times each patient was observed.

```
how many time points per patient?
table(table(toes$patient))
```

1	2	3	4	5	6	7
5	3	7	6	10	39	224

We have 224 patients observed at all 7 time points, and the rest of the patients are observed at fewer time points, between 1 and 6.

We can write some code to look at the patterns of missingness across the waves of data collection:

```
define function
summarise.patient = function(patient) {
 pat = rep(".", 7)
 pat[patient$visit] = 'X'
 paste(pat, collapse="")
}

For each patient, this code makes a string of "."
then it replaces all dots with an "X" if we have data for that visit

summarize missingness
miss = toes %>% group_by(patient) %>%
 do(pattern = summarise.patient(.)) %>%
 unnest(cols = c(pattern))
Group the data by patient
Then use the do() command on each chunk of our dataframe
The "." means "the chunk" (it is a pronoun, essentially).
This code creates a list of character vectors
The unnest() takes our character vector out of this list made by "do"

head(miss)

A tibble: 6 x 2
 patient pattern
 <dbl> <chr>
1 1 XXXXXXXX
2 2 XXXXXX.
3 3 XXXXXXXX
4 4 XXXXXXXX
5 6 XXXXXXXX
6 7 XXXXXXXX
```

Here we see the different patterns of missing outcomes, i.e., when patients leave and if they come back. When patients leave and never come back, regardless of the time point (see lines 4 and 5), we have “monotone missingness.”

```
sort missing patterns in decreasing order
starting with no missingness
sort(table(miss$pattern), decreasing=TRUE)
```

```
XXXXXXX XXXXX.X XXXX.XX XXX.... X..... XXXXX.. XXXX... XX..... XXX.XXX XXXXXX.
 224 21 10 6 5 5 4 3 3 3
XXX.X.. XXXX..X X.XXXXX XX..X.. XX.XXX. XX.XXXX XXX..XX XXX.X.X
 2 2 1 1 1 1 1 1
```

```
summarize number of data patterns
miss = miss %>% group_by(pattern) %>%
 summarise(n=n())
miss = arrange(miss, -n)
miss
```

```
A tibble: 18 x 2
 pattern n
 <chr> <int>
1 XXXXXXXX 224
2 XXXXX.X 21
3 XXXX.XX 10
4 XXX.... 6
5 X..... 5
6 XXXXX.. 5
7 XXXX... 4
8 XX..... 3
9 XXX.XXX 3
10 XXXXXX. 3
11 XXX.X.. 2
12 XXXX..X 2
13 X.XXXXX 1
14 XX..X.. 1
15 XX.XXX. 1
16 XX.XXXX 1
17 XXX..XX 1
18 XXX.X.X 1
```

```
percent missing data (224 complete cases)
224 / sum(miss$n)
```

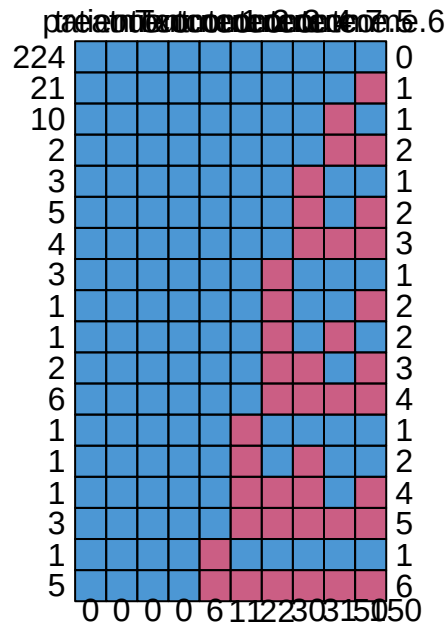
```
[1] 0.762
```

76% (or 224) of the patients have complete data

We can also look at patterns of missing outcomes by reshaping to wide, to have one column per time point, and then looking at the missing data patterns with `md.pattern()` from the `mice` package:

```
reshape data to wide
dat.wide = reshape(toes2, direction="wide", v.names="outcome",
 idvar="patient", timevar = "visit")

looking at missing data with mice package
pat <- md.pattern(dat.wide)
```



## 45.8 Further reading

Some further reading on handling missing data. But this is really a course into itself.

- Gelman & Hill Chapter 25 has a more detailed discussion of missing data imputation.
- White IR, Royston P, Wood AM. Multiple imputation using chained equations: issues and guidance for practice. *Statistics in Medicine* 2011;30: 377-399.
- Graham, JW, Olchowski, AE, Gilreath, TD, 2007. How Many Imputations are Really Needed? Some Practical Clarifications of Multiple Imputation Theory 206–213. <https://doi.org/10.1007/s11121-007-0070-9>



- van Buurin S, Groothuis-Oudshoorn K, MICE: Multivariate Imputation by Chained Equations. Journal of Statistical Software. 2011;45(3):1-68.
- Grund S, Lüdtke O, Robitzsch A. Multiple Imputation of Missing Data for Multilevel Models: Simulations and Recommendations. DOI: 10.1177/1094428117703686

## 45.9 Appendix: More about the mice package

The mice package gives back a very complex object that has a lot of information about how values were imputed, which values were imputed, and so forth. In the following we unpack the `imp` variable from above a bit more.

### Looking at the imputation object

In the following code, we look at the object we get back from `mice()`. It has lots of parts that we can peek into.

First, the `imp` list inside of `imp` stores all of our newly imputed data. It is itself a list of each variable with their imputed values:

```
imp$imp
```

	1	2	3	4	5
5	6	19	14	8	14
10	12	12	7	23	23
25	14	19	14	19	14
26	37	18	32	32	18
27	11	1	18	13	18
32	65	45	13	28	29
33	22	36	12	18	16
34	13	18	1	13	13
35	63	35	45	52	71
36	23	39	20	59	96
37	24	16	12	34	18
39	64	135	85	80	91
42	115	76	115	37	91
43	66	122	78	64	122
45	44	28	45	23	16
46	23	45	46	45	35
52	20	52	63	47	47
53	59	59	48	115	37
54	40	16	35	37	63

55	40	35	48	39	49
56	23	39	59	59	16
57	44	52	40	52	20
58	30	30	27	14	23
59	45	32	16	16	46
60	44	27	34	28	30
61	89	64	80	37	64
65	16	16	14	23	29
72	46	52	65	45	35
75	35	64	71	18	78
83	20	40	71	46	59
84	28	63	37	29	63
102	115	78	78	37	66
103	46	29	31	23	40
107	16	30	13	14	22
115	41	12	44	7	22
119	50	78	122	85	50
150	24	12	27	21	12

\$Solar.R

	1	2	3	4	5
5	7	313	82	13	314
6	322	187	222	24	238
11	66	274	139	135	112
27	20	24	7	238	193
96	175	223	284	197	220
97	51	139	274	237	98
98	98	203	220	188	276

\$Wind

[1] 1 2 3 4 5  
 <0 rows> (or 0-length row.names)

\$Temp

[1] 1 2 3 4 5  
 <0 rows> (or 0-length row.names)

\$Month

[1] 1 2 3 4 5  
 <0 rows> (or 0-length row.names)

\$Day

[1] 1 2 3 4 5

```
<0 rows> (or 0-length row.names)
```

```
str(imp$imp)
```

```
List of 6
```

```
$ Ozone : 'data.frame': 37 obs. of 5 variables:
 ..$ 1: int [1:37] 6 12 14 37 11 65 22 13 63 23 ...
 ..$ 2: int [1:37] 19 12 19 18 1 45 36 18 35 39 ...
 ..$ 3: int [1:37] 14 7 14 32 18 13 12 1 45 20 ...
 ..$ 4: int [1:37] 8 23 19 32 13 28 18 13 52 59 ...
 ..$ 5: int [1:37] 14 23 14 18 18 29 16 13 71 96 ...
```

```
$ Solar.R: 'data.frame': 7 obs. of 5 variables:
 ..$ 1: int [1:7] 7 322 66 20 175 51 98
 ..$ 2: int [1:7] 313 187 274 24 223 139 203
 ..$ 3: int [1:7] 82 222 139 7 284 274 220
 ..$ 4: int [1:7] 13 24 135 238 197 237 188
 ..$ 5: int [1:7] 314 238 112 193 220 98 276
```

```
$ Wind : 'data.frame': 0 obs. of 5 variables:
 ..$ 1: logi(0)
 ..$ 2: logi(0)
 ..$ 3: logi(0)
 ..$ 4: logi(0)
 ..$ 5: logi(0)
```

```
$ Temp : 'data.frame': 0 obs. of 5 variables:
 ..$ 1: logi(0)
 ..$ 2: logi(0)
 ..$ 3: logi(0)
 ..$ 4: logi(0)
 ..$ 5: logi(0)
```

```
$ Month : 'data.frame': 0 obs. of 5 variables:
 ..$ 1: logi(0)
 ..$ 2: logi(0)
 ..$ 3: logi(0)
 ..$ 4: logi(0)
 ..$ 5: logi(0)
```

```
$ Day : 'data.frame': 0 obs. of 5 variables:
 ..$ 1: logi(0)
 ..$ 2: logi(0)
 ..$ 3: logi(0)
 ..$ 4: logi(0)
 ..$ 5: logi(0)
```

```
str(impimpOzone)
```

```
'data.frame': 37 obs. of 5 variables:
 $ 1: int 6 12 14 37 11 65 22 13 63 23 ...
 $ 2: int 19 12 19 18 1 45 36 18 35 39 ...
 $ 3: int 14 7 14 32 18 13 12 1 45 20 ...
 $ 4: int 8 23 19 32 13 28 18 13 52 59 ...
 $ 5: int 14 23 14 18 18 29 16 13 71 96 ...
```

We see that Ozone and Solar.R have imputed values, and the other variables do not.

Next, we see two missing observations in our original data and then see the two imputed values for these two missing observations.

```
airqualitysub$Ozone

[1] 41 36 12 18 NA 28 23 19 8 NA

impimpOzone[,1]

[1] 6 12 14 37 11 65 22 13 63 23 24 64 115 66 44 23 20 59 40
[20] 40 23 44 30 45 44 89 16 46 35 20 28 115 46 16 41 50 24
```

We can make (the hard way) a vector of Ozone by plugging our missing values into the original data. But the `complete()` method, above, is preferred.

```
oz = airqualitysub$Ozone
oz[is.na(oz)] = impimpOzone[,1]

Warning in oz[is.na(oz)] = impimpOzone[, 1]: number of items to replace is
not a multiple of replacement length

oz

[1] 41 36 12 18 6 28 23 19 8 12
```

What else is there in `imp`?

```
names(imp)

[1] "data" "imp" "m" "where"
[5] "blocks" "call" "nmis" "method"
[9] "predictorMatrix" "visitSequence" "formulas" "calltype"
[13] "post" "blots" "ignore" "seed"
[17] "iteration" "lastSeedValue" "chainMean" "chainVar"
[21] "loggedEvents" "version" "date"
```

What was our imputation method?

```
imp$method
```

```
Ozone Solar.R Wind Temp Month Day
"pmm" "pmm" "" "" "" ""
```

Mean imputation for each variable with missing values. Later this will say other thing.

What was used to impute what?

```
imp$predictorMatrix
```

```
 Ozone Solar.R Wind Temp Month Day
Ozone 0 1 1 1 1 1
Solar.R 1 0 1 1 1 1
Wind 1 1 0 1 1 1
Temp 1 1 1 0 1 1
Month 1 1 1 1 0 1
Day 1 1 1 1 1 0
```

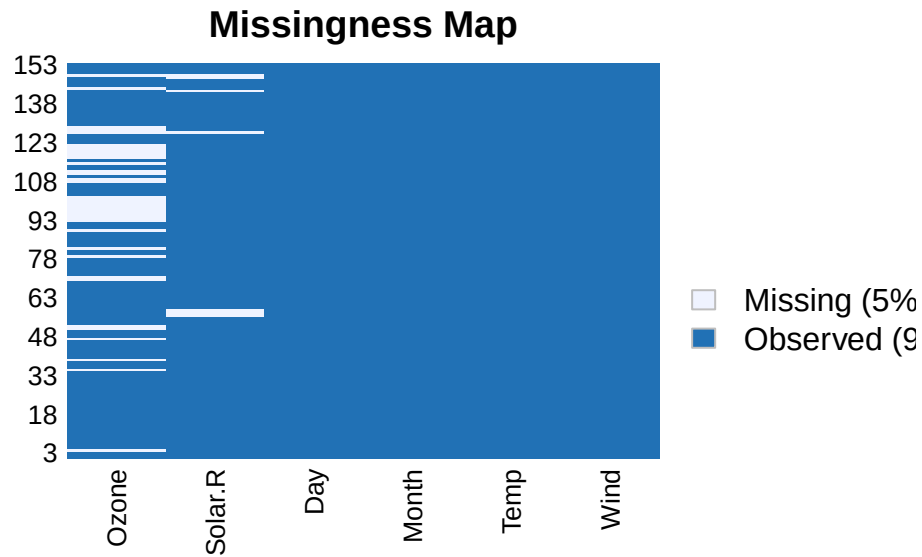
## 45.10 Appendix: The amelia package

Amelia is another multiple imputation and missing data package. We do not prefer it, but have some demonstration code in the following, for reference.

```
library(Amelia)
```

For missingness we can make the following:

```
missmap(airquality)
```



Each row of the plot is a row of the data, and missing values are shown in brown. But ugly!  
And hard to see any trends in the missingness.

You can use the `Amelia` package to do mean imputation.

```
library(dplyr)

exclude variables that do not vary
a.airquality = airquality %>% dplyr::select(-Month)

impute data
a.imp <- amelia(a.airquality, m=5)

-- Imputation 1 --

 1 2 3 4 5 6 7

-- Imputation 2 --

 1 2 3 4 5 6

-- Imputation 3 --

 1 2 3 4 5 6 7

-- Imputation 4 --
```

```

1 2 3 4

-- Imputation 5 --

1 2 3 4 5 6

a.imp

```

Amelia output with 5 imputed datasets.  
 Return code: 1  
 Message: Normal EM convergence.

```

Chain Lengths:

Imputation 1: 7
Imputation 2: 6
Imputation 3: 7
Imputation 4: 4
Imputation 5: 6

```

We can plot our imputed values against our observed values to check that they make sense. We will do this for just one of five datasets we just imputed using Amelia.

```

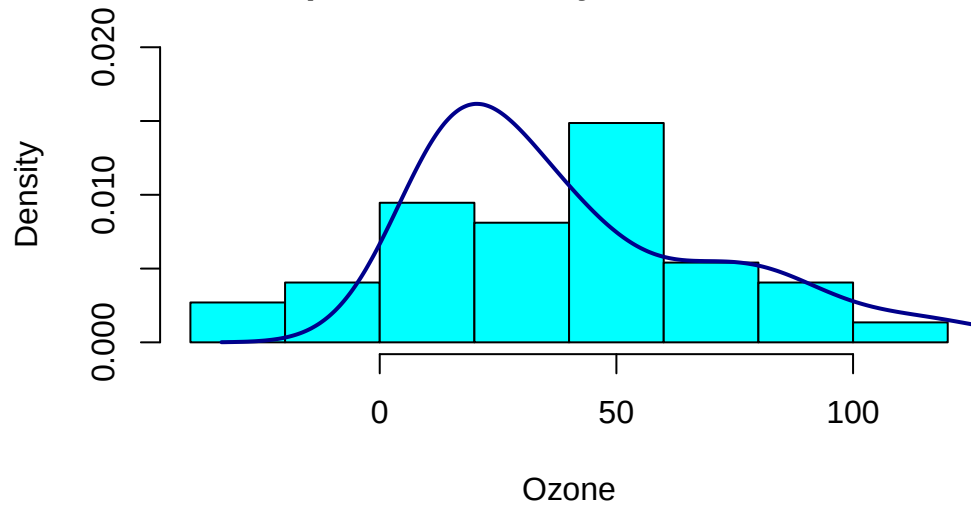
put imputed values from the third dataset in an object
one_imp <- a.imp$imputations[[3]]$Ozone

make object with observed values
from observations without missing Ozone values
obs_data <- a.airquality$Ozone

make a plot overlaying observed and imputed values
hist(one_imp[is.na(obs_data)], prob=TRUE, xlab="Ozone",
 main="Histogram of Imputed Values in 3rd Imputation \nCompared to Density in Observed",
 col="cyan", ylim=c(0,0.02))
lines(density(obs_data[!is.na(obs_data)]), col="darkblue", lwd=2)

```

### Histogram of Imputed Values in 3rd Imputation Compared to Density in Observed Data



You can also do multiple imputation in **Amelia**. However, **Amelia** does not have an easy way to combine the estimates from the imputed datasets (no analogue of `with()` in **mice**). You can write a function that fits your model of interest in each imputed dataset and then use a package like **mitools** to pool the estimates and variances.

Much easier to use **mice**!

**Aside:** A more important limitation of **Amelia** is that the algorithm it uses to impute missing values assumes multivariate normality, which is often questionable, especially when you have binary variables.



## 46 Using mice to handle categorical data

Non-continuous data is a bit tricky because we can't directly do mean imputation, since there is nothing to take a mean of. That said, all the ideas shown in the prior intro to missing data chapter can be extended to non-continuous (categorical) variables.

We have a few options:

- Add a Missing category
- Do mean imputation on dummy variables
- Use mice to impute values. This is probably the easiest option.

We walk through these next, using the `nhanes` data (another built-in dataset in the `mice` package) for our illustration. In `nhanes`, one of the variables is an `age` variable that is coded as an ordered categorical variable (e.g. 20-39, 40-59, etc). Another is the binary `hyp` variable. To illustrate imputation, we are just going to delete some of the `age` data so we can try to impute it back.

```
load data and create some missing values for an ordered categorical
variable
```

```
data(nhanes2)
nhanes2$age[1:5] = NA
head(nhanes2)
```

```
 age bmi hyp chl
1 <NA> NA <NA> NA
2 <NA> 22.7 no 187
3 <NA> NA no 187
4 <NA> NA <NA> NA
5 <NA> 20.4 no 113
6 60-99 NA <NA> 184
```

Ok, we have our data. Next we go through the way of handling it.

## 46.1 Adding a missing category

Adding a missing category is a common approach for categorical variables, and it is easy to do.

For example, for the `age` variable, we can simply add a new level called "Missing" to cover any NA observations, using `fct_explicit_na()` from the `forcats` package (loaded automatically with `tidyverse`):

```
nhanes2_cat <- nhanes2 %>%
 mutate(age = fct_na_value_to_level(age, level="Missing"))

levels(nhanes2_cat$age)

[1] "20-39" "40-59" "60-99" "Missing"

table(nhanes2_cat$age)
```

20-39	40-59	60-99	Missing
9	6	5	5

The `age` variable now has a “Missing” level, so it can be passed directly into a regression or other model. The same approach works for the binary `hyp` variable:

```
nhanes2_cat <- nhanes2_cat %>%
 mutate(hyp = fct_na_value_to_level(hyp, level="Missing"))

table(nhanes2_cat$hyp)
```

no	yes	Missing
13	4	8

The appeal of this approach is its simplicity: no imputation model is required, and the "Missing" category can sometimes capture meaningful signal (e.g., if a certain type of respondent systematically skips a question). The downside is that it adds a category whose interpretation is not always clear, and if the data are not MCAR it could introduce bias in downstream analysis just like listwise deletion would.

## 46.2 Mean imputation on dummy variables

A common approach is to convert a categorical variable into a set of dummy (0/1) variables and then do ordinary mean imputation on each dummy.

This works especially naturally for a binary variable like `hyp`, which takes values "yes" and "no". Converting `hyp` to a 0/1 dummy and then mean-imputing is equivalent to replacing each missing value with the sample proportion of "yes" responses among non-missing cases:

```
nhanes2_dum <- nhanes2 %>%
 mutate(hyp_num = as.integer(hyp == "yes"))

head(nhanes2_dum)
```

	age	bmi	hyp	chl	hyp_num
1	<NA>	NA	<NA>	NA	NA
2	<NA>	22.7	no	187	0
3	<NA>	NA	no	187	0
4	<NA>	NA	<NA>	NA	NA
5	<NA>	20.4	no	113	0
6	60-99	NA	<NA>	184	NA

```
hyp_mean <- mean(nhanes2_dum$hyp_num, na.rm = TRUE)
hyp_mean
```

```
[1] 0.2352941
```

```
nhanes2_dum <- nhanes2_dum %>%
 mutate(hyp_imp = if_else(is.na(hyp_num), hyp_mean, as.double(hyp_num)))

head(nhanes2_dum)
```

	age	bmi	hyp	chl	hyp_num	hyp_imp
1	<NA>	NA	<NA>	NA	NA	0.2352941
2	<NA>	22.7	no	187	0	0.0000000
3	<NA>	NA	no	187	0	0.0000000
4	<NA>	NA	<NA>	NA	NA	0.2352941
5	<NA>	20.4	no	113	0	0.0000000
6	60-99	NA	<NA>	184	NA	0.2352941

```
summary(nhanes2_dum$hyp_imp)
```

Min.	1st Qu.	Median	Mean	3rd Qu.	Max.
0.0000	0.0000	0.0000	0.2353	0.2353	1.0000

The originally-missing `hyp` rows now have the value 0.235, which is the proportion of hypertensive respondents in the observed data.

For an ordered categorical variable like `age`, you would create a separate dummy for each level (dropping one to avoid perfect collinearity) and mean-impute each one.

However, just as with continuous variables, mean imputation has well-known limitations: it attenuates relationships between variables and underestimates variance. For categorical variables, it also produces fractional imputed values (e.g., 0.4) that do not correspond to any valid category. E.g., what does it mean to say our first observation has 0.23 for “having hypertension”?

## 46.3 Using Mice to impute values

`Mice` will automatically detect and handle non-continuous (e.g. dichotomous or categorical) variables on its own. That said, it will also allow you to specify the type of variable you are imputing yourself, if you want more control over the process.

Imputing missing values is easy:

```
imp.cat <- mice(nhanes2, m = 1, print=FALSE)
full.cat = mice::complete(imp.cat)
head(full.cat)
```

	age	bmi	hyp	chl
1	60-99	26.3	yes	218
2	60-99	22.7	no	187
3	20-39	30.1	no	187
4	60-99	22.0	yes	204
5	60-99	20.4	no	113
6	60-99	25.5	yes	184

We now have categorical values in our `age` column and `hyp` column, along with the continuous imputations for the other variables.

The `mice` package uses a different model for each variable, and the model can depend on what kind of data we are imputing. We can check which imputation method `mice` used as so:

```
imp.cat$method
```

age	bmi	hyp	chl
"polyreg"	"pmm"	"logreg"	"pmm"

`mice` used the `polyreg` imputation method for the variable *age*, which means it treated it as an unordered categorical variable. We also see that a logistic regression was used for our binary variable *hyp*. But we know our *age* variable is an ordered variable: higher values categories signified older age. We might want to use a model that takes that ordering into account.

One very important method often used is “`polr`” (Proportional odds model) which handles *ordered* categories, instead of unordered. This allows it to say some general range of categories is more or less likely, given the other variables.

In general, the more structure you can impose (based on your domain knowledge) means better imputation.

More broadly, to know what methods for imputation are available, see the various methods described under `mice`’s help file (or check out the long but readable `mice` paper by Van Buuren and Groothuis-Oudshoorn (2011) (also see the Canvas site for the reference).

We can manually force `mice` to treat *age* as an ordered as so:

```
imp.cat2 <- mice(nhanes2,
 meth=c("polr", "pmm", "logreg", "pmm"),
 m=1, print=FALSE)
head(mice::complete(imp.cat2, 1))
```

	age	bmi	hyp	chl
1	20-39	35.3	no	204
2	40-59	22.7	no	187
3	20-39	33.2	no	187
4	20-39	30.1	no	187
5	60-99	20.4	no	113
6	60-99	25.5	yes	184

```
imp.cat2$method
```

age	bmi	hyp	chl
"polr"	"pmm"	"logreg"	"pmm"

### 46.3.1 A coding trick to get the defaults

As a coding tip, we can use `mice` with `maxit=0` to get what the defaults are first, and then we can easily change the ones we want, and run `mice` for real. This is useful if we have a lot of variables, and do not want to type in all the defaults:

```
imp <- mice(nhanes2, maxit = 0)
imp$method
```

```
 age bmi hyp chl
"polyreg" "pmm" "logreg" "pmm"
```

```
mth = imp$method
mth[1] = "polr"
mth
```

```
 age bmi hyp chl
"polr" "pmm" "logreg" "pmm"
```

And now run as before:

```
imp <- mice(nhanes2, method=mth, print=FALSE)
```

Easy as pie!

## 46.4 References

**Part VI**

**On Machine Learning**

## **47 A demonstration of different machine learning methods all fit to the same data**



## 48 Overview

This document showcases a bunch of different machine learning tools, all used on the same data set. At the end we compare the different rmse on the validation set.

For context, we will use a classic data set (e.g., Almond et al., 2005) on child birthweight. This data set was originally constructed to estimate the causal effect of maternal smoking on child birthweight; that is not what we are up to now. Our goal is to instead *predict* child birthweight directly based on observable characteristics prior to birth.

The target of interest is the child's birthweight, stored as `child_birthwt`. This is a continuous outcome. All other variables are fair game as predictors.

While this is obviously a simplified data set, the prediction problem is very real – many medical insurers and providers target services based on algorithms similar to what you will put together.

**Note:** For the purposes of illustration we are going to use a proportionally small training set of 10,000 observations (so we need to regularize and use fancy stuff) and a very large validation set (so we see the real comparison of our different choices). In practice we would try to use as much of our data as possible for training.

## 49 Setup

Load our libraries, set our random seed for reproducibility:

```
library(MASS)
library(rattle)
library(glmnet)
library(tidyverse)
library(modelr)
library(broom)
library(caret)
library(ranger)
set.seed(8675309)

knitr::opts_chunk$set(echo = TRUE,
 fig.width = 5,
 out.width = "5in",
 fig.align = "center")
options(list(dplyr.summarise.inform = FALSE))
theme_set(theme_classic())
```

### 49.0.1 Load in the birthweight data.

We have two files: training and holdout. We keep our holdout set for our final validation after we pick our final models. This allows a fair comparison of all the methods at the end as we do not use the holdout set AT ALL for building our models.

```
Load the data
ca_training = read.csv("data/lbw_training.csv")
ca_holdout = read.csv("data/lbw_holdout.csv")
```

If you are pasting this code into your own computer, you can [download lbw\\_holdout.csv](#) and [lbw\\_training.csv](#) from the course website.

Let's peek at our list of variables:

```
names(ca_training)
```

[1]	"child_birthwt"	"use_tobacco"
[3]	"born_in_hospital"	"MD_at_birth"
[5]	"any_drink_on_avg"	"mother_foreign_born"
[7]	"mother_age"	"father_age"
[9]	"num_prenatal_visits"	"mother_hispanic_other"
[11]	"mother_black"	"father_hispanic_other"
[13]	"father_black"	"mother_educ_less_than_hs"
[15]	"mother_educ_hs"	"father_educ_less_than_hs"
[17]	"father_educ_hs"	"adequate_care"
[19]	"birth_month"	"birth_weekday"
[21]	"child_female"	"mother_anemic"
[23]	"mother_cardiac_disease"	"mother_lung"
[25]	"mother_herpes"	"mother_diabetes"

## 50 Baseline linear model (OLS)

How do we do with simple linear regression models? We try two versions, one with just the main effects and one with all the pairwise interactions.

```
model_linear <- lm(child_birthwt ~ ., data = ca_training)
```

```
model_linear_int <- lm(child_birthwt ~ .^2, data = ca_training)
```

```
rmse(model_linear, ca_holdout)
```

```
[1] 559.436
```

```
rmse(model_linear_int, ca_holdout)
```

```
[1] 591.2767
```

Note the worse performance of the linear regression with interactions. We are likely overfitting.

## 51 Forward stepwise selection

Here we search for a good linear model by iteratively adding the “best” covariate until we are not improving on our model performance measure.

```
library(MASS)

set up simplest and most complex to consider:
`~ 1` is the "intercept only" model. Our max model
has everything (but no interactions).
mod_simple <- lm(child_birthwt ~ 1, data = ca_training)
mod_max <- lm(child_birthwt ~ ., data = ca_training)

use forward stepwise selection to pick an optimal model (in terms of AIC)
`trace=0` makes it not print out a lot of stuff to the screen about what it
is doing.
mod_forward <- stepAIC(mod_simple,
 scope = list(lower = formula(mod_simple),
 upper = formula(mod_max)),
 direction = "forward",
 trace = 0)

summary(mod_forward)
```

Call:

```
lm(formula = child_birthwt ~ use_tobacco + mother_black + num_prenatal_visits +
 child_female + adequate_care + mother_age + born_in_hospital +
 any_drink_on_avg + mother_foreign_born + mother_educ_less_than_hs +
 mother_anemic + MD_at_birth + father_hispanic_other + father_black,
 data = ca_training)
```

Residuals:

Min	1Q	Median	3Q	Max
-3038.05	-304.73	26.42	352.09	2399.93

Coefficients:

	Estimate	Std. Error	t value	Pr(> t )
(Intercept)	3254.332	60.917	53.423	< 2e-16 ***
use_tobacco	-225.548	15.441	-14.607	< 2e-16 ***
mother_black	-148.518	40.371	-3.679	0.000236 ***
num_prenatal_visits	27.124	2.088	12.989	< 2e-16 ***
child_female	-127.642	11.152	-11.445	< 2e-16 ***
adequate_careInadequate	208.301	33.101	6.293	3.25e-10 ***
adequate_careIntermediate	78.113	16.530	4.726	2.32e-06 ***
mother_age	4.934	1.065	4.634	3.63e-06 ***
born_in_hospital	-187.136	45.238	-4.137	3.55e-05 ***
any_drink_on_avg	-242.372	59.992	-4.040	5.38e-05 ***
mother_foreign_born	-63.806	27.741	-2.300	0.021466 *
mother_educ_less_than_hs	-54.475	18.572	-2.933	0.003363 **
mother_anemic	-132.998	56.122	-2.370	0.017817 *
MD_at_birth	-28.245	16.289	-1.734	0.082948 .
father_hispanic_other	-80.786	48.069	-1.681	0.092870 .
father_black	-62.116	38.409	-1.617	0.105865

---

Signif. codes: 0 '\*\*\*' 0.001 '\*\*' 0.01 '\*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 556.9 on 9984 degrees of freedom

Multiple R-squared: 0.08228, Adjusted R-squared: 0.0809

F-statistic: 59.68 on 15 and 9984 DF, p-value: < 2.2e-16

`coef( mod_forward )`

(Intercept)	use_tobacco	mother_black
3254.331679	-225.548150	-148.518044
num_prenatal_visits	child_female	adequate_careInadequate
27.124442	-127.642279	208.301177
adequate_careIntermediate	mother_age	born_in_hospital
78.113446	4.934118	-187.136158
any_drink_on_avg	mother_foreign_born	mother_educ_less_than_hs
-242.371983	-63.806385	-54.475462
mother_anemic	MD_at_birth	father_hispanic_other
-132.998442	-28.244929	-80.785733
father_black		
-62.115737		

We can examine how many coefficients we zeroed out with this approach:

```
Total number of coefficients (remember to subtract off intercept)
length(coef(model_linear)) - 1
```

```
[1] 41
```

```
length(coef(mod_forward)) - 1
```

```
[1] 15
```

We have dropped several covariates.

Finally, how well do we do in terms of predictive performance?

```
rmse(mod_forward, ca_holdout)
```

```
[1] 558.9989
```

## 52 Ridge Regression

How do we do with ridge? To use `glmnet` we need to make our data into a matrix with no categorical covariates (i.e., we need to convert those to dummy variables). This “design matrix” or “model matrix” is used by those ML packages (in particular `glmnet`) that do not like formulae. We make this as follows:

```
x <- model.matrix(child_birthwt ~ ., ca_training)[,-1]
y <- as.numeric(ca_training$child_birthwt)

how many covariates do we have now that we fleshed out our categorical
ones?
dim(x)

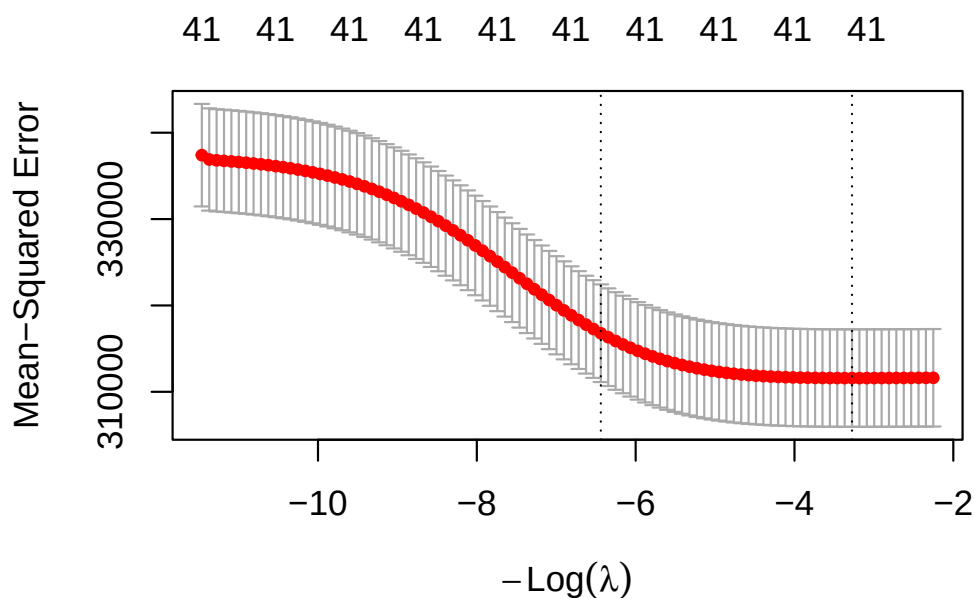
[1] 10000 41
```

We are going to cross validate to pick the best tuning parameter.

```
model_ridge <- cv.glmnet(x = x, y = y, alpha = 0)

Plot to see how our CV estimate of performane changes as our tuning parameter changes.
plot(model_ridge)
```





Now let's look at the model corresponding to the "1 SE" rule.

```
Calculate coefficients at optimal lambda
```

```
predict(model_ridge, "coefficients", newx = x, s = "lambda.1se")[,1]
```

(Intercept)	use_tobacco	born_in_hospital
3369.117138	-109.333424	-110.752801
MD_at_birth	any_drink_on_avg	mother_foreign_born
-18.307264	-150.301619	-25.061873
mother_age	father_age	num_prenatal_visits
2.387578	1.087979	9.380462
mother_hispanic_other	mother_black	father_hispanic_other
-9.908412	-73.839709	-42.635734
father_black	mother_educ_less_than_hs	mother_educ_hs
-63.432580	-35.164414	3.169975
father_educ_less_than_hs	father_educ_hs	adequate_careInadequate
-17.728130	-11.615696	9.880843
adequate_careIntermediate	birth_monthM10	birth_monthM11
-4.005582	-12.728006	12.571847
birth_monthM12	birth_monthM2	birth_monthM3
-15.496326	15.208849	-10.498643
birth_monthM4	birth_monthM5	birth_monthM6
3.210140	13.952109	2.536463
birth_monthM7	birth_monthM8	birth_monthM9
7.638260	-3.250663	-11.412587

birth_weekdayD2	birth_weekdayD3	birth_weekdayD4
-2.340281	5.611307	17.808560
birth_weekdayD5	birth_weekdayD6	birth_weekdayD7
2.112503	8.272195	-8.147729
child_female	mother_anemic	mother_cardiac_disease
-62.913159	-72.776220	4.197639
mother_lung	mother_herpes	mother_diabetes
12.464810	14.774745	31.563450

```
Re-fit model at optimal
model_ridge_optimal <- glmnet(x = x, y = y, alpha = 0, lambda = model_ridge$lambda.1se)
```

### RMSE on holdout:

We first make our holdout data into a matrix just like the training data:

```
Do the same for holdout.
x_holdout <- model.matrix(child_birthwt ~ ., ca_holdout)[-1]
y_holdout <- as.numeric(ca_holdout$child_birthwt)
```

Unfortunately, `glmnet` doesn't work with the `rmse()` method, so we have to write our own `rmse`. Sad!

```
Function to calculate RMSE given our design matrix rather
than original data.
rmse_by_hand <- function(this_model, x_holdout, y_holdout){
 this_pred <- predict(this_model, newx = x_holdout)

 sqrt(mean((this_pred - y_holdout)^2))
}
```

```
Calculate RMSE
rmse_by_hand(model_ridge_optimal, x_holdout, y_holdout)
```

```
[1] 565.0963
```

We can also use the helper function in `caret` that we saw in the random forest case study that takes observed and actual outcomes:

```
preds = as.numeric(predict(model_ridge_optimal, x_holdout))
rs = data.frame(obs = y_holdout,
 pred = preds)
caret::defaultSummary(rs)
```

RMSE	Rsquared	MAE
565.09626065	0.07662296	420.94054253

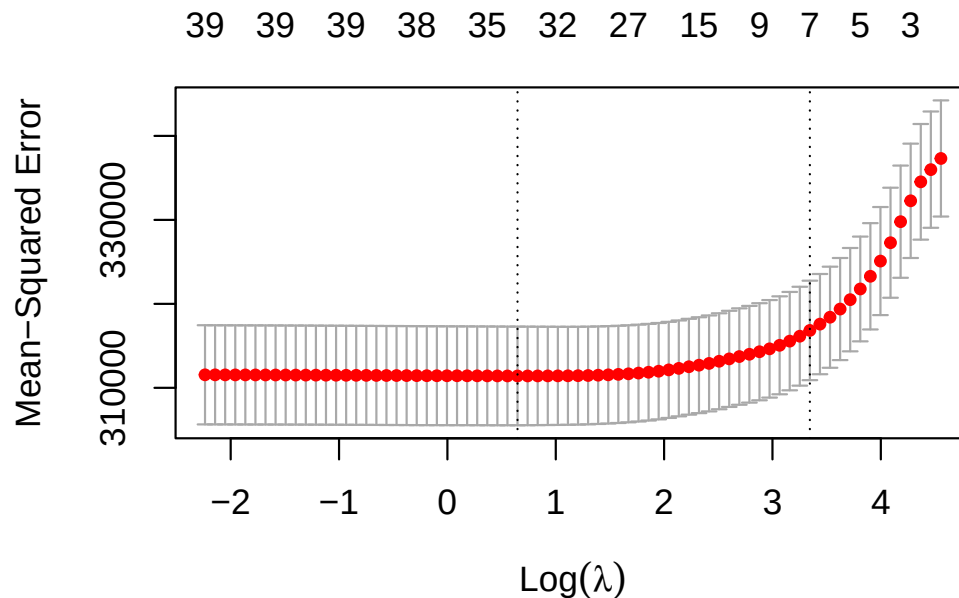
**Note:** The `predict()` method for `glmnet` gives back a matrix, not a vector of numbers. Annoying. We have to use `as.numeric` to get it to change to a list of numbers.

## 53 Lasso Regression

How do we do with Lasso? It is the same as ridge, except we need to set `alpha=1`.

```
model_lasso <- cv.glmnet(x = x, y = y, alpha = 1)
```

```
Plot output
plot(model_lasso)
```



```
Calculate coefficients at optimal lambda. We only look at the non-zero ones.
```

```
coef <- predict(model_lasso, "coefficients", newx = x, s = "lambda.1se")[,1]
```

```
coef[coef != 0]
```

(Intercept)	use_tobacco	born_in_hospital	mother_age
3288.089967	-167.324623	-13.895767	1.568242
num_prenatal_visits	mother_black	father_black	child_female
12.063002	-106.815386	-26.432899	-74.649746

```
Re-fit model at optimal
model_lasso_optimal <- glmnet(x = x, y = y, alpha = 1, lambda = model_lasso$lambda.1se)

Calculate RMSE for Lasso
rmse_by_hand(model_lasso_optimal, x_holdout, y_holdout)

[1] 564.7121
```

## 54 Single Tree (a CART)

We first do a single tree and prune it, and then we will do random forests later on.

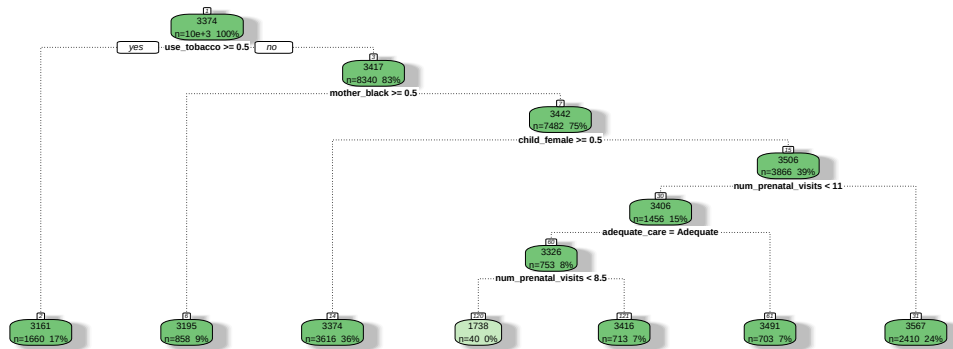
To fit a single tree we use `rpart` (Recursive Partition).

```
library(rpart)
```

```
model_tree <- rpart(child_birthwt ~ ., data = ca_training)
```

We can then plot it:

```
library(rattle)
fancyRpartPlot(model_tree)
```



Rattle 2026-Mar-14 12:21:04 Imiratrix

Our RMSE:

```
How well do we do on the test set?
rmse(model_tree, ca_holdout)
```

```
[1] 558.7839
```

Let's use cross validation to pick our `cp` tuning parameter (which controls the depth and complexity of our tree):

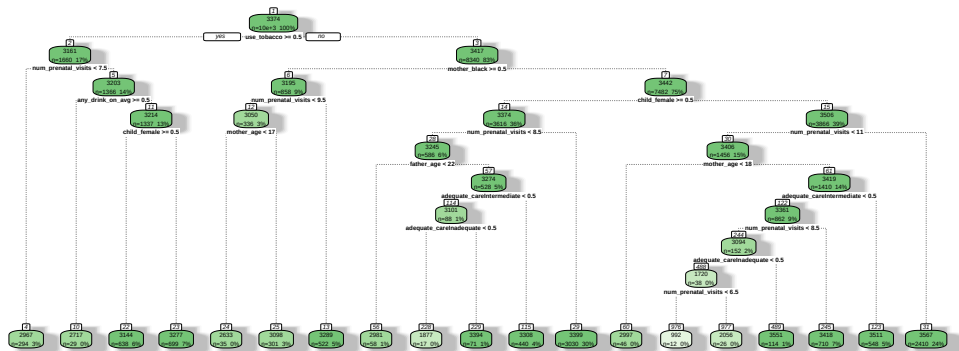
```
train_control <-
 trainControl(method = "cv",
 number = 10)

tune_grid <- data.frame(
 cp = exp(seq(log(0.001), log(0.5), length.out=100)))

rpart_cv <-
 train(child_birthwt ~ ., data = ca_training,
 method = "rpart",
 na.action = "na.omit",
 tuneGrid = tune_grid,
 trControl = train_control)
```

Warning in nominalTrainWorkflow(x = x, y = y, wts = weights, info = trainInfo,  
: There were missing values in resampled performance measures.

```
Plot the final model (fit using the best chosen tuning parameter automatically)
fancyRpartPlot(rpart_cv$finalModel)
```



Rattle 2024-Dec-26 17:42:49 Imiratrix

Now let's look at the RMSE of our final model. We use `predict` on the entire `rpart_cv` object since it will automatically give a tree refit to the entire training data using the best selected tuning parameter:

```
rmse(rpart_cv, ca_holdout)
```

```
[1] 548.8857
```

To get the predictions themselves we would use `predict`, handing over `rpart_cv` *not* `rpart_cv$finalModel`:

```
preds = predict(rpart_cv, ca_holdout)
summary(preds)
```

Min.	1st Qu.	Median	Mean	3rd Qu.	Max.
992	3289	3399	3373	3511	3567



## 55 Random Forests

We fit a random forest with the `ranger` package as so:

```
fit a random forest model
model_rf <- ranger(child_birthwt ~ ., data = ca_training,
 importance = "permutation")
```

```
model_rf
```

Ranger result

Call:

```
ranger(child_birthwt ~ ., data = ca_training, importance = "permutation")
```

Type:	Regression
Number of trees:	500
Sample size:	10000
Number of independent variables:	25
Mtry:	5
Target node size:	5
Variable importance mode:	permutation
Splitrule:	variance
OOB prediction error (MSE):	302410.2
R squared (OOB):	0.1037941

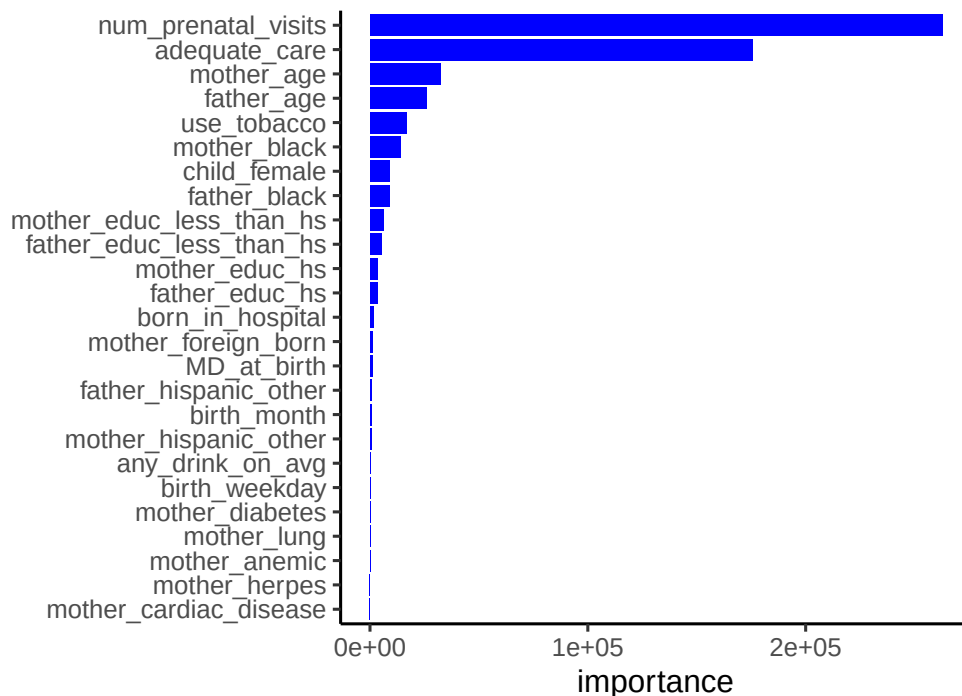
```
Table of variable importance
importance(model_rf)
```

use_tobacco	born_in_hospital	MD_at_birth
16588.89005	1693.18649	1116.55033
any_drink_on_avg	mother_foreign_born	mother_age
448.65352	1328.86719	32323.92626
father_age	num_prenatal_visits	mother_hispanic_other
25935.76551	262591.71389	573.88371
mother_black	father_hispanic_other	father_black
13993.30014	792.15976	8945.04138

mother_educ_less_than_hs	6254.52673	mother_educ_hs	3419.14652	father_educ_less_than_hs	5492.35051
father_educ_hs	3398.24303	adequate_care	175591.38932	birth_month	614.54015
birth_weekday	380.24615	child_female	8977.13883	mother_anemic	105.88669
mother_cardiac_disease	-325.34716	mother_lung	144.81925	mother_herpes	-46.14257
mother_diabetes	313.14919				

Annoyingly, with the `ranger` package we have to make a variable importance plot by hand. (We could instead use the `randomForest` package, which has some default importance plot code. See prior scripts illustrating this if desired.) The following code makes our plot:

```
imps = importance(model_rf)
imps = tibble(var = names(imps),
 importance = imps)
imps = imps %>% arrange(importance)
ggplot(imps, aes(x = reorder(var, importance), y = importance)) +
 geom_col(fill = "blue") +
 coord_flip() +
 labs(x = "")
```



**Note:** The `reorder()` line in `ggplot` sets the order of our x variable so we get our variables from least to most important. The `coord_flip` makes our x-axis our y-axis and vice-versa, so we get nice horizontal bars.

The `ranger` package gets fancy with its `predict` which makes it a bit harder to calculate out of sample RMSE. We do it like so:

```
rf_preds = predict(model_rf, data=ca_holdout)
rf_preds
```

Ranger prediction

```
Type: Regression
Sample size: 118805
Number of independent variables: 25
```

```
head(predictions(rf_preds))
```

```
[1] 3481.162 3594.208 3517.143 3432.828 3335.656 3394.036
```

```
RMSE(predictions(rf_preds), ca_holdout$child_birthwt)
```

```
[1] 546.1326
```

Note how we get a package of predictions from `predict` and then need to get the actual predictions out of the package with `predictions()`. Also note the capital RMSE method in `caret` is different from the `rmse` from the `modelr` package.

Different packages are all slightly different. Keep reference code like this to easily remember how to do basic coding tasks.

## 55.1 Tuning the random forest

We should tune our random forest, to figure out which specification is best. We use `caret`'s `train()` to do this (borrowing code from the case-study walk-through from the Data Science in Education textbook):

```
setting a seed for reproducibility
set.seed(2020)
```

```
Create a grid of different values of mtry, splitrule, and min.node.size,
the three tuning parameters for our random forest.
```

```
tune_grid <-
 expand.grid(
 mtry = c(2, 3, 7, 10),
 splitrule = c("variance"),
 min.node.size = c(10, 20, 50)
)

Fit a new model, using the tuning grid we created above. This will take
awhile to run.
rf_tuned <-
 train(child_birthwt ~ ., data = ca_training,
 method = "ranger",
 tuneGrid = tune_grid)
```

```
rf_tuned
```

Random Forest

10000 samples  
25 predictor

No pre-processing

Resampling: Bootstrapped (25 reps)

Summary of sample sizes: 10000, 10000, 10000, 10000, 10000, 10000, ...

Resampling results across tuning parameters:

mtry	min.node.size	RMSE	Rsquared	MAE
2	10	560.5576	0.08226263	421.6399
2	20	560.7701	0.08165209	421.7262
2	50	561.0561	0.08155258	421.8141
3	10	557.0580	0.08455732	420.1078
3	20	557.0095	0.08532631	419.9823
3	50	557.4392	0.08517044	419.9752
7	10	557.9592	0.08025062	423.9294
7	20	556.6819	0.08320896	422.4458
7	50	555.3160	0.08691731	420.4274
10	10	559.3643	0.07946736	426.1047
10	20	557.4722	0.08297020	424.0431
10	50	555.1987	0.08782704	421.2257

Tuning parameter 'splitrule' was held constant at a value of variance  
RMSE was used to select the optimal model using the smallest value.

The final values used for the model were `mtry = 10`, `splitrule = variance` and `min.node.size = 50`.

We then take our final random forest, fit to all of our training data and using the winning tuning parameters, by simply using `predict` from the result of our `train()` call:

```
rmse(rf_tuned, ca_holdout)
```

```
[1] 547.0557
```

## 56 Comparing our machines

Now let's compare the RMSE for everyone! We add in a null model of predicting the grand mean to give a reference of how well we do when we do nothing. We can even compare all the RMSEs to the one of the mean as a point of reference:

```
model_null = lm(child_birthwt ~ 1, data=ca_training)
rmse_mean = rmse(model_null, ca_holdout)

RMSEs = c(mean = rmse_mean,
 OLS = rmse(model_linear, ca_holdout),
 OLS.quad = rmse(model_linear_int, ca_holdout),
 forward = rmse(mod_forward, ca_holdout),
 ridge = rmse_by_hand(model_ridge_optimal, x_holdout, y_holdout),
 lasso = rmse_by_hand(model_lasso_optimal, x_holdout, y_holdout),
 CART = rmse(model_tree, ca_holdout),
 CART.prune = rmse(rpart_cv, ca_holdout),
 RF = RMSE(predictions(rf_preds), ca_holdout$child_birthwt),
 RF_tuned = rmse(rf_tuned, ca_holdout))
```

```
Warning in predict.lm(model, data): prediction from rank-deficient fit; attr(*,
"non-estim") has doubtful cases
```

```
results = tibble(model = names(RMSEs),
 RMSE = RMSEs,
 perDecr = 100 * RMSE / rmse_mean)
knitr::kable(results, digits=1)
```

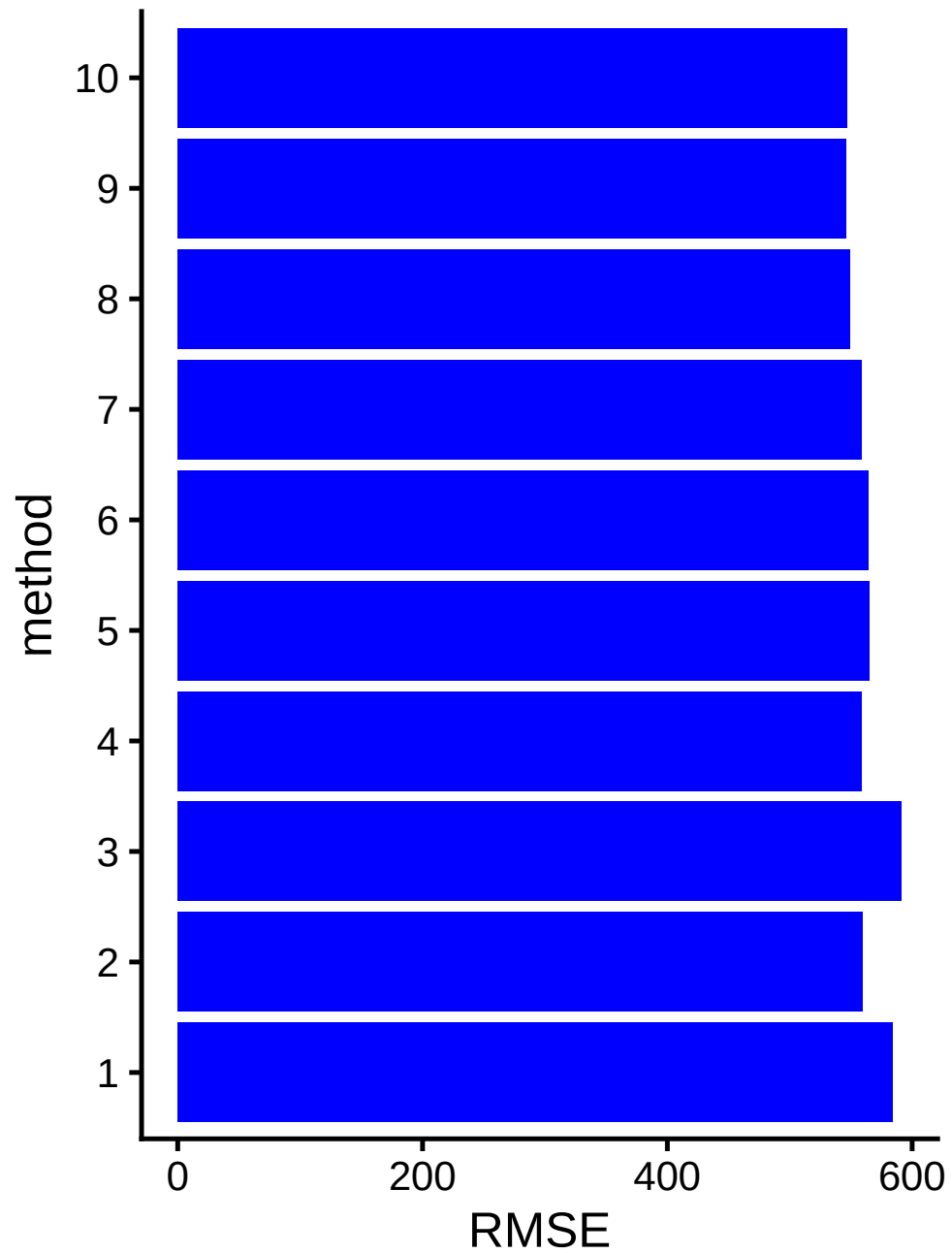
model	RMSE	perDecr
mean	584.1	100.0
OLS	559.4	95.8
OLS.quad	591.3	101.2
forward	559.0	95.7
ridge	565.1	96.8
lasso	564.7	96.7
CART	558.8	95.7

model	RMSE	perDecr
CART.prune	548.9	94.0
RF	546.1	93.5
RF_tuned	547.1	93.7

None of them are spectacular, but of them all, random forest wins, with a reduction of 6.5% in the RMSE over doing nothing. Interestingly, the tuned random forest was a tiny bit worse than the out-of-box random forest. The difference is small, so this is likely just due to instability in the tuning.

A plot of the comparisons follows. Note how the bars are similar heights, indicating that nothing is a slam dunk over, even, just predicting the mean. The interacted OLS model is so overfit it does *worse* than the mean!

```
results = rownames_to_column(results, var="method")
results$method = reorder(results$method, 1:nrow(results))
ggplot(results, aes(x = method, y = RMSE)) +
 geom_col(fill = "blue") +
 coord_flip()
```



A different way of thinking about the RMSE is that it is a measure of baseline variation:

```
sd(ca_holdout$child_birthwt)
```

```
[1] 584.053
```



This motivates why it is a reasonable baseline RMSE to compare the other RMSEs to.

## **57 Reflections on using machine learning (for final projects)**

## 58 Test/train splits, test/train/validate, and cross validation

It is important to track what your machine learning methods are doing in terms of splitting your data inside the call. For example, the `caret` package's `train()` method will often be doing cross validation (or something similar) inside of it. You thus do not need to always do a test/train split outside of it!

Also, don't do a test/train split if you don't have a lot of data. It is too expensive! This is where cross validation is particularly useful, since it lets all of your data be used for testing, and it uses most of your data for fitting your models. It is kind of like a "have your cake and eat it too" situation.

Finally, the full test/train/validate trifecta is only if you are really, really concerned about predicting your future performance of a model. It turns out that if you use cross-validation to select the best set of tuning parameters, the final estimated accuracy from the cross-validation will generally be pretty close to what a final validation set would tell you.

Let's see how the case study in [Data Science in Ed, Chapter 14](#) went about things. They first split the data into a training set and a testing set. They then made a grid of tuning parameters related to tree size and so forth, and then used `train()` to identify what set of tuning parameters was best for the data. In particular, `train()` used this grid and a variant of cross validation (bootstrap resampling) to repeatedly divide the training data into a part used to fit one random forest for each tuning parameter combo and a part used to evaluate all of those random forests on out of sample data. The final table produced gives the estimated error rates, and finally `train()` selects the tuning parameter combo with the best rates.

The book then took the final model (via `rf_fit2$finalModel`), which is a last model fit by `train()` to all the training data using the best found parameter combo, as their answer. To evaluate the quality of their answer, they used `finalModel` to predict outcomes for the test data they set aside at the beginning of their case study.

In our language, their "test" data was used for the final validation step, and the `train()` method was doing little test/train splits inside of it to get the estimates of out of sample error as part of the tuning process.

Note that they also found the predicted error of the finally selected final model was about the same as reported by the out of sample error by `train()`: this is not unusual. If we are not

fitting too many different combos of tuning parameter and so forth, then the estimated error from this process will often be close.

## 58.1 Cross validation options for caret

The `caret` package does cross validation internally as part of `train()`, if you tell it. If you don't, it does a different kind of internal repeated test/train splitting that it calls bootstrap. This is not bootstrap for inference! What it is doing is resampling the data with replacement to get a training dataset of the same size as it was passed. It will then have about a third of the data not in the training set, and it will use this for out-of-sample testing to estimate the performance of all the models fit. Finally, it repeats this a bunch of times and averages: this means each observation will be in most training sets, but will be used for testing some of the time. This is just like cross validation (except more random)!

Also, `caret`'s cross-validation is, by default, a random cross-validation: repeatedly take 10% of the data as testing, and use the rest for training. Repeat 10 times. This means each observation will be used about once for testing and about nine times for training, but not exactly. Functionally, this will generally be nearly the same as the classic CV where we divide all the data into 10 parts systematically.

It basically does not matter which form of cross-validation or splitting you use. The number of iterations will impact running time: 25 iterations will be 2.5 times longer to run than 10!

## 58.2 Bootstrap vs. Cross validation

Don't confuse bootstrapping with cross-validation. Bootstrapping is a way of doing statistical inference: you use it to ask how much an estimate would change if you happened to get a different data set from the same source. This allows you to decide if, for example, a coefficient is "really" positive—if your bootstrapping doesn't really give you any negative estimates, then you can be sure that your estimate is probably not positive due to random chance.

Cross validation, by contrast, is a way of doing a lot of test/train splits because you want to know how a fit model would work on new data. It is focusing on estimating future predictive accuracy, not statistical inference.

## 59 Determining what variables are important

The Lasso is a sparse regression approach: as part of fitting a lasso model, you are given a subset of your original variables deemed important enough to include in the predictive model. Random forests, by contrast, allow for a second step of generating a variable importance plot, where you get a measure of how useful each variable was for making predictions.

Both these tools can be useful for identifying factors particularly tied to your outcome. In general, you would expect those variables kept by Lasso to also have high scores in a variable importance plot. Both approaches, however, have some caveats that one should think about when interpreting results.

First, the Lasso approach gives you a final complete model built out of only a few variables. Each kept variable comes with a coefficient, and some of those coefficients will be tiny and others larger. Before directly comparing coefficients, however, spend a moment to think about how the scale of each variable matters.

For example, in the following fake dataset each variable (other than the last) has the same impact on the outcome. Put another way, they are all about the same in terms of their correlation with the outcome, as shown by the last column of the output.

```
library(glmnet)

tb = data.frame(X1 = rnorm(1000),
 X2 = rnorm(1000, sd = 10),
 X3 = rnorm(1000, sd = 100),
 X4 = rnorm(1000),
 X5 = rnorm(1000))
tb$X4_proxy = tb$X4 + rnorm(1000, sd=0.2)
tb$Y = with(tb, X1 + 0.1 * X2 + 0.01 * X3 + X4 + rnorm(1000))

cor(tb) %>%
 knitr::kable(digits=2)
```

	X1	X2	X3	X4	X5	X4_proxy	Y
X1	1.00	-0.02	-0.01	0.02	-0.02	0.02	0.44
X2	-0.02	1.00	0.02	0.03	-0.02	0.02	0.47
X3	-0.01	0.02	1.00	0.00	0.03	0.00	0.43

	X1	X2	X3	X4	X5	X4_proxy	Y
X4	0.02	0.03	0.00	1.00	0.01	0.98	0.48
X5	-0.02	-0.02	0.03	0.01	1.00	0.01	0.00
X4_proxy	0.02	0.02	0.00	0.98	0.01	1.00	0.47
Y	0.44	0.47	0.43	0.48	0.00	0.47	1.00

We can fit a Lasso model as so:

```
Xmat = model.matrix(Y ~ ., data=tb)

mod <- cv.glmnet(x = Xmat[,-1], y = tb$Y)

coef(mod)
```

```
7 x 1 sparse Matrix of class "dgCMatrix"
 lambda.1se
(Intercept) -0.005395787
X1 0.908085579
X2 0.091508205
X3 0.008841278
X4 0.728966264
X5 .
X4_proxy 0.181158483
```

Note how we have the proxy, a little bit, and also X3. The proxy looks more important than X3 (but actually isn't important). X3 looks unimportant because the scale of X3 is so large. One fix is to standardize your covariates before putting them into your lasso:

```
tb2 <- tb %>%
 mutate(X2 = scale(X2),
 X3 = scale(X3))
Xmat2 = model.matrix(Y ~ ., data=tb)
mod2 <- cv.glmnet(x = Xmat2[,-1], y = tb2$Y)
coef(mod2)
```

```
7 x 1 sparse Matrix of class "dgCMatrix"
 lambda.1se
(Intercept) -0.005395787
X1 0.908085579
X2 0.091508205
X3 0.008841278
```

```

X4 0.728966264
X5 .
X4_proxy 0.181158483

```

If we use random forests and a variable importance plot, we get this: ::: {.cell layout-align="center"}

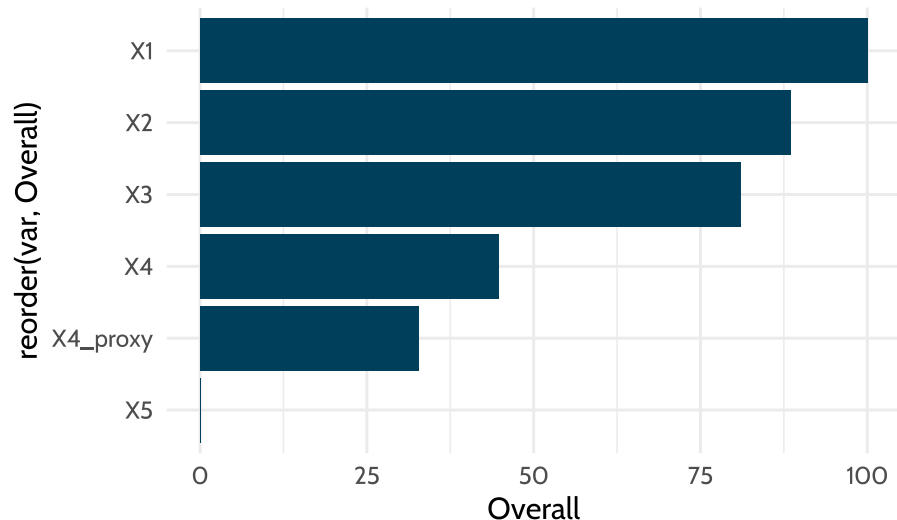
```

set.seed(2020)
library(caret)

rf_fit2_imp <- train(Y ~ .,
 data = tb,
 method = "ranger",
 importance = "permutation")

varImp(rf_fit2_imp) %>%
 pluck(1) %>%
 rownames_to_column("var") %>%
 ggplot(aes(x = reorder(var, Overall), y = Overall)) +
 geom_col(fill = dataedu_colors("darkblue")) +
 coord_flip() +
 theme_dataedu()

```



:::

The variable importance plot correctly puts X1, X2, and X3 all as about the same importance (they are: the larger variance perfectly balances out the smaller coefficient). The X4 variable

gets dinged a bit because the `X4_proxy` variable can serve almost as well for prediction, so they both are deemed important.



## 60 Sensitivity check on trees

Say you want to present a tree in your final project to show how an outcome relates to different variables of interest. Trees, unfortunately, are notoriously unstable and thus you should see if you would get a very different tree with slight changes to your data.

Fortunately, it is easy to check this with the following:

Once you have your final tuning parameters and so forth set, repeat the following 5 times: \* Bootstrap your data. \* Fit a tree to the bootstrapped data, using the tuning parameters you selected. \* Make a plot of your bootstrapped tree.

Now check to see if your five trees are basically similar, or entirely different, from each other. This is called a *stability check* and is a useful way of seeing if your results are particularly sensitive to mild changes in data.

## 61 A cool and easy way to do causal inference type stuff

Consider the problem of comparing Lyft vs Uber prices given a dataset of rides. You can't just compare the average price in each type because it is possible that, for example, Lyft rides are systematically shorter than Uber.

One direction to take is to fit two random forests, one for your Lyft data and one for your Uber data, to predict price given distance, time, weather, starting location, and anything else you have as a covariate. You then use each random forest to predict price for each observation in your entire dataset. You now have two predictions, one from your Lyft model and one from your Uber model, for each observation. Finally, you compare to see if the predictions are different on average. This gives you a “controlled comparison” where you are asking if there are systematic shifts in cost for otherwise similar things.

## 62 Interpreting LASSO models, and some other stuff

Back in Lab 10, or thereabouts, we walked through a LASSO model fit to student level data from the PISA exam. For our predictive task, we imagined that these students come from a single country, but in fact, we combined data from multiple nations. I sourced these data using the `learningtower` package.

## 63 The Story

You are newly hired analyst working in the State Ministry of Education. This morning, you open your email to find a request from your supervisor, your first project in the new job. How exciting!

Your boss writes that the Ministry has received funding to provide a one-on-one tutoring intervention for a modest number of struggling 8th grade math students. It's important that we select the right students for the intervention because we can't afford to give it to everyone. Unfortunately, we don't have previous math scores for students in the state. The first standardized math test they take is at the *end* of 8th grade.

Your boss proposes that you build a predictive model, using the other data the ministry has about students, to estimate who will perform poorly in math at the end of the 8th grade.

The only other requirement is that the model can be explained to policymakers and other higherups in the ministry. No black-box models.

Her email includes an attachment with the data on last year's 8th graders.

Our goal is to understand what is predictive of math scores. More specifically, we want to predict who the struggling students will be. There are a lot of things we could do. Remember whatever we do must be interpretable/explainable. Trees might work. Of course we'll settle on LASSO here. Generally speaking, LASSO is more interpretable than Ridge? (LASSO shrinks many predictors to zero, so we only have to explain the non-zero coefficients, which hopefully will be a small number.)

## 64 Fitting and interpreting LASSO

Here is a block of code that (without any actual exploration, etc.—please see the lab for that) fits the LASSO model.

```
stu_data <- read_csv('data/student_data.csv')
stu_data = stu_data[,-c(1:2)]
stu_data <- na.omit(stu_data)

x <- model.matrix(math ~ ., stu_data)[,-1]
y <- stu_data$math

set.seed(42120)

cross_validation_results_lasso <- cv.glmnet(x = x,
 y = y,
 alpha = 1, # 1 for LASSO, 0 for ridge
 nfolds = 10,
 type.measure = "mse")

print(cross_validation_results_lasso)
```

Call: `cv.glmnet(x = x, y = y, type.measure = "mse", nfolds = 10, alpha = 1)`

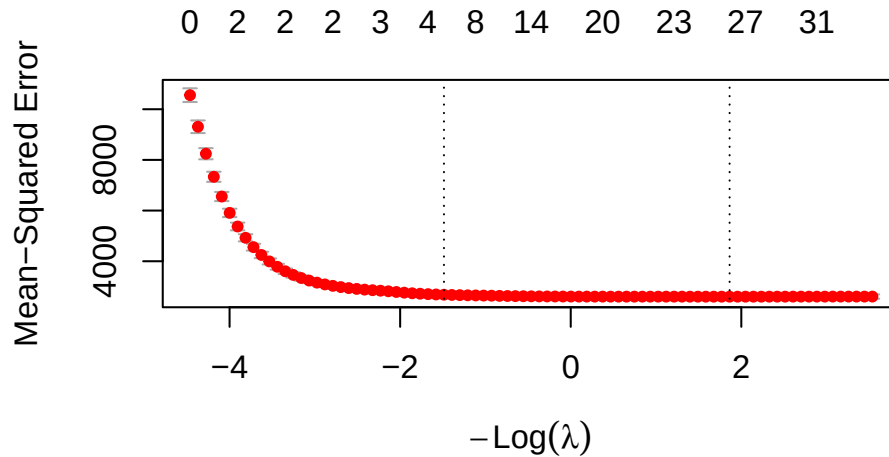
Measure: Mean-Squared Error

	Lambda	Index	Measure	SE	Nonzero
min	0.155	69	2601	85.10	27
1se	4.422	33	2678	87.96	5

In the above we loaded data, dropped anything with missing data (sad!) and fit LASSO, tuning with cross validation. We get an estimated Lambda of 4.422 using the 1SE rule.

We like plots, so we generate this one:

```
plot(cross_validation_results_lasso)
```



What does it tell us? Mainly, it says that small amounts of regularization are about the same as larger amounts—note the long flat part of the curve. This means that we don't lose much if we regularize more (up to where it curves up). Regularizing more means fewer nonzero coefficients, and easier interpretation! If we regularize too much, however, our RMSE goes up quite fast.

But this plot is not particularly useful for our eventual consumers. They don't want to know that you tuned your model; they want to know what your eventual model says! These plots, therefore, do not belong in a final report.

What we want is our final model:

```
best_lambda_lasso <- cross_validation_results_lasso$lambda.1se
final_model_lasso <- glmnet(x = x,
 y = y,
 alpha = 1,
 lambda = best_lambda_lasso)

coef(final_model_lasso)
```

```
32 x 1 sparse Matrix of class "dgCMatrix"
 s0
(Intercept) 78.3051308
mother_educISCED 2 .
mother_educISCED 3A .
mother_educISCED 3B, C .
mother_educless than ISCED1 .
father_educISCED 2 .
father_educISCED 3A .
father_educISCED 3B, C .
father_educless than ISCED1 .
```

gendermale	9.0804121
computeryes	0.5051019
internetyes	.
read	0.3432561
science	0.4877308
deskyes	.
roomyes	.
television1	.
television2	.
television3+	.
computer_n1	.
computer_n2	.
computer_n3+	.
car1	.
car2	.
car3+	.
book101-200	.
book11-25	.
book201-500	.
book26-100	.
bookmore than 500	.
wealth	.
escs	3.0051742

We can print only the nonzero elements as so:

```
cc <- coef(final_model_lasso)
cc <- cc[cc[,1] != 0,]
cc
```

(Intercept)	gendermale	computeryes	read	science	escs
78.3051308	9.0804121	0.5051019	0.3432561	0.4877308	3.0051742

LASSO selected only a few of the many predictors available. In particular, the main predictors of math score are reading in science score, the student's economic, social and cultural status (ESCS), whether they have a computer, and whether they are assigned male in the data. Given this information, the Lasso model is saying that knowing how many televisions, cars, books, or computers are in the family is not useful for predicting math score. Neither is the mothers or fathers education level.

As an aside, it's generally hard to use categorical variables – continuous variables are easier to select because a single variable can describe a lot of variation. So this might be why these other variables are not being picked.

## 64.1 What variables to include?

We might not want to put all of our available variables into our model. For example, if we are trying to predict math score, we might not want to include other test scores from that same testing occasion, because we're looking for contextual factors that lead to a particular outcome. Thus, we might rerun everything we've done without including reading and science, which would force the model to use the contextual variables about the students rather than other measures of their academic ability.

## 64.2 Coefficient size

The SIZE of the coefficients is going to be too small, in general, because LASSO shrinks coefficients towards zero to stabilize. We should not, for example, expect that the increase in math for each unit increase in reading is 0.3432561. One trick some folks do is *refit* the selected coefficients using OLS to get a new, final model:

```
Identify nonzero coefficients
coef_indices <- which(colnames(x) %in% names(cc))
coef_indices
```

```
[1] 9 10 12 13 31
```

```
Subset data to nonzero covariates
x_sub <- x[, coef_indices]
head(x_sub)
```

	gendermale	computeryes	read	science	escs
1	1	1	347.710	273.827	0.7018
2	0	1	427.958	460.357	0.2091
3	1	1	271.743	337.150	-1.1074
4	1	1	336.271	368.131	-0.4944
5	1	1	402.166	322.163	-0.1840
6	0	1	350.848	342.939	-0.6022

```
Fit OLS model to nonzero covariates
fit_sub <- lm(y ~ ., data = as.data.frame(x_sub))
```

```
Print summary of refit model
summary(fit_sub)
```



```

Call:
lm(formula = y ~ ., data = as.data.frame(x_sub))

Residuals:
 Min 1Q Median 3Q Max
-189.522 -32.514 -0.457 32.687 239.605

Coefficients:
 Estimate Std. Error t value Pr(>|t|)
(Intercept) 51.00114 5.30280 9.618 < 2e-16 ***
gendermale 19.31204 1.86074 10.379 < 2e-16 ***
computeryes 8.01858 2.71953 2.949 0.00322 **
read 0.38790 0.01883 20.602 < 2e-16 ***
science 0.47816 0.01944 24.596 < 2e-16 ***
escs 4.52125 0.97928 4.617 4.04e-06 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 51.17 on 3309 degrees of freedom
Multiple R-squared: 0.7529, Adjusted R-squared: 0.7525
F-statistic: 2016 on 5 and 3309 DF, p-value: < 2.2e-16

```

We can compare how much shrinkage there was:

```

ols_coef = coef(fit_sub)
ctbl = tibble(coef = names(ols_coef),
 OLS = ols_coef,
 Lasso = cc)
ctbl

```

```

A tibble: 6 x 3
 coef OLS Lasso
<chr> <dbl> <dbl>
1 (Intercept) 51.0 78.3
2 gendermale 19.3 9.08
3 computeryes 8.02 0.505
4 read 0.388 0.343
5 science 0.478 0.488
6 escs 4.52 3.01

```

## 64.3 Assessing stability (a kind of inference)

We might worry that the Lasso model is picking these variables only due to random chance. We could bootstrap this process to see if the same variables tend to get picked each time. Here is a single bootstrap iteration to illustrate:

```
datstar = slice_sample(stu_data, n = nrow(stu_data), replace=TRUE)
xstar <- model.matrix(math ~ ., datstar)[,-1]
ystar <- datstar$math

modstar <- glmnet(x = xstar,
 y = ystar,
 lambda = best_lambda_lasso,
 alpha = 1)

cc <- coef(modstar)
cc[cc[,1] != 0,]
```

(Intercept)	father_educless	than ISCED1
83.6514190		-2.6175776
gendermale		read
6.5947801		0.3550822
science		escs
0.4687787		3.3871490

We generally got the same answers, suggesting stability in what variables are selected. But we should do this a lot, and see how often each variable shows up. To do this, we need a function! And, for our function, life is always easier if we get a data frame back:

```
one_boot <- function() {
 datstar = slice_sample(stu_data, n = nrow(stu_data), replace=TRUE)
 xstar <- model.matrix(math ~ ., datstar)[,-1]
 ystar <- datstar$math

 modstar <- glmnet(x = xstar,
 y = ystar,
 lambda = best_lambda_lasso,
 alpha = 1)

 cc <- coef(modstar)
 cc <- cc[cc[,1] != 0,]
 cc = tibble(coef = names(cc), est = cc)
}
```

Then replicate:

```
rps = map_df(1:100, ~ one_boot())
rps %>% group_by(coef) %>%
 summarise(n = n()) %>%
 arrange(-n) %>%
 knitr::kable()
```

coef	n
(Intercept)	100
gendermale	100
read	100
science	100
escs	99
computeryes	58
wealth	44
car2	26
internetyes	14
father_educless than ISCED1	13
deskyes	6
mother_educless than ISCED1	5
book201-500	2
computer_n3+	1
mother_educISCED 3A	1

The things at the top were *always* selected—they are clearly important. The things near the bottom are less so. Our initial “computer” variable seems somewhat important, but not as reliably selected as things such as reading and science.

## 64.4 Last words on writing up results

In writing up your results, consider the following elements:

1. Introduction: Begin by introducing the purpose of your analysis and the dataset you used. For example:

“We fit a Lasso regression model to predict the response variable [name of response variable] based on [list of predictor variables]. The dataset used was [name of dataset] and consisted of [number of observations] observations.”

2. Model Selection: Explain the process you used to select the best model, including the choice of the regularization parameter. For example:

“We used cross-validation to select the value of lambda that minimized the mean squared error. The selected lambda was [value].”

3. Model Performance: Report the performance of the Lasso model, including the model’s R-squared value, the root mean squared error (RMSE), and any other relevant metrics. For example:

“The Lasso model achieved an R-squared value of [R-squared value], indicating that [percentage of variation in response variable] of the variation in the response variable can be explained by the predictor variables. The RMSE of the model was [RMSE value], indicating an average error of [RMSE value] units in predicting the response variable.”

You can calculate R<sup>2</sup> by comparing the variance of the predictions to the variance of the original outcome:

$$R^2 = 1 - \frac{Var(predictions)}{Var(Y)}$$

4. Interpretation of Coefficients: Report the estimated coefficients. For example:

“Table XX shows the estimated coefficients for the predictor variables.”

You might include OLS values as well, on this table.

5. Conclusion: Summarize the key findings of your analysis and any insights gained from the Lasso model. For example:

“In conclusion, we found that [some variables] predicted our outcome, suggesting [something]. On the other hand, we expected [something else] to be selected, but it never was across the bootstrap samples, indicating that, given the other variables, there is no strong connection between [this thing] and the outcome.”

Remember to provide enough detail for the reader to understand your analysis, but keep your report concise and focused on the key points.

**Part VII**

**On the Bootstrap**

## **65 Demonstration of the Cluster Bootstrap**

## 66 The Cluster Bootstrap

The cluster bootstrap is a resampling technique commonly used in statistics and machine learning for estimating the variability of statistical estimators, such as mean, variance, or regression coefficients. It is especially useful when the underlying data have a complex dependence structure.

The main idea of the cluster bootstrap is to resample clusters of data points rather than individual data points. A cluster is a group of data points that are correlated with each other, for example, spatially adjacent data points or data points within the same experimental unit.

To perform the cluster bootstrap, we follow these steps:

1. Divide the original data set by the clustering variable, with each cluster containing correlated data points.
2. Randomly select clusters from the original data set with replacement, forming a bootstrap sample with the same number of clusters (but not necessarily the same number of individuals, if cluster sizes vary). If a cluster gets resampled twice, each time would be given a different, new, cluster id, so the data still has the right number of clusters and so clusters don't end up larger than in the original data.
3. Compute the statistical estimator of interest on the bootstrap sample.
4. Repeat steps 2 and 3 a large number of times, say 1000 times, to obtain a distribution of the estimator.
5. Use the distribution of the estimator to construct confidence intervals or calculate standard errors.

The cluster bootstrap is particularly useful when the underlying data have a complex correlation structure that cannot be easily accounted for using standard resampling methods, such as the simple random sampling bootstrap.

The following script is used to explore data from the Tenn Star education randomized controlled trial (RCT). It loads student-level and teacher-level data, merges them by the class ID variable, deletes observations with missing data, and creates an indicator variable for whether the student was in a small class (this is the treatment). The script then fits a linear regression model to estimate the impact of being in a small class on math scores after kindergarten, using small class status, student birth quarter, teacher education level, and teacher experience as predictors.

However, the model is not adjusted for clustering at the class level, so the standard errors may be wrong. To correct for this, we offer two solutions. First, the sandwich package is used to calculate cluster-robust standard errors. This is the classic econometric solution for clustering in a regression. Second, we demonstrate cluster bootstrapping for estimating the standard errors of the model coefficients.

## 66.1 The Tenn Star Data

The Tennessee Star Experiment was a large-scale randomized controlled trial (RCT) conducted in Tennessee, USA, in the mid-1980s. The experiment aimed to evaluate the effectiveness of class size reduction on student outcome. Students and teachers were randomized to different class sizes, making the treatment assignment effectively assigned at the cluster level when looking at student outcomes.

We load and prepare the data as so:

```
stud <- read.csv('data/tenn_stud_dat.csv') ## student-level data
teach <- read.csv('data/tenn_teach_dat.csv') ## teacher-level data
dat <- merge(stud, teach, by="clid")
dat <- na.omit(dat) ## for simplicity, we'll delete everyone missing any variables

create an indicator for being in a small class
dat$small_class <- ifelse(dat$cltypek == 'small class', "yes", "no")

Drop some extra variables
dat <- dplyr::select(dat, -id, -cltypek, -sesk)

head(dat)
```

	clid	ssex	race		sbirthq	sbirthy	treadssk	tmathssk
1	1	male	black	2nd qtr - april,may,june	1980		445	489
2	1	male	black	3rd qtr - july,aug,sept	1980		393	429
3	1	male	black	4th qtr - oct,nov,dec	1978		395	429
4	1	male	black	2nd qtr - april,may,june	1980		403	405
5	1	female	black	2nd qtr - april,may,june	1980		424	500
6	1	male	black	3rd qtr - july,aug,sept	1980		414	444
	hdegk	cladk	totexpk	tracek	small_class			
1	bachelors	apprentice	1	black	no			
2	bachelors	apprentice	1	black	no			
3	bachelors	apprentice	1	black	no			
4	bachelors	apprentice	1	black	no			
5	bachelors	apprentice	1	black	no			



```
6 bachelors apprentice 1 black no
```

```
length(unique(dat$clid)) ## 196 classes
```

```
[1] 196
```

```
nrow(dat) ## 3219 students
```

```
[1] 3219
```

Our research question is whether student math achievement was higher for kids in small classrooms vs. large ones.

## 66.2 The Wrong Method

If we use simple regression we are not taking the correlation of students within a given teacher into account. I.e., say a teacher happened to be effective. Then this teacher being assigned to a small class would have a positive impact on a bunch of treated students due to the single teacher. The student outcomes are correlated with each other by the single teacher. We would need to take this into account when calculating uncertainty: the students are not independent.

```
mod <- lm(tmathssk ~ small_class + sbirthq + hdegk + totexpk,
 data = dat)
res_OLS <- tidy(mod)
knitr::kable(res_OLS, digits = 2)
```

term	estimate	std.error	statistic	p.value
(Intercept)	486.49	2.33	208.38	0.00
small_classyes	6.39	1.69	3.79	0.00
sbirthq2nd qtr - april,may,june	-5.64	2.33	-2.42	0.02
sbirthq3rd qtr - july,aug,sept	-13.85	2.27	-6.11	0.00
sbirthq4th qtr - oct,nov,dec	10.19	2.49	4.08	0.00
hdegkmaster +	-2.71	5.44	-0.50	0.62
hdegkmasters	-5.25	1.82	-2.89	0.00
hdegkspecialist	16.98	7.62	2.23	0.03
totexpk	0.58	0.15	3.88	0.00

To be clear, the above regression gives a reasonable *estimate* of the *impact* of small class size, but the standard errors, and therefore p-values, etc., are wrong.

## 66.3 Cluster robust standard errors

One statistical way of handling this is with the `lm_robust` package that uses the `sandwich` package that does cluster robust standard errors:

```
library(estimatr)
mod_CRVE <- lm_robust(tmathssk ~ small_class + sbirthq + hdegk + totexpk,
 clusters = dat$clid,
 data = dat) ## another way to get the same result (more or less)
res_CRVE <- tidy(mod_CRVE) %>%
 dplyr::select(term, estimate, std.error, p.value)
knitr::kable(res_CRVE, digits = 2)
```

term	estimate	std.error	p.value
(Intercept)	486.49	4.26	0.00
small_classyes	6.39	3.86	0.10
sbirthq2nd qtr - april,may,june	-5.64	2.27	0.01
sbirthq3rd qtr - july,aug,sept	-13.85	2.35	0.00
sbirthq4th qtr - oct,nov,dec	10.19	2.48	0.00
hdegkmaster +	-2.71	11.50	0.83
hdegkmasters	-5.25	4.15	0.21
hdegkspecialist	16.98	15.10	0.37
totexpk	0.58	0.35	0.11

## 66.4 Cluster Bootstrap

Another way is to use the cluster bootstrap. It is a versatile method for getting standard errors on data that is clustered, as it keeps clusters intact.

We write an analysis function as follows:

```
my_analysis <- function(the_dat) {
 mod <- lm(tmathssk ~ small_class + sbirthq + hdegk + totexpk,
 data = the_dat)
 broom::tidy(mod)
}
```

```
my_analysis(dat)
```

```
A tibble: 9 x 5
```

term	estimate	std.error	statistic	p.value
------	----------	-----------	-----------	---------

	<chr>	<dbl>	<dbl>	<dbl>	<dbl>
1	"(Intercept)"	486.	2.33	208.	0
2	"small_classyes"	6.39	1.69	3.79	0.000156
3	"sbirthq2nd qtr - april,may,june"	-5.64	2.33	-2.42	0.0155
4	"sbirthq3rd qtr - july,aug,sept"	-13.9	2.27	-6.11	0.00000000110
5	"sbirthq4th qtr - oct,nov,dec"	10.2	2.49	4.08	0.0000452
6	"hdegkmaster + "	-2.71	5.44	-0.497	0.619
7	"hdegkmasters"	-5.25	1.82	-2.89	0.00393
8	"hdegkspecialist"	17.0	7.62	2.23	0.0259
9	"totexpk"	0.579	0.149	3.88	0.000108

We then nest our data so each row is an entire cluster:

```
dat_nst <- dat %>%
 group_by(clid) %>%
 nest() %>%
 ungroup()
dat_nst
```

```
A tibble: 196 x 2
 clid data
 <int> <list>
1 1 <tibble [20 x 11]>
2 2 <tibble [22 x 11]>
3 3 <tibble [21 x 11]>
4 5 <tibble [23 x 11]>
5 6 <tibble [21 x 11]>
6 7 <tibble [18 x 11]>
7 9 <tibble [23 x 11]>
8 11 <tibble [19 x 11]>
9 13 <tibble [17 x 11]>
10 14 <tibble [17 x 11]>
i 186 more rows
```

We can then bootstrap our data as so:

```
one_cluster_boot <- function() {

 dat_nst_star = slice_sample(dat_nst, n = nrow(dat_nst), replace=TRUE)
 dat_nst_star$clid = 1:nrow(dat_nst_star)

 dat_star <- unnest(dat_nst_star, cols="data")
```

```

 my_analysis(dat_star)
}

```

Note we are regenerating the cluster ID so if we have the same cluster multiple times, each time gets a new ID.

We bootstrap and analyze a bunch of times and get standard errors:

```
boots = map_df(1:1000, ~ one_cluster_boot(), .id = "boot")
```

```

res_boot <- boots %>% group_by(term) %>%
 summarise(SE = sd(estimate))
res_boot

```

```

A tibble: 9 x 2
 term SE
 <chr> <dbl>
1 "(Intercept)" 4.38
2 "hdegkmaster + " 12.7
3 "hdegkmasters" 4.08
4 "hdegkspecialist" 14.0
5 "sbirthq2nd qtr - april,may,june" 2.34
6 "sbirthq3rd qtr - july,aug,sept" 2.34
7 "sbirthq4th qtr - oct,nov,dec" 2.37
8 "small_classyes" 3.77
9 "totexpk" 0.363

```

We can compare to the original (WRONG) OLS estimates, the CRVE standard errors, and the bootstrap standard errors:

```

CRVE_sub <- res_CRVE %>%
 dplyr::select(-estimate, -p.value) %>%
 rename(SE_CRVE = std.error)
OLS_sub <- res_OLS %>%
 dplyr::select(-estimate, -p.value) %>%
 rename(SE_OLS = std.error)

Make the table
tbl <- left_join(res_boot, OLS_sub,
 by = "term") %>%
 left_join(CRVE_sub, by = "term") %>%
 relocate(term, statistic) %>%

```

```
mutate(boot_v_OLS = SE / SE_OLS,
 boot_v_CRVE = SE / SE_CRVE)

knitr::kable(tbl, digits=2)
```

term	statistic	SE	SE_OLS	SE_CRVE	boot_v_OLS	boot_v_CRVE
(Intercept)	208.38	4.38	2.33	4.26	1.88	1.03
hdegkmaster +	-0.50	12.75	5.44	11.50	2.34	1.11
hdegkmasters	-2.89	4.08	1.82	4.15	2.24	0.98
hdegkspecialist	2.23	13.96	7.62	15.10	1.83	0.92
sbirthq2nd qtr -	-2.42	2.34	2.33	2.27	1.00	1.03
april,may,june						
sbirthq3rd qtr - july,aug,sept	-6.11	2.34	2.27	2.35	1.03	1.00
sbirthq4th qtr - oct,nov,dec	4.08	2.37	2.49	2.48	0.95	0.95
small_classyes	3.79	3.77	1.69	3.86	2.23	0.98
totexpk	3.88	0.36	0.15	0.35	2.43	1.03

A ratio of 1 means the estimated SEs are about the same. More than 1 means the bootstrap is returning larger SEs.

Generally, we see that the bootstrap is increasing the SEs for the level-2 coefficients (those that are talking about how clusters are different). This is good: the OLS SEs are way too small since they are not taking clustering into account.

The CRVE is basically the same as the bootstrap here, with some mild differences. All these methods are for estimating standard errors; they do not change the estimated coefficients themselves.

Bootstrapping is a simple way of getting uncertainty when you don't know how to do that with math or a package. When we can do a mathematical approximation, the bootstrapping might not be worth it, due to the extra computation. But bootstrapping, by direct simulation, can also account for things like heteroskedasticity, outliers, or other weirdness that the mathematical approximations cannot. It is worth using in many cases due to this general applicability, versatility, and robustness.

**Part VIII**

**Extra Stuff**

## 67 Cleaning Excel files

How do you cleaning excel files? This “handout” (actually an R script) loads in a csv file made from a dirty excel file and cleans it. This is a data cleaning problem I have seen a few final project groups face. Download these files, look at the original csv file, and then source the script and see what you get!

Resources

- [The dataset](#)
- [The script](#)

Here is the code, for reference. But download the script!

```

This script walks through a common type of data prep process.

If you have an excel file with merged cells and lots of tables in a big ugly
mass, then it can be hard to get those data "tidy". One way forward is to
first save the excel file as a CSV file, which breaks all the merged cells.
The content of the merged cell will usually show up in the leftmost cell,
which will give you a column of data that is a mix of table headers and table
content.

At this point, you read the ugly table into R and start identifying which
tables start and stop where. You can then slice out the relevant rows, and
reformat those to get a final table that you could use.

This script does the latter step, working with a sample csv file.

library(tidyverse)
dat = read_csv("sample_ugly_data.csv", col_names = FALSE)

names(dat)

dat

dat = filter(dat, !is.na(dat$X1))
```

```

which(str_detect(dat$X3, "junk word"))
dat = filter(dat, is.na(X3) | !str_detect(X3, "junk word"))
dat

Locate the rows that start a table, somehow.
#
table_rows = which(str_detect(dat$X1, "Table"))
table_rows
dat$X1[table_rows]

start = table_rows[[2]] + 1
stop = table_rows[[3]] - 1
subdat = dat[start:stop,]
subdat

Note: You might also detect specific rows via targeted searching. E.g.,:
start_row = which(str_detect(dat$X1, "Alpha"))
start_row

This will copy over the region names into a new "region" variable. We know a
region name when the X2 data are missing (table header rows often do not have
data in other cells in that row).
subdat = mutate(subdat,
 region = ifelse(is.na(X2), X1, NA))
subdat

This propagates the regions to all rows.
subdat = fill(subdat, region)
subdat

Now we drop the region header rows since we have a new column of that variable!
subdat = filter(subdat, !is.na(X2))
subdat

We next start dropping empty columns. Any column with a lot of missing data
will be considered junk. This counts the number of NAs in each column.
missings = apply(is.na(subdat), 2, sum)
missings

This keeps only those columns with fewer than 10 missing values.

```



```

subdat = subdat[, missings < 10]
subdat

We now name our columns by pulling off the first row that has the names. We
have to put our hand-built rowname back in, and then we drop the row and set
our column names!
names = as.character(subdat[1,])
names
names[length(names)] = "region"
names
names(subdat) = names

subdat = subdat[-1,]
subdat

second table, just like above

One might imagine making a function to do all this stuff...

start = table_rows[[3]] + 1
stop = nrow(dat)
subdat2 = dat[start:stop,]
subdat2

subdat2 = mutate(subdat2,
 region = ifelse(is.na(X2), X1, NA))
subdat2 = fill(subdat2, region)
subdat2 = filter(subdat2, !is.na(X2))
subdat2

missings = apply(is.na(subdat2), 2, sum)
missings

subdat2 = subdat2[, missings < 10]
subdat2

names = as.character(subdat2[1,])
names
names[length(names)] = "region"
names

```

```

names(subdat2) = names

subdat2 = subdat2[-1,]
subdat2

Merge our two tables

We merge by both region and location since we have duplicate location names
within different regions.
final_dat = left_join(subdat, subdat2, by=c("region", "Location"))
final_dat

final_dat = relocate(final_dat, region)

We finally convert our numeric columns to numbers.
final_dat = mutate(final_dat,
 Grade = parse_number(Grade),
 Size = parse_number(Size))

final_dat

Save your file and use the cleaned file for all your actual analysis!
write_csv(final_dat, file = "cleaned_data.csv")

```

## 68 Main effects, interactions in linear models, and prediction

This document is designed to address a problem that I have seen several times in the final projects. In particular, people will fit a sequence of models, and noticed that the main effect changes massively when they include an interaction term. This document shows a possible solution for this “problem,” a solution where you fit a model, and predict two outcomes for each observation, one with a dummy variable of interest and one without. By then looking at the average difference between these predictions, we can recover what we should think of as a main effect.

### 68.1 Getting the data ready

We use a data set of a bunch of behavioral outcomes measured for how a mother interacts with a child. One question of interest is whether mothers systematically interact with girls differently than they do with boys.

We first load tidyverse and load the data.

```
qm = read.table("data/KONG.txt")
names(qm) = c("wealth", "age", "fed", "med", "bookread", "hwhelp", "talkchld", "female", "sibling")
head(qm)
```

	wealth	age	fed	med	bookread	hwhelp	talkchld	female	sibling
1	11830	11.333330	9	9	2	2	2	0	1
2	8500	12.333330	2	5	2	2	2	1	1
3	20047	13.416670	9	8	2	2	2	1	1
4	18650	11.166670	0	12	2	2	2	1	1
5	6760	11.666670	5	0	1	2	1	0	1
6	4580	9.416667	8	8	2	2	2	1	1

```
nrow(qm)
```

```
[1] 1976
```

We make an aggregate measure of interaction by adding our three interview questions together. There are better ways of doing this, but this will work for now.

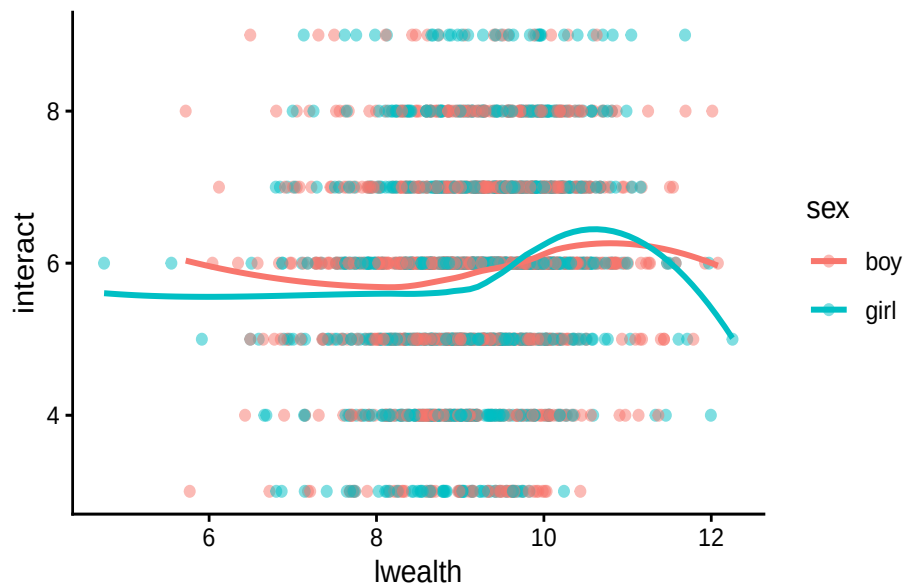
```
qm = mutate(qm, lwealth = log(wealth),
 interact = bookread + hwhelp + talkchld,
 sex = factor(female, levels=c(0,1), labels=c("boy","girl")))
```

To simplify things, we are going to take the middle 80% of the data based on wealth. (There is odd tail behavior that clouds the trends that I want to avoid for pedagogical purposes.)

```
qm = filter(qm, wealth >= quantile(wealth, 0.1), wealth <= quantile(wealth, 0.9)) nrow(qm)
```

Lets see what we have:

```
ggplot(qm, aes(x=lwealth, y=interact, col=sex)) +
 geom_point(alpha=0.5) +
 stat_smooth(aes(group=sex), method="loess", se=FALSE)
```



Lets look at a simple main effect of sex

```
M0 = lm(interact ~ sex + lwealth + sibling, data=qm)
summary(M0)
```

Call:

```
lm(formula = interact ~ sex + lwealth + sibling, data = qm)
```

Residuals:

Min	1Q	Median	3Q	Max
-3.4280	-0.9432	0.0462	0.9583	3.7106

Coefficients:

	Estimate	Std. Error	t value	Pr(> t )
(Intercept)	4.365853	0.311989	13.994	< 2e-16 ***
sexgirl	0.005984	0.062663	0.095	0.924
lwealth	0.197600	0.032674	6.048	1.75e-09 ***
sibling	-0.219954	0.043481	-5.059	4.62e-07 ***

---

Signif. codes: 0 '\*\*\*' 0.001 '\*\*' 0.01 '\*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 1.367 on 1972 degrees of freedom

Multiple R-squared: 0.03351, Adjusted R-squared: 0.03203

F-statistic: 22.79 on 3 and 1972 DF, p-value: 1.679e-14

## 68.2 Fitting an interacted model

Now let's fit a model where we interact sex and wealth. The plot does not look particularly linear, but we will proceed for illustrative purposes.

```
M1 = lm(interact ~ sex * lwealth + sibling, data=qm)
summary(M1)
```

Call:

```
lm(formula = interact ~ sex * lwealth + sibling, data = qm)
```

Residuals:

Min	1Q	Median	3Q	Max
-3.3704	-0.9534	0.0387	0.9547	3.7785

Coefficients:

	Estimate	Std. Error	t value	Pr(> t )
(Intercept)	4.80629	0.41588	11.557	< 2e-16 ***
sexgirl	-0.95389	0.60286	-1.582	0.113750
lwealth	0.14987	0.04422	3.389	0.000715 ***
sibling	-0.22096	0.04347	-5.083	4.06e-07 ***
sexgirl:lwealth	0.10468	0.06539	1.601	0.109569

---

```
Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
Residual standard error: 1.366 on 1971 degrees of freedom
Multiple R-squared: 0.03476, Adjusted R-squared: 0.0328
F-statistic: 17.74 on 4 and 1971 DF, p-value: 2.543e-14
```

Notice the coefficient for sex changes a lot. This is because this is the estimated difference of girls and boys *for those with a lwealth of 0*.

To recover our “main effect” estimate we need to see what the average predicted difference would be for all the individuals in our dataset. We do this by doing the following steps:

- (1) Copy our dataset to make two new ones

```
qm.g = qm.b = qm
```

- (2) Now make all the folks in the `qm.g` dataset girls, and all folks in the other boys.

```
qm.g$sex = "girl"
qm.b$sex = "boy"
```

- (3) Predict interaction assuming *everyone* is a girl and *everyone* is a boy:

```
pred.g = predict(M1, qm.g)
pred.b = predict(M1, qm.b)
```

- (4) Then the difference of the predictions is our predicted difference in interaction for two kids with the same covariates except sex:

```
deltas = pred.g - pred.b
sex.diff = mean(deltas)
sex.diff
```

```
[1] 0.006737327
```

See? Now it matches more closely with the main effect estimate of `M0`. The core idea here is that the real interpretation of the main effect is a difference in average outcomes (when looking at a dummy variable) between two groups, when holding all else equal. When you have interactions, the main effect is the predicted difference *for those with zeros for the interaction terms*, which might not be very sensible. We can get back to the difference between the two groups by predicting for everyone and comparing the averages.

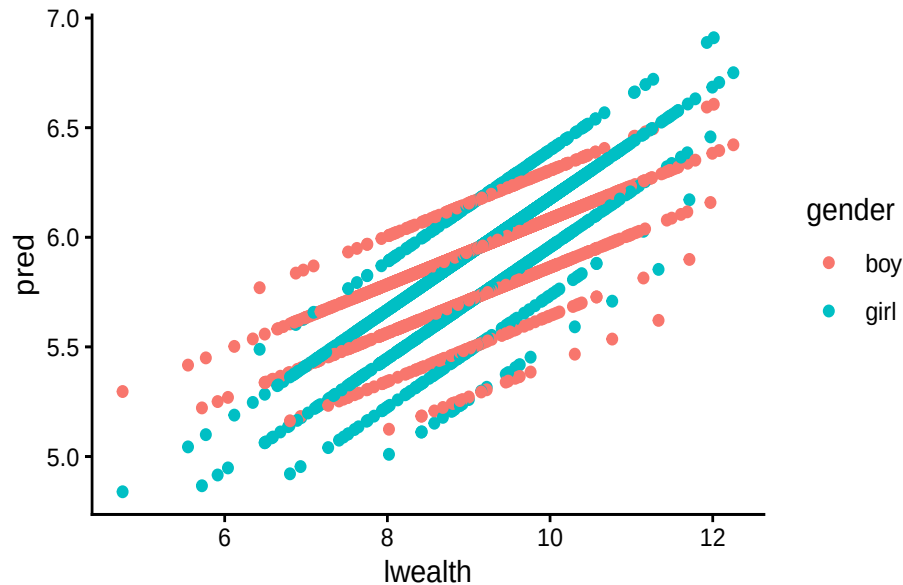
## 68.3 Plotting

As a reminder, we can plot with our predictions in a nice way. The cleanest would be to put all our predictions into a dataframe, convert to long form, and plot.

(Note we use 'gender' as our key, in the following, to avoid colliding with the 'sex' variable name.)

```
qm$girl = pred.g
qm$boy = pred.b
qml = qm %>%
 pivot_longer(cols = c(girl, boy),
 names_to = "gender",
 values_to = "pred")

ggplot(qml, aes(x=lwealth, y= pred, col=gender)) +
 geom_point()
```



We see different stripes for the different numbers of siblings.

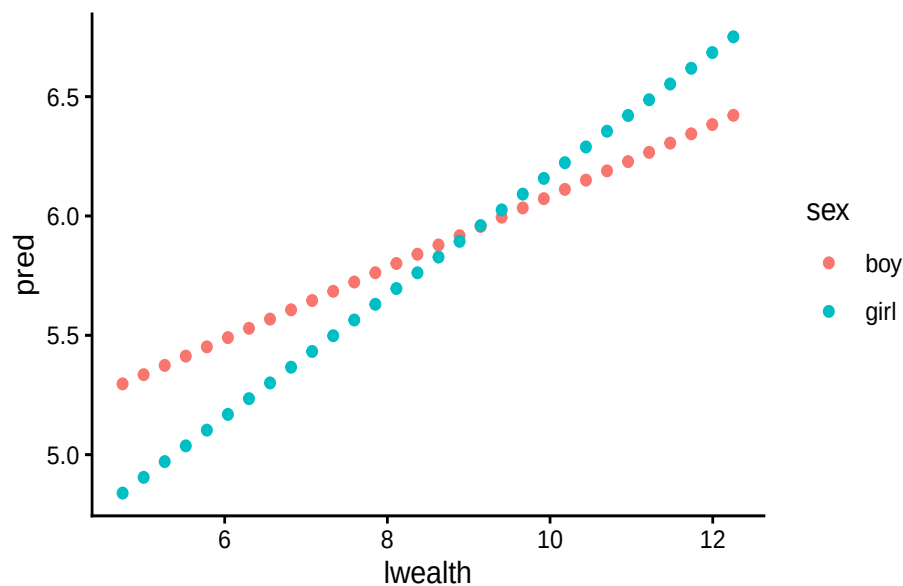
As a better way, we can get fancier with the data grid stuff.

```
library(modelr)
grid = data_grid(qm, lwealth = seq_range(lwealth, 30), sex, .model = M1)
head(grid)
```

```
A tibble: 6 x 3
 lwealth sex sibling
 <dbl> <fct> <dbl>
1 4.74 boy 1
2 4.74 girl 1
3 5.00 boy 1
4 5.00 girl 1
5 5.26 boy 1
6 5.26 girl 1

grid = add_predictions(grid, M1, "pred")

ggplot(grid, aes(x=lwealth, y= pred, col=sex)) +
 geom_point()
```



I used `geom_point()` to show the individual predictions. Normally we would use `geom_line()`

## 68.4 Centering, an alternative approach

You can also center your continuous variable, which means your coefficient for your main effect on your dummy variable will correspond to the average value of the continuous. Much more interpretable!



```
qm = mutate(qm, lwealth.cent = lwealth - mean(lwealth))
M1b = lm(interact ~ sex * lwealth.cent + sibling, data=qm)
```

```
coef(M0)
```

```
(Intercept) sexgirl lwealth sibling
4.365852906 0.005984015 0.197600022 -0.219953588
```

```
coef(M1b)
```

```
(Intercept) sexgirl lwealth.cent
6.181712978 0.006737327 0.149873199
sibling sexgirl:lwealth.cent
-0.220960207 0.104675040
```

See? Much more interpretable!

## 68.5 Confidence intervals for the gap?

A bootstrap is the easiest here.

```
reps = replicate(1000, {
 qm.star = mosaic::sample(qm, replace=TRUE)

 # refit the original model with the bootstrap data
 M1.star = update(M1, data=qm.star)

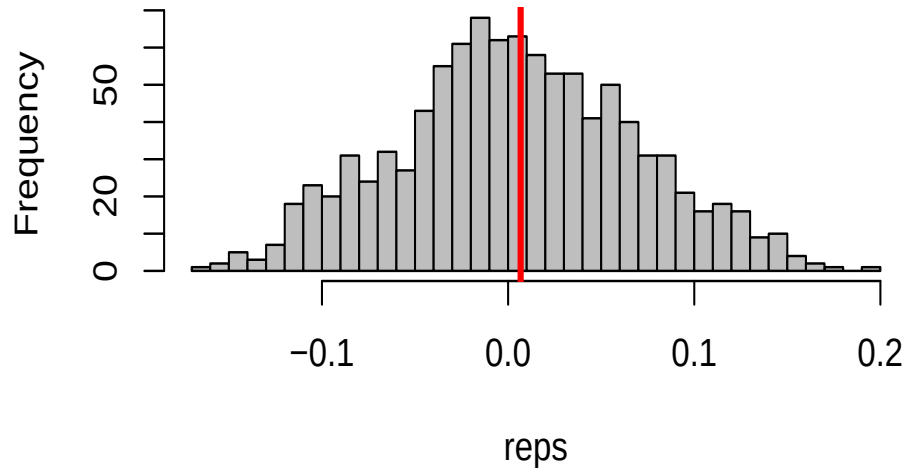
 pred.g = predict(M1.star, qm.g)
 pred.b = predict(M1.star, qm.b)

 deltas = pred.g - pred.b
 mean(deltas)
})
```

Our bootstrap confidence interval: quantile( reps, c( 0.025, 0.975 ) )

```
hist(reps, breaks=30, col="grey")
abline(v=sex.diff, col="red", lwd=3)
```

## Histogram of reps



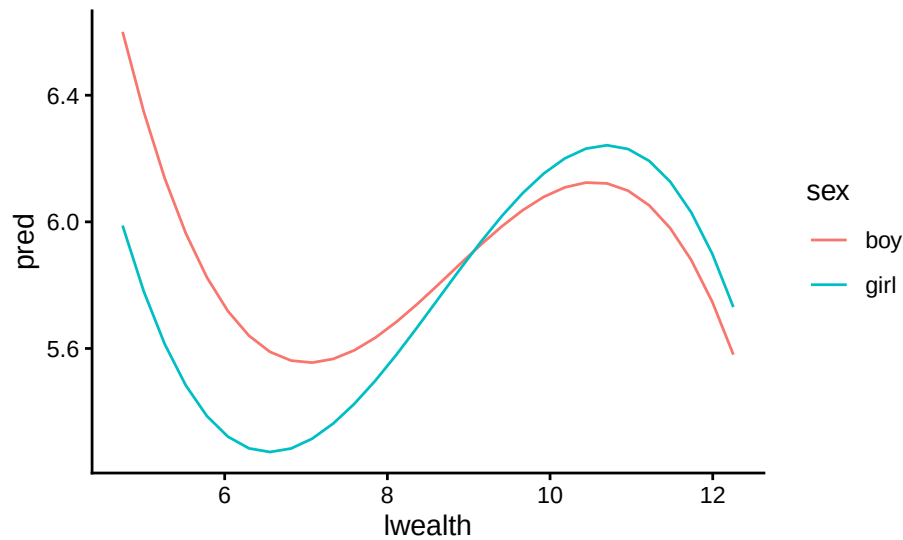
## 68.6 A final investigation

I don't like the linear relationship. Better to have, perhaps a cubic for both girl and boy?

```
M3 = lm(interact ~ (lwealth + I((lwealth-mean(lwealth))^2) +
 I((lwealth-mean(lwealth))^3)) * sex + sibling,
 data=qm)
coef(M3)
```

(Intercept)	3.9677413630	lwealth	0.2376064190
I((lwealth - mean(lwealth))^2)	0.0226463368	I((lwealth - mean(lwealth))^3)	-0.0264912833
sexgirl	-1.0082023177	sibling	-0.2159902502
lwealth:sexgirl	0.1117568699	I((lwealth - mean(lwealth))^2):sexgirl	-0.0122453285
I((lwealth - mean(lwealth))^3):sexgirl	-0.0007258967		

```
grid = add_predictions(grid, M3, "pred")
ggplot(grid, aes(x=lwealth, y= pred, col=sex)) +
 geom_line()
```



In truth, splines would be the best. A topic for another day (or prior handout).

## 68.7 Disclaimer

The coefficients of the above are not significant, and we are not finding a real difference between girls and boys in this case. But the code is designed to illustrate the core concept of using predict to get access to a real estimate of a main effect when you are fitting a model with an interaction term.

To check if a complex model is an improvement, use `anova()`

```
anova(M0, M1, M3)
```

### Analysis of Variance Table

Model 1: `interact ~ sex + lwealth + sibling`

Model 2: `interact ~ sex * lwealth + sibling`

Model 3: `interact ~ (lwealth + I((lwealth - mean(lwealth))^2) + I((lwealth - mean(lwealth))^3)) * sex + sibling`

	Res.Df	RSS	Df	Sum of Sq	F	Pr(>F)
1	1972	3684.0				
2	1971	3679.2	1	4.7838	2.5691	0.10913
3	1967	3662.7	4	16.5032	2.2157	0.06503 .

---

Signif. codes: 0 '\*\*\*' 0.001 '\*\*' 0.01 '\*' 0.05 '.' 0.1 ' ' 1

Nope, the interactions (or the more flexible cubic) are not helping us (other than helping our understanding of how to model interactions).

# 69 Finding and Merging Data Online

An Example with IPEDS Data

## 69.1 Datasets Posted on the Web

Downloading and merging publicly posted datasets is a very important skill. This handout walks through an example case of downloading datasets posted on a government website, cleaning them, merging them together, and using the merged dataset for an analysis. It addresses some of the common challenges of getting acquainted with a data portal.

No guide can cover all cases, but this example should give you a sense of what to look for/think about when wrangling data posted on websites. Note that this is *not about scraping*. What we're considering here are datasets posted for download, often - but not always - found on government sites.

## 69.2 IPEDS

This example uses data from IPEDS, the Integrated Postsecondary Education Data System. These data are made available by NCES, the National Center for Education Statistics, a great place to start looking for education-related data in the US. The data represent a set of surveys, conducted every year, of all colleges, universities, and other post-secondary institutions that take federal student aid money (which is like, many of them). Data are available going back to 1986 (some scattered earlier years also have data). The surveys cover enrollments, graduation rates, prices, and many other things.

There are 12 total survey components that cover 9 major topics:

- Academic Libraries
- Admissions
- Completions

- Enrollment (Fall and 12-Month)
- Finance
- Graduation Rates and Outcome Measures
- Human Resources
- Institutional Characteristics
- Student Financial Aid

It's worth noting that some info is only collected every other year. In addition, the data are not consistent over time. New survey items get added. Old items get dropped. Definitions change. Identifying codes, like the Classification of Instructional Program (CIP) codes or the Standard Occupational Classification (SOC), are periodically reviewed and updated. Being aware of (and dealing with) these issues is part of our job as data scientists. When you're working with new data, take some time to identify these kinds of challenges before you dig too deeply into analysis.

## 69.3 Finding IPEDS Data

The landing page for IPEDS is [here](#). When you open the page, you'll see many links. NCES provides a lot of information about their data (and different ways to access it). Sorting through surfeits of info like this is often the biggest challenge of working with public data sources. My description of IPEDS (in the above section) came from digging around these links.<sup>1</sup>

On the IPEDS page, there's a link to their [data explorer](#). When you hit this page, you'll see links to the most recently released surveys. You can filter the datasets by year or by which of the 12 survey components you're interested in. These data are aggregated, but it's a good way to get a sense of what information is available in the individual survey components.

To *get* the survey data, look for this section of the landing page:

---

<sup>1</sup>Along the way, I encountered some broken pages, like [this link to the IPEDS data release calendar](#). Maybe they'll fix it, but you can often find the information you need on some other page. When that doesn't work, ask someone who's familiar with the data. When *that* doesn't work, you can email someone who works for the agency responsible for the data. In my experience, people get back to you reasonably quickly and are happy to be helpful, as long as you write politely to them.



## Survey Data

Data are available starting with the 1980-81 collection year for the *Complete data files* and *Custom data files* functions, which zip the data into comma separated value (\*.csv). Beginning with the 2004-05 collection year, data for each collection year are compiled into an *Access database*.

Select download option



In the dropdown menu, select “Complete Data Files”. This will take you to a page with download links for each survey component. The files we want are found in the “Data File” column. Let’s download this admissions and test scores file,

2021	Admissions and Test Scores	Admission considerations, applications, admissions, enrollees and test scores, fall 2021	<a href="#">ADM2021</a>
------	----------------------------	------------------------------------------------------------------------------------------	-------------------------

When we download the file, we should put it in a folder on our computer that’s dedicated to this project. That’ll make things easier once we start writing code. On my laptop, I’ve made a folder called “ipeds\_handout”. Inside *that* folder, I made another folder called “data”. I put the downloaded file into that “data” folder. When I start writing code, I’ll store it in the parent folder (“ipeds\_handout”).

The downloaded file is a zipped csv. We know how to work with a csv in R, but before we can do that, we have to unzip the file. If you haven’t done this before, [here’s a page](#) that walks through how to unzip files on PC or Mac. Once you have the unzipped csv, you can delete the original zip file to tidy things up.

While we’re at it, let’s grab one more file, data on instructional staff salaries,

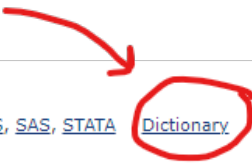
2021	Instructional Staff/Salaries	Number and salary outlays for full-time nonmedical instructional staff, by gender, and academic rank: Academic year 2021-22	<a href="#">SAL2021 IS</a>
------	------------------------------	-----------------------------------------------------------------------------------------------------------------------------	----------------------------

Like before, unzip the file and put the csv in a project-specific folder.<sup>2</sup> You should also download and unzip the dictionary files for both datasets.

---

<sup>2</sup>To be clear, this should be the same folder as the first dataset. Also, if you’re working with many datasets for a project, it’s a good idea to rename them at this stage. Give the datasets clear, descriptive names so you know what they contain. This will make it easier to write and read code later.

2021	Admissions and Test Scores	Admission considerations, applications, admissions, enrollees and test scores, fall 2021	<a href="#">ADM2021</a>	<a href="#">ADM2021_STATA</a>	<a href="#">SPSS, SAS, STATA</a>	<a href="#">Dictionary</a>
------	----------------------------	------------------------------------------------------------------------------------------	-------------------------	-------------------------------	----------------------------------	----------------------------



The dictionaries contain codebooks that explain what each variable and value in the dataset means. These usually aren't big files, so we can open and browse them directly in Excel. I'm going to use them soon to figure out how to find the information I'm interested in.

Now we're ready to get started.

## 69.4 Our Example Case

Let's start by loading our libraries and data.

```
library(tidyverse)
library(ggplot2)

admit <- read_csv('data/adm2021.csv')
```

Rows: 1981 Columns: 68

```
-- Column specification -----
Delimiter: ","
chr (29): XAPPLCN, XAPPLCNM, XAPPLCNW, XADMSSN, XADMSSNM, XADMSSNW, XENRLT, ...
dbl (39): UNITID, ADMCON1, ADMCON2, ADMCON3, ADMCON4, ADMCON5, ADMCON6, ADMC...
```

i Use ``spec()`` to retrieve the full column specification for this data.  
i Specify the column types or set ``show_col_types = FALSE`` to quiet this message.

```
sal <- read_csv('data/sal2021_is.csv')
```

Rows: 15382 Columns: 110

```
-- Column specification -----
Delimiter: ","
chr (54): XSAINSTT, XSAINSTM, XSAINSTW, XSA_9MCT, XSA_9MCM, XSA_9MCW, XSATOT...
dbl (56): UNITID, ARANK, SAINSTT, SAINSTM, SAINSTW, SA_9MCT, SA_9MCM, SA_9MC...
```

i Use ``spec()`` to retrieve the full column specification for this data.  
i Specify the column types or set ``show_col_types = FALSE`` to quiet this message.



With the datasets we've downloaded, we can explore the relationship between assistant professor salaries and the math ability of incoming Freshmen. To begin, we narrow down each dataset to the variables of interest. To identify these, I looked at the dictionaries we downloaded from IPEDS.

```
Keep only school IDs and average salary for 9-month
Contract instructors
sal <- sal %>% filter(ARANK==3) %>%
 # The dictionary tells us that these are assistant profs
 select(UNITID, SA09MAT)

Keep school IDs, # of applicants, % that submit SATs,
and 75th pctile of SAT scores
admit <- admit %>% select(UNITID, APPLCN, SATPCT, SATMT75)
```

In other cases, you might have to do much more cleaning before you merge these datasets together. In general, the process will be to clean each dataset so they can be easily merged. (Of course, more data cleaning may be necessary, post-merge.)

#### 69.4.1 Merging the Admissions and Salary Data

It's a good idea to do a full join. This will keep all rows from both datasets and allow us to explore which rows did and did not match, post-merge. That said, I often start with a left or right join just as a quick check of how many records match.

```
df <- left_join(sal, admit, by='UNITID')
sum(is.na(df$SATPCT))
```

```
[1] 1298
```

About half (around 1,300 institutions) don't have a match. This should be explored. Maybe it's from particular types of institutions (e.g. Vocational/Technical Schools). We might have to merge in additional data to explore the matching behavior. We might figure it out by reading carefully through the dictionaries or other documentation on IPEDS. You can take these steps by following the framework laid out above.

For now, we can run a preliminary regression and call it a day.

```
First let's mean-center both variables
df <- df %>% mutate(across(c(SA09MAT, SATMT75), ~.x-mean(.x, na.rm=TRUE)))

summary(lm(SA09MAT ~ SATMT75, df))
```

```

Call:
lm(formula = SA09MAT ~ SATMT75, data = df)

Residuals:
 Min 1Q Median 3Q Max
-49118 -8410 -889 8346 52056

Coefficients:
 Estimate Std. Error t value Pr(>|t|)
(Intercept) 2829.703 459.431 6.159 1.18e-09 ***
SATMT75 164.646 6.186 26.616 < 2e-16 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 12740 on 771 degrees of freedom
(1529 observations deleted due to missingness)
Multiple R-squared: 0.4788, Adjusted R-squared: 0.4782
F-statistic: 708.4 on 1 and 771 DF, p-value: < 2.2e-16

```

It looks like assistant professors make more at schools with higher SAT math scores (as measured by the 75th percentile of the distribution). Assistant profs at schools with 100 points higher scores earn about 10k more on average. There are lots of reasons we shouldn't put much weight on this result. Lots of rows didn't merge. We haven't investigated missingness in the data. But the above steps walk through the process of a first, rough-cut analysis with IPEDS data.

## 70 Applied Prediction Papers

Below is a list of papers related to predictive data science and machine learning in public policy (including education). These are organized into: (1) surveys and reviews and (2) original applications. You may find many of these papers helpful as you are working on your final project and interpreting results - please peruse them with this use case in mind as well as for your general understanding of how these methods are used in the real world.

## 71 *Surveys and reviews*

The following list includes review articles, textbooks, and textbook chapters that provide an overview of the application of machine learning in various fields related to public policy. These fields include education, public health, urban planning, economics, and urban planning, among many others. Many of these works discuss or cite specific applications of these methods within these fields as well (including some of the references that follow after this section).

**71.0.0.1** Athey, Susan. “Beyond Prediction.” *Science* (American Association for the Advancement of Science), vol. 355, no. 6324, 2017, pp. 483–85, <https://doi.org/10.1126/science.aal4321>.

Discussion of the challenges of using machine learning predictions to improve public policies. Includes many references to real-world uses of predictive data science.

**71.0.0.2** Athey, Susan. The Impact of Machine Learning on Economics. In: *The Economics of Artificial Intelligence: An Agenda* [Internet]. University of Chicago Press; 2018 [cited 2023 Mar 31]. p. 507–47. Available from: <https://www.nber.org/books-and-chapters/economics-artificial-intelligence-agenda/impact-machine-learning-economics>

This chapter discusses use of machine learning in economics and includes a discussion of “prediction policy” applications for informing economic decision-making.

**71.0.0.3** Casali, Ylenia, et al. Machine learning for spatial analyses in urban areas: a scoping review. *Sustainable Cities and Society*. 2022 Oct 1;85:104050.

Authors review the use of machine learning in city/urban planning. Specific attention is given to machine learning applications involving spatial data.

**71.0.0.4 Hu, Xindi C., et al. “The Utility of Machine Learning Models for Predicting Chemical Contaminants in Drinking Water: Promise, Challenges, and Opportunities.” *Current Environmental Health Reports*. 2023 Mar;10(1):45–60.**

Review of the use and challenges of machine learning models for predicting chemical contaminants in drinking water. Also discusses their frequent use to guide sampling efforts by prioritizing at-risk areas.

**71.0.0.5 Payedimarri, Anil Babu, et al. Prediction Models for Public Health Containment Measures on COVID-19 Using Artificial Intelligence and Machine Learning: A Systematic Review. *International Journal of Environmental Research and Public Health*. 2021 Jan;18(9):4499.**

A brief review of the use of machine learning (and artificial intelligence) methods to evaluate public health interventions to contain the spread of SARS-CoV-2.

**71.0.0.6 Perry, Walt L. 2013. Predictive Policing: The Role of Crime Forecasting in Law Enforcement Operations. Rand Corporation.**

Open access book gives an overview of common predictive policing practices.

**71.0.0.7 Williamson, Ben. 2016. “Digital education governance: data visualization, predictive analytics, and ‘real-time’ policy instruments.” *Journal of Education Policy* 31(2):123-141.**

Broad overview of uses of modern education data. Includes a section on predictive analytics. See first full paragraph on p. 136.

**71.0.0.8 Baker, R. S., Martin, T., & Rossi, L. M. (2016). Educational Data Mining and Learning Analytics. In *The Wiley Handbook of Cognition and Assessment* (pp. 379–396). John Wiley & Sons, Ltd. <https://doi.org/10.1002/9781118956588.ch16>**

This chapter written by experts in the field provides a nice overview of educational data mining and learning analytics. Data obtained through naturally occurring log data (e.g., from learning management systems) or specifically procured sources (e.g., eyetracking) can be used to both make inferences and predictions on learning behavior and outcomes.

## 72 *Original applications*

The following list includes papers that apply machine learning methods to specific research questions. All are related to prediction and public policy, but span a similarly wide range of disciplines. While reading these, focus on the descriptions of the methods and results. This may be useful for your final project (and, of course, more generally!).

**72.0.0.1 Bansak, Kirk, et al. “Improving Refugee Integration through Data-Driven Algorithmic Assignment.” *Science* (American Association for the Advancement of Science), vol. 359, no. 6373, 2018, pp. 325–29.**

In this paper, the authors propose a model that aims to predict where refugees will integrate best. They suggest governments take up their approach to make their refugee programs more efficient.

**72.0.0.2 Goel S, Rao JM, Shroff R. Precinct or prejudice? Understanding racial disparities in New York City’s stop-and-frisk policy. *The Annals of Applied Statistics*. 2016 Mar;10(1):365–94.**

The authors of this paper used machine learning to estimate the probability that a detained individual truly has a weapon for stops related to suspicion of criminal weapon possession in New York City. Disproportionate stops by racial/ethnic groups, and the factors resulting in these disparities, are also discussed.

**72.0.0.3 Hino, M, et al. Machine learning for environmental monitoring. *Nature Sustainability*. 2018 Oct;1(10):583–8.**

Demonstration of how machine learning methods can help allocate resources to more efficiently conduct inspections for violations of the Clean Water Act.

**72.0.0.4 Kelly, Sean, et al. “Automatically Measuring Question Authenticity in Real-World Classrooms.” *Educational Researcher*, vol. 47, no. 7, 2018, pp. 451–64, Available from: <https://doi.org/10.3102/0013189X18785613>.**

This team uses regression trees to predict which teacher questions are “authentic”.

**72.0.0.5 Lee Kwang-Sig, et al. Association of Preterm Birth with Depression and Particulate Matter: Machine Learning Analysis Using National Health Insurance Data. *Diagnostics*. 2021 Mar;11(3):555.**

Demonstrates a use of machine learning to predict and identify the majors determinants of preterm birth in South Korea. Authors discuss that strategies to reduce air pollution could be an effective intervention based on these findings.

**72.0.0.6 Yoo, Sanglim. Investigating important urban characteristics in the formation of urban heat islands: a machine learning approach. *Journal of Big Data*. 2018 Jan 24;5(1):2.**

The author presents and discusses use of random forest to predict the formation of urban heat islands in Indianapolis, Indiana.

**72.0.0.7 <https://chicago.github.io/food-inspections-evaluation/>**

The city of Chicago uses a predictive model to decide which food establishments are inspected first.

**72.0.0.8 Romero, C., López, M. I., Luna, J. M., & Ventura, S. (2013). Predicting students' final performance from participation in on-line discussion forums. *Computers & Education*, 68, 458-472.**

The researchers used classification and clustering to predict student performance based on a collection of features from an online discussion forum. This paper is a good example of the complicated data cleaning and feature pre-processing usually required for learning analytics (LA) work. It also demonstrates the various models that can be used in the process; plus, the paper actually explains what the models are and how they work before going into the results. Overall, it is a good entry paper into the field.

**72.0.0.9 Bozick, Robert and Dalton, Benjamin. Balancing Career and Technical Education With Academic Coursework: The Consequences for Mathematics Achievement in High School. *Educational Evaluation and Policy Analysis*. 2013 Jun 1;35(2):123-38.**

These researchers assessed the ability of career and technical education courses to improve student learning. Their work provides an example of the use of bootstrapping methods to calculate standard errors of regression coefficients.

Van Buuren, Stef, and Karin Groothuis-Oudshoorn. 2011. "Mice: Multivariate Imputation by Chained Equations in r." *Journal of Statistical Software* 45: 1–67.